Thinking in R

Gilles Colling

First Edition

2026

Table of contents

Preface 1

Foreword 3

I. Part I: Foundations 5

1. Two models of computation 7
1.1. Instructions and state . . . 8
1.2. Expressions and functions . . . 9
1.3. What this has to do with R . . . 10

2. R's family tree 13
2.1. Church's lambda calculus becomes a programming language . . . 14
2.2. Scheme strips it down . . . 15
2.3. S makes it practical . . . 16
2.4. R starts in Auckland . . . 17
2.5. What R inherited . . . 17

3. Expressions and names 19
3.1. The REPL . . . 19
3.2. Names and binding . . . 20
3.3. Expressions all the way down . . . 22
3.4. Calling functions . . . 23
3.5. The workspace . . . 25

4. Vectors 27
4.1. There are no scalars . . . 27
4.2. Atomic vectors . . . 28
4.3. Coercion . . . 29
4.4. Vectorized operations . . . 31
4.5. Indexing . . . 34
4.6. Missing values . . . 35
4.7. Meet penguins . . . 37

5. Your first function 39
5.1. Anatomy of a function . . . 39
5.2. Arguments and defaults . . . 40
5.3. Return values . . . 41
5.4. Functions are just values . . . 43
5.5. Scope: a first look . . . 44
5.6. Guarding your inputs . . . 46
5.7. Recursion . . . 47
5.8. Programs and proofs . . . 50

6. Numerical computing 53
6.1. How R stores numbers . . . 53

6.2. Floating-point traps . . . 54
6.3. Catastrophic cancellation . . . 56
6.4. Integers . . . 57
6.5. Numerical stability in practice . . . 58
6.6. Matrix algebra . . . 61
6.7. When R lies and when it is precise . . . 63

7. Functions are values **65**
7.1. Passing functions to functions . . . 65
7.2. Anonymous functions . . . 66
7.3. Storing functions in lists . . . 67
7.4. Functions that return functions . . . 69
7.5. Operators are functions . . . 71

8. Logic and control flow **73**
8.1. Logical vectors . . . 73
8.2. Comparison operators . . . 74
8.3. Boolean operators . . . 75
8.4. How numbers are stored . . . 82
8.5. `if/else` as an expression . . . 83
8.6. Vectorized conditionals . . . 84
8.7. `switch()` . . . 86

9. Complexity and algorithms **87**
9.1. Not all operations are equal . . . 87
9.2. Big-O notation . . . 87
9.3. Hash tables: O(1) lookup from scratch . . . 90
9.4. Space complexity . . . 92
9.5. Common algorithms in R . . . 94
9.6. Computability . . . 96

II. Part II: Working with data **101**

10. Lists **103**
10.1. Why lists exist . . . 103
10.2. Creating and accessing lists . . . 104
10.3. Nested lists . . . 107
10.4. Lists as the backbone of R . . . 108
10.5. Modifying lists . . . 109
10.6. Linked lists . . . 110
10.7. Looking ahead . . . 112

11. Data frames **113**
11.1. A list of equal-length vectors . . . 113
11.2. Creating data frames . . . 115
11.3. Subsetting data frames . . . 116
11.4. Adding and modifying columns . . . 118
11.5. Combining data frames . . . 119

11.6. Tibbles . . . 120
11.7. Exploring data frames . . . 122
11.8. Putting it together . . . 124

12. Strings, factors, and dates 125
12.1. Character vectors . . . 125
12.2. stringr: consistent string operations . . . 127
12.3. Regex essentials . . . 128
12.4. Why factors exist . . . 130
12.5. forcats: taming factors . . . 132
12.6. Dates and times . . . 133
12.7. lubridate: dates for humans . . . 135
12.8. Summary . . . 136

13. Reading and writing data 139
13.1. File paths . . . 139
13.2. Reading CSV files . . . 140
13.3. Writing CSV files . . . 142
13.4. Excel files . . . 143
13.5. Databases . . . 144
13.6. Binary and columnar formats . . . 145
13.7. Encodings . . . 146

14. Data transformation 149
14.1. The idea: verbs for data . . . 149
14.2. `filter()`: keep rows . . . 150
14.3. `select()`: pick columns . . . 152
14.4. `mutate()`: compute new columns . . . 153
14.5. `summarise()` and `group_by()`: aggregate . . . 155
14.6. `arrange()`: order rows . . . 158
14.7. More verbs . . . 158
14.8. The base R equivalents . . . 160

15. Pipes and composition 163
15.1. The problem with nesting . . . 163
15.2. The pipe: `|>` . . . 164
15.3. Building a pipeline . . . 165
15.4. Function composition . . . 166
15.5. The placeholder `_` . . . 167
15.6. `|>` vs `%>%` . . . 168
15.7. When not to pipe . . . 169

16. Tidy data 173
16.1. What tidy means . . . 173
16.2. `pivot_longer()`: columns to rows . . . 175
16.3. `pivot_wider()`: rows to columns . . . 178
16.4. When data isn't tidy (and that's fine) . . . 179
16.5. Joins: combining tables . . . 180

16.6. Other tidyr tools . . . 182
16.7. Summary . . . 184

17. Visualization **185**
17.1. The grammar of graphics . . . 185
17.2. Aesthetics: data to visuals . . . 186
17.3. Geoms: shapes for data . . . 189
17.4. Scales: controlling the mapping . . . 193
17.5. Facets: small multiples . . . 196
17.6. Themes: non-data styling . . . 197
17.7. Putting it together . . . 199
17.8. ggplot2 as lambda calculus . . . 201
17.9. Common mistakes . . . 202
17.10Saving plots . . . 203

III. Part III: Thinking functionally **205**

18. Closures and scope **207**
18.1. Environments . . . 207
18.2. Lexical scoping . . . 209
18.3. Execution environments . . . 211
18.4. Closures . . . 213
18.5. `<<-` and mutable state . . . 214
18.6. Closures as portable state . . . 216
18.7. Inspecting environments . . . 218

19. Iteration without loops **221**
19.1. The problem with `for` loops . . . 221
19.2. `lapply()` and `sapply()`: base R functionals . . . 223
19.3. `purrr::map()`: the tidyverse functional . . . 224
19.4. Typed variants . . . 226
19.5. Iterating over multiple inputs . . . 227
19.6. Side effects: `walk()` . . . 228
19.7. When loops are fine . . . 230
19.8. `across()` revisited . . . 230

20. Function factories **233**
20.1. The pattern . . . 233
20.2. The lazy evaluation trap . . . 234
20.3. Practical factories . . . 236
20.4. Memoization . . . 238
20.5. Function operators . . . 239
20.6. Composing factories and operators . . . 241

21. Reduce and accumulate **243**
21.1. Left fold . . . 243
21.2. The init argument . . . 244
21.3. Right fold . . . 245

21.4. accumulate: keeping the intermediates . . . 245
21.5. purrr::reduce() and purrr::accumulate() . . . 246
21.6. Practical patterns . . . 247
21.7. Row-wise folds with c_across() . . . 248
21.8. When to fold and when not to . . . 249

22. Recursion and fixed points 251
22.1. Recursive functions . . . 251
22.2. The call stack . . . 252
22.3. Tail recursion and accumulators . . . 254
22.4. Trampolining . . . 255
22.5. Divide and conquer . . . 256
22.6. Recursive list processing . . . 258
22.7. Mutual recursion . . . 259
22.8. Fixed-point combinators . . . 260
22.9. When not to recurse . . . 261
22.10Historical notes . . . 262

23. Lazy evaluation 265
23.1. Promises . . . 265
23.2. Consequences of laziness . . . 267
23.3. What is non-standard evaluation? . . . 269
23.4. Data masking . . . 271
23.5. Tidy evaluation basics . . . 272
23.6. The trade-offs of NSE . . . 274

IV. Part IV: The type system 275

24. S3, S4, and S7 277
24.1. What dispatch means . . . 277
24.2. S3 basics . . . 279
24.3. Writing S3 classes properly . . . 280
24.4. Inheritance . . . 281
24.5. Why S3 works . . . 282
24.6. S4 in brief . . . 283
24.7. S7 . . . 283

25. Contracts and defensive code 287
25.1. The case for validation . . . 287
25.2. `stop()`, `warning()`, and `message()` . . . 289
25.3. Writing informative errors . . . 290
25.4. Common validation patterns . . . 292
25.5. The checkmate package . . . 293
25.6. `tryCatch()` and error handling . . . 294
25.7. `rlang::abort()` and cli . . . 296
25.8. Designing function interfaces . . . 297
25.9. Summary . . . 298

26. Metaprogramming **299**
26.1. Code is data . . . 299
26.2. Abstract syntax trees . . . 300
26.3. Capturing and evaluating expressions . . . 302
26.4. Building expressions programmatically . . . 304
26.5. Formulas as expressions . . . 305
26.6. When to use metaprogramming . . . 307
26.7. The metaprogramming toolkit . . . 308

27. Building a DSL **309**
27.1. R's existing DSLs . . . 309
27.2. How `aes()` works . . . 311
27.3. Building a unit-aware DSL . . . 312
27.4. Techniques for DSL construction . . . 315
27.5. `eval(parse(text = ...))`: the anti-pattern . . . 316
27.6. Design principles . . . 317
27.7. Putting it all together . . . 318

V. Part V: Building on R **321**

28. Performance **323**
28.1. Profile before you optimize . . . 323
28.2. Vectorization . . . 325
28.3. Memory: pre-allocate and avoid copies . . . 328
28.4. data.table . . . 330
28.5. Columnar engines: Arrow and DuckDB . . . 331
28.6. When and how to call compiled code . . . 333
28.7. The optimization checklist . . . 335

29. R internals **337**
29.1. Stack and heap . . . 337
29.2. Every object is a SEXP . . . 337
29.3. Inspecting objects . . . 339
29.4. Memory layout of vectors . . . 341
29.5. Reference counting and copy-on-modify . . . 342
29.6. Garbage collection . . . 344
29.7. The evaluator . . . 346
29.8. Promises at C level . . . 347
29.9. Closures at C level . . . 348
29.10. Three roads to C . . . 349
29.11. Reading R's source code . . . 350
29.12. Putting it together . . . 351

30. R as mathematics **353**
30.1. Lambda calculus in one page . . . 353
30.2. Church booleans . . . 354
30.3. Church numerals . . . 356

30.4. Church pairs and lists . . . 358
30.5. The Z combinator . . . 360
30.6. The punchline . . . 362
30.7. Types as propositions . . . 362
30.8. Functors, monoids, and friends . . . 363
30.9. Practical mathematics in R . . . 365
30.10 The language is still moving . . . 367
30.11 The historical thread . . . 367
30.12 References and sources . . . 368

31. Connecting to other languages **371**
31.1. C via `.Call()` . . . 371
31.2. C++ via Rcpp . . . 373
31.3. Rust via extendr . . . 375
31.4. Python via reticulate . . . 376
31.5. Fortran . . . 378
31.6. When to use what . . . 379
31.7. Packages worth studying . . . 380
31.8. References and sources . . . 381

32. Building packages **383**
32.1. Why packages . . . 383
32.2. The anatomy of a package . . . 384
32.3. Creating a package . . . 385
32.4. Writing documentation with roxygen2 . . . 386
32.5. Dependencies . . . 388
32.6. Testing with testthat . . . 389
32.7. Vignettes . . . 390
32.8. Basic usage . . . 391
32.9. Sharing your package . . . 392

33. Reproducibility and workflow **395**
33.1. What reproducibility means . . . 395
33.2. Projects and file paths . . . 396
33.3. renv: locking dependencies . . . 397
33.4. Quarto: code and prose together . . . 398
33.5. targets: pipeline automation . . . 401
33.6. Version control with Git . . . 403
33.7. set.seed() and session info . . . 405
33.8. The reproducibility stack . . . 406
33.9. Beyond renv: full-stack reproducibility . . . 406

Afterword **409**

Index **411**

Preface

R descends from the lambda calculus, by way of Scheme and a Bell Labs language called S. That ancestry shaped every part of the language. All of it traces back to a single design decision made in the 1970s.

You'll learn R from scratch: how to store and manipulate data, write your own functions, transform messy tables into clean results, and build visualizations. You'll also learn *why* R works the way it does. By the end, you'll understand functional programming, know how to work with real data, and see ideas (closures, composition, immutability) that show up in every modern language. No prior programming experience is required.

Who this book is for

R beginners who want to understand the language properly will find the full path here. Intermediate R users who learned the tidyverse first and want to fill in the foundational layer underneath it will find it too. Programmers coming from Python, JavaScript, or another language who need to pick up R will see how it connects to ideas they already know. The book assumes you are comfortable with a text editor and a terminal; it does not teach those from zero.

How the book is organized

The book has five parts. Part I (Chapters 1–9) builds the foundations: expressions, vectors, functions, control flow, and algorithmic thinking. Part II (Chapters 10–17) covers data; lists, data frames, strings, reading files, transformation with dplyr, tidy data, and visualization with ggplot2. Part III (Chapters 18–23) covers closures, higher-order functions, map/reduce, recursion, and lazy evaluation. Part IV (Chapters 24–27) introduces the type system; S3, S7, defensive coding, metaprogramming, and building a small DSL. Part V (Chapters 28–33) covers performance, R internals, connecting to other languages, package development, and reproducible workflows.

Setup

You need two things installed: R itself and an editor.

1. Install R from https://cran.r-project.org/ (download the version for your operating system).
2. Install RStudio Desktop (free) or use VS Code with the R extension.

The book uses base R throughout; several chapters also use tidyverse packages. Install them once by running in the R console:

```
install.packages("tidyverse")
```

Foreword

> "You know, people think mathematics is complicated. Mathematics is the simple bit. It's the stuff we can understand. It's cats that are complicated."
>
> — John Horton Conway

Type `TRUE + TRUE` into an R console. You get `2`. Why? R is not doing arithmetic on the word "true." It converts a logical value to the number `1`, twice, and adds. That conversion follows a rule, and the rule traces back to a design decision that predates R by sixty years and programming itself by two decades. Where did it come from?

In 1936, a logician named Alonzo Church defined booleans as functions that choose: TRUE picks the first argument, FALSE picks the second. No special boolean type, no `if` keyword — just functions calling functions. John McCarthy built a programming language from that idea in 1958 and called it Lisp. Guy Steele and Gerald Sussman distilled it into Scheme in 1975. John Chambers designed S at Bell Labs in the 1980s with Scheme's scoping rules in his head, and two statisticians in Auckland built R from S in the 1990s. The `TRUE + TRUE` in your console still carries Church's fingerprint, and so does nearly everything else you type into R.

Part I

Part I: Foundations

1 Two models of computation

By 1930, the dream was nearly complete. Three centuries of work — formalizing numbers, formalizing logic, formalizing proof — had brought mathematicians to the edge of something unprecedented: a mechanical method that could settle any mathematical question. Feed in a statement, turn the crank, get the answer. The greatest living mathematician stood before a crowd in Königsberg and declared it would be done. *Wir müssen wissen. Wir werden wissen.* We must know. We shall know.

But to build the machine that turns the crank, someone first had to answer a question that nobody had thought to ask. What does it mean to compute?

David Hilbert had spent thirty years building toward that question. To finish his life's work, he needed someone to define what "turning the crank" actually meant.

Once upon a time, four numbers had been left on the table: 3, 5, 2, 8.

Mr. Tape came in through the front door. He was long and thin, and he walked in a straight line from the door to the table. Behind him came Mr. State, who took his place behind Mr. Tape and waited, as he always did. But they were not the only ones there. At the far end of the table someone was already sitting with his hands flat on the surface.

Mr. Tape touched the first number and read it aloud.

"Three."

Mr. State was holding a 3. No one had seen where it came from. On the table, the 3 was still there. He seemed pleased.

Mr. Tape touched the next.

"Five."

Mr. State was holding an 8. The 3 had slipped his mind. He looked at the 8 as though he had always been holding it.

"Two."

He was holding a 10.

"Eight."

He was holding an 18. He could not have told you what he'd had a moment before.

"Eighteen," said Mr. Tape, as was his duty.

The man at the far end of the table had been watching.

"Let's see," said Mr. Fold at last.

Mr. Fold picked up the 2 and the 8. He pressed their edges together and creased them flat. A 10 sat on the table. The 2 and the 8 were gone.

That seemed reasonable.

He folded the 10 into the 5 the same way. Crease and press again. Then the 15 into the 3.

Mr. State stepped forward and picked it up. He turned it over.

It was the same number.

1.1. Instructions and state

Mr. Tape and Mr. State are a computing machine. In 1936, a twenty-four-year-old graduate student at Cambridge proved that their method — read a symbol, update a state, move to the next — is enough to carry out any computation that can be precisely described. The graduate student was Alan Turing, who three years later would crack the German military's Enigma cipher in World War II.

A Turing machine *does* things: it reads, writes, moves, changes. It has memory (the tape), and that memory changes over time. A program, in this view, is a sequence of instructions that progressively modify stored values until the answer emerges.

Here is what that looks like as pseudocode, for a simple sum of four numbers:

```
total = 0
for each number in [3, 5, 2, 8]:
    total = total + number
return total
```

`total` starts at 0, becomes 3, becomes 8, becomes 10, becomes 18. Mr. Tape read the numbers; Mr. State held the running total, forgetting each previous value the moment the next one arrived. The variable *varies*; the answer lives in its final state. This pattern dominated programming for the next seventy years.

Turing's tape machine was an abstraction, but the first real computers were not far off. ENIAC, built at the University of Pennsylvania, weighed 30 tons, stretched 100 feet long, and filled an entire room. It was programmed by wiring plugboards and flipping switches, one instruction at a time. The language that crystallized this model for a generation of programmers was C, designed by Dennis Ritchie at Bell Labs in the early 1970s. In C, the programmer allocates memory, reads from it, writes to it, and frees it when done. The language does not manage any of this for you. R itself is written in C. And every popular language that followed inherited the same picture, even when they made it friendlier. Python handles memory for you and reads almost like pseudocode, yet underneath, variables are still containers and a program is still a sequence of instructions changing what's inside them.

The `for` loop above could be Python with one character changed. JavaScript, created at Netscape (Brendan Eich wrote the first version in ten days), has since become the most widely used programming language in the world. All of them share the same underlying model of what computation is. It dominates so thoroughly that you might assume it's the only one there is.

So is it?

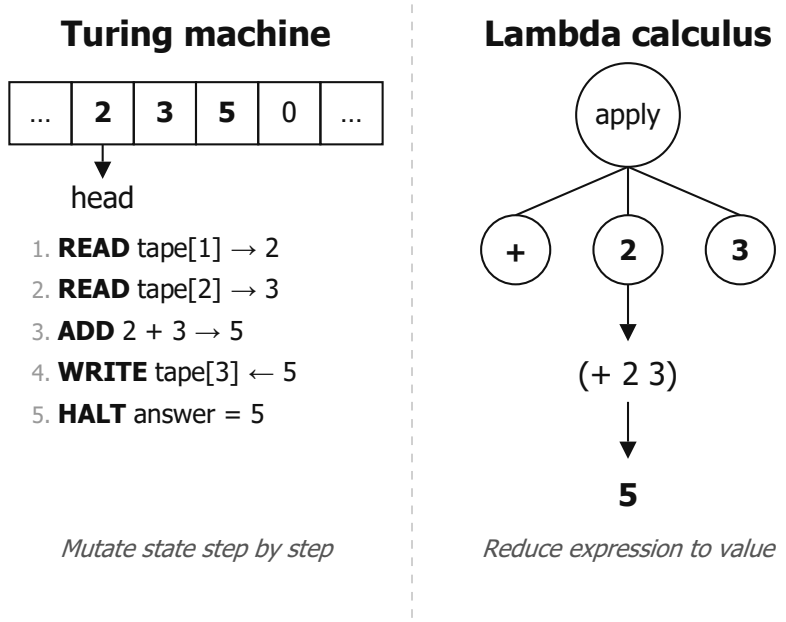


Figure 1.1.: Two models of computation: a Turing machine mutates state step by step; Church's lambda calculus reduces an expression to a value.

1.2. Expressions and functions

Mr. Fold used no tape and kept no state. He pressed numbers together, pair by pair, each pair replaced by its sum and the originals destroyed, until one remained. Replacement instead of memory, and yet the same answer as Mr. Tape and Mr. State. Can Mr. Fold match their infinite reading and rewriting power?

Alonzo Church, who happened to be Turing's doctoral advisor at Princeton, was working on the same problem at the same time but from a completely different angle. Church built a notation called the *lambda calculus*. It has three things:

1. Variables: `x`
2. Functions: `x. x + 1` (take `x`, return `x + 1`)
3. Application: `(x. x + 1)(5)` gives `6`

There is no tape, no memory, no state that changes over time. You write an expression and simplify it by applying functions to arguments until you reach a value that can't be simplified further. Church proved that this system, despite having no concept of memory

or instructions, can compute exactly the same class of functions as Turing's machine. The equivalence, known as the Church-Turing thesis, meant that memory and instructions were never necessary for computation in the first place. Everything a Turing machine can do, pure expression can do without a single mutable cell.

What does the same sum look like in this world?

```
sum([3, 5, 2, 8])
= add(3, sum([5, 2, 8]))
= add(3, add(5, sum([2, 8])))
= add(3, add(5, add(2, 8)))
= add(3, add(5, 10))
= add(3, 15)
= 18
```

The expression unfolds, each `add` consuming its arguments, until only 18 remains. This was Mr. Fold's method — no running total, no memory, just pairs collapsing until one number was left. Nothing was overwritten.

1.3. What this has to do with R

R descends from Church's model. The chain runs through four decades and several languages (Chapter 2 tells the full story), but the result is visible right now, without knowing any of that history. Compare the *shape* of the code with the two models from earlier.

Suppose you have a list of exam scores and you want to know how many students scored above 70. In the instruction-and-state style, you would create a counter, loop through the scores, and increment each time you find one above the threshold:

```r
scores <- c(65, 82, 71, 90, 55, 78)

count <- 0
for (s in scores) {
  if (s > 70) {
    count <- count + 1
  }
}
count
```

This works, and R can do it. But there is a variable (`count`) that gets modified in a loop, one step at a time. `count` starts at 0, becomes 1, becomes 2, becomes 3, becomes 4. One slate, always overwritten. Here is the same problem in the expression-and-function style:

```r
scores <- c(65, 82, 71, 90, 55, 78)

sum(scores > 70)
```

One line. `scores > 70` produces a vector of `TRUE` and `FALSE` values; `sum()` counts the `TRUE`s. The expression describes the answer, and R evaluates it.

Both give the answer 4. But the second version is shorter, easier to read, and (as you'll learn in Chapter 4) faster, because R was *designed* for it. The loop version works against the grain; the expression version works with it.

The same pattern runs through the rest of the language:

- `if/else` returns a value, so you can assign its result directly. In most languages, `if` is a statement that *does* something; in R, it's an expression that *produces* something (Chapter 3):

  ```
  y <- if (x > 0) "positive" else "negative"
  ```

- Functions are ordinary values. You can store one in a variable, pass it to another function, or create one on the fly (Section 5.4).
- `x * 2` multiplies every element of `x` at once, with no loop and no index variable. Operations that in other languages require a loop are built into R as expressions on whole vectors (Chapter 4).
- Pipe chains compose functions left to right, each one receiving the result of the previous one. This is Church's core operation, function composition, made into syntax (Chapter 15):

  ```
  data |> filter(x > 5) |> summarize(mean(y))
  ```

But how did Church's 1936 notation, written decades before computers existed, end up inside a language used by statisticians? Chapter 2 follows the chain: Church to Lisp to Scheme to S to R, and what each step added along the way.

2 R's family tree

Consider this small program in R:

```
add_tax <- function(price, rate) price * (1 + rate)

prices <- c(10, 25, 8, 42)
with_tax <- sapply(prices, add_tax, rate = 0.2)
with_tax
```

That code used four ideas R did not invent: first-class functions, vectorized operations, higher-order functions, and lexical scope. They arrived from different languages and different decades.

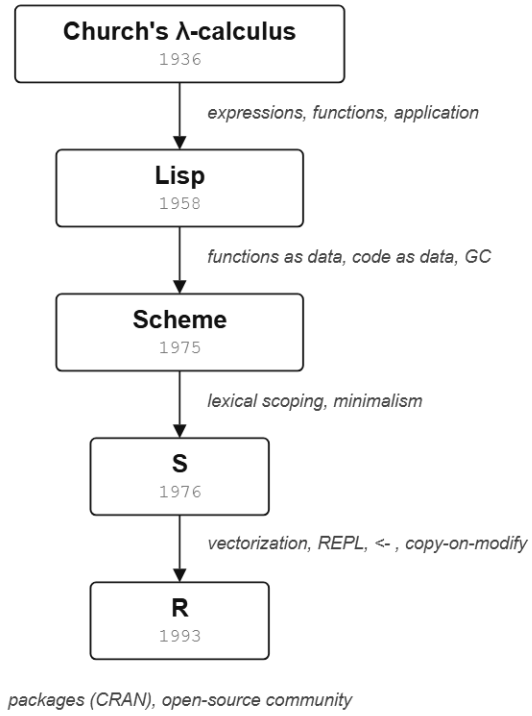


Figure 2.1.: R's lineage: each language passed a core idea to the next.

2.1. Church's lambda calculus becomes a programming language

By 1958, the only real option for writing programs was Fortran, which John Backus and his team at IBM had released the year before. Before Fortran, programming meant writing instructions in the machine's own notation: numbers and abbreviations that corresponded directly to hardware operations. Backus wanted scientists to write something closer to mathematical formulas, and Fortran (short for "Formula Translation") was the result, the first high-level programming language, fast at arithmetic and accessible to people who were not hardware specialists. But a researcher at MIT wanted to write programs for artificial intelligence research, and he needed something Fortran could not do: a language that could manipulate symbols, build up complex structures, and treat its own programs as data.

He found his answer in Church's lambda calculus (Section 1.2). McCarthy took the idea (functions that take arguments and return values, nothing else) and turned it into a programming language called Lisp, short for "list processing."

Lisp introduced ideas that had never existed in a programming language before:

- **Functions as data.** A Lisp program could create a function, store it in a variable, pass it to another function, or return it as a result. No other language did this in 1958.
- **Code as data.** Every Lisp program is itself a list, the same data structure the language manipulates. Programs can inspect, rewrite, and generate other programs, which is why Lisp macros remain unmatched in most languages sixty years later.
- **Garbage collection.** The language manages memory automatically. The programmer never has to say "I'm done with this piece of data; free the memory."

McCarthy had written a mathematical description of how Lisp should evaluate expressions (a function called `eval`), intending it purely as a theoretical exercise. His graduate student Steve Russell read the paper and realized he could translate `eval` directly into machine code for the IBM 704. McCarthy later recalled: "Steve Russell said, look, why don't I program this eval… and I said to him, ho, ho, you're confusing theory with practice, this eval is intended for reading, not for computing. But he went ahead and did it." The result was a working Lisp interpreter, born from the same insight that underpins this whole book: the boundary between mathematical notation and executable code is thinner than it looks.

What does this look like in R?

```
my_function <- function(x) x + 1
my_function(5)
```

```
#> [1] 6
```

`my_function` is a value. You could put it in a list, pass it to another function, or replace it with something else entirely. That idea came straight from Lisp. But Lisp itself had a problem: by the 1970s, it had grown enormous, splintered into competing dialects, and accumulated features nobody could agree on. Two people at MIT decided to strip it back to the bones.

2.2. Scheme strips it down

Gerald Jay Sussman was a professor at MIT's AI Lab; Guy Lewis Steele Jr. was his graduate student. They had been studying Carl Hewitt's Actor model, a theory of concurrent computation where independent "actors" send messages to each other, and to understand it better they decided to implement a small version in Lisp. So they built their own tiny dialect, stripped down to the essentials. Along the way, the distinction between actors and lambda calculus functions dissolved: an actor that receives a message and responds is a function that takes an argument and returns a value.

Out of that realization came Scheme, first described in a 1975 AI Memo at MIT. Between 1975 and 1980, Sussman and Steele published a series of papers now known as "the Lambda Papers," working out the consequences of taking the lambda calculus seriously as a foundation for programming.

Lexical scoping. When a function refers to a variable, where does it look? Older Lisps used *dynamic scoping*, where the meaning of a variable depends on whatever happens to be in scope at the moment the function is called. The same function could behave differently depending on who called it. Scheme chose *lexical scoping*: a variable means what it meant where the function was *defined*, regardless of where it runs. This sounds like a technicality, but it determines whether you can use functions as building blocks at all. R uses lexical scoping, inherited through S from Scheme. Chapter 18 uses this to build function factories and closures.

Minimalism. Scheme showed that you don't need a big language. A small set of well-chosen primitives, all consistent with the lambda calculus, is enough to build anything. This philosophy shaped S and through it R: R's core is surprisingly small, and most of what feels "built in" is actually implemented as ordinary R functions. Even `+` is a function you can call as `` `+`(1, 2) ``, override, or pass to `Reduce`. That uniformity comes from Scheme.

Here is lexical scoping in R. Don't worry about the details yet; just notice that it works:

```
make_adder <- function(n) {
  function(x) x + n
}

add_ten <- make_adder(10)
add_ten(3)
```

```
#> [1] 13
```

`add_ten` remembers that `n` was 10 when it was created, even though `make_adder` has already finished running. The function looks for `n` where it was *defined* (inside `make_adder`), not where it's *called*. That's lexical scoping. Chapter 18 builds on this to create functions that remember things.

Lisp and Scheme were built by computer scientists for computer scientists. Neither had anything to say about data, about columns of measurements and rows of observations, about

the daily work of someone trying to understand an experiment. What neither language had was anything to say about data.

2.3. S makes it practical

In 1976 at Bell Labs, the computer center had a Honeywell 645 mainframe running an operating system called GCOS. If you wanted to do statistics, you used a Fortran library called SCS (Statistical Computing Subroutines): write a Fortran program, call the library functions, compile it, submit it to the batch queue, wait. Rick Becker, John Chambers, Doug Dunn, and Allan Wilks held a series of meetings in the spring of that year, and the question they asked was simple: could they build something interactive, where a statistician types a command and gets an answer back immediately?

They could. The first working version of S ran on the Honeywell 645 that same year. The early system was essentially an interactive front end to the Fortran library, but it grew quickly; by 1988 S had been rewritten in C (version 3), and by 1998 it had become a full programming language (version 4, described in Chambers' book *Programming with Data*). In 1998, S won the ACM Software System Award, the same award given to Unix, TeX, and the World Wide Web.

S made design decisions that R carries to this day:

- **Vectorized operations.** In S, `x * 2` multiplies every element of `x`. There is no loop, no index variable. The operation applies to the whole vector at once because S was built for people who think about columns of data, not individual numbers.
- **The assignment arrow `<-`.** On the terminals available at Bell Labs in the 1970s, there was a key that typed a left arrow as a single character. Chambers used it for assignment. The `=` sign was reserved for named arguments in function calls. R kept both conventions.
- **Copy-on-modify.** When you "modify" a value in S (and later R), the language quietly makes a copy. Your original data is never destroyed. This is a functional programming idea (values don't change), made practical for interactive data analysis: you can always go back.
- **Functions from Scheme.** When Chambers needed to decide how functions should work in S, he looked at Scheme. Functions in S are first-class values with lexical scoping, exactly as in Scheme.

Try this:

```
temperatures <- c(72, 85, 61, 90, 78)
temperatures - 32
```

```
#> [1] 40 53 29 58 46
```

No loop. You subtracted 32 from five numbers in one expression. In Fortran or C you would write a loop, manage an index, and store the result somewhere; S (and R after it) made this the default way of working with data.

S was eventually commercialized as S-PLUS, sold first by StatSci, then by Insightful Corporation, and finally by TIBCO, which acquired Insightful for $25 million in 2008. By then, S had a free competitor that had already overtaken it, one that started with a corridor conversation in New Zealand.

2.4. R starts in Auckland

By 1991, S-PLUS was the only way to use the S language, and it was commercial software — expensive for a university department that needed every student to have a copy. In the Department of Statistics at the University of Auckland, students were stuck with clunky programs that made data analysis feel like filing taxes. Two lecturers in that department had a corridor conversation about the problem, and decided to solve it themselves by writing a new implementation of S from scratch.

The language was not a port of S; it was a new implementation with a different memory model and eventually a different package system, though it followed S's design decisions closely (and through S, Scheme's and Church's). They named it R, a play on their first initials and a nod to S. Ross Ihaka had studied at Berkeley and knew the S language inside out; Robert Gentleman brought computational statistics. Two years after release, Martin Mächler at ETH Zurich convinced them to license it under the GNU General Public License. That decision made R free and open-source, which mattered more than anyone realized at the time: a core development group formed, CRAN started collecting contributed packages, and by the mid-2000s R had become the standard tool for statistical computing in academia.

R looks almost identical to S on the surface, but under the hood it is semantically closer to Scheme. So which pieces came from where?

2.5. What R inherited

Here is the chain, and what each link contributed:

Ancestor	Year	What R inherited
Church's lambda calculus	1936	Everything is an expression. Functions take arguments and return values.
Lisp	1958	Functions as data. Code as data. Garbage collection.
Scheme	1975	Lexical scoping. Minimalism. Taking lambda calculus seriously.
S	1976	Vectorized operations. Interactive data analysis. `<-` assignment. Copy-on-modify. Formula objects.

Ancestor	Year	What R inherited
R	1991	Free implementation. Package system (CRAN). Open-source community.

Look again at the five lines from the top of this chapter:

```
add_tax <- function(price, rate) price * (1 + rate)

prices <- c(10, 25, 8, 42)
with_tax <- sapply(prices, add_tax, rate = 0.2)
with_tax
```

`function(price, rate)` creates a function and stores it in a variable, the way you'd store a number: Lisp, 1958. `prices * (1 + rate)` inside the function operates on the whole vector at once, no loop: S, 1976. `sapply(prices, add_tax, rate = 0.2)` passes a function *to* another function: Church, 1936. And `add_tax` finds `rate` based on where it was defined, not where it's called: Scheme, 1975.

The rest of this book is about what you can do with these features once you understand them, starting with the most fundamental one: in R, everything is an expression, and every expression has a value.

3 Expressions and names

So you can name values and use them later. But what exactly counts as an expression?

Open R. Type `1 + 1` and press Enter.

```
1 + 1
#> [1] 2
```

You just wrote a complete R program. It has no variables, no functions, no setup. It is an expression, and R evaluated it and printed the result. Every piece of R code you will ever write works this way: you give R an expression, R gives you back its value.

What makes this work? Three things: R's interactive loop, expressions themselves, and a mechanism for giving names to values so you can use them later.

3.1. The REPL

When you open R, you see a prompt: `>`. R is waiting. When you type something and press Enter, four things happen in sequence:

1. R **reads** what you typed.
2. R **evaluates** it (figures out the value).
3. R **prints** the result.
4. R **loops** back to step 1 and waits again.

This is called a REPL (Read-Eval-Print Loop). There is no "compile" step, no "run" button; you type an expression, you see its value immediately.

In compiled languages (C, Java, and Go, which Google created in 2009 because even their best engineers were waiting too long for C++ to compile), you write the whole program first, then a compiler translates it into machine code before anything runs. R is *interpreted*: it reads and evaluates one expression at a time, immediately. Python and JavaScript work the same way. The tradeoff is speed: a compiler can optimize the whole program in advance, so compiled code runs faster. But interpreted languages let you experiment and see results as you go, which is why they dominate in data analysis.

In practice, the line between compiled and interpreted is blurry. R itself is written in C, and many R functions are thin wrappers around compiled C or Fortran code. When you type

`sum(x)`, the actual arithmetic happens in compiled machine code underneath. You get the interactive interface; the heavy computation still runs at full speed.

Try a few:

```
42
#> [1] 42
```

```
"hello"
#> [1] "hello"
```

```
TRUE
#> [1] TRUE
```

```
sqrt(9)
#> [1] 3
```

Each of these is an expression. `42` is an expression whose value is 42; `"hello"` evaluates to the text string "hello"; `sqrt(9)` calls the function `sqrt` with the argument `9` and returns 3. The `[1]` you see before each result is R telling you "this is the first element of the output." You can ignore it for now; it becomes useful when results have many elements (Chapter 4).

Notice the pattern: R evaluates expressions and prints their values, exactly the way the calculator in Chapter 1 evaluated `3 + 5 + 2 + 8`. This is Church's model of computation (Section 1.2), made interactive through the REPL loop. But evaluating expressions and forgetting them immediately is like a calculator with no memory. What if you want to keep a result?

Exercises

1. Type `2 + 3 * 4` into R. What do you get? Does R follow the usual order of operations (multiplication before addition)?
2. What happens when you type `"hello" + 1`? Read the error message. What is R telling you?
3. Try `pi`. This is a name that R already knows. What value does it have?

3.2. Names and binding

You want to compute something, save the result, and use it later. In R, you do this by giving a value a name:

```
x <- 3
```

The arrow `<-` is R's assignment operator. It binds the name `x` to the value `3`. After this line runs, typing `x` gives you back 3:

```
x
#> [1] 3
```

You can then use `x` in other expressions:

```
x + 10
#> [1] 13
```

```
x * x
#> [1] 9
```

It is tempting to think of `x` as a box that contains 3, and if you have used C or Python, that is exactly the model you carry around: the variable is a container, and assignment puts a value into it. R works differently. `x <- 3` does not put 3 into a box; it attaches the label `x` to a value that already exists. The value 3 is out there. `x` is just a name for it.

Why does that distinction matter?

```
y <- x
```

Now both `x` and `y` refer to 3. If you later write `x <- 7`, that changes what `x` points to, but `y` still points to 3:

```
x <- 7
x
#> [1] 7
y
#> [1] 3
```

This "names are labels, not containers" model is a consequence of R's functional design. Values in R are immutable; when you "change" something, R creates a new value and moves the name to point at it. The old value stays untouched (and gets cleaned up automatically when nothing points to it anymore; R's *garbage collector* periodically scans memory and reclaims anything that no name points to anymore, a mechanism R inherited from Lisp (Section 2.1); Section 29.1 covers how this works at the implementation level). If names were containers, you would need to worry about two containers sharing the same contents and one of them mutating behind your back. With labels and immutable values, that problem vanishes.

Think about what vanishes with it. In C, you learn to watch for *aliasing*: two pointers to the same block of memory, one of them writing while the other reads stale data. In Python, you learn that `a = b` for a list means both names see the same mutable object, and modifying through one surprises anyone holding the other. Entire chapters of software engineering textbooks are devoted to defensive copying, synchronization, and ownership protocols —

machinery whose sole purpose is to keep shared mutable state from corrupting your program. R sidesteps all of it by removing the condition that makes the checks necessary. Values do not change, so there is nothing to corrupt, no mutable state for two references to fight over. That category of bugs — aliasing, stale reads, defensive-copy failures — is structurally absent, which frees you to think about the problem you sat down to solve instead of the bookkeeping.

> **Opinion**
>
> Use `<-` for assignment, not `=`. Both work, but `<-` is the convention in R, and for good reason: on the Bell Labs terminals where S was designed, there was a single key that typed `<-` (Section 2.3). The convention stuck, and almost all R code uses it. Reserve `=` for naming arguments inside function calls, like `log(8, base = 2)`.

Exercises

1. Predict what this code prints, then run it to check:

```
a <- 10
b <- a
a <- 20
b
```

2. What happens if you type a name that doesn't exist, like `zzz`? Read the error message.
3. Names in R are case-sensitive. If you run `X <- 5`, can you get the value back by typing `x`?

3.3. Expressions all the way down

In most programming languages, there are two kinds of code: *expressions* (which produce a value) and *statements* (which do something but don't produce a value). In Python, `1 + 1` is an expression, but `if x > 0: print("yes")` is a statement. You cannot write `result = if x > 0: "yes" else: "no"` because `if` doesn't produce a value.

R has no statements. Everything is an expression, and every expression returns a value.

`if/else` returns a value:

```
x <- 3
result <- if (x > 0) "positive" else "non-positive"
result
#> [1] "positive"
```

You can store the result of `if/else` in a name because `if/else` *is* an expression that evaluates to a value, just like `1 + 1`. In Python or C, you cannot do this because `if` is a statement that performs an action rather than producing a result.

A block of code in curly braces `{ }` returns the value of its last expression:

```
result <- {
  a <- 2
  b <- 3
  a + b
}
result
#> [1] 5
```

The block evaluated three expressions, and the value of the whole block is the value of the last one: `a + b`, which is 5.

Even assignment returns a value (though R hides it). You can reveal it with parentheses:

```
(z <- 42)
#> [1] 42
```

Without the parentheses, R assigns 42 to `z` and prints nothing. With them, R prints the value that the assignment expression returned.

This "everything is an expression" rule comes directly from Church's lambda calculus (Section 1.2), where there are only expressions: no instructions, no statements, no side-channel commands. R followed that design faithfully, and the payoff is that you can compose pieces of R code in ways that would be impossible in a language built around statements. If every piece of code produces a value, every piece of code can be nested inside every other piece of code. What happens when you apply that principle to the thing that does the most work in R: functions?

Exercises

1. What does this return? Predict first, then run it: `if (FALSE) 1 else 2`
2. What value does this block produce? `{ 10; 20; 30 }`
3. Can you store the result of `for (i in 1:3) i` in a name? Try it. What happens? (This is one of the very few places where R behaves more like a statement language.)

3.4. Calling functions

You have already called a function: `sqrt(9)`. But what is actually happening when R sees those parentheses?

`sqrt` is a name bound to a function, the same way `x` was a name bound to 3. The parentheses `()` tell R to *call* the function, passing `9` as an argument. R evaluates the call and returns the result:

```
sqrt(9)
#> [1] 3
```

What happens if you type `sqrt` without parentheses?

```
sqrt
#> function (x)  .Primitive("sqrt")
```

R prints the function itself. `sqrt` is a value, just like `3` or `"hello"`, and the parentheses are what make R actually run it. This distinction between the function *itself* and *calling* the function may seem academic right now, but it will matter enormously when you start passing functions to other functions in later chapters.

Functions can take multiple arguments. `log` computes a logarithm, and you can specify the base:

```
log(8, base = 2)
#> [1] 3
```

`8` is a *positional* argument (R knows it's the first one), while `base = 2` is a *named* argument (you're saying explicitly which parameter gets the value 2). Named arguments can go in any order:

```
log(base = 2, x = 8)
#> [1] 3
```

To learn what arguments a function takes, use `?`:

```
?log
```

This opens the help page for `log`, which lists its arguments, explains what they do, and gives examples.

Here is where things get interesting. `+` is a function:

```
`+`(3, 5)
#> [1] 8
```

So is `<-`:

```
`<-`(w, 99)
w
```

```
#> [1] 99
```

Everything that happens in R is a function call. When you write `3 + 5`, R translates it to `` `+`(3, 5) ``: a function called `+`, applied to two arguments. There is no special syntax for arithmetic, no operators baked into the language at a level below functions. `+` is a function, `3 + 5` is function application, and in the lambda calculus the only thing you can do is apply a function to an argument. R took that idea literally.

Exercises

1. Use `round()` to round 3.14159 to two decimal places. (Hint: check `?round` for the argument name.)
2. What does `nchar("functional")` return? What about `nchar("")`?
3. Try `sum(1, 2, 3, 4, 5)`. Now try `sum(1:5)`. Do they give the same result?

3.5. The workspace

Every name you create lives in your *workspace* (also called the global environment). You can see all the names you've defined with `ls()`:

```
ls()
```

And you can remove a name with `rm()`:

```
rm(x)
```

After `rm(x)`, the name `x` is gone. The value it pointed to gets cleaned up by garbage collection if nothing else refers to it.

The workspace is simple on the surface: `ls()` shows what's in it, `rm()` removes things. But there is more going on underneath. Functions create their own local environments when they run, and R looks up names by searching through a chain of environments from the inside out. You have been working in one environment so far, the global one. When you start writing functions (Chapter 18), the picture gets richer.

> **Saving your workspace**
>
> When you quit R, it may ask whether to "save your workspace image." This saves all names and values currently in your workspace to a file, so they reappear next time you open R. In the early days, when computations could take hours, this made sense. Today, most computations are fast enough that you're better off keeping your work in a script file (`.R` or `.qmd`) and rerunning it from scratch. That way, every value in your workspace traces back to a line of code you can point to, and you never wonder where a mysterious leftover variable came from.

Try this:

```
environment()
```

R returns `<environment: R_GlobalEnv>`. The workspace is an object, the same kind of thing as the numbers and strings you have been binding to names. You can pass it to `ls.str()` and get back every binding in it. The container that holds your names is itself a value with a name.

4 Vectors

Type `42` in R.

```
42
#> [1] 42
```

That `[1]` has been there since Chapter 1, quietly announcing something you probably didn't notice: you never made a number. You made a vector.

4.1. There are no scalars

In C or Python, a single number and a collection of numbers are fundamentally different types: one is a scalar, the other is an array or list. R erases that distinction. Every value in R is a vector: `42` is a numeric vector of length 1, `"hello"` is a character vector of length 1, and `TRUE` is a logical vector of length 1.

```
length(42)
#> [1] 1
length("hello")
#> [1] 1
length(TRUE)
#> [1] 1
```

Why would a language erase the distinction between "one thing" and "many things"? Because S (Section 2.3), the language R descends from, was built for people who work with columns of data, not individual numbers. A single temperature reading is rarely interesting on its own; you care about the whole column. So S made the column the basic unit of computation, and R inherited that decision wholesale, which means every operation you write, from arithmetic to comparison, has to reckon with that choice.

`x * 2` works the same whether `x` has one element or a million, because `x` is always a vector and R doesn't need a special case for scalars; it just operates on vectors. But how do you build them?

4.2. Atomic vectors

You make a vector with `c()`, short for "combine":

```
temperatures <- c(72, 85, 61, 90, 78)
temperatures
#> [1] 72 85 61 90 78
```

`c()` can combine vectors too:

```
morning <- c(58, 62, 55)
afternoon <- c(75, 80, 71)
all_temps <- c(morning, afternoon)
all_temps
#> [1] 58 62 55 75 80 71
```

It flattens everything into a single vector; there are no nested vectors in R, so `c(c(1, 2), c(3, 4))` gives you `c(1, 2, 3, 4)`. Look at what's happening in `c(morning, afternoon)`:

```
c(morning, afternoon)
= c(c(58, 62, 55), c(75, 80, 71))
= c(58, 62, 55, 75, 80, 71)
= [58, 62, 55, 75, 80, 71]
```

Each step replaces a name or function call with its value. `morning` becomes `c(58, 62, 55)`, the nested `c()` calls flatten into one, and the last step is where term replacement ends and the computer takes over: R allocates a contiguous block of memory, stores six numbers side by side, and the expression has been reduced to a value that lives in memory. This is term replacement, the logic at the heart of Church's lambda calculus (Section 1.2): you have an expression, you replace each piece with its value until nothing is left to replace. In Chapter 1, `add(3, add(5, 10))` reduced step by step until only `18` remained. Here, `c(morning, afternoon)` reduces to a six-element vector. Even building a data structure is a function call that reduces to a value. (Term replacement is one of the oldest ideas in logic and a cornerstone of computer science. Mathematicians were studying it decades before electronic computers existed; it turns out that "replace an expression with its value" is enough to define all of computation.)

> **For the curious**
>
> `c()` has two properties worth noticing. First, it is *associative*: `c(c(1, 2), c(3, 4))` and `c(1, c(2, c(3, 4)))` both produce `c(1, 2, 3, 4)`. Grouping doesn't matter. Second, `c(x, NULL)` returns `x` unchanged, so `NULL` acts as an *identity element*: combining with it does nothing. An operation that is associative and has an identity element is called a *monoid*. Addition with 0 is a monoid. String concatenation with `""` is a monoid. `c()` with `NULL` is a monoid. The pattern keeps showing up: dplyr pipelines (Chapter 14), function composition (Chapter 15), and `Reduce()` (Chapter 21) all rely on it.

A few shortcuts for common patterns:

```
1:5
#> [1] 1 2 3 4 5
```

```
seq(0, 1, by = 0.25)
#> [1] 0.00 0.25 0.50 0.75 1.00
```

```
rep("hello", 3)
#> [1] "hello" "hello" "hello"
```

`1:5` generates integers from 1 to 5, `seq()` generates a sequence with a specified step size, and `rep()` repeats a value.

R has four atomic types you will use constantly, and every one of them follows a pecking order that R enforces behind your back. Two more exist but rarely matter:

```
typeof(3.14)
#> [1] "double"
typeof(42L)
#> [1] "integer"
typeof("hello")
#> [1] "character"
typeof(TRUE)
#> [1] "logical"
```

`double` is a decimal number (even `42` is stored as a double by default), while `integer` needs an `L` suffix: `42L`. `character` is text, `logical` is `TRUE` or `FALSE`. The other two types, `complex` and `raw`, exist for specialized work; you can ignore them.

Every element in a vector must be the same type. `typeof()` and `length()` answer the two most basic questions about any vector: what type are the elements, and how many are there? But what happens when you try to break that same-type rule?

Exercises

1. What does `c(1, c(2, c(3, 4)))` produce? Predict, then check.
2. Create a vector of the first 10 even numbers using `seq()`.
3. What is `typeof(1)`? What about `typeof(1L)`?

4.3. Coercion

Try mixing types in a vector:

```
c(1, "two", 3)
#> [1] "1"   "two" "3"
```

R converted the numbers to text. No warning, no error. This is called *coercion*, and R follows a fixed hierarchy: logical < integer < double < character. When you mix types, everything gets promoted to the most general one.

```
c(TRUE, 1, "hello")
#> [1] "TRUE"  "1"     "hello"
```

`TRUE` became `"1"` (first promoted to the integer 1, then to the string `"1"`), and `1` became `"1"`.

This hierarchy explains a trick from Chapter 1 that might have seemed like magic. `sum(scores > 70)` works because `scores > 70` produces a logical vector, and `sum()` coerces the logicals to numbers: `TRUE` becomes 1, `FALSE` becomes 0, so the count of passing values falls out automatically:

```
x <- c(65, 82, 71, 90, 55, 78)
x > 70
#> [1] FALSE  TRUE  TRUE  TRUE FALSE  TRUE
sum(x > 70)
#> [1] 4
```

Opinion

R will never warn you about coercion. `c(1, 2, "3")` quietly becomes all character, and your arithmetic breaks ten lines later with an error message that points nowhere near the real problem. Memorize the hierarchy: logical < integer < double < character. It will save you hours.

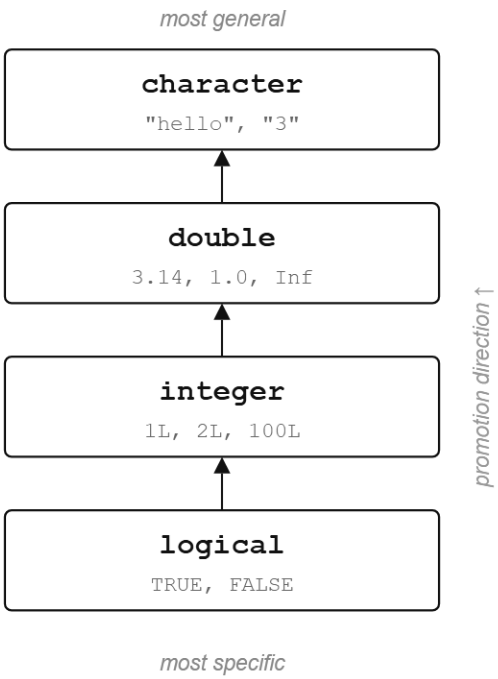


Figure 4.1.: R's coercion hierarchy: mixing types promotes everything to the most general type.

Exercises

1. What does `TRUE + 1` return? What about `FALSE * 5`?
2. What is `typeof(c(1L, 1.5))`? Which type won?
3. What does `as.numeric("hello")` return? Read the warning.

4.4. Vectorized operations

Say you have five temperatures in Fahrenheit and want Celsius. In most languages, you write a loop: go through each element, apply the formula, store the result. In R:

```
temperatures <- c(72, 85, 61, 90, 78)
(temperatures - 32) * 5/9
#> [1] 22.22222 29.44444 16.11111 32.22222 25.55556
```

One expression, no loop. The subtraction, multiplication, and division each apply to every element at once. This is a *vectorized operation*, and it is the default way of working in R.

Comparisons are vectorized too:

```
temperatures > 75
#> [1] FALSE  TRUE FALSE  TRUE  TRUE
```

The result is a logical vector of the same length: `TRUE` where the condition holds, `FALSE` where it doesn't.

You can combine comparisons with `&` (and) and `|` (or):

```
temperatures > 60 & temperatures < 85
#> [1]  TRUE FALSE  TRUE FALSE  TRUE
```

Arithmetic, comparisons, and most built-in functions all work this way, returning a vector of the same length for each input vector. `sqrt()`, `log()`, `round()`, `abs()`: give them a vector and they return a vector.

```
sqrt(c(1, 4, 9, 16, 25))
#> [1] 1 2 3 4 5
```

In lambda calculus (Section 1.2), `sqrt` is a function `f = x. √x`. Even `√` itself is not a primitive; it can be computed by repeated function application. You start with a rough guess and improve it over and over until it's close enough:

```
√x = iterate( guess. (guess + x / guess) / 2, x)
```

Start with a guess, apply the refinement function `guess. (guess + x / guess) / 2` repeatedly, and the result converges to `√x`. Square root reduces to division, addition, and repeated application of a lambda, which are themselves reducible further. At every level, the building blocks are functions applied to arguments.

> **Under the hood**
>
> In practice, R calls C's `sqrt()`, which on modern CPUs maps to a hardware instruction. The chip uses a lookup table for a rough estimate, then refines it with the same `(guess + x / guess) / 2` step, finishing in 10-20 clock cycles. The abstraction and the silicon converge on the same algorithm.

For our purposes, `f = x. √x` is enough. R applies `f` to each element by term replacement:

```
f = x. √x

f(c(1, 4, 9, 16, 25))
= c(f(1), f(4), f(9), f(16), f(25))          # distribute f over the vector
= c(( x. √x)(1), ( x. √x)(4), ..., ( x. √x)(25))  # replace f with its definition
= c(√1, √4, √9, √16, √25)                   # substitute each x
= c(1, 2, 3, 4, 5)
```

R distributes the function over the vector automatically; you write one application, R performs five. This is Church's model extended to collections: a function applied to a vector produces a vector of results. `sqrt(c(1, 4, 9, 16, 25))` is a single function applied to a single argument, and the fact that the argument contains five numbers is R's business, not yours.

> **For the curious**
>
> This pattern has a name: a *functor*. A functor maps a function over a structure while preserving the structure's shape. `sqrt` applied to a five-element vector returns a five-element vector; the function changes the contents, but the container stays intact. Vectors are functors, lists are functors (via `lapply`), and data frame columns are functors (via `mutate`). The word comes from category theory, but the idea is concrete: apply inside, preserve outside. You will see it everywhere in this book, and by the end it will feel as natural as vectorization itself.

Consider what `temperatures * 9/5 + 32` actually did. A function was applied to a structure and produced a structure of the same shape — a functor operation, driven by the same term-replacement mechanism that Section 1.2 introduced as lambda calculus. And when you wrote `c(morning, afternoon)`, the names were substituted for their values and the expression reduced to a single vector. That is *beta reduction*: replacing a name with its value and simplifying (Section 1.2 covers the formal version). The theoretical machinery from Chapter 1 has been running under every line of code in this chapter, not as background context but as the engine. By the time you typed `42` into the console and got `42` back, you were already doing functional programming — you just did not have a name for it yet.

4.4.1. Recycling

What happens when you operate on two vectors of different lengths?

```
c(1, 2, 3, 4) + c(10, 20)
#> [1] 11 22 13 24
```

R *recycled* the shorter vector, reusing `10, 20` to match the length of the longer one: `1+10, 2+20, 3+10, 4+20`. This is also how `temperatures - 32` works: `32` is a vector of length 1, and R recycles it five times to match the length of `temperatures`.

When the lengths don't divide evenly, R warns you:

```
c(1, 2, 3, 4, 5) + c(10, 20)
#> Warning in c(1, 2, 3, 4, 5) + c(10, 20): longer object length is not a
#> multiple of shorter object length
#> [1] 11 22 13 24 15
```

> **Opinion**
>
> Recycling is useful when the shorter vector has length 1 (e.g., `x * 2`, `x > 0`). For anything else, match your vector lengths explicitly. A recycling warning almost always means a bug.

Exercises

1. What does `c(1, 2, 3) * c(0, 1)` produce? Does R warn you?
2. Why does `x - mean(x)` work without a loop? (Hint: what length is `mean(x)`?)
3. What does `c(TRUE, FALSE) + c(1, 2, 3, 4)` produce?

4.5. Indexing

You often need specific elements from a vector, and the `[` operator extracts them. There are four ways to use it.

By position:

```
temperatures <- c(72, 85, 61, 90, 78)
temperatures[1]
#> [1] 72
temperatures[c(1, 3, 5)]
#> [1] 72 61 78
```

By exclusion (negative indices):

```
temperatures[-2]
#> [1] 72 61 90 78
```

Everything except the second element.

By logical vector:

```
temperatures[temperatures > 75]
#> [1] 85 90 78
```

`temperatures > 75` produces `c(FALSE, TRUE, FALSE, TRUE, TRUE)`, and `[` keeps only the `TRUE` elements. You're asking a yes/no question of every element and keeping the ones that pass. This is filtering, and it will come back in a more powerful form when you learn `dplyr::filter()` (Chapter 14).

```
hot <- temperatures > 75
temperatures[hot]
```

```
#> [1] 85 90 78
```

By name:

```
scores <- c(alice = 92, bob = 78, carol = 85)
scores["bob"]
#> bob
#>  78
scores[c("alice", "carol")]
#> alice carol
#>    92    85
```

You can also use `[` to replace elements:

```
x <- c(10, 20, 30)
x[2] <- 99
x
#> [1] 10 99 30
```

This looks like it's changing the vector in place, but R is actually making a copy with the new value and pointing the name `x` at the copy. The old vector (with 20 in position 2) is gone. This is copy-on-modify, the same idea from Section 3.2: values in R don't change; names get moved to point at new values. So far, every element in our vectors has been present and accounted for, but real data has gaps.

Exercises

1. Extract the last element of `temperatures` without hard-coding the position. (Hint: `length()`.)
2. What does `temperatures[c(TRUE, FALSE)]` return? How does recycling interact with logical indexing?
3. Create a named vector of three cities and their populations. Extract two by name.

4.6. Missing values

A sensor fails, a patient skips a visit, a field is left blank. Real data has gaps, and R represents them with `NA` ("Not Available"):

```
readings <- c(23.1, NA, 22.5, 24.0, NA, 23.8)
readings
#> [1] 23.1   NA 22.5 24.0   NA 23.8
```

`NA` is not zero, and it is not an empty string. It means "I don't know." And that uncertainty is contagious: any computation involving an unknown value produces an unknown result.

```
sum(readings)
#> [1] NA
mean(readings)
#> [1] NA
```

R is being honest. If you don't know some of the values, you don't know the sum. If you want to ignore the missing values, you have to say so explicitly:

```
sum(readings, na.rm = TRUE)
#> [1] 93.4
mean(readings, na.rm = TRUE)
#> [1] 23.35
```

`na.rm = TRUE` means "remove NAs before computing." Many R functions have this argument.

Find which elements are missing with `is.na()`:

```
is.na(readings)
#> [1] FALSE  TRUE FALSE FALSE  TRUE FALSE
```

Filter them out with `!` (not):

```
readings[!is.na(readings)]
#> [1] 23.1 22.5 24.0 23.8
```

One common trap: you cannot test for `NA` with `==`.

```
NA == NA
#> [1] NA
```

This returns `NA`, not `TRUE`. Think about it: if you don't know what `x` is and you don't know what `y` is, you can't know whether they're equal. Always use `is.na()`.

This contagious behavior is not arbitrary; R is implementing a discipline where a value is either present or absent, and any operation on an absent value produces absence. The principle is older than R.

Haskell, a language that emerged in 1990 when a group of functional programming researchers decided to consolidate their dozens of incompatible dialects, takes the strictest possible position: no variable can ever change its value, and no function can have a side effect. The compiler enforces these rules, which means the type system carries more information than in most languages. (The language is named after Haskell Curry, a logician who worked on combinatory logic in the 1930s and 1940s; he will reappear in Section 5.8.)

One consequence: in Haskell, the possibility of absence is built into the type system as `Maybe`. A value is either `Just 42` or `Nothing`, and any operation on `Nothing` propagates `Nothing`

forward. `1 + NA` returning `NA` in R is the same logic as Haskell's `Nothing >>= f` returning `Nothing`.

Rust adopted the same idea for different reasons. Graydon Hoare started Rust at Mozilla in 2006 because C and C++ gave you direct control over memory but no protection against the bugs that come with it: buffer overflows, use-after-free, data races. He borrowed heavily from Haskell's type system, and Rust's equivalent of `Maybe` is called `Option`: a value is `Some(42)` or `None`, and `None.map(f)` returns `None`. The compiler rejects any code that forgets to check whether a value is present.

R's approach is looser: nothing stops you from ignoring `NA`, and you will not find out until your results look wrong. But the underlying rule is the same. If any input is unknown, the output is unknown. R threads this uncertainty through every arithmetic and comparison operation automatically, which is why `NA` feels contagious. It is. That's the point. The question is what this contagion looks like when you have 344 measurements and a handful of gaps.

Exercises

1. What does `c(1, NA, 3) > 2` return? What happens to the `NA`?
2. How many missing values are in `readings`? (Hint: combine `sum()` and `is.na()`.)
3. What does `NA | TRUE` return? What about `NA & FALSE`? One of each pair is not `NA`. Why?

4.7. Meet penguins

R 4.5 and later includes a dataset called `penguins` (from the Palmer Archipelago in Antarctica), and it's a good place to see everything from this chapter working on real measurements. If you're running an older version, install it with `install.packages("palmerpenguins")` and load it with `library(palmerpenguins)`.

```
head(penguins)
```

`penguins` is a data frame (a table), and we'll learn about those properly in Chapter 11. For now, the important thing is that you can pull out a single column with `$`, and that column is a vector:

```
bill <- penguins$bill_length_mm
```

Now you have a numeric vector of 344 bill length measurements, with some `NA`s mixed in. Everything from this chapter applies:

```
length(bill)
typeof(bill)
```

```
mean(bill, na.rm = TRUE)
range(bill, na.rm = TRUE)
```

```
sum(is.na(bill))
```

```
long_bills <- bill[bill > 50 & !is.na(bill)]
length(long_bills)
```

How many penguins have a bill longer than 50 mm? Logical indexing filters the values, `is.na()` handles missing data, and `length()` counts the result. One line each, no loops. You wrote five operations in this chapter that would each require explicit iteration in most languages; in R, the vector carried the iteration for you. Data frames, which we turn to next, are built from exactly these vectors, stacked side by side like columns in a spreadsheet. The machinery you just learned doesn't go away; it becomes the engine underneath everything else.

Exercises

1. What is the mean body mass (`penguins$body_mass_g`) of the penguins? Don't forget the `NA`s.
2. How many penguins have a flipper length (`penguins$flipper_length_mm`) greater than 200?
3. What is the shortest bill in the dataset? (Hint: `min()` with `na.rm = TRUE`.)

5 Your first function

`sqrt(9)`, `c(1, 2, 3)`, `sum(x > 70)`: every piece of R code so far has relied on functions that someone else wrote. This chapter shows you how to write your own.

5.1. Anatomy of a function

A function in R has three parts: a name, arguments, and a body.

```
double <- function(x) x * 2
```

`function(x)` declares one parameter called `x`. The body, `x * 2`, is the expression R evaluates when you call the function, and `double` is the name you are binding this function to (the same way `x <- 3` binds a name to a number).

Call it:

```
double(5)
#> [1] 10
```

```
double(c(1, 2, 3))
#> [1] 2 4 6
```

The second call works because `*` is vectorized (Section 4.4): the body `x * 2` already operates element by element, so `double` handles vectors without a loop, without any extra effort on your part. But what happens when the body grows beyond a single expression?

A function with a longer body uses curly braces:

```
describe <- function(x) {
  n <- length(x)
  m <- mean(x)
  paste("n =", n, ", mean =", round(m, 2))
}

describe(c(72, 85, 61, 90, 78))
#> [1] "n = 5 , mean = 77.2"
```

The body is a block of expressions that R evaluates in order, returning the value of the last one. This is the same rule from Section 3.3: a block in curly braces is itself an expression, and its value is whatever the final line produces.

Exercises

1. Write a function `square` that takes a number and returns its square.
2. Write a function `to_celsius` that converts a Fahrenheit temperature to Celsius (formula: `(f - 32) * 5/9`). Test it with `to_celsius(212)`.
3. Does `to_celsius(c(32, 72, 212))` work? Why?

5.2. Arguments and defaults

Functions can have multiple arguments, and some can have default values:

```
greet <- function(name, greeting = "Hello") {
  paste(greeting, name)
}
```

`name` is required; `greeting` is optional, defaulting to `"Hello"` if you leave it out:

```
greet("R")
#> [1] "Hello R"
```

```
greet("R", "Hey")
#> [1] "Hey R"
```

You can also pass arguments by name, in any order:

```
greet(greeting = "Bonjour", name = "R")
#> [1] "Bonjour R"
```

This is the same positional-versus-named distinction from Section 3.4, when we used `log(8, base = 2)`. The rule is uniform across R because function calls all work the same way, whether you wrote the function or someone at R Core did.

Some functions accept a variable number of arguments using `...` (pronounced "dot-dot-dot"):

```
shout <- function(...) {
  words <- c(...)
  toupper(paste(words, collapse = " "))
}
```

```
shout("hello", "world")
#> [1] "HELLO WORLD"
```

`...` collects every argument you pass, however many there are, and `c(...)` combines them into a vector. This is a surprisingly deep idea. In Church's lambda calculus (Section 1.2), a function takes exactly one argument; if you want two, you write a function that returns another function:

```
x. y. x + y
```

This reads: "a function that takes `x` and returns *another function* that takes `y` and returns `x + y`." To add 3 and 5, you apply it twice: `( x. y. x + y)(3)(5)` = `( y. 3 + y)(5)` = `3 + 5` = `8`. Each application replaces one argument, one step at a time. This is called *currying*, and R can do it too (Chapter 20). `...` is R's practical alternative: instead of nesting functions, you let a single function accept any number of arguments and decide what to do with them inside. `c()`, `paste()`, `sum()`: they all use `...`, which is why `c(1, 2, 3)` and `c(1, 2, 3, 4, 5, 6, 7)` both work. More on `...` in Chapter 23.

Exercises

1. Write a function `power` with arguments `x` and `n = 2` that computes `x^n`. Test it with `power(3)` and `power(3, 3)`.
2. What happens if you call `greet()` with no arguments? Read the error message.
3. Rewrite `power` so the default exponent is 3. What does `power(2)` return now?

5.3. Return values

R returns the value of the last expression in the body. No keyword needed:

```
add_one <- function(x) x + 1
add_one(10)
#> [1] 11
```

The body is `x + 1`. R evaluates it and returns the result. In Python or Java you would need to write `return` explicitly, but R treats the last expression as the return value. A function is an expression that evaluates to a value, the same idea from the lambda calculus in Section 2.1.

When you call `add_one(10)`, R substitutes `10` for `x` in the body, turning `x + 1` into `10 + 1`, then evaluates it. Lambda calculus calls this *beta reduction*: `( x. x + 1)(10)` reduces to `10 + 1`. Every function call in R is a beta reduction.

R does have a `return()` function, and it is useful for early exits:

```
safe_log <- function(x) {
  if (any(x <= 0)) return(NA)
  log(x)
}

safe_log(5)
#> [1] 1.609438
safe_log(-1)
#> [1] NA
```

If `x` has any non-positive values, the function exits immediately and returns `NA`. Otherwise, it falls through to `log(x)`.

> **Opinion**
>
> Use `return()` only for early exits like `safe_log` above. If the function runs to the end, let the last expression be the return value; writing `return(x)` on the last line adds nothing but noise.

One subtlety: assignment returns its value invisibly. When a function's last line is an assignment, R returns the value but does not print it:

```
quiet <- function(x) {
  result <- x + 1
}

quiet(5)
```

Nothing printed. The function returned 6, but invisibly. If you want to see it:

```
(quiet(5))
#> [1] 6
```

Or store it:

```
y <- quiet(5)
y
#> [1] 6
```

The general rule: if you want your function to show a result, make sure the last expression is the value itself, not an assignment. But there is a deeper question lurking here: if a function is just a value, can you treat it the way you treat a number or a string?

Exercises

1. What does this function return? Predict, then check:

```
mystery <- function(x) {
  a <- x + 1
  b <- a * 2
  b
}
mystery(3)
```

2. What about this one?

```
mystery2 <- function(x) {
  a <- x + 1
  b <- a * 2
}
mystery2(3)
```

3. Write a function `clamp` that takes a value `x`, a `lo`, and a `hi`, and returns `lo` if `x < lo`, `hi` if `x > hi`, and `x` otherwise. Use `return()` for the early exits.

5.4. Functions are just values

Type `sqrt` without parentheses and R prints the function itself, not a result. A function is a value, the same kind of thing as a number or a string: you can assign it, put it in a list, or interrogate its type.

```
f <- double
f(5)
#> [1] 10
```

```
is.function(double)
#> [1] TRUE
```

```
typeof(double)
#> [1] "closure"
```

`double` is a name bound to a value, and that value happens to be a function. `f <- double` gives the same function a second name, which is exactly what happened in Section 3.2 when `y <- x` gave the number 3 a second name.

You can even create a function without giving it a name at all:

```
(function(x) x + 1)(10)
#> [1] 11
```

`function(x) x + 1` creates a function; the parentheses around it and `(10)` immediately call it with the argument `10`. This is an *anonymous function*: `x. x + 1` applied to `10` (Section 1.2). Chapter 19 uses them heavily.

You can do anything with a function that you can do with a number: store it in a variable, pass it to another function, return it as a result. This is what McCarthy built into Lisp in 1958 (Section 2.1), and R inherits it directly. So functions are values, and values live somewhere. Where do the variables inside a function live?

Exercises

1. Assign `sqrt` to a new name `s`. Does `s(16)` work?
2. What does `is.function(42)` return? What about `is.function(is.function)`?
3. Run `(function(a, b) a + b)(3, 4)`. What happens?

5.5. Scope: a first look

What happens to variables created inside a function?

```
add_ten <- function(x) {
  y <- 10
  x + y
}

add_ten(5)
#> [1] 15
```

`y` was created inside `add_ten`. Can you access it from outside?

```
y
#> [1] 6
```

No. Variables created inside a function live in the function's own local environment and vanish when the function finishes. This is what makes functions safe: they do not leak their internal state into your workspace. But the boundary is not symmetric.

```
multiplier <- 3

scale <- function(x) {
  x * multiplier
}
```

```
scale(10)
#> [1] 30
```

`scale` found `multiplier` in the workspace because it was not defined inside the function body. R looks for names first in the local environment, then in the environment where the function was defined, a strategy called lexical scoping that R inherited from Scheme (Section 2.2).

If a function can read outer variables, can it change them?

```
n <- 0

increment <- function() {
  n <- n + 1
  n
}

increment()
#> [1] 1
n
#> [1] 0
```

`increment` created its own local `n`, set it to 1, and returned it, but the `n` in the workspace is still 0. The function read the outer `n` without modifying it; it created a new local binding with the same name. This is R's copy-on-modify philosophy: values do not change, you create new values.

The practical rule is simple: what happens inside a function stays inside the function. The full machinery behind this (environments, closures, and the scope chain) is in Chapter 18.

Exercises

1. Predict the output, then run it:

```
x <- 100
f <- function() {
  x <- 1
  x
}
f()
x
```

2. What does this return?

```
x <- 10
g <- function(x) {
```

```
  x + 1
}
g(20)
```

Is it 11 or 21? Why?

3. Can a function call another function you've defined? Try it: write `triple <- function(x) x * 3`, then write `triple_and_add <- function(x) triple(x) + 1`. Does `triple_and_add(5)` work?

5.6. Guarding your inputs

What happens when someone passes the wrong type to your function?

```
to_celsius <- function(f) (f - 32) * 5/9
to_celsius("hot")
#> Error in `f - 32`:
#> ! non-numeric argument to binary operator
```

The error says `non-numeric argument to binary operator`, which is technically accurate but unhelpful: you have to read the function body to figure out what went wrong. Better to catch the problem at the door:

```
to_celsius <- function(f) {
  stopifnot(is.numeric(f))
  (f - 32) * 5/9
}

to_celsius("hot")
#> Error in `to_celsius()`:
#> ! is.numeric(f) is not TRUE
```

`stopifnot()` checks a condition and stops with an error if it is `FALSE`. The message now tells you exactly what failed: `is.numeric(f)` is not TRUE. You can stack multiple checks in the same call:

```
safe_divide <- function(x, y) {
  stopifnot(is.numeric(x), is.numeric(y), y != 0)
  x / y
}

safe_divide(10, 0)
#> Error in `safe_divide()`:
#> ! y != 0 is not TRUE
```

A function that fails immediately with a clear message saves you from chasing wrong outputs later. Chapter 25 covers input validation in depth; for now, `stopifnot()` at the top of your functions is enough. But validation only protects against bad inputs. What about the structure of the computation itself?

Exercises

1. Add a `stopifnot()` to your `power` function that checks `x` is numeric and `n` is numeric.
2. What happens if you write `stopifnot(is.numeric(x), length(x) == 1)` and pass a vector? Try it.

5.7. Recursion

A function can call other functions. Can it call *itself*?

```
factorial <- function(n) {
  if (n == 0) return(1)
  n * factorial(n - 1)
}

factorial(5)
#> [1] 120
```

`factorial(5)` calls `factorial(4)`, which calls `factorial(3)`, and so on, until `factorial(0)` returns 1 and the whole chain collapses back:

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 * factorial(0)
= 5 * 4 * 3 * 2 * 1 * 1
= 120
```

This is *recursion*: a function defined in terms of itself. Look at the unfolding carefully. Each line replaces a function call with its value, the same term replacement from Section 4.2, and no variable is being updated, no counter is ticking. The answer was implicit in the original expression `factorial(5)`; evaluation simply made it explicit.

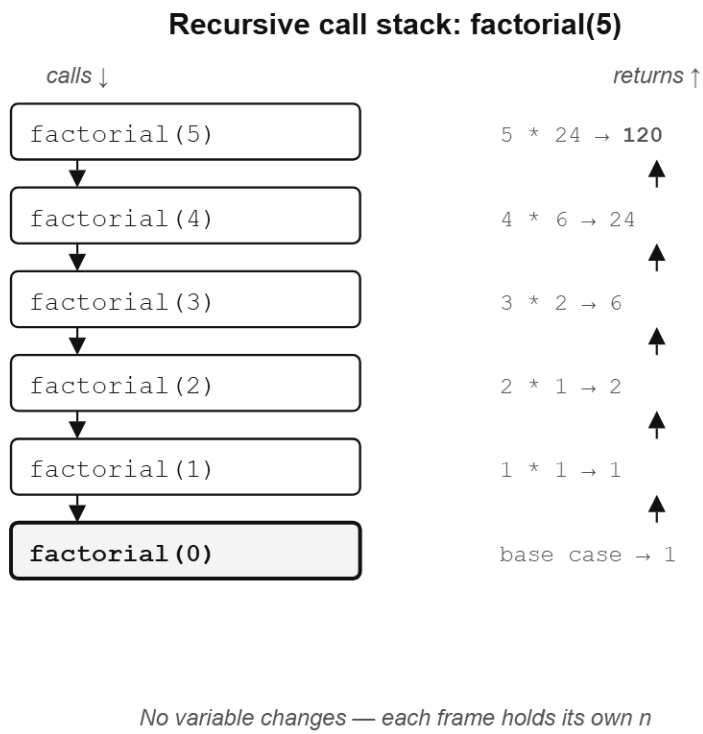


Figure 5.1.: Recursive call stack for `factorial(5)`: each frame waits for the one below it. No variable changes — each frame holds its own `n`.

In lambda calculus notation, factorial looks like this:

```
fact =  n. if n = 0 then 1 else n * fact(n - 1)
```

(This self-reference is informal shorthand; pure lambda calculus has no names, so `fact` cannot refer to itself directly. Section 30.5 shows how recursion works without them.)

Wait. In Section 1.2, the lambda calculus had only three things: variables, functions, and application. Where did `if/then/else` come from? Church showed that you can encode true, false, and conditional branching as pure lambda functions:

```
TRUE  = x.  y. x      # a function that takes two arguments and returns the first
FALSE = x.  y. y      # a function that takes two arguments and returns the second
IF    = cond.  then.  else. cond(then)(else)
```

`TRUE` is a function that picks its first argument; `FALSE` picks its second. `IF` just hands both options to the condition and lets it choose. So `IF(TRUE)(1)(2)` becomes `TRUE(1)(2)` becomes `( x.  y. x)(1)(2)` becomes `1`. The entire conditional is built from functions choosing between arguments, and the `if n = 0 then 1 else ...` notation above is shorthand for this encoding.

The reduction works as before, one substitution at a time:

```
fact(3)                                      # apply fact with n = 3
= ( n. if n=0 then 1 else n * fact(n-1))(3) # replace fact with its definition
= if 3=0 then 1 else 3 * fact(3-1)           # substitute n = 3
= 3 * fact(2)                                # 3 0, so take the else branch
= 3 * 2 * fact(1)                            # same pattern: n=2, substitute, reduce
= 3 * 2 * 1 * fact(0)                        # n=1
= 3 * 2 * 1 * 1                              # n=0, take the then branch: 1
= 6
```

R's `function(n)` is the same as `n`. The reduction above is what R does when you call `factorial(3)`: substitute `3` for `n`, evaluate the body, hit `factorial(2)`, substitute again, keep going until the base case. Recursion is how Church's lambda calculus (Section 1.2) handles repetition. There is no loop counter; the function calls itself with a smaller argument until it stops.

```r
fibonacci <- function(n) {
  if (n <= 1) return(n)
  fibonacci(n - 1) + fibonacci(n - 2)
}

fibonacci(10)
#> [1] 55
```

Recursion is elegant but not always practical. `fibonacci(30)` is already slow because it recomputes the same values over and over: `fibonacci(10)` calls `fibonacci(9)` and `fibonacci(8)`, but `fibonacci(9)` also calls `fibonacci(8)`, so that subtree gets computed twice. Further down, `fibonacci(7)` gets computed three times, `fibonacci(6)` five times, and the work grows exponentially. Is there a way to keep the recursive structure without paying the exponential cost?

A smarter version avoids this by carrying the last two values forward as arguments:

```r
fibonacci_fast <- function(n, a = 0, b = 1) {
  if (n == 0) return(a)
  fibonacci_fast(n - 1, b, a + b)
}

fibonacci_fast(10)
#> [1] 55
fibonacci_fast(50)
#> [1] 12586269025
```

Instead of branching into two recursive calls, `fibonacci_fast` makes one call per step, passing the accumulated values along. Watch the reduction:

```
fibonacci_fast(4, 0, 1)
= fibonacci_fast(3, 1, 1)    # n=4→3, a=0→1, b=1→0+1=1
```

```
= fibonacci_fast(2, 1, 2)    # n=3→2, a=1→1, b=1→1+1=2
= fibonacci_fast(1, 2, 3)    # n=2→1, a=1→2, b=2→1+2=3
= fibonacci_fast(0, 3, 5)    # n=1→0, a=2→3, b=3→2+3=5
= 3                           # n=0, return a
```

No variable is being mutated; each call just substitutes new values for `n`, `a`, and `b` and hands them to the next call. This form is called *tail recursion* because the recursive call is the last thing the function does. The naive version cannot compute `fibonacci(50)` in any reasonable time. This one handles it instantly:

Any function with multiple recursive calls can be rewritten as a single recursive call with extra arguments that carry the intermediate results. For Fibonacci, the state to pass forward is the last two numbers; for a tree traversal, it might be an accumulator list.

R has a limit on how deep recursion can go (around 1000 calls by default), and for most day-to-day R work, vectorized operations and functions like `map()` (Chapter 19) are better tools. But there is something strange about `factorial` that has nothing to do with performance: its base case and inductive step look exactly like a proof by mathematical induction.

Exercises

1. Trace through `factorial(3)` by hand, writing out each step like the example above.
2. Write a recursive function `sum_to` that adds up all integers from 1 to `n`. Check: `sum_to(100)` should return 5050.
3. What happens if you call `factorial(-1)`? Why? Add a `stopifnot()` to guard against it.

5.8. Programs and proofs

Look at `factorial` again:

```r
factorial <- function(n) {
  if (n == 0) return(1)
  n * factorial(n - 1)
}
```

Two parts: a base case (when `n = 0`, return 1) and an inductive step (assume `factorial(n - 1)` gives the right answer, and use it to compute `factorial(n)`). If you have seen mathematical induction, you recognize the shape immediately. A proof by induction has a base case and an inductive step too. The resemblance is not a coincidence.

In the 1930s and 1940s, the logician Haskell Curry (after whom the Haskell programming language is named, and from whom the term "currying" above comes) noticed something strange: the rules for combining types in a programming language looked identical to the rules for combining propositions in formal logic. A function that takes an `A` and returns a `B` had the same structure as a logical proof that "if A, then B." In 1969, William Howard made this precise: every type corresponds to a logical proposition, and every program of

that type corresponds to a proof of that proposition. A recursive function that terminates is a proof by induction. This result is called the *Curry-Howard correspondence*.

Writing `factorial` constructs a proof by induction that the product `1 * 2 * ... * n` exists for every non-negative integer. The transformation from double-recursive `fibonacci` to tail-recursive `fibonacci_fast` is the same move a logician makes when they strengthen an inductive hypothesis to make a proof go through: the extra arguments (`a` and `b`) *are* the stronger hypothesis. Double induction reduces to simple induction with stronger invariants, which is why the same trick turns a slow recursive function into a fast one.

This was the landscape of the 1930s: Church, Godel, Curry, and Turing were all studying what could be *proved*, before computers existed, and they found that the answer was identical to what could be *computed*.

Today, the Curry-Howard correspondence is the foundation of *formal verification*: software that proves other software is correct.

The first practical tool to exploit this idea was Coq, developed at INRIA, the French national computer science institute, starting in the late 1980s. Thierry Coquand and Gerard Huet designed it around a type theory called the Calculus of Constructions (the "C-o-C" that gives Coq part of its name). In Coq, every program must come with a machine-checkable proof that it does what its type signature claims; if the proof has a gap, the code does not compile. This makes Coq painful for ordinary programming but powerful for building software where correctness matters more than convenience.

In 2013, Leonardo de Moura at Microsoft Research started Lean, a newer *proof assistant* with a more accessible syntax and a growing library of formalized mathematics called Mathlib. Coq grew out of theoretical computer science; Lean has attracted a broader community of mathematicians who are formalizing results from analysis, algebra, and number theory. In Lean, a proof that adding zero to any natural number gives back the same number looks like this:

```
theorem add_zero (n : Nat) : n + 0 = n := by
  induction n with
  | zero => rfl
  | succ n ih => simp [Nat.add_succ, ih]
```

The structure is identical to `factorial`: a base case (`zero`), an inductive step (`succ n`), and the result from the smaller case (`ih`, the induction hypothesis). If the proof is wrong, the code does not compile. The proof *is* a program, and the program *is* a proof.

The definition of `factorial` says that `factorial(n)` and `n * factorial(n - 1)` are the same expression for arbitrary `n`, an equivalence that holds by term replacement alone, without evaluating either side. It is true by definition. The base case says `factorial(0)` is `1`, also by definition. Those two statements are the entire proof: you show that the program in state `n` and the program in state `n - 1` are the same program up to one substitution, and that the chain has a starting point. That is a proof by induction, and it covers every natural number at once. In lambda calculus notation:

```
fact =  n. if n = 0 then 1 else n * fact(n - 1)
```

`fact` is the function. `fact(n)` is the function applied to some `n`. Replacing the name `fact` with its definition gives:

```
fact(n)
= ( n. if n = 0 then 1 else n * fact(n - 1))(n)   # replace fact with its definition
= if n = 0 then 1 else n * fact(n - 1)              # substitute n
```

You might think: of course they are the same, we just wrote the definition. But `fact` and `fact(n)` are not the same thing. `fact` is a definition, a rule that says what to do with an argument; `fact(n)` is that rule applied to a specific (but arbitrary) `n`. The reduction above shows what happens when you apply the rule: you get a term that contains `fact(n - 1)`, the same program applied to a smaller argument. That relationship between `fact(n)` and `fact(n - 1)` is not something we assumed or proved; it fell out of one substitution. A tautology. And yet that tautology, together with the base case, constitutes a proof by induction that `fact` produces the right answer for every natural number. When `n = 0`, the expression reduces to `1`. The substitution gives you the step from `n - 1` to `n`.

This is what Curry-Howard means in practice. The Lean proof above has the same structure as `factorial`: a base case (`zero`), a step that relates `n` to `n - 1` by definitional equivalence (`succ n`), and nothing else. Both work because the equivalence of two program states is a tautology that requires only substitution.

The seL4 microkernel (used in military helicopters and autonomous vehicles) has a machine-checked proof that certain classes of errors in its code cannot occur, for any input. Not "we haven't found a bug yet": a mathematical guarantee, verified by a computer. The CompCert C compiler has a similar proof that it will never miscompile your code. Both proofs were written as programs in Coq. Traditional testing covers a finite set of inputs; a proof by induction, because it operates on the structure of the program rather than on specific values, covers all of them.

R does not enforce this connection the way Coq or Lean do. But the structure is there underneath, in every recursive function you write, with its base case and its inductive step, carrying the same shape as a proof that holds for all natural numbers. You have been writing proofs this whole chapter. You just did not need to call them that.

6 Numerical computing

Type `0.1 + 0.2 == 0.3` in R. It returns `FALSE`.

```
0.1 + 0.2 == 0.3
#> [1] FALSE
```

This is not a bug. Every number you type into R is an approximation, and operations on those approximations do not always behave the way real arithmetic does. Most of the time the approximation is so good that you never notice it. This chapter shows you the cases where it matters, how to recognize the traps, and how to work with R's matrix algebra tools. The first four sections (storage, traps, cancellation, integers) are for every R user. The rest covers numerical stability and matrix algebra; a callout box marks where you can skip ahead if you are new to R.

6.1. How R stores numbers

R uses IEEE 754 double-precision floating point for all numeric values by default, packing each number into 64 bits: 1 for the sign, 11 for the exponent, 52 for the significand (the significant digits). That buys you roughly 15 to 16 significant decimal digits and a range from about 5e-324 to 1.8e308.

Two machine constants are worth knowing:

```
.Machine$double.eps
#> [1] 2.220446e-16
```

This is *machine epsilon*: the smallest number such that `1 + eps` is not equal to `1`. It defines the granularity of floating point near 1; any difference smaller than this disappears.

```
.Machine$integer.max
#> [1] 2147483647
```

The largest integer R can represent with the integer type. Beyond this, integer arithmetic produces `NA` with a warning.

R also has three special floating-point values:

```
1 / 0
#> [1] Inf
-1 / 0
#> [1] -Inf
0 / 0
#> [1] NaN
```

`Inf` and `-Inf` result from overflow and division by zero, while `NaN` ("Not a Number") results from undefined operations like `log(-1)`. These values propagate: any arithmetic involving `NaN` returns `NaN`, and any comparison with `NaN` returns `NA`.

So R's numbers are approximations, and for most work the approximation is excellent. But "excellent" is not "exact."

> **The lambda calculus connection**
>
> Lambda calculus and mathematical logic operate on exact symbolic terms. Physical computers operate on finite approximations. The gap between `0.1` (the mathematical real number) and `0.1` (the closest IEEE 754 double) is where the two diverge.

How big is that gap? You can see the stored representation with `sprintf()`:

```
sprintf("%.20f", 0.1)
#> [1] "0.10000000000000000555"
sprintf("%.20f", 0.2)
#> [1] "0.20000000000000001110"
sprintf("%.20f", 0.3)
#> [1] "0.29999999999999998890"
```

None of these are the decimal values you typed; they are the closest 64-bit floating-point numbers. Every trap in this chapter traces back to that sliver of space between what you write and what R stores.

6.2. Floating-point traps

The classic:

```
0.1 + 0.2 == 0.3
#> [1] FALSE
```

Neither 0.1 nor 0.2 nor 0.3 can be represented exactly in binary floating point, so the sum of the stored representations of 0.1 and 0.2 differs from the stored representation of 0.3 by a tiny amount (about 5.6e-17). Because `==` checks for exact equality, it returns `FALSE`. This is a specific instance of a deeper fact: floating-point operations do not obey real arithmetic. Addition is not even associative; `(a + b) + c` may differ from `a + (b + c)` because each intermediate result gets rounded independently. David Goldberg's 1991 paper

"What Every Computer Scientist Should Know About Floating-Point Arithmetic" remains the definitive reference.

The fix is `all.equal()`, which compares with a tolerance:

```
all.equal(0.1 + 0.2, 0.3)
#> [1] TRUE
```

The default tolerance is `sqrt(.Machine$double.eps)`, about 1.5e-8. For vectorized comparisons, use `dplyr::near()`:

```
dplyr::near(0.1 + 0.2, 0.3)
#> [1] TRUE
```

Opinion

Never use `==` to compare computed floating-point numbers. Use `all.equal()` for tests and `dplyr::near()` for vectorized comparisons. If you find yourself writing `x == 0.3`, stop and ask whether `x` was computed or typed.

Accumulation is subtler. Adding a small number to a large number can lose the small number entirely:

```
1e16 + 1 - 1e16
#> [1] 0
```

The result should be 1. It is 0, because `1e16 + 1` cannot be distinguished from `1e16` in 64-bit floating point; the 1 simply vanishes. When you call `sum()` on a million numbers, these rounding errors accumulate with each addition. R's `sum()` uses pairwise summation (splitting the vector in half and summing each half recursively) to reduce this drift, though for extreme precision requirements compensated summation algorithms like Kahan's (1965) track the accumulated error in a separate variable and correct for it at each step.

Sequences suffer the same problem:

```
x <- seq(0, 1, by = 0.1)
sum(x == 0.3)
#> [1] 0
```

The sequence has 11 elements, and one of them is "close to" 0.3, but none of them is exactly equal to `0.3` as stored in memory.

Subtraction of nearby values is another trap. Computing `sqrt(x^2 + 1) - x` for large `x` should give a value near `1/(2*x)`, but for large enough `x` the subtraction loses all significant digits:

```
x <- 1e15
sqrt(x^2 + 1) - x   # should be tiny and positive
```

```
#> [1] 0
```

The result is 0, because `sqrt(x^2 + 1)` rounds to `x` before the subtraction happens. This is called *catastrophic cancellation*, and it deserves its own section.

Exercises

1. Check whether `sqrt(2)^2 == 2` returns `TRUE` or `FALSE`. Then use `all.equal()` to compare them.
2. Compute `1e16 + 1 == 1e16` and explain the result.
3. Use `dplyr::near()` to find which elements of `seq(0, 1, by = 0.1)` are close to 0.3.

6.3. Catastrophic cancellation

When you subtract two nearly equal numbers, the significant digits cancel, and what remains is dominated by rounding error.

The textbook example is the variance formula. The mathematically correct identity is:

$$\mathrm{Var}(x) = E[x^2] - (E[x])^2$$

In code:

```
naive_var <- function(x) mean(x^2) - mean(x)^2
```

For well-separated values, this works fine:

```
x <- c(1, 2, 3, 4, 5)
naive_var(x)
#> [1] 2
var(x) * (length(x) - 1) / length(x)  # population variance for comparison
#> [1] 2
```

Now add a large offset so the values are large and close together:

```
y <- x + 1e8
naive_var(y)
#> [1] 2
var(y) * (length(y) - 1) / length(y)
#> [1] 2
```

The naive formula returns a wildly wrong answer (possibly negative, depending on the data), because `mean(y^2)` and `mean(y)^2` are both enormous and nearly equal; their difference is noise.

R's built-in `var()` uses a numerically stable one-pass algorithm that avoids this trap. The lesson: mathematically equivalent expressions are not numerically equivalent. Prefer formulations that avoid subtracting nearly equal quantities. When you see unexpected negative values for something that should be positive (a variance, a squared distance), suspect cancellation.

The same problem appears when computing `sqrt(x^2 + 1) - 1` for small `x`. As `x` approaches zero, `sqrt(x^2 + 1)` rounds to `1`, and the subtraction gives `0`, even though the true answer is a small positive number. The algebraically equivalent form `x^2 / (sqrt(x^2 + 1) + 1)` avoids the cancellation:

```
x <- 1e-8
sqrt(x^2 + 1) - 1          # cancellation
#> [1] 0
x^2 / (sqrt(x^2 + 1) + 1)  # stable
#> [1] 5e-17
```

Exercises

1. Compute `naive_var(x + 1e10)` for `x <- 1:10`. Compare with `var(x) * 9/10`. What happens?
2. For `x <- 1e-12`, compare `sqrt(x^2 + 1) - 1` with `x^2 / (sqrt(x^2 + 1) + 1)`. Which is more accurate?

6.4. Integers

R has 32-bit integers, created with the `L` suffix:

```
typeof(1L)
#> [1] "integer"
typeof(1)
#> [1] "double"
```

`1L` is an integer; `1` without the suffix is a double. The colon operator produces integers:

```
typeof(1:10)
#> [1] "integer"
```

But `c()` coerces to double unless every element carries the `L` suffix:

```
typeof(c(1, 2, 3))
#> [1] "double"
typeof(c(1L, 2L, 3L))
#> [1] "integer"
```

Integer arithmetic is exact within its range, with no rounding and no floating-point surprises. That range, however, is limited:

```
.Machine$integer.max
#> [1] 2147483647
.Machine$integer.max + 1L
#> Warning in .Machine$integer.max + 1L: NAs produced by integer overflow
#> [1] NA
```

Overflow produces `NA` with a warning. R does not silently wrap around to negative numbers the way C does.

When should you use integers? For counts, indices, anything naturally discrete: use `as.integer()` or the `L` suffix. For most data analysis, doubles are fine. Integer precision matters in package code, algorithms, and large-scale computation where the 15-digit limit of doubles becomes a concern for exact counting.

One subtlety: mixing integers and doubles promotes the result to double:

```
typeof(1L + 1.0)
#> [1] "double"
typeof(1L + 1L)
#> [1] "integer"
```

Usually harmless, but it means you cannot always predict the type of an arithmetic result. If exact integer arithmetic matters, keep all operands as integers.

Exercises

1. What is `typeof(1:10 * 1.5)`? Why?
2. What happens when you compute `.Machine$integer.max + 1L`? What about `.Machine$integer.max + 1` (without the `L`)?

6.5. Numerical stability in practice

> **Reading guide**
>
> The sections above (storage, floating-point traps, cancellation, integers) are knowledge every R user needs. The sections below (condition numbers, matrix algebra, log-space) are more specialized. If you are working through the book sequentially, you can skip ahead to Chapter 7 and return here when you encounter these topics in practice, for instance when reading Chapter 28 or Chapter 31.

Individual floating-point traps lose a few digits. But when traps compound across a chain of operations, the entire result can become meaningless. How do you tell in advance?

6.5.1. Condition numbers

The *condition number* of a matrix measures how sensitive the solution of a linear system is to small perturbations in the input. R computes it with `kappa()`:

```
# A well-conditioned matrix
A_good <- matrix(c(2, 1, 1, 3), nrow = 2)
kappa(A_good)
#> [1] 3.333333
```

A condition number near 1 means the system is well-behaved: small changes in the input produce small changes in the output. Large condition numbers mean trouble.

The Hilbert matrix is the classic example of an ill-conditioned matrix, with entries $H_{ij} = 1/(i + j - 1)$:

```
hilbert <- function(n) {
  i <- 1:n
  1 / outer(i, i, function(a, b) a + b - 1)
}

kappa(hilbert(5))
#> [1] 640356.4
kappa(hilbert(10))
#> [1] 2.41618e+13
kappa(hilbert(15))
#> [1] 2.185423e+17
```

The condition number grows exponentially. For a 15x15 Hilbert matrix it is astronomical, meaning that solving $Hx = b$ with 64-bit floats produces answers that are essentially noise. You can verify:

```
H <- hilbert(8)
x_true <- rep(1, 8)
b <- H %*% x_true
x_solved <- solve(H, b)
max(abs(x_solved - x_true))  # should be 0, will not be
#> [1] 4.386696e-07
```

The error can be many orders of magnitude larger than machine epsilon. The matrix is not singular, the code is correct, and yet the problem itself amplifies rounding errors. What, then, can an algorithm guarantee?

6.5.2. Forward and backward stability

A *forward-stable* algorithm produces answers close to the true answer. A *backward-stable* algorithm does something subtler: it produces the exact answer to a slightly perturbed

problem. The computed solution $\hat{x}$ satisfies $(A+\delta A)\hat{x} = b+\delta b$ where δA and δb are small.

Most LAPACK routines (which R calls internally for `solve()`, `qr()`, `svd()`, `chol()`) are backward-stable. For well-conditioned problems, backward stability guarantees accurate results. For ill-conditioned problems, no algorithm can save you, but you can sometimes reformulate the problem so that it becomes well-conditioned.

6.5.3. Polynomial fitting: a practical demonstration

Fitting a high-degree polynomial shows both the problem and the reformulation in action. The naive approach constructs a Vandermonde matrix (columns $1, x, x^2, \dots, x^{15}$), which becomes badly conditioned as the degree grows:

```
set.seed(42)
x <- seq(0, 1, length.out = 50)
y <- sin(2 * pi * x) + rnorm(50, sd = 0.1)

# Unstable: raw powers form an ill-conditioned design matrix
X_raw <- outer(x, 0:15, "^")
kappa(X_raw)
#> [1] 201550293198

# Stable: orthogonal polynomials
X_ortho <- model.matrix(~ poly(x, 15))
kappa(X_ortho)
#> [1] 7.014364
```

The raw Vandermonde matrix has a condition number many orders of magnitude larger than the orthogonal polynomial basis. In practice:

```
# Unstable fit (may produce wild coefficients)
fit_raw <- lm(y ~ X_raw[, -1])

# Stable fit (well-behaved coefficients)
fit_ortho <- lm(y ~ poly(x, 15))

# Compare fitted values: they should be similar but may diverge
max(abs(fitted(fit_raw) - fitted(fit_ortho)))
#> [1] 0.07816089
```

`poly()` exists for a reason. Whenever you fit polynomials, use orthogonal bases. More generally, if a computation involves a matrix, check its condition number before trusting the result.

Exercises

1. Compute the condition number of `hilbert(5)`, `hilbert(10)`, and `hilbert(15)`. How does it grow?
2. Create a 20x20 random matrix with `matrix(rnorm(400), 20, 20)`. Compute its condition number. Is it well-conditioned? Now create `matrix(rnorm(400), 20, 20) + diag(100, 20)` (adding a large diagonal). How does the condition number change?
3. Fit a degree-12 polynomial to `sin(x)` on `[0, 1]` using both `poly(x, 12)` and raw powers. Compare the fitted values. At what degree do the two approaches start to disagree noticeably?

6.6. Matrix algebra

Matrices in R are vectors with a `dim` attribute:

```
A <- matrix(1:6, nrow = 2, ncol = 3)
A
#>      [,1] [,2] [,3]
#> [1,]    1    3    5
#> [2,]    2    4    6
```

Every matrix operation in R eventually calls LAPACK or BLAS — Fortran libraries that have been tested, tuned, and argued over for decades. When you write `solve(A, b)`, you are asking those routines to find the vector x that satisfies $Ax = b$ exactly, using numbers that are not exact. LAPACK promises *backward stability*: the x it hands back is the exact solution to a slightly perturbed version of your problem, $A + \ A$. For well-conditioned matrices, A is negligible — a few ulps, invisible in any application. The Hilbert matrix from the previous section is a different story; there, A is large enough to drown the answer in noise, because the matrix itself amplifies every perturbation and no amount of algorithmic care can undo that. The condition number you computed is the magnification factor, and it is the single number that determines whether `solve()` gives you a trustworthy answer or an expensive random number.

The key operations:

Matrix multiplication uses `%*%`, not `*` (which is element-wise):

```
B <- matrix(c(1, 0, 0, 1, 1, 0), nrow = 3, ncol = 2)
A %*% B
#>      [,1] [,2]
#> [1,]    1    4
#> [2,]    2    6
```

Transpose with `t()`:

```
t(A)
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
#> [3,]    5    6
```

Cross products are faster than explicit transpose-multiply:

```
X <- matrix(rnorm(6), nrow = 3, ncol = 2)
crossprod(X)     # t(X) %*% X, but faster
#>           [,1]      [,2]
#> [1,] 3.2009566 0.5723763
#> [2,] 0.5723763 0.4978568
tcrossprod(X)    # X %*% t(X)
#>            [,1]       [,2]       [,3]
#> [1,]  0.5169554 -0.1946305  0.6850608
#> [2,] -0.1946305  0.6224605 -1.2102932
#> [3,]  0.6850608 -1.2102932  2.5593975
```

Solving linear systems: `solve(A, b)` solves $Ax = b$ for x:

```
A <- matrix(c(2, 1, 1, 3), nrow = 2)
b <- c(5, 7)
x <- solve(A, b)
x
#> [1] 1.6 1.8
A %*% x  # verify: should equal b
#>      [,1]
#> [1,]    5
#> [2,]    7
```

Opinion

Use `solve(A, b)` to solve systems, not `solve(A) %*% b`. Computing the inverse is slower and less numerically stable. The only reason to compute `solve(A)` explicitly is if you need the inverse itself, which is rare.

Decompositions: R provides `eigen()` for eigenvalues, `svd()` for singular value decomposition, `qr()` for QR decomposition, and `chol()` for the Cholesky decomposition of symmetric positive-definite matrices, all backed by LAPACK and BLAS (battle-tested Fortran libraries). R's matrix operations are fast for moderate sizes, up to thousands of rows and columns.

```
eigen(A)$values
#> [1] 3.618034 1.381966
svd(A)$d
#> [1] 3.618034 1.381966
```

A practical note: R fills matrices column by column by default, which catches people who expect row-by-row filling:

```
matrix(1:6, nrow = 2)
#>      [,1] [,2] [,3]
#> [1,]    1    3    5
#> [2,]    2    4    6
matrix(1:6, nrow = 2, byrow = TRUE)
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
```

The `byrow = TRUE` argument changes the fill order. Forgetting this is a common source of bugs in matrix code.

Exercises

1. Create a 3x3 matrix and compute its determinant with `det()`. Then compute the inverse with `solve()` and verify that `A %*% solve(A)` is close to the identity matrix (use `round()` or `all.equal()`).
2. Solve the system: $2x + y = 5$, $x + 3y = 7$. Verify the solution by multiplying `A %*% x`.
3. Use `crossprod()` and `t() %*%` on the same matrix. Verify the results are identical. (Try `bench::mark()` on a larger matrix to see the speed difference.)

6.7. When R lies and when it is precise

A short guide to where you can trust R's numbers and where you should be cautious:

Precise: integer arithmetic, logical operations, string operations, counting. These are exact.

Approximate: any computation involving doubles. Sums, means, variances, regression coefficients. The approximation is excellent (15 digits), but it is still an approximation.

Dangerous spots: comparing doubles for equality, subtracting nearly equal values, summing many numbers of very different magnitudes, numbers near overflow or underflow.

Log-space arithmetic: when numbers get very small (probabilities in statistical models, likelihoods), work in log-space. Compute `log(a) + log(b)` instead of `a * b`. For `log(exp(a) + exp(b))`, use a `logSumExp` function that avoids the overflow of `exp(a)`:

```
log_sum_exp <- function(a, b) {
  m <- pmax(a, b)
  m + log(exp(a - m) + exp(b - m))
}

# Direct computation overflows
```

```
a <- 1000
b <- 1001
# exp(a) + exp(b) would be Inf
log_sum_exp(a, b)
#> [1] 1001.313
```

Most data analysis is safe. The traps bite in numerical algorithms, simulation, and optimization. Be aware of them, but don't be paranoid. One principled approach to tracking error is interval arithmetic: instead of storing a single float, you track `[lower, upper]` bounds, and every operation propagates the interval so that the result is a proof that the true answer lies within the bounds. R has no built-in interval arithmetic, but the idea is worth knowing about when precision is critical.

> **The lambda calculus connection**
>
> Interval arithmetic connects to dependent types in type theory, where a value is annotated with its error bound, turning "how wrong could this be?" from a question into a quantity the computation tracks automatically.

7 Functions are values

In Section 5.4, you saw that a function is a value: you can assign `sqrt` to a new name and call it, the same way you'd assign a number. If functions are values, you can pass them to other functions, store them in lists, and build functions that create new functions.

7.1. Passing functions to functions

`sapply()` takes a vector and a function, and applies the function to every element:

```
sapply(1:5, sqrt)
#> [1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

Notice that `sqrt` has no parentheses. You are not calling `sqrt` here; you are handing the function itself to `sapply`, which calls it five times, once per element. This is the same distinction from Section 5.8: `fact` is the definition (a rule), while `fact(n)` is the rule applied to an argument. When you write `sapply(1:5, sqrt)`, you pass the definition, and `sapply` decides when to apply it.

Here is the pattern behind most of R's power: instead of writing a loop that processes each element, you hand a function to something that knows how to apply it. The loop is still running inside `sapply`, but you never had to write it. You described *what* to compute, not *how* to iterate.

```
sapply(c(-3, 0, 4, -1, 7), abs)
#> [1] 3 0 4 1 7
```

```
sapply(c("hello", "world"), nchar)
#> hello world
#>     5     5
```

Any function that takes a single argument works here: `abs` computes absolute values, `nchar` counts characters, and you can pass either to `sapply` with the same syntax.

`sapply` takes each element of the input, applies your function, and tries to simplify the results into a vector. When all the results have the same length (as they do here, since `sqrt` returns one number per input), you get a numeric vector back. When results differ in length or

type, `sapply` returns a list instead. `lapply()` is the stricter sibling that always returns a list. (Chapter 19 covers the distinction in detail; for now, the simple cases are enough.)

A function that takes another function as an argument is called a *higher-order function*. `sapply` is one; most of R's power comes from others. The pattern is the same one from Section 4.4: apply a function to each element of a container, get back a container of the same shape. In Section 4.4, R did this implicitly with vectorization (`sqrt(c(1,4,9))`). Here, `sapply` does it explicitly: you pick the function, `sapply` handles the iteration.

> **The lambda calculus connection**
>
> In lambda calculus notation (Section 1.2), passing a function to another function is just application:
>
> ```
> sapply = xs. f. [f(xs ), f(xs ), ..., f(xs )]
> sapply([1,2,3,4,5])(sqrt) = [sqrt(1), sqrt(2), sqrt(3), sqrt(4), sqrt(5)]
> ```
>
> Replace each `f(xs )` with its value, and the expression reduces to the result vector. This is the same term replacement from Section 4.2.

Exercises

1. Use `sapply()` to compute `log()` of the numbers 1 through 10.
2. What happens if you write `sapply(1:5, sqrt())` with parentheses? Try it and read the error.
3. Use `sapply()` and `is.na()` to check which elements of `c(1, NA, 3, NA, 5)` are missing.

7.2. Anonymous functions

You can pass a named function like `sqrt` to `sapply`. But suppose you need to square every element: there is no built-in `square` function sitting in base R, and creating one just to use it once feels like overkill.

```
sapply(1:5, function(x) x^2)
#> [1]  1  4  9 16 25
```

`function(x) x^2` creates a function right where it is needed, with no name and no assignment; it exists only for this one call to `sapply`, and then it vanishes. This is an *anonymous function*, and you already saw a brief example in Section 5.4.

The `function(x)` prefix is verbose enough that many R programmers avoided anonymous functions for years, defaulting to named helpers even for throwaway operations. The language eventually responded.

R 4.1 introduced a shorter syntax:

```
sapply(1:5, \(x) x^2)
#> [1]  1  4  9 16 25
```

`\(x)` is shorthand for `function(x)`, and the backslash is meant to evoke the Greek letter lambda ( ), which is where the whole idea comes from (Section 1.2). In lambda calculus, this function is written `x. x²`; in R before 4.1, it is `function(x) x^2`; in R 4.1+, it is `\(x) x^2`.

Anonymous functions shine when the operation is short and you only need it once:

```
sapply(c(10, 20, 30), \(x) x / sum(c(10, 20, 30)))
#> [1] 0.1666667 0.3333333 0.5000000
```

```
sapply(c("alice", "bob"), \(name) paste("Hello,", name))
#>          alice            bob
#> "Hello, alice"   "Hello, bob"
```

When the function gets longer than one line, or when you need it in more than one place, give it a name. A function called `normalize` is easier to read than an anonymous function that divides by the sum. A function called `greet` is easier to maintain than a lambda pasted into three different `sapply` calls.

> **Opinion**
>
> If you use a function twice, name it. Anonymous functions belong inside `sapply`, `lapply`, and similar one-off contexts. The moment you catch yourself copying the same `\(x) ...` expression, that function is asking for a name.

Exercises

1. Use `sapply()` with an anonymous function to add 10 to each element of `c(1, 5, 9)`.
2. Rewrite `sapply(1:5, \(x) x^2)` using the older `function(x)` syntax. Verify you get the same result.
3. Write a named function `to_celsius` that converts Fahrenheit to Celsius. Then use `sapply()` to convert `c(32, 72, 100, 212)`.

7.3. Storing functions in lists

Consider a data frame with mixed column types: some numeric, some character, some logical. You want a one-number summary of each column, but `mean` is meaningless for character data and `length(unique(...))` is meaningless for logicals. The brute-force solution is a chain of `if`/`else if` blocks that checks each column's type and calls the right function. It works for three types. It gets ugly at six. At twelve, it is unreadable, and every new type means hunting through the chain to find the right insertion point.

The logic is fine. The problem is that it's tangled with the control flow. If functions are values, you can separate the two: store the logic in a list, keyed by type, and let a single lookup replace the entire chain.

```
summarizers <- list(
  numeric = \(x) mean(x, na.rm = TRUE),
  character = \(x) length(unique(x)),
  logical = \(x) sum(x, na.rm = TRUE)
)

summarizers$numeric(c(1, 2, 3, NA, 5))
#> [1] 2.75
summarizers$character(c("a", "b", "a", "c"))
#> [1] 3
summarizers$logical(c(TRUE, FALSE, TRUE, TRUE))
#> [1] 3
```

`summarizers` is a list of three functions, accessed with `$` like any named list element. A list of functions keyed by name is a *dispatch table*: given a key, it returns the function you need, no conditionals required. Adding a new type means adding one entry to the list. No `if`/`else` to restructure, no risk of breaking the branches above or below. Package internals, Shiny applications, and testing frameworks all use this pattern to select behavior at runtime.

Notice what moved. In the `if`/`else` version, control flow is in charge: it interrogates the type, then calls the right function as a subordinate. In the dispatch table, the functions exist independently as values in a list, and the control flow shrinks to a single lookup. The operations no longer live inside the conditional; the conditional lives inside the data structure.

An `if`/`else` chain is closed: extending it means opening the chain and editing its guts, hoping you do not break the branches above and below. A dispatch table you extend by dropping a new entry into a list; the entries that already work never see the change.

> **Why dispatch tables feel natural in R**
>
> In Church's lambda calculus, functions were values from the start. Packaging them inside control structures was a habit inherited from imperative languages, not a necessity. The dispatch table recovers the default: data holds the operations, and a lookup replaces the branching.

Here is the same idea in a simpler form:

```
transforms <- list(
  double = \(x) x * 2,
  triple = \(x) x * 3,
  negate = \(x) -x
)
```

```
transforms$double(5)
#> [1] 10
transforms$triple(5)
#> [1] 15
transforms$negate(5)
#> [1] -5
```

The dispatch table stores functions that already exist. But what if you need to manufacture a function that does not exist yet?

Exercises

1. Create a list with two functions: `square` and `cube`. Apply both to the number 4.
2. Write a list of functions where each one converts a temperature from Celsius to a different scale (Fahrenheit, Kelvin). Use them to convert 100 degrees Celsius.

7.4. Functions that return functions

A function can return a number, a string, a vector, or a data frame. It can also return a function.

```
make_adder <- function(n) {
  function(x) x + n
}
```

`make_adder` takes a number `n` and returns a *new function* that adds `n` to its argument:

```
add5 <- make_adder(5)
add5(3)
#> [1] 8
add5(100)
#> [1] 105
```

```
add10 <- make_adder(10)
add10(3)
#> [1] 13
```

`add5` and `add10` are both functions, built by `make_adder` with different values of `n`, and each one remembers the value it was created with. `add5` remembers 5; `add10` remembers 10. How?

When `make_adder(5)` runs, it creates a local environment where `n = 5`. The returned function `function(x) x + n` was defined inside that environment. When you later call `add5(3)`, R finds `n` through lexical scoping (Section 5.5): it looks in the environment where the function was

defined and finds `n = 5`. But that environment was created during a function call that has already returned. The call is gone. So where does the environment live?

A function that travels with its creation environment is called a *closure*. Every function in R is technically a closure, but the term earns its weight when a function returned by another function captures variables from the enclosing scope. Closures are powerful enough to deserve their own chapter (Chapter 18); the mechanism that makes `make_adder` work has consequences far beyond adding numbers.

> **The lambda calculus connection**
>
> In lambda calculus notation:
>
> ```
> make_adder =  n.  x. x + n
> make_adder(5) = ( n.  x. x + n)(5) =  x. x + 5
> ```
>
> Replacing `n` with `5` gives a new function, `x. x + 5`, which is `add5`. The returned function has one argument already filled in. This is *currying*: fixing one argument to produce a simpler function. Each call to a function factory is a beta reduction (Section 1.2); the returned function is the reduced term.

```r
make_multiplier <- function(factor) {
  function(x) x * factor
}

double <- make_multiplier(2)
triple <- make_multiplier(3)

double(7)
#> [1] 14
triple(7)
#> [1] 21
```

`make_multiplier` is another function factory, producing `double` and `triple` from the same template with different values baked in. Function factories appear throughout R: custom plot themes, parameterized statistical tests, configured data transformations. Chapter Chapter 20 covers them in depth. For now, notice that the functions we have been passing around, storing in lists, and returning from factories all share a quiet assumption: that operators like `+` and `*` are themselves functions. Are they?

Exercises

1. Write a function `make_power` that takes an exponent `n` and returns a function that raises its argument to the `n`th power. Create `square <- make_power(2)` and `cube <- make_power(3)` and test them.
2. What does `make_adder(0)` return? Is it the identity function?

3. Write a function `make_greeter` that takes a greeting string (like `"Hello"` or `"Bonjour"`) and returns a function that takes a name and produces the greeting. Test: `greet_en <- make_greeter("Hello"); greet_en("Alice")` should return `"Hello, Alice"`.

7.5. Operators are functions

Try this:

```
`+`(2, 3)
#> [1] 5
```

`+` is a function. It takes two arguments and returns their sum, and when you write `2 + 3`, R translates it to `` `+`(2, 3) ``. The infix notation is syntactic sugar; underneath, it is a function call.

The same is true for every operator:

```
`*`(4, 5)
#> [1] 20
`>`(10, 3)
#> [1] TRUE
`&`(TRUE, FALSE)
#> [1] FALSE
```

It goes further than arithmetic. Indexing with `[` is a function call:

```
x <- c(10, 20, 30)
`[`(x, 2)
#> [1] 20
```

Accessing a list element with `$` is a function call. Even the curly brace `{` is a function call (it evaluates each expression inside and returns the last one). Assignment with `<-` is a function call.

John Chambers, the creator of S (Section 2.3), summarized R's design with two principles:

> "Everything that exists is an object. Everything that happens is a function call."

Numbers, strings, vectors, functions, even `NULL` are all objects. Addition, comparison, indexing, assignment, and control flow are all function calls. Since functions are themselves objects, the two principles fold into each other.

You can even override built-in operators. (This is a demonstration, not a recommendation.)

```
`+` <- function(a, b) a * b
2 + 3
#> [1] 6
```

```
rm(`+`)
2 + 3
#> [1] 5
```

After redefining `+` to mean multiplication, `2 + 3` returns 6. `rm()` removes the custom definition and restores the original. The fact that you *can* redefine `+` shows that it really is just a name bound to a function, like any other name. R's entire syntax is built on function calls, and those functions are values you can inspect, replace, and pass around.

> **Opinion**
>
> Never redefine built-in operators in real code. The example above exists to show you what R is made of, not to suggest a workflow. Code that redefines `+` is code that nobody, including you six months later, will be able to read.

Exercises

1. Rewrite `10 - 3` as an explicit function call using backticks.
2. Rewrite `x[1]` as an explicit function call. (Hint: `` `[`(x, 1) ``.)
3. What does `` `{`(1, 2, 3) `` return? Why?

8 Logic and control flow

In Section 4.3, you saw that `TRUE` becomes 1 and `FALSE` becomes 0 when R needs a number. That was a hint. Logical values are not second-class citizens in R; they are a full type with their own operators, and they drive every decision your code makes. This chapter covers logical vectors, comparisons, boolean operators, and `if/else`. In R, `if/else` is an expression that returns a value, not a statement that directs traffic.

8.1. Logical vectors

`TRUE` and `FALSE` are R's logical values. You already met them in Section 4.2 as one of the four main types (double, integer, character, logical).

```
x <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
typeof(x)
#> [1] "logical"
length(x)
#> [1] 5
```

Every comparison you write, every filter you apply, every conditional check you run produces one of these vectors. The trick from Section 4.3 is what makes expressions like `sum(x > 5)` work: `x > 5` produces a logical vector, `sum()` coerces `TRUE` to 1 and `FALSE` to 0, and suddenly you are counting how many elements pass the test without writing a single loop.

A logical value is one of exactly two things: `TRUE` or `FALSE`. No third option, no grey area, no maybe. In type theory this is a *sum type* (also called a tagged union or variant): a type defined by listing its variants, so that `Bool = TRUE | FALSE`. Contrast this with a data frame row, which combines fields with AND (a penguin has a species AND a mass AND a flipper length); a sum type chooses one variant with OR. You will see sum types surface again when factors define a fixed set of levels (Section 12.4) and when S3 dispatch selects a method based on class (Section 24.2). Where product types say "all of these together," sum types say "exactly one of these." What happens, then, when someone overrides one of the two options?

R has `T` and `F` as shortcuts for `TRUE` and `FALSE`. Do not use them.

```
T <- 42
T
```

```
#> [1] 42
```

```
rm(T)
```

`T` is a regular variable name, so you can overwrite it with anything, and R will not complain. `TRUE` is a reserved word, locked and immutable. Code that relies on `T` meaning `TRUE` will break silently the moment someone (or some package) assigns to `T`, and the same applies to `F`.

> **Opinion**
>
> Never use `T` and `F`. Always write `TRUE` and `FALSE` in full. If someone reassigns `T`, your filter returns the wrong rows and nothing throws an error.

Exercises

1. What does `sum(c(TRUE, FALSE, TRUE, FALSE, TRUE))` return? Why?
2. What is `typeof(TRUE)`? What about `typeof(T)` (before any assignment)?
3. Try `TRUE <- 42`. What happens?

8.2. Comparison operators

Six operators compare values:

```
3 == 3
#> [1] TRUE
3 != 4
#> [1] TRUE
3 < 5
#> [1] TRUE
3 > 5
#> [1] FALSE
3 <= 3
#> [1] TRUE
3 >= 4
#> [1] FALSE
```

All six are vectorized:

```
x <- c(1, 5, 3, 8, 2)
x > 3
#> [1] FALSE  TRUE FALSE  TRUE FALSE
```

Five values in, five logical values out, following the same vectorization from Section 4.4 but applied to comparisons instead of arithmetic.

`%in%` tests membership in a set:

```
x <- c("cat", "dog", "bird", "cat")
x %in% c("cat", "bird")
#> [1]  TRUE FALSE  TRUE  TRUE
```

Compare `x == "cat" | x == "bird"` to `x %in% c("cat", "bird")`: both produce the same result, but `%in%` scales gracefully to larger sets without forcing you to chain `==` and `|` over and over.

One trap deserves early mention. Try this:

```
0.1 + 0.2 == 0.3
#> [1] FALSE
```

`FALSE`. Computers store numbers in binary, and most decimal fractions have no exact binary representation (the same way 1/3 has no exact decimal representation). The stored values of `0.1 + 0.2` and `0.3` differ by about `5.6e-17`, an invisible speck of rounding error that `==` faithfully detects. Use `all.equal()` for approximate comparison:

```
all.equal(0.1 + 0.2, 0.3)
#> [1] TRUE
```

Chapter 6 covers floating-point arithmetic in depth, including catastrophic cancellation, condition numbers, and log-space arithmetic. For now, the rule is simple: never use `==` to compare computed decimal numbers. But if you cannot trust `==` for decimals, how do you combine multiple conditions safely?

Exercises

1. What does `c(1, 2, 3) == c(1, 5, 3)` return?
2. Use `%in%` to check which elements of `c("a", "b", "c", "d")` are vowels.
3. What does `0.1 + 0.1 + 0.1 == 0.3` return? Is it the same as `0.1 + 0.2 == 0.3`? Why?

8.3. Boolean operators

Logical values combine with `&` (and), `|` (or), and `!` (not):

```
TRUE & FALSE
#> [1] FALSE
TRUE | FALSE
#> [1] TRUE
!TRUE
#> [1] FALSE
```

These are vectorized, like arithmetic:

```
x <- c(1, 5, 3, 8, 2)
x > 2 & x < 6
#> [1] FALSE  TRUE  TRUE FALSE FALSE
```

Each element is tested independently: is it greater than 2 *and* less than 6?

R also has `&&` and `||`, which look similar but behave very differently. They examine only the first element of each operand and use *short-circuit evaluation*: if the left side of `&&` is `FALSE`, the right side is never touched, because the result must be `FALSE` regardless of what the right side contains. The same logic applies to `||` when the left side is `TRUE`.

```
FALSE && stop("this is never reached")
#> [1] FALSE
```

No error. `stop()` was never called because `&&` saw `FALSE` on the left and skipped everything to the right. Compare that to the single-ampersand version:

```
FALSE & stop("this IS reached")
#> Error:
#> ! this IS reached
```

`&` evaluated both sides, so `stop()` fired.

> **Opinion**
>
> Use `&` and `|` when working with vectors (filtering data, logical indexing). Use `&&` and `||` inside `if()` conditions, where you have a single logical value and want short-circuit behavior.

`xor()` (exclusive or) returns `TRUE` when exactly one of its arguments is `TRUE`:

```
xor(TRUE, FALSE)
#> [1] TRUE
xor(TRUE, TRUE)
#> [1] FALSE
```

You won't need `xor()` often, but it exists. These four operations, `&`, `|`, `!`, and `xor()`, are not just software conventions; they correspond directly to the logic gates wired into every processor, described by the same truth tables George Boole formalized in 1854.

Exercises

1. What does `c(TRUE, FALSE, TRUE) & c(TRUE, TRUE, FALSE)` return?
2. What does `TRUE | stop("error")` do? What about `TRUE || stop("error")`? Explain the difference.

3. Write a logical expression that tests whether `x` is between 10 and 20 (inclusive).

From logic gates to circuits

The `&` you typed a moment ago to combine filter conditions is the same AND operation that electrical engineers wire into silicon, described by the same truth tables George Boole formalized in 1854. In his 1937 master's thesis, Claude Shannon showed that these truth tables could be implemented with electrical switches, and that insight bridged the gap between abstract algebra and physical circuitry. Every digital circuit is built from these operations.

The `&`, `|`, `!`, and `xor()` you use in R correspond to physical circuits called *logic gates* inside every processor.

An AND gate has two input wires and one output wire; the output is 1 (high voltage) only when both inputs are 1. An OR gate outputs 1 when at least one input is 1. A NOT gate (also called an inverter) has one input and flips it: 1 becomes 0, 0 becomes 1. An XOR gate outputs 1 when the inputs differ.

```
AND gate        OR gate         XOR gate        NOT gate
A  B  out       A  B  out       A  B  out       A  out
0  0   0        0  0   0        0  0   0        0   1
0  1   0        0  1   1        0  1   1        1   0
1  0   0        1  0   1        1  0   1
1  1   1        1  1   1        1  1   0
```

These are the same truth tables as R's `&`, `|`, `xor()`, and `!`. When you write `TRUE & FALSE` in R, the CPU evaluates it using an AND gate (or a chain of them, for vectors). When you write `TRUE & FALSE` in R, the CPU evaluates the same Boolean operation using an AND gate.

Everything a computer does, from adding numbers to rendering video, is built from combinations of these four gates. We can design a circuit the way electrical engineers do: start with inputs and outputs, build a truth table, find the minimal Boolean expression, then wire the gates.

Designing a half adder

The simplest possible addition: two single bits, A and B. Two outputs: a sum bit and a carry bit (because 1 + 1 = 10 in binary, which is 0 with a carry of 1).
Step 1: truth table. List every possible input combination and the desired output.

```
A  B  | Sum  Carry
0  0  |  0     0
0  1  |  1     0
1  0  |  1     0
1  1  |  0     1
```

Step 2: Boolean expression. For each output, read off the rows where it equals 1. Write each row as a product (AND) of the inputs, using NOT for 0s. Then combine the rows with OR. This is called the *sum of minterms*, the canonical form for any Boolean function:

```
Sum   = (NOT A AND B) OR (A AND NOT B)
Carry = A AND B
```

8.4. How numbers are stored

R has `double` and `integer` types (Section 4.2), and for most day-to-day work you never think about the difference. But the computer thinks about it constantly, because the two types are stored in fundamentally different ways.

Everything is bits: 0s and 1s. An integer is stored as a binary number, so the integer 42 in binary is `101010`:

```
 1   0   1   0   1   0
2   2   2³  2²  2¹  2
32  16   8   4   2   1

32 + 0 + 8 + 0 + 2 + 0 = 42
```

Each position represents a power of 2: a `1` means "include this power," a `0` means "skip it." R uses 32 bits for an integer, so the largest integer is 2^{31} - 1 = 2,147,483,647 (one bit is reserved for the sign, positive or negative).

```
.Machine$integer.max
#> [1] 2147483647
```

A double (also called a floating-point number) is stored differently, using 64 bits split into three parts: 1 bit for the sign, 11 bits for the exponent, and 52 bits for the fraction (called the significand):

```
[sign]  [exponent: 11 bits]  [significand: 52 bits]
  1 bit     determines           determines
  +/-       the scale            the precision
```

The number is reconstructed as: $\pm$ fraction $\times$ 2^{exponent}. This format (IEEE 754) can represent very large numbers (up to about 1.8×10^{308}) and very small ones (down to about 2.2×10^{-308}), but not all numbers exactly, which is exactly the trap that bit us with `0.1 + 0.2`.

```
.Machine$double.xmax
#> [1] 1.797693e+308
.Machine$double.xmin
#> [1] 2.225074e-308
```

The 52-bit significand stores the closest binary approximation of each decimal number; when you add two approximations, rounding errors accumulate. Chapter 6 covers these traps in detail, including how to detect them with `sprintf("%.20f", ...)` and how to avoid them with `all.equal()` and `dplyr::near()`.

R defaults to double for all numbers unless you add the `L` suffix:

```
typeof(42)
#> [1] "double"
typeof(42L)
#> [1] "integer"
```

`42` is a 64-bit floating-point number; `42L` is a 32-bit integer. For most data analysis, the distinction is invisible, until you compare computed values with `==` or work with very large counts. But now that you understand how R stores and compares values, the question shifts: how does R act on the results of those comparisons?

8.5. `if/else` as an expression

In most languages, `if/else` is a *statement*: it controls which code runs but does not produce a value. In R, `if/else` is an *expression*: it evaluates to the value of whichever branch is taken, and you can assign that value directly.

```
x <- -3
y <- if (x > 0) "positive" else "non-positive"
y
#> [1] "non-positive"
```

`if (x > 0) "positive" else "non-positive"` evaluated to `"non-positive"`, and that value was assigned to `y`. This follows from Section 3.4: everything in R is an expression, and expressions return values. `if/else` is no exception.

Multi-line branches use curly braces, and the value of each branch is the last expression evaluated inside it:

```
classify <- function(x) {
  if (x > 0) {
    label <- "positive"
    label
  } else if (x == 0) {
    "zero"
  } else {
    "negative"
  }
}

classify(5)
#> [1] "positive"
classify(0)
#> [1] "zero"
classify(-2)
#> [1] "negative"
```

R has no `elif` keyword; you chain conditions with `else if`, which is just an `else` followed by another `if`. Long chains get hard to read, and when you have more than two or three branches, consider `switch()` (Section 8.7) or `dplyr::case_when()` (Section 8.6).

The connection to lambda calculus runs deeper than it might seem. In Chapter 22, we saw Church's encoding of booleans, where `TRUE = x. y. x` (pick the first) and `FALSE = x. y. y` (pick the second). R's `if/else` does the same thing: it takes a condition and two branches, and returns whichever branch the condition selects, just as Church's booleans select an argument. The Curry-Howard correspondence takes this further, mapping logical propositions to types and proofs to programs; under this lens, `TRUE` and `FALSE` are proof and refutation, and an `if/else` expression is a case analysis on evidence. Typed functional languages like Haskell make this connection explicit; R keeps it implicit, but the structure is the same.

```
if (c(TRUE, FALSE)) "yes"
#> Error in `if (c(TRUE, FALSE)) ...`:
#> ! the condition has length > 1
```

`if` expects a single logical value. If you give it a vector, R uses only the first element and warns you. But data analysis rarely involves a single value; you usually need to test every element of a column at once. So what do you reach for when `if/else` won't scale?

Exercises

1. What does `if (TRUE) 1 else 2` return? What about `if (FALSE) 1 else 2`?
2. Write a function `sign_label` that takes a number and returns `"positive"`, `"zero"`, or `"negative"`.
3. What does `if (NA) "yes" else "no"` produce? Why?

8.6. Vectorized conditionals

`if/else` works on a single value. `ifelse()` works on vectors:

```
x <- c(-2, 0, 3, -1, 5)
ifelse(x > 0, "pos", "neg")
#> [1] "neg" "neg" "pos" "neg" "pos"
```

For each element, `ifelse()` checks the condition and returns the corresponding value from the second or third argument, vectorized in the same way `+` and `>` are (Section 4.4).

But `ifelse()` has a weakness: it doesn't check types.

```
ifelse(TRUE, 1, "no")
#> [1] 1
ifelse(FALSE, 1, "no")
```

```
#> [1] "no"
```

One call returns a number, the other a string. The return type depends on which branch is taken, and that can change silently when your data changes. `dplyr::if_else()` is stricter:

```
dplyr::if_else(TRUE, 1, "no")
#> Error in `dplyr::if_else()`:
#> ! Can't combine `true` <double> and `false` <character>.
```

It refuses to mix types, catching bugs early rather than letting them propagate downstream.

For multiple conditions, `dplyr::case_when()` replaces nested `ifelse()` chains:

```
x <- c(-2, 0, 3, -1, 5)
dplyr::case_when(
  x > 0  ~ "positive",
  x == 0 ~ "zero",
  x < 0  ~ "negative"
)
#> [1] "negative" "zero"     "positive" "negative" "positive"
```

Each line is a condition-value pair, evaluated in order; `case_when()` returns the value for the first condition that matches. Compare this to the nested version:

```
ifelse(x > 0, "positive", ifelse(x == 0, "zero", "negative"))
```

Both produce the same result, but `case_when()` is easier to read, easier to extend, and harder to get wrong.

> **Opinion**
>
> Prefer `dplyr::case_when()` over nested `ifelse()`. Nesting `ifelse()` calls creates code that is hard to read and easy to break when you add a condition; `case_when()` scales cleanly to any number of branches.

Exercises

1. Use `ifelse()` to replace negative values in `c(-3, 5, -1, 8, 0)` with zero.
2. Use `dplyr::case_when()` to classify `penguins$body_mass_g` into `"light"` (under 3500), `"medium"` (3500-5000), and `"heavy"` (over 5000). Don't forget `NA`s.
3. What happens if no condition matches in `case_when()`? Test it.

8.7. switch()

When you need to dispatch on a single string value, `switch()` is cleaner than a chain of `if/else if`:

```
describe <- function(day) {
  switch(day,
    Monday    = "start of the week",
    Friday    = "almost there",
    Saturday  = ,
    Sunday    = "weekend",
    "just another day"
  )
}

describe("Friday")
#> [1] "almost there"
describe("Saturday")
#> [1] "weekend"
describe("Wednesday")
#> [1] "just another day"
```

Each name is matched against the input. `Saturday =` with no value falls through to the next case (`Sunday`), so both return `"weekend"`. The unnamed last entry is the default.

`switch()` only works with a single string (or number, but string dispatch is the common use). For vector operations, use `case_when()`. For complex branching logic, use `if/else if`. `switch()` fills the narrow gap where you have one value and several named options, and it fills it well. But the real lesson of this chapter is not about any single function. From logical vectors through comparison operators, boolean algebra, and conditional expressions, you have traced a single thread: the idea that every computation reduces to choosing between two values, `TRUE` or `FALSE`, 1 or 0, high voltage or low. The `if/else` in your R script and the AND gate in your CPU are expressions of the same underlying logic, separated by layers of abstraction but never by kind. The next chapter picks up where that thread leads: what happens when you need to make the same choice not once but thousands of times, across every row of a data frame, without writing a single loop.

9 Complexity and algorithms

You write a loop that takes three seconds on your test data. You deploy it on the full dataset, ten times larger, and it takes five minutes. Not thirty seconds. Five minutes. You double-check the code; nothing has changed, nothing is obviously wrong. The loop is doing the same thing it did before, just on more rows. So why did a 10x increase in data produce a 100x increase in running time?

The answer is not about your code being "bad," and no amount of hardware will fix it. It is about the *shape* of growth: how running time scales with input size, and why some operations hit a wall that scales steeper than you expected. Big-O notation, hash tables, the limits of computation: these ideas draw the line between code that scales and code that chokes. They also explain why that line exists at all.

9.1. Not all operations are equal

Consider two ways to find a value in a vector. `x[5]` jumps directly to the fifth memory address, regardless of how long the vector is. `which(x == 5)` checks every element one by one until it finds a match. For a vector of length 10, you would not notice the difference. For a vector of length 10 million, you would.

That difference is not a constant factor that a faster computer could erase; it *grows* with input size, because one operation ignores the size of the vector entirely while the other touches every element in it. A constant-time operation stays constant whether the input has ten elements or ten billion, while a linear operation scales proportionally, doing twice the work on twice the data. How running time scales with input size: that is the core idea of computational complexity, and once you see it, you start noticing it everywhere.

9.2. Big-O notation

Big-O notation formalizes a single question: how does running time grow with input size? We say f(n) is O(g(n)) if, past some input size, f grows no faster than a constant multiple of g.

The notation strips away constant factors and lower-order terms, which means an algorithm that takes $3n^2 + 100n + 7$ steps is simply $O(n^2)$. The constants matter in practice (a factor of 10 is still a factor of 10), but Big-O captures the *shape* of growth, and that shape dominates once n gets large enough. What about when n is small?

9.2.1. Common complexity classes

O(1): constant time. The operation takes the same time regardless of input size. `length(x)` is O(1) because R stores the length as an attribute and never counts the elements; indexing by position (`x[5]`) is also O(1), since R computes a memory offset and jumps directly to it. A less obvious example, and a more interesting one, is hash table lookup, which gets its own section below (Section 9.3).

O(log n): logarithmic time. Each step halves the search space. Binary search is the classic example: `findInterval(0.5, sorted_x)` finds where 0.5 falls in a sorted vector by repeatedly splitting it in half, which on a million elements takes roughly 20 comparisons and on a billion elements about 30. Logarithmic growth is *extremely* slow. So slow it barely registers.

O(n): linear time. You touch every element once. `sum(x)` walks the entire vector, adding each element: one pass, n additions. `which(x > 0)` also scans every element once, selecting those that match. Linear is often the best you can do, because you need to at least *read* all the input.

O(n log n): linearithmic time. The cost of sorting. `sort(x)` for a vector of length n makes O(n log n) comparisons, which is only slightly worse than linear: sorting a million numbers takes about 20x more work than summing them, not 1,000,000x more. Can you do better? For comparison-based sorting, no: there is a proof (via decision trees) that any algorithm which sorts by comparing pairs of elements must make at least O(n log n) comparisons in the worst case. Radix sort sidesteps this bound by not comparing elements at all (it distributes them into buckets by digit) but it only works when the keys have bounded range (integers, fixed-length strings).

O(n²): quadratic time. Nested loops where both loops run over the input. This is where performance falls apart. The classic R example is growing a vector with `c()`:

```
# Each c() call copies the entire existing vector
result <- c()
for (i in seq_len(n)) {
  result <- c(result, i)  # copies 1, then 2, then 3, ... elements
}
# Total copies: 1 + 2 + ... + n = n(n+1)/2 = O(n^2)
```

Chapter 28 revisits this bottleneck in detail. The quadratic cost is not inevitable for append-style operations: if R used a doubling strategy (as Python's `list.append` does), each individual append would be O(1) *amortized*. The idea is to allocate double the space when the array fills up; most appends just write to the next slot (O(1)), and the occasional O(n) resize happens rarely enough that the average cost per operation is constant. R does not do this for `c()`, which is why pre-allocation matters. A naive distance matrix computation is also $O(n^2)$: for n points, you compute n(n-1)/2 pairwise distances.

O(2^n): exponential time. Brute-force enumeration of all subsets. A set of n elements has 2^n subsets: at n = 20 that is about a million, at n = 30 a billion, at n = 50 over a quadrillion. Exponential algorithms are effectively unusable for large inputs, and no clever constant-factor tricks will save you.

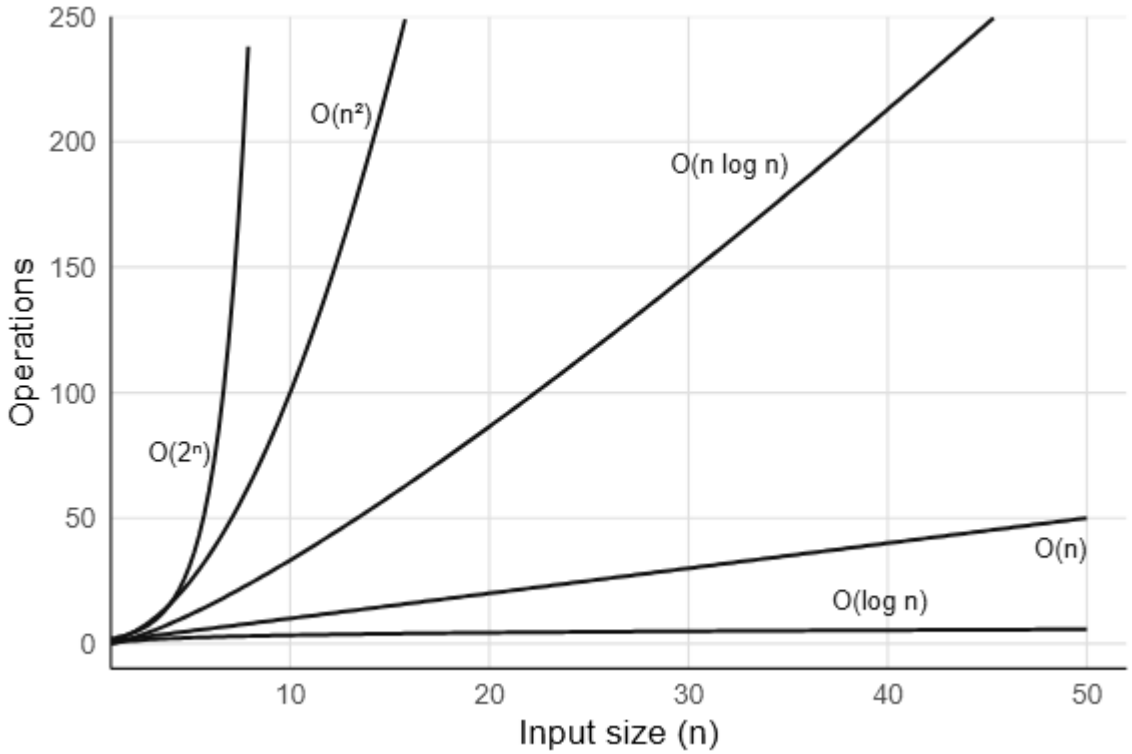


Figure 9.1.: Common complexity classes: how operations scale with input size.

9.2.2. How to tell empirically

The simplest test: double n and see what happens to the time. If the time roughly doubles, you have O(n); if it roughly quadruples, O(n^2). Try this with `sum(rnorm(n))` (linear) versus the `c()` growth loop (quadratic) for n = 1000, 2000, 5000, 10000. The ratio test works because Big-O captures the exponent, and doubling the input raises a power-law relationship to the surface. What happens when the ratio is somewhere between 2 and 4? A ratio near 2.8 when you double n suggests O(n^1.5), and in general a ratio of r implies a growth exponent of log2(r). You can also fit a log-log regression: plot log(time) against log(n), and the slope estimates the exponent directly.

Exercises

1. What is the Big-O complexity of `rev(x)` for a vector of length n? Why?
2. You have a sorted vector of length n. Compare the time of `x[1]` (get the minimum) versus `min(x)` (scan for the minimum). What are their complexities?
3. The function `outer(x, y, "-")` computes all pairwise differences between two vectors. If both have length n, what is the complexity in time and space?

9.3. Hash tables: O(1) lookup from scratch

Suppose you have a list of 10,000 names and you need to check whether `"beta"` is among them. The naive approach scans the list from the beginning, comparing each entry, and on average you check half the entries before finding a match (or all of them if the name isn't there). That is O(n).

If the list is sorted, binary search gets you to O(log n). But there is a way to get O(1), constant time, regardless of how many names are in the list.

The trick is to turn the name into a number. A *hash function* takes a string (or any value) and produces an integer. For example, a simple hash might sum the character codes of each letter:

```
simple_hash <- function(key, n_slots) {
  codes <- utf8ToInt(key)
  (sum(codes) %% n_slots) + 1   # map to a slot between 1 and n_slots
}

simple_hash("beta", 10)
#> [1] 3
simple_hash("gamma", 10)
#> [1] 6
simple_hash("alpha", 10)
#> [1] 9
```

Now allocate a fixed-size array (say 10 slots). To store `"beta" = 2`, compute `simple_hash("beta", 10)` and put the value in that slot; to look it up later, compute the same hash, jump directly to that slot, and read the value. No scanning. The hash function converts a search problem into an arithmetic problem.

```
# Build a hash table by hand
n_slots <- 10
slots <- vector("list", n_slots)

# Insert three key-value pairs
insert <- function(slots, key, value, n_slots) {
  i <- simple_hash(key, n_slots)
  slots[[i]] <- list(key = key, value = value)
  slots
}

slots <- insert(slots, "alpha", 1, n_slots)
slots <- insert(slots, "beta",  2, n_slots)
slots <- insert(slots, "gamma", 3, n_slots)

# Look up "beta": one hash computation, one slot access
```

```
lookup <- function(slots, key, n_slots) {
  i <- simple_hash(key, n_slots)
  if (!is.null(slots[[i]])) slots[[i]]$value else NA
}

lookup(slots, "beta", n_slots)
#> [1] 2
```

One hash computation, one array access, O(1). That is the entire idea.

The problem is *collisions*: two different keys can hash to the same slot, and our `simple_hash` sums character codes, so any two strings with the same character-code sum will collide. When that happens, the slot needs to hold multiple entries, and the standard fix is *chaining*, where each slot stores a linked list of key-value pairs. On lookup, you hash to the slot, then scan the short list:

```
# A hash table with chaining
insert_chain <- function(slots, key, value, n_slots) {
  i <- simple_hash(key, n_slots)
  slots[[i]] <- c(slots[[i]], list(list(key = key, value = value)))
  slots
}

lookup_chain <- function(slots, key, n_slots) {
  i <- simple_hash(key, n_slots)
  chain <- slots[[i]]
  for (entry in chain) {
    if (entry$key == key) return(entry$value)
  }
  NA
}

slots2 <- vector("list", 10)
slots2 <- insert_chain(slots2, "alpha", 1, 10)
slots2 <- insert_chain(slots2, "beta",  2, 10)
slots2 <- insert_chain(slots2, "gamma", 3, 10)

lookup_chain(slots2, "beta", 10)
#> [1] 2
lookup_chain(slots2, "missing", 10)
#> [1] NA
```

In the worst case, every key hashes to the same slot, the chain has n entries, and lookup degrades to O(n). But a good hash function distributes keys evenly across slots, keeping each chain short: with n keys and n slots, the average chain length is 1, and lookup is O(1) on average.

R uses exactly this mechanism for environments. Every environment is a hash table mapping

names to values, so when you write `e$beta`, R hashes the string `"beta"`, jumps to the right slot, and returns the value. This is also why `match()`, `duplicated()`, `%in%`, `unique()`, and `table()` are fast: they build a temporary hash table internally, making membership tests O(1) per query instead of O(n).

```
# R environments are hash tables
e <- new.env(hash = TRUE, parent = emptyenv())
e$alpha <- 1
e$beta  <- 2
e$gamma <- 3

# This does NOT scan through "alpha" first.
# The hash of "beta" points directly to the right slot.
e$beta
#> [1] 2
```

The constant in O(1) depends on the hash function's speed and the collision rate. Real hash functions (R uses one internally for its environments) are more sophisticated than summing character codes, distributing keys more uniformly and handling edge cases. But the principle is always the same: convert the key to an integer, use it as an index, handle collisions with chaining. What happens when the table fills up? The chains get longer, and lookup degrades toward O(n). Real hash tables track the *load factor* (number of entries divided by number of slots) and resize, typically doubling the slot count, when it crosses a threshold (often 0.75). Resizing costs O(n) because every key must be rehashed into the new array, but it happens rarely enough that the amortized cost per insertion stays O(1).

Exercises

1. What happens to `simple_hash` if two keys have the same character-code sum? Try `simple_hash("ab", 10)` and `simple_hash("ba", 10)`. Why is this a problem, and how does chaining solve it?
2. Our hand-built hash table has a fixed number of slots. What happens as you insert many more keys than slots? How does the average chain length change? (This is why real hash tables *resize*: when the load factor exceeds a threshold, they allocate a larger array and rehash all keys.)
3. Use `system.time()` to compare `x %in% y` versus a loop with `any(y == x[i])` for `x <- sample(1e6)` and `y <- sample(1e4)`. The first builds a hash table; the second does linear scans.

9.4. Space complexity

Time is not the only resource that scales. Memory matters too, especially in R where data frames can be gigabytes, and running out of RAM kills your session more decisively than a slow loop ever could.

A full copy of a 1 GB data frame uses 1 GB of additional memory; two copies and you are at 3 GB total. R's garbage collector reclaims unused memory, but if multiple large objects are live simultaneously, you can exhaust RAM. How does R decide when to copy?

9.4.1. Copy-on-modify

R uses *copy-on-modify* semantics: when you assign an object to a new name, R does not immediately copy it. Both names point to the same underlying data, and a copy is triggered only when one reference is modified:

```
x <- rnorm(1e6)
y <- x              # no copy yet: x and y share the same memory
y[1] <- 0           # NOW R copies the entire vector, then modifies element 1
```

You can verify this with `lobstr::obj_addr()`:

```
library(lobstr)
x <- rnorm(1e6)
y <- x
obj_addr(x) == obj_addr(y)  # TRUE: same memory

y[1] <- 0
obj_addr(x) == obj_addr(y)  # FALSE: y is now a separate copy
```

Copy-on-modify is why passing large objects to functions is cheap in R: the function receives a reference, not a copy, and a copy happens only if the function modifies the object. But when modification *does* happen (say, inside a loop) the cost can compound quickly, turning what looks like an O(n) operation into something much worse.

9.4.2. Pre-allocation vs. growing

The `c()` loop from the quadratic example above is slow in time and wasteful in memory. Each iteration allocates a new vector of size i, copies the old data, and discards the old allocation, so the total memory allocated across all iterations is $1 + 2 + 3 + \ldots + n = O(n^2)$, even though the final result uses only O(n) space.

Pre-allocation fixes both problems:

```
# O(n²) total memory allocated
result <- c()
for (i in seq_len(n)) result <- c(result, i)

# O(n) total memory allocated
result <- numeric(n)
for (i in seq_len(n)) result[i] <- i
```

9.4.3. Measuring memory

```
x <- rnorm(1e6)
object.size(x)
#> 8000048 bytes
```

`object.size()` gives a rough estimate. For more accurate measurements, including shared memory between objects:

```
lobstr::obj_size(x)

# Shared memory: two references to the same object
y <- x
lobstr::obj_size(x, y)  # same as obj_size(x), not double
```

Exercises

1. Create a list of 10 data frames, each with 1000 rows and 5 columns. Use `lobstr::obj_size()` to measure the total. Is it 10 times the size of one data frame?
2. Explain why `x <- 1:1e8` uses far less memory than `x <- as.numeric(1:1e8)`. (Hint: check with `object.size()`. R stores the compact sequence as just a start, end, and step, exploiting the structure. This connects to Kolmogorov complexity: the shortest program that produces a given output. "Random" data is incompressible, while structured data like `rep(1, 1e6)` can be described in a few bytes. Compression algorithms approximate Kolmogorov complexity from above.)
3. Write a function that takes a data frame and returns it unchanged. Use `lobstr::obj_addr()` to verify that no copy is made.

9.5. Common algorithms in R

You rarely implement algorithms from scratch in R, but the built-in functions make algorithmic choices on your behalf, and knowing what they chose helps you predict when they will be fast and when they will not.

9.5.1. Sorting

`sort()` uses a hybrid introsort: quicksort for the fast average case, switching to heapsort if quicksort degrades, and insertion sort for small sub-arrays. Worst-case time is O(n log n).

`order()` returns the permutation that would sort the vector, also O(n log n), and for data frames it is the basis of `dplyr::arrange()`. R also offers `sort(x, method = "radix")` for integer and character vectors; radix sort is O(n) for bounded-range integers, which is why `data.table::setorder()` is fast: it uses radix sort by default.

9.5.2. Searching and matching

`match()` and `%in%` build a hash table from one vector, then probe it for each element of the other. Building the table is O(n); each lookup is O(1) on average, so `x %in% y` is O(n + m) where n and m are the lengths of `x` and `y`.

`findInterval()` does binary search on a sorted vector: O(log n) per query. If you have many queries against the same sorted vector, `findInterval` can be faster than building a hash table, because the sorted structure is already there and hashing has overhead.

9.5.3. Hashing

`duplicated()`, `unique()`, and `table()` all use hash tables internally. That is why `duplicated(x)` is O(n), not the $O(n^2)$ you would get from comparing every pair of elements.

9.5.4. String matching

`grep()` and `grepl()` compile the pattern into a finite automaton, then scan the input strings. For fixed strings, `fixed = TRUE` skips regex compilation entirely:

```
# Regex: compiles pattern, then scans
grep("item_5", x)

# Fixed string: faster, no regex overhead
grep("item_5", x, fixed = TRUE)
```

Using `fixed = TRUE` avoids regex compilation and can be noticeably faster for simple substring matches; for repeated matching against the same pattern, pre-compiling with `stringi::stri_detect_regex()` avoids recompilation. But what happens when the patterns themselves grow complex, when you need backtracking or lookaheads?

Exercises

1. You need to check whether each element of a vector `x` (length 1 million) appears in a lookup table `y` (length 100). Compare `x %in% y` with a loop using `any(y == x[i])`. What are their complexities?
2. Why is `duplicated(x)` faster than `length(unique(x)) < length(x)` for simply checking whether duplicates exist? (Hint: `duplicated` can short-circuit in principle, though R's implementation does not.)
3. You have a sorted numeric vector of length 10 million. You need to find where 1000 query values fall. Compare `findInterval(queries, sorted_vec)` with `match(queries, sorted_vec)`. Which is appropriate here, and why?

9.6. Computability

Everything above assumes the problem *can* be solved, that some algorithm exists, however slow, that will eventually produce the answer. Not all problems have that property.

9.6.1. The halting problem

By the early twentieth century, mathematicians had been on a winning streak that stretched back decades. Cantor had tamed infinity. Frege had formalized logic. Peano had reduced arithmetic to five axioms. Every time someone asked "can we make this rigorous?" the answer had been yes. David Hilbert had watched this happen for thirty years, and in 1928 he posed the question that felt like the natural finish line: the *Entscheidungsproblem*, the decision problem. Could you build a mechanical procedure that takes any mathematical statement and tells you whether it is true or false? If such a procedure existed, mathematical truth would be computable. Every conjecture decidable, every proof checkable by turning a crank, but then in 1936 a twenty-three-year-old graduate student would shock the whole mathematical worldview.

He proved that no algorithm can decide, for an arbitrary program and input, whether that program will eventually halt or run forever. It was Alan Turing, the same graduate student whose tape machine we met in Section 1.1, now proving that his own machine had a boundary it could never cross. The specific question he settled is called the *halting problem*. The proof is one of the most beautiful arguments in mathematics, it fits in half a page, and it works by turning the assumption against itself. We give it in full, twice: once in R and once in lambda calculus.

The proof by contradiction (in R). Assume, for the sake of contradiction, that a function `halts(f, x)` exists. It takes any function `f` and any input `x`, and returns `TRUE` if `f(x)` terminates, `FALSE` if `f(x)` runs forever. It always returns one or the other; it never crashes or loops itself. That is the assumption we will break.

Given `halts`, define a new function:

```
trouble <- function(x) {
  if (halts(trouble, x)) {
    while (TRUE) {}   # loop forever
  } else {
    return(x)          # halt
  }
}
```

Now ask: does `trouble(trouble)` halt?

Case 1: Suppose `halts(trouble, trouble)` returns `TRUE`. Then `trouble(trouble)` enters the `if` branch and loops forever. But `halts` said it would halt. Contradiction.

Case 2: Suppose `halts(trouble, trouble)` returns `FALSE`. Then `trouble(trouble)` enters the `else` branch and returns `trouble`. It halts. But `halts` said it would not. Contradiction.

Both cases contradict the assumption, and there is no third case (we assumed `halts` always returns `TRUE` or `FALSE`). Therefore `halts` cannot exist. No algorithm, in any language, can solve the halting problem for arbitrary programs.

The argument is a *diagonal argument*, the same technique Cantor used in 1891 to prove that the real numbers are uncountable, and the same structure Godel used in 1931 for the incompleteness theorems. The trick is always the same: assume a complete listing (or decision procedure) exists, then construct something that disagrees with every entry on the diagonal.

The proof in lambda calculus. Seeing the same proof in lambda calculus is worth the detour, because it shows that undecidability is not a quirk of loops or mutable state or any particular language's execution model. The result falls out of self-application alone, which means it holds for any system powerful enough to pass functions to functions — R included.

Church proved the same result independently in 1936, before Turing's paper, using the *lambda calculus* rather than a machine model. The argument has the same shape but uses different language.

In lambda calculus, every value is a function and every computation is function application. There are no loops; non-termination means a term *reduces forever* without reaching a normal form. For instance, the term (x. x x)(x. x x) applies itself to itself, producing the same term again, endlessly, never halting. This term is called Ω.

Church's question: is there a lambda term H that, given any term M, returns `true` if M has a normal form (halts) and `false` if M reduces forever?

Assume H exists. Define:

```
D =  m. if (H (m m)) then Ω else I
```

where Ω is the non-terminating term from above and I = x. x is the identity (which halts immediately). Now apply D to itself:

- If H(D D) returns `true` (meaning D D halts), then D D reduces to Ω and loops forever. Contradiction.
- If H(D D) returns `false` (meaning D D diverges), then D D reduces to I and halts. Contradiction.

Same diagonal structure, same conclusion: H cannot exist. No lambda term can decide whether an arbitrary term has a normal form.

The mechanism that breaks H is self-application: a term applied to itself. That mechanism is not exotic. When you write `function(f) f(f)` in R, you are writing self-application. When you pass a function to `sapply`, you are handing a value to a function that will call it, one step from the same recursive knot. R has the same capability: functions are values, values can be passed to functions, and nothing prevents a function from receiving itself. Closures, recursion, and higher-order functions all depend on that freedom. The halting problem is the price. Any language expressive enough to let functions consume themselves is expressive enough to construct the diagonal argument, and that includes every language in this book's

lineage from Lisp onward. When a linter warns you about unreachable code or an optimizer tries to eliminate dead branches, it is working inside the limits Church proved in 1936: necessarily incomplete, always capable of missing something.

Church published this in April 1936 ("An Unsolvable Problem of Elementary Number Theory"); Turing published his version in November 1936 ("On Computable Numbers"). They later proved that their two formalisms, lambda calculus and Turing machines, compute exactly the same class of functions. The halting problem is not an artifact of one model. It is a boundary.

Why it matters in practice. Rice's theorem (1953) generalizes the result: *any* non-trivial semantic property of programs is undecidable. You cannot write a program that decides whether another program is "correct," "efficient," or "pure." Every static analyzer is necessarily incomplete, meaning it will either miss some bugs or raise false alarms. Tools like lintr and mypy use sound overapproximations, rejecting some correct programs to guarantee they never accept an incorrect one. The question is not whether your analyzer has blind spots, but where.

9.6.2. Practical workarounds

You cannot solve the halting problem in general, but you can set time limits:

```
# setTimeLimit: kill a computation after 5 seconds
setTimeLimit(elapsed = 5, transient = TRUE)
tryCatch(
  {
    # potentially non-terminating computation
    result <- some_function(x)
  },
  error = function(e) message("Computation timed out")
)
setTimeLimit(elapsed = Inf)  # reset
```

The R package `R.utils` provides `withTimeout()` for a cleaner interface. These are engineering solutions to a mathematically unsolvable problem: you cannot know whether the computation will finish, but you can decide how long you are willing to wait.

9.6.3. The Church-Turing thesis

In 1936, Alonzo Church proved that the *Entscheidungsproblem* (the decision problem for first-order logic) is unsolvable, using lambda calculus as his model of computation. Alan Turing, working independently, proved the same result using his abstract machine. Turing then went further: in an appendix added after learning of Church's work, he proved that the two formalisms compute exactly the same class of functions. The *Church-Turing thesis* states that any function computable by an effective procedure can be computed by a Turing machine, or equivalently, expressed as a lambda calculus term.

This has a concrete consequence: R, Python, C, Haskell, and every other general-purpose language can compute exactly the same set of functions. No language can solve the halting problem; no language can compute a function that another general-purpose language cannot. The differences between languages are speed, expressiveness, and safety, not computational power.

R's functional core (functions as values, closures, recursion) is directly descended from lambda calculus, so when you write `function(x) x + 1`, you are writing a lambda term. The equivalence means R's functional style and C's imperative loops are two notations for the same mathematics.

9.6.4. P vs NP

Some problems are easy to *verify* but (probably) hard to *solve*. Given a proposed solution to the traveling salesman problem, you can check its total distance in O(n), but finding the optimal solution appears to require exponential time.

The class **P** contains problems solvable in polynomial time; the class **NP** contains problems whose solutions can be *verified* in polynomial time. Whether P = NP is the most famous open question in computer science, and most researchers believe P NP, which would mean some problems are fundamentally harder to solve than to check.

The subset sum problem makes the gap concrete. Given a set of integers, does any subset sum to a target value? Verifying a proposed subset is O(n), just add the numbers. Finding one by brute force means checking all 2^n subsets:

```
# Brute force: check all 2^n subsets (exponential)
subset_sum_brute <- function(x, target) {
  n <- length(x)
  for (i in seq_len(2^n) - 1) {
    bits <- as.logical(intToBits(i)[seq_len(n)])
    if (sum(x[bits]) == target) return(x[bits])
  }
  NULL
}

# A greedy approximation: sort descending, take what fits (polynomial)
subset_sum_greedy <- function(x, target) {
  x <- sort(x, decreasing = TRUE)
  picked <- logical(length(x))
  remaining <- target
  for (i in seq_along(x)) {
    if (x[i] <= remaining) {
      picked[i] <- TRUE
      remaining <- remaining - x[i]
    }
  }
  if (remaining == 0) x[picked] else NULL
```

```
}
```

At n = 20 the brute-force version checks about a million subsets and finishes in under a second. At n = 40 it checks a trillion — you will not wait long enough. The greedy version runs in O(n log n) on either input but is not guaranteed to find a solution that exists. That is the P-vs-NP gap in miniature: polynomial verification, (apparently) exponential search, and a fast heuristic that trades correctness for speed.

In practice, R encounters NP-hard problems in optimization. `optim()` does not guarantee a global optimum for non-convex problems; it uses heuristic methods (Nelder-Mead, simulated annealing) that find good-enough solutions without exhaustive search. This is the pragmatic response to intractability: give up on perfection and settle for "good enough, fast."

The loop from the opening, the one that took five minutes instead of thirty seconds — you now have the vocabulary to diagnose it: somewhere in that loop, an $O(n^2)$ operation was hiding inside what looked like O(n) code, and the 10x increase in data exposed the quadratic scaling that small test sets had concealed. The growth rate was there from the start; faster hardware only postpones when you hit it. But complexity is only half the story — the other half is the data structure you choose to hold your values, and so far the only container you have is the atomic vector, which insists that everything inside it be the same type.

Exercises

1. Write a function that takes another function `f` and an argument `x`, calls `f(x)` with a 2-second time limit, and returns `NA` if the computation does not finish.
2. Can a program determine whether a given R function is pure (no side effects)? Why or why not?
3. Is sorting in P or NP? What about finding the shortest path that visits all cities exactly once (traveling salesman)?

Part II

Part II: Working with data

10 Lists

You have a penguin. It has a species (character), a body mass (double), and a vector of bill and flipper measurements (also doubles). Try cramming all of that into a single atomic vector and R will coerce everything to character, because vectors are strict: one type, no exceptions (Section 4.3). The species survives, the numbers become strings, and the measurements lose their identity as a group. What you need is a container that keeps each piece intact, preserving its type and its shape, without flattening or coercing anything. R calls that container a list.

10.1. Why lists exist

Where a vector is homogeneous, a list is heterogeneous: each element can be a number, a string, a vector of arbitrary length, a function, or even another list, and all of them coexist without interference.

```
penguin <- list(
  species = "Adelie",
  mass = 3750,
  measurements = c(39.1, 18.7, 181)
)
penguin
#> $species
#> [1] "Adelie"
#>
#> $mass
#> [1] 3750
#>
#> $measurements
#> [1]  39.1  18.7 181.0
```

`species` is a character scalar, `mass` is a numeric scalar, `measurements` is a numeric vector of length 3. No coercion, no flattening. A vector could not hold these together without reducing them all to a single type; a list simply keeps each element as it was.

Technically, a list is a *recursive vector*, where "recursive" means its elements can themselves be lists, allowing arbitrary nesting. R's documentation calls lists "generic vectors" to

distinguish them from atomic vectors (Section 4.2), and `typeof()` confirms:

```
typeof(penguin)
#> [1] "list"
```

You already used a list in Section 7.3 when you stored functions as named elements. Lists are the same structure whether they hold numbers, strings, or functions, but accessing the right element requires care.

The lambda calculus already had this: `a. b. f. f a b` is a pair encoded as a closure, with selectors `x. y. x` and `x. y. y` to get each element back. `cons`/`car`/`cdr` in Lisp; `list()`/`[[`/`$` in R.

Exercises

1. Create a list with your name (character), your age (numeric), and your three favourite colours (a character vector). Print it.
2. What does `typeof(list(1, "a", TRUE))` return?
3. What happens if you use `c()` instead of `list()` to combine `1`, `"a"`, and `TRUE`? Why is the result different?

10.2. Creating and accessing lists

10.2.1. Creating lists

`list()` creates a list, and elements can be named or unnamed:

```
named <- list(a = 1, b = "hello", c = TRUE)
unnamed <- list(1, "hello", TRUE)
```

Named elements are easier to work with because you can refer to them by meaning rather than position; unnamed elements force you to remember which slot holds what.

10.2.2. The train analogy

Think of a list as a train where each carriage holds its own cargo, and your choice of accessor determines whether you detach the carriage or reach inside it:

- `x[1]` returns the first carriage still attached to the train. The result is a list of length 1.
- `x[[1]]` opens the first carriage and pulls out what's inside. The result is the element itself.
- `x$name` is shorthand for `x[["name"]]`.

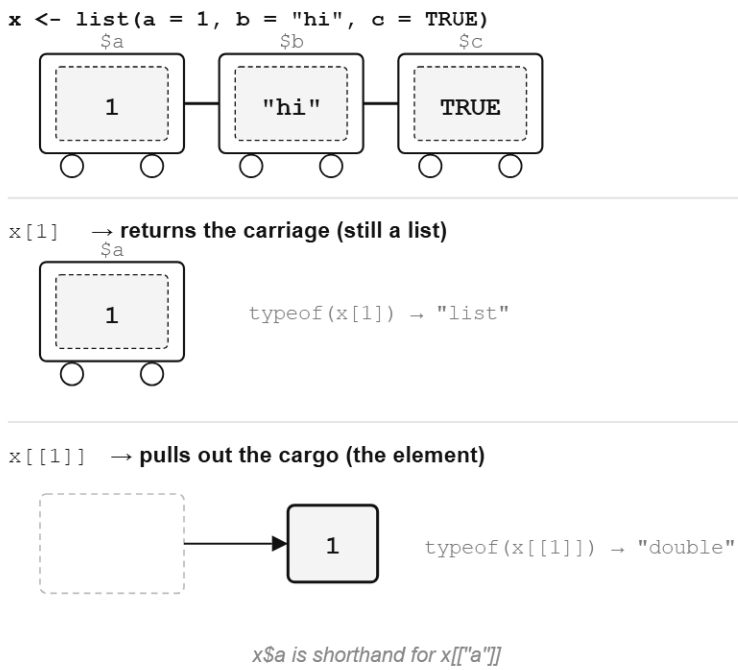


Figure 10.1.: `x[1]` returns the carriage (still a list); `x[[1]]` pulls out the cargo (the element itself).

```
x <- list(a = 10, b = c(1, 2, 3), c = "hello")
```

`[` returns a sub-list:

```
x[1]
#> $a
#> [1] 10
typeof(x[1])
#> [1] "list"
```

Still a list, just a shorter one.

`[[` extracts the element:

```
x[[1]]
#> [1] 10
typeof(x[[1]])
#> [1] "double"
```

Now it's the number 10, not a list containing 10. This distinction is the single most common source of confusion with lists: `[` keeps the container, `[[` removes it.

`$` works with names:

```
x$b
#> [1] 1 2 3
x[["b"]]
#> [1] 1 2 3
```

Both return the same thing. `$` is convenient for interactive use, but `[[` is necessary when the name is stored in a variable:

```
key <- "b"
x[[key]]
#> [1] 1 2 3
```

```
x$key
#> NULL
```

`x$key` looks for a literal element named `"key"`, not for the value stored in the variable `key`. Whenever the name is computed or passed as an argument, you need `[[`.

You can select multiple elements with `[`:

```
x[c(1, 3)]
#> $a
#> [1] 10
#>
#> $c
#> [1] "hello"
x[c("a", "c")]
#> $a
#> [1] 10
#>
#> $c
#> [1] "hello"
```

But `[[` only works with a single index, extracting one element at a time.

> **Opinion**
>
> Default to `[[` and `$` for extracting elements. Use `[` only when you need a sub-list. If you find yourself writing `x[1][[1]]`, you wanted `x[[1]]` all along.

Exercises

1. Given `x <- list(a = 10, b = 20, c = 30)`, predict the output of `x[2]`, `x[[2]]`, and `x$b` before running them.

2. What is `typeof(x[1])` versus `typeof(x[[1]])`? Explain the difference.
3. Create a variable `name <- "c"`. Use it to extract the element `"c"` from the list `x`. Which accessor works?

10.3. Nested lists

A list element can be another list, which means that a single list can grow into a tree of arbitrary depth:

```
study <- list(
  site = "Palmer Station",
  years = 2007:2009,
  species = list(
    list(name = "Adelie", count = 152),
    list(name = "Gentoo", count = 124),
    list(name = "Chinstrap", count = 68)
  )
)
```

`str()` reveals the shape:

```
str(study)
#> List of 3
#>  $ site   : chr "Palmer Station"
#>  $ years  : int [1:3] 2007 2008 2009
#>  $ species:List of 3
#>   ..$ :List of 2
#>   .. ..$ name : chr "Adelie"
#>   .. ..$ count: num 152
#>   ..$ :List of 2
#>   .. ..$ name : chr "Gentoo"
#>   .. ..$ count: num 124
#>   ..$ :List of 2
#>   .. ..$ name : chr "Chinstrap"
#>   .. ..$ count: num 68
```

`str()` is the single most useful function for lists. It works on any R object, but with lists it shows you the nesting, the types, and the lengths in one compact summary. Get in the habit of calling it on anything you did not build yourself.

Every nested list is a tree: sub-lists are branches, atomic values are leaves. File systems, the DOM, and JSON are all trees too, which is why R's lists map onto them so naturally.

Inspecting is only half the problem. How do you reach into the tree and pull out what you need? Chain `[[` or `$`:

```
study$species[[1]]$name
#> [1] "Adelie"
study[["species"]][[2]][["count"]]
#> [1] 124
```

Each `[[` or `$` steps one level deeper: `study$species` is a list of three lists, `study$species[[1]]` is the first of those, and `study$species[[1]]$name` is the string `"Adelie"`. JSON from a web API, the output of `lm()`, and configuration files are all nested lists or objects built on them, and the access pattern is always the same: `$` or `[[` to step down one level, repeated as many times as needed.

Exercises

1. Given the `study` list above, extract the count for Chinstrap penguins.
2. Use `str()` on the result of `lm(mpg ~ wt, data = mtcars)`. How many top-level elements does the model object have?
3. Create a nested list representing a book: title, author, and a list of chapters (each with a number and a title). Extract the title of the second chapter.

10.4. Lists as the backbone of R

You might think of lists as a container you reach for occasionally, but in truth they are the foundation that most of R's familiar structures are built on.

A data frame is a list:

```
df <- data.frame(x = 1:3, y = c("a", "b", "c"))
typeof(df)
#> [1] "list"
is.list(df)
#> [1] TRUE
```

Each column is one element of that list, and the data frame simply adds a constraint: all columns must have the same length. Underneath, `df$x` works exactly like list access because it *is* list access.

A linear model is a list:

```
fit <- lm(mpg ~ wt, data = mtcars)
typeof(fit)
#> [1] "list"
names(fit)
#>  [1] "coefficients"  "residuals"     "effects"       "rank"
#>  [5] "fitted.values" "assign"        "qr"            "df.residual"
#>  [9] "xlevels"       "call"          "terms"         "model"
```

`fit$coefficients`, `fit$residuals`, `fit$fitted.values` — these are ordinary `$` extractions on a list. The `"lm"` class attribute changes how R prints and summarizes the object, not how you get data out of it.

Data frames (Chapter 11), model objects, and environments are all lists (or list-like structures) with different rules about what they can contain. The most interesting rule is also the simplest: what happens when every element is a vector of the same length?

Exercises

1. Run `typeof()` on a data frame you create. Then run `is.list()` on it. What do you conclude?
2. Fit a model with `lm(Sepal.Length ~ Petal.Length, data = iris)`. Use `names()` to see its elements, then extract the R-squared from `summary()` of the fit. (Hint: `str(summary(fit))` will help.)
3. What does `length()` return for a data frame with 5 columns and 100 rows? Why?

10.5. Modifying lists

Adding an element is as simple as assigning to a new name:

```
x <- list(a = 1, b = 2)
x$c <- 3
x[["d"]] <- "new"
str(x)
#> List of 4
#>  $ a: num 1
#>  $ b: num 2
#>  $ c: num 3
#>  $ d: chr "new"
```

Removing an element means setting it to `NULL`:

```
x$b <- NULL
str(x)
#> List of 3
#>  $ a: num 1
#>  $ c: num 3
#>  $ d: chr "new"
```

`b` is gone, not set to `NULL`. This is a common trap: assigning `NULL` to a list element deletes it entirely. If you actually want to store `NULL` as a value, you need `x["e"] <- list(NULL)`:

```
x["e"] <- list(NULL)
str(x)
```

```
#> List of 4
#>  $ a: num 1
#>  $ c: num 3
#>  $ d: chr "new"
#>  $ e: NULL
```

Now `e` exists and its value is `NULL`. The single-bracket assignment with `list(NULL)` is the only way to store an actual `NULL` in a list, an asymmetry worth memorizing because it bites everyone at least once.

Replacing an element is just assignment to an existing name:

```
x$a <- 100
x$a
#> [1] 100
```

R uses copy-on-modify for lists, the same semantics as for vectors. When you modify a list, R copies only the parts that change, not the entire structure; for practical purposes you can treat assignment as modifying in place, since the copying is an implementation detail that rarely affects your code.

Exercises

1. Create a list with elements `x = 1` and `y = 2`. Add an element `z = 3`, then delete `x`. Print the result.
2. What does `length()` return after you delete an element from a list?
3. Try `x$a <- NULL` on a list where `a` exists. Then try `x["a"] <- list(NULL)`. What is the difference?

10.6. Linked lists

R's built-in lists are arrays of pointers, so indexing is O(1) and prepending is O(n) (every pointer in the array gets copied). A linked list is a chain of nodes, each holding a value and a pointer to the next. Prepending is O(1); reaching element k means following k pointers. You build one in R with nested lists:

```
cons <- function(head, tail) list(head = head, tail = tail)
car  <- function(lst) lst$head
cdr  <- function(lst) lst$tail
```

`cons` constructs a node, `car` extracts the first element, and `cdr` extracts the rest. These names come from Lisp (1958), where `car` and `cdr` referred to hardware registers on the IBM 704.

Build a linked list of three elements:

```
ll <- cons(1, cons(2, cons(3, NULL)))
str(ll)
#> List of 2
#>  $ head: num 1
#>  $ tail:List of 2
#>   ..$ head: num 2
#>   ..$ tail:List of 2
#>   .. ..$ head: num 3
#>   .. ..$ tail: NULL
```

To traverse it, recurse until you hit `NULL`:

```
ll_to_vector <- function(lst) {
  if (is.null(lst)) return(c())
  c(car(lst), ll_to_vector(cdr(lst)))
}
ll_to_vector(ll)
#> [1] 1 2 3
```

So why doesn't R use linked lists for everyday work? Because R needs contiguous memory. Functions like `sum()` and the BLAS routines behind matrix algebra expect a single block of doubles that a C function can walk with a pointer, and a linked list scatters its elements across the heap (the region of memory where R allocates objects; Section 29.1 explains the distinction). R's built-in lists (VECSXP) are arrays of pointers: O(1) random access, contiguous pointer storage. Linked lists trade that for O(1) prepend and natural recursive processing — peel off the head, recur on the tail. Lisp was built on that kind of processing; R was built on vectorized computation over contiguous arrays.

From Church encoding to cons cells

The `cons`/`car`/`cdr` structure is a direct translation of Church encoding. In lambda calculus, a pair is `h. t. f. f h t`: a function that captures two values and waits for a selector. `car` passes a selector that returns the first value; `cdr` passes one that returns the second. A linked list is a chain of such pairs, terminated by a special "nil" value. Lisp was built on exactly this encoding, and `cons`/`car`/`cdr` remain the standard names for these operations across functional languages.

Both R and Lisp descend from the same lambda calculus, but the engineering decisions diverged early. Lisp committed to recursive decomposition: peel from the head, recur on the tail, build up an answer one `cons` at a time. R went the other way — contiguous arrays, vectorized operations, the C runtime walking a flat block of memory at speed. That fork propagated through every design decision that followed. R has `sapply` where Lisp has `mapcar`. R's `[` extracts by position; Lisp peels from the front. The entire language favors breadth over depth, and the linked list code above shows what depth-first design looks like from the inside.

Exercises

1. Using `cons`, `car`, and `cdr` defined above, build the linked list `(10, 20, 30)` and extract the second element without converting to a vector.
2. Write a function `ll_length` that counts the number of nodes in a linked list by recursing through `cdr` until `NULL`.
3. Write a function `ll_map` that takes a linked list and a function, and returns a new linked list with the function applied to each element. Test it by doubling every element of `cons(1, cons(2, cons(3, NULL)))`.

10.7. Looking ahead

Lists hold anything, including other lists. Constrain that freedom (require every element to be a vector of the same length) and the general-purpose container becomes a table.

11 Data frames

You have a hundred measurements of penguin body mass, another hundred of bill length, and a character vector of species names. Three vectors, all the same length, all describing the same 100 birds. You could keep them as separate objects, indexing carefully, hoping that nobody reorders one without reordering the others. Or you could store them together in a structure that enforces the constraint for you: every column the same length, every row one observation, accessible with the list operations from Chapter 10.

That structure is the data frame: a list where every element is a vector of the same length. Lists hold anything; a data frame constrains that freedom to equal-length columns, and once you see this, subsetting, column operations, and the entire tidyverse make sense. The idea predates R by decades: Edgar Codd's 1970 paper at IBM defined relations as sets of tuples, each row a typed record, and R's data frames, SQL tables, pandas DataFrames, and Spark datasets all descend from that same relational algebra.

So what does this constraint actually buy you?

11.1. A list of equal-length vectors

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
```

Ask R what a data frame actually is:

```
typeof(penguins)
#> [1] "list"
is.list(penguins)
#> [1] TRUE
```

Type `"list"`, because it *is* a list. The difference from an ordinary list is that every element (every column) must be a vector of the same length, so each row lines up across all columns; one observation, one position in every vector. Build one yourself:

```
df <- data.frame(
  name = c("Adelie", "Gentoo"),
  mass = c(3750, 5200)
)
df
#>     name mass
#> 1 Adelie 3750
#> 2 Gentoo 5200
```

You are building a list with two elements (`name` and `mass`), both of length 2. R adds a `class` attribute (`"data.frame"`) and row numbers, but the underlying structure is just a named list with bookkeeping on top. You can verify this with `str()`:

```
str(df)
#> 'data.frame':    2 obs. of  2 variables:
#>  $ name: chr  "Adelie" "Gentoo"
#>  $ mass: num  3750 5200
```

Two columns, two rows, and nothing more than a list wearing a label.

Type theory calls this a *product type*. A data frame with columns `name` (character) and `mass` (double) has type `character × double`: each row pairs one character value with one double value. Add a third column and the type grows: `character × double × integer`.

The equal-length constraint is doing more work than it appears. A plain list lets you pair a five-element vector with a three-element vector and says nothing about the mismatch. A data frame refuses. Why? Because the constraint *is* the guarantee: row i across all columns describes the same observation. Break the rectangle — one column longer than another — and that guarantee disappears, and every function that touches the data must re-verify alignment on its own. Codd's 1970 relational model imposed the same rule: every tuple in a relation has the same attributes. SQL inherited it, pandas and Arrow enforce it today, and R's data frame sits in that same lineage. The trade is flexibility for a promise: if the data is rectangular, row alignment is automatic and every subsetting operation preserves it. Logical indexing, `merge()`, and `rbind()` all depend on that promise holding.

So what goes into those columns when you build one from scratch?

Exercises

1. Create a data frame with three columns: `species` (character), `island` (character), and `bill_length` (numeric), with at least three rows. Check its `typeof()`.
2. What does `length(penguins)` return? Why? (Hint: a data frame is a list.)
3. What does `names(penguins)` return?

11.2. Creating data frames

`data.frame()` is the base R workhorse. Column names come from the argument names:

```
measurements <- data.frame(
  species = c("Adelie", "Chinstrap", "Gentoo"),
  mass_g  = c(3750, 3800, 5200),
  island  = c("Torgersen", "Dream", "Biscoe")
)
measurements
#>     species mass_g    island
#> 1    Adelie   3750 Torgersen
#> 2 Chinstrap   3800     Dream
#> 3    Gentoo   5200    Biscoe
```

Before R 4.0, `data.frame()` converted character vectors to factors by default (`stringsAsFactors = TRUE`), which meant that strings that looked like text silently became categorical variables with integer codes underneath. This caused confusion for over two decades.

> **Opinion**
>
> The `stringsAsFactors` default was wrong for 23 years and finally fixed in R 4.0 (2020). If someone's tutorial still warns about it, the tutorial is outdated.

11.2.1. Row names

Some built-in datasets carry a feature you might not expect:

```
head(mtcars, 3)
#>                mpg cyl disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4     21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710    22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

The car names ("Mazda RX4", etc.) are row names, not a column. They look like data, they smell like data, but they live in a separate metadata slot that can't contain duplicates, gets silently dropped by many functions, and behaves nothing like a real column. You can't filter by row name without extra syntax, and you can't have two rows with the same label.

> **Opinion**
>
> Pretend row names don't exist. If something is data, put it in a column. `tibble::rownames_to_column()` converts row names to a proper column when you inherit a dataset that uses them.

Exercises

1. Create a data frame where one column is logical (e.g., `passed = c(TRUE, FALSE, TRUE)`). What does `str()` show?
2. Run `row.names(mtcars)` and `row.names(penguins)`. What's the difference?

11.3. Subsetting data frames

Because a data frame is a list, the same accessors work:

```
penguins$species[1:3]
#> [1] Adelie Adelie Adelie
#> Levels: Adelie Chinstrap Gentoo
```

`$` extracts a column and returns a vector, the same way `[[` extracts an element from a list:

```
penguins[["species"]][1:3]
#> [1] Adelie Adelie Adelie
#> Levels: Adelie Chinstrap Gentoo
```

`[[` does the same thing but takes a string, which matters when the column name lives in a variable rather than in your source code:

```
col <- "species"
penguins[[col]][1:3]
#> [1] Adelie Adelie Adelie
#> Levels: Adelie Chinstrap Gentoo
```

Single brackets, by contrast, return a one-column data frame (a sub-list), not a vector:

```
penguins["species"][1:3, ]
#> # A tibble: 3 x 1
#>   species
#>   <fct>
#> 1 Adelie
#> 2 Adelie
#> 3 Adelie
```

Same rule as lists: `[` keeps the container, `[[` extracts the contents. But data frames are rectangular, which opens a second dimension.

11.3.1. Two-dimensional indexing

Because a data frame has rows and columns, it also accepts two indices: `[row, column]`.

```
penguins[1, ]
#> # A tibble: 1 x 8
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>             <dbl>         <dbl>             <int>       <int>
#> 1 Adelie  Torgers~           39.1          18.7               181        3750
#> # i 2 more variables: sex <fct>, year <int>
```

First row, all columns, returned as a data frame. Now try the reverse:

```
penguins[, 1]
#> # A tibble: 344 x 1
#>    species
#>    <fct>
#>  1 Adelie
#>  2 Adelie
#>  3 Adelie
#>  4 Adelie
#>  5 Adelie
#>  6 Adelie
#>  7 Adelie
#>  8 Adelie
#>  9 Adelie
#> 10 Adelie
#> # i 334 more rows
```

All rows, first column. Here is the gotcha: by default, this returns a *vector*, not a data frame. R "drops" the data frame structure when the result has a single column.

```
class(penguins[, 1])
#> [1] "tbl_df"     "tbl"        "data.frame"
```

To keep the data frame structure, use `drop = FALSE`:

```
class(penguins[, 1, drop = FALSE])
#> [1] "tbl_df"     "tbl"        "data.frame"
```

This `drop` behavior is a common source of bugs. You write a function that subsets a data frame, test it with two columns, and it works; someone passes one column, and your function breaks because it suddenly receives a vector instead of a data frame.

```
penguins[1, 2]
#> # A tibble: 1 x 1
#>   island
#>   <fct>
#> 1 Torgersen
```

Row 1, column 2: a single value (a vector of length 1). `[` with two indices reaches any cell, row, or column, but the `drop` gotcha still applies whenever your selection narrows to one column.

11.3.2. Logical subsetting

Logical indexing from Section 4.5 works on data frames too. Put the logical vector in the row position:

```
heavy <- penguins[penguins$body_mass_g > 5000 & !is.na(penguins$body_mass_g), ]
nrow(heavy)
#> [1] 61
head(heavy, 3)
#> # A tibble: 3 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>           <dbl>         <dbl>             <int>       <int>
#> 1 Gentoo  Biscoe           50            16.3               230        5700
#> 2 Gentoo  Biscoe           50            15.2               218        5700
#> 3 Gentoo  Biscoe           47.6          14.5               215        5400
#> # i 2 more variables: sex <fct>, year <int>
```

`penguins$body_mass_g > 5000` produces a logical vector, `TRUE` for heavy penguins and `FALSE` (or `NA`) for the rest, and placing it before the comma selects the matching rows. `dplyr::filter()` (Chapter 14) does the same filtering without repeating the data frame name; base R makes you write it twice, once for the condition and once for the subset.

Exercises

1. Predict the class of each result before running:

```
penguins$bill_length_mm
penguins["bill_length_mm"]
penguins[["bill_length_mm"]]
penguins[, "bill_length_mm"]
penguins[, "bill_length_mm", drop = FALSE]
```

2. Extract all penguins from Biscoe island using logical subsetting. How many rows?
3. What does `penguins[1:5, c("species", "island")]` return?

11.4. Adding and modifying columns

Assign to a new name with `$`:

```
penguins$mass_kg <- penguins$body_mass_g / 1000
head(penguins$mass_kg)
#> [1] 3.75 3.80 3.25   NA 3.45 3.65
```

This adds (or overwrites) a column. The right-hand side must be a vector with the same number of rows, or a single value that gets recycled (Section 4.4.1).

For modifying multiple columns at once, base R provides `within()`:

```
df2 <- within(penguins, {
  mass_kg <- body_mass_g / 1000
  bill_ratio <- bill_length_mm / bill_depth_mm
})
head(df2[, c("mass_kg", "bill_ratio")])
#> # A tibble: 6 x 2
#>   mass_kg bill_ratio
#>     <dbl>      <dbl>
#> 1    3.75       2.09
#> 2    3.8        2.27
#> 3    3.25       2.24
#> 4   NA         NA
#> 5    3.45       1.90
#> 6    3.65       1.91
```

`within()` evaluates the expressions inside the data frame's environment, so you can refer to columns by name without the `df$` prefix, and the result is a new data frame. `dplyr::mutate()` (Chapter 14) has almost entirely replaced `within()` in modern R. You will still see it in older scripts.

Exercises

1. Add a column `bill_area` to `penguins` that multiplies `bill_length_mm` by `bill_depth_mm`. What is `mean(penguins$bill_area, na.rm = TRUE)`?
2. What happens if you assign a vector of the wrong length to a new column? Try `penguins$test <- 1:10`.

11.5. Combining data frames

`rbind()` stacks rows; the column names and types must match:

```
batch1 <- data.frame(species = "Adelie", mass = 3750)
batch2 <- data.frame(species = "Gentoo", mass = 5200)
rbind(batch1, batch2)
#>   species mass
#> 1  Adelie 3750
```

```
#> 2  Gentoo 5200
```

`cbind()` glues columns side by side; the row counts must match:

```
ids <- data.frame(id = c("A1", "G1"))
masses <- data.frame(mass = c(3750, 5200))
cbind(ids, masses)
#>   id mass
#> 1 A1 3750
#> 2 G1 5200
```

When they fail, the error messages mention dimensions rather than the real problem, which is almost always mismatched column names in `rbind()` or mismatched row counts in `cbind()`. Check `names()` and `nrow()` before combining, and you will save yourself a frustrating debugging session.

Exercises

1. Create two data frames with the same columns but different data. Stack them with `rbind()`.
2. What happens if the column names don't match? Try it.
3. Use `cbind()` to attach a new `id` column to `penguins`. What class is the result?

11.6. Tibbles

Print a large data frame in R and thousands of rows flood the console. Subset a single column: you silently get a vector instead of a data frame. Type a column name wrong and partial matching returns the wrong column.

A tibble is the tidyverse's answer to all three problems:

```
library(tibble)
tbl <- tibble(
  species = c("Adelie", "Gentoo", "Chinstrap"),
  mass = c(3750, 5200, 3800)
)
tbl
#> # A tibble: 3 x 2
#>   species    mass
#>   <chr>     <dbl>
#> 1 Adelie     3750
#> 2 Gentoo     5200
#> 3 Chinstrap  3800
```

Three differences matter.

Printing. A tibble shows the first 10 rows, column types, and fits the screen width. No more flooding the console because you forgot `head()`.

Subsetting. `tbl[, 1]` always returns a tibble, never a vector, so the `drop` gotcha is gone:

```
class(tbl[, 1])
#> [1] "tbl_df"     "tbl"        "data.frame"
```

No partial matching. With a data frame, `df$sp` might match `df$species`. A tibble refuses:

```
tbl$sp
#> Warning: Unknown or uninitialised column: `sp`.
#> NULL
```

> **Opinion**
>
> Partial matching is autocomplete for bugs. You type `df$sp`, R guesses you meant `df$species`, and your code works until someone adds a column called `spine_count`. Then `df$sp` becomes ambiguous and silently returns `NULL`. A tibble catches this immediately.

A tibble is still a data frame, which means every function that accepts a data frame accepts a tibble:

```
is.data.frame(tbl)
#> [1] TRUE
```

Convert between them with `as_tibble()` and `as.data.frame()`:

```
as_tibble(mtcars)
#> # A tibble: 32 x 11
#>      mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
#>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#>  1  21       6  160    110  3.9   2.62  16.5     0     1     4     4
#>  2  21       6  160    110  3.9   2.88  17.0     0     1     4     4
#>  3  22.8     4  108     93  3.85  2.32  18.6     1     1     4     1
#>  4  21.4     6  258    110  3.08  3.22  19.4     1     0     3     1
#>  5  18.7     8  360    175  3.15  3.44  17.0     0     0     3     2
#>  6  18.1     6  225    105  2.76  3.46  20.2     1     0     3     1
#>  7  14.3     8  360    245  3.21  3.57  15.8     0     0     3     4
#>  8  24.4     4  147.    62  3.69  3.19  20       1     0     4     2
#>  9  22.8     4  141.    95  3.92  3.15  22.9     1     0     4     2
#> 10  19.2     6  168.   123  3.92  3.44  18.3     1     0     4     4
#> # i 22 more rows
```

Opinion

Use tibbles by default. Use plain data frames when a package demands one (rare) or when you want zero dependencies.

Exercises

1. Create a tibble and a data frame with the same data. Compare what `[, 1]` returns for each.
2. Create a data frame with a column called `temperature`. Access it with `df$temp`. Then try the same with a tibble.
3. What does `is.list()` return for a tibble?

11.7. Exploring data frames

`str()` is the first thing to run on a new dataset:

```
str(penguins)
#> tibble [344 x 8] (S3: tbl_df/tbl/data.frame)
#>  $ species          : Factor w/ 3 levels "Adelie","Chinstrap",..: 1 1 1 1 1 1 1
1 1 1 ...
#>  $ island           : Factor w/ 3 levels "Biscoe","Dream",..: 3 3 3 3 3 3 3 3 3
3 ...
#>  $ bill_length_mm   : num [1:344] 39.1 39.5 40.3 NA 36.7 39.3 38.9 39.2 34.1 42
...
#>  $ bill_depth_mm    : num [1:344] 18.7 17.4 18 NA 19.3 20.6 17.8 19.6 18.1 20.2
...
#>  $ flipper_length_mm: int [1:344] 181 186 195 NA 193 190 181 195 193 190 ...
#>  $ body_mass_g      : int [1:344] 3750 3800 3250 NA 3450 3650 3625 4675 3475
4250 ...
#>  $ sex              : Factor w/ 2 levels "female","male": 2 1 1 NA 1 2 1 2 NA
NA ...
#>  $ year             : int [1:344] 2007 2007 2007 2007 2007 2007 2007 2007 2007
2007 ...
```

344 rows, 8 columns, the type of each column, and a preview of the values. This is the first thing to run when you load a new dataset.

Other useful functions:

```
nrow(penguins)
#> [1] 344
ncol(penguins)
#> [1] 8
```

```
dim(penguins)
#> [1] 344   8
```

```
names(penguins)
#> [1] "species"           "island"            "bill_length_mm"
#> [4] "bill_depth_mm"     "flipper_length_mm" "body_mass_g"
#> [7] "sex"               "year"
```

```
head(penguins, 3)
#> # A tibble: 3 x 8
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>             <dbl>         <dbl>             <int>       <int>
#> 1 Adelie  Torgers~           39.1          18.7               181        3750
#> 2 Adelie  Torgers~           39.5          17.4               186        3800
#> 3 Adelie  Torgers~           40.3          18                 195        3250
#> # i 2 more variables: sex <fct>, year <int>
```

```
summary(penguins)
#>       species          island    bill_length_mm  bill_depth_mm
#>  Adelie   :152   Biscoe   :168   Min.   :32.10   Min.   :13.10
#>  Chinstrap: 68   Dream    :124   1st Qu.:39.23   1st Qu.:15.60
#>  Gentoo   :124   Torgersen: 52   Median :44.45   Median :17.30
#>                                  Mean   :43.92   Mean   :17.15
#>                                  3rd Qu.:48.50   3rd Qu.:18.70
#>                                  Max.   :59.60   Max.   :21.50
#>                                  NA's   :2       NA's   :2
#>  flipper_length_mm  body_mass_g       sex           year
#>  Min.   :172.0     Min.   :2700   female:165   Min.   :2007
#>  1st Qu.:190.0     1st Qu.:3550   male  :168   1st Qu.:2007
#>  Median :197.0     Median :4050   NA's  : 11   Median :2008
#>  Mean   :200.9     Mean   :4202                Mean   :2008
#>  3rd Qu.:213.0     3rd Qu.:4750                3rd Qu.:2009
#>  Max.   :231.0     Max.   :6300                Max.   :2009
#>  NA's   :2         NA's   :2
```

`summary()` shows min, max, quartiles, and mean for numeric columns, counts for factors, and NA counts at the bottom of each column. RStudio's `View()` gives you a spreadsheet instead, useful for browsing but it gets sluggish past a few thousand rows and does nothing outside interactive sessions.

Exercises

1. How many `NA` values are in the `sex` column of `penguins`? Use `sum()` and `is.na()`.
2. What is the mean body mass of penguins? Use `mean()` with `na.rm = TRUE`.
3. Run `str()` on the `mtcars` dataset. How many columns are numeric?

11.8. Putting it together

A data frame is a list (Section 11.1), each column is a vector (Section 4.2), and subsetting uses the same `[`, `[[`, and `$` operators you learned for lists (Chapter 8) and vectors (Section 4.5). Logical indexing filters rows (Section 11.3.2) with the same mechanism from Section 4.4, just applied to a two-dimensional structure. Everything connects.

One consequence of this design matters more than any syntax detail: R stores data frames column by column, each column a single contiguous vector in memory. Column operations like `mean(df$x)` walk sequential memory and run fast, while row operations like `apply(df, 1, sum)` jump between columns on every row and run slow. This is the same trade-off databases make: columnar stores (DuckDB, Parquet) optimize for analytical aggregation; row stores (PostgreSQL, MySQL) optimize for single-record transactions.

Chapter 12 covers the column types you will encounter most often: strings, factors, and dates. Chapter 14 introduces `dplyr`, which replaces the `df[df$x > 5, ]` syntax with readable verbs like `filter()`, `mutate()`, and `summarize()`. Those verbs work because data frames are lists of vectors. You started this chapter with three loose vectors and a prayer that they stayed aligned; you end it knowing why R binds them together and what that binding makes possible.

12 Strings, factors, and dates

You have a column of dates in three different formats, a species column that plots in the wrong order, and a free-text field full of trailing whitespace and inconsistent capitalization. Numbers behave; text, categories, and time do not. This chapter covers the three types that derail an afternoon: strings, factors, and dates. Each has a dedicated tidyverse package (`stringr`, `forcats`, `lubridate`), each comes with traps that catch beginners, and none of them will feel fully tamed by the end of a single chapter. The three topics are independent; skip to whichever one is causing you trouble right now.

We will use the `palmerpenguins` dataset throughout. Load it now:

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
library(stringr)
library(forcats)
library(lubridate)
#>
#> Attaching package: 'lubridate'
#> The following objects are masked from 'package:base':
#>
#>     date, intersect, setdiff, union
```

12.1. Character vectors

A string in R is a character vector of length 1. There is no special string type: `"hello"` is `character(1)`, the same kind of atomic vector you met in Section 4.2.

```
x <- "hello"
typeof(x)
#> [1] "character"
length(x)
#> [1] 1
```

Double quotes and single quotes both work. Pick one and stick with it.

> **Opinion**
>
> Use double quotes. R's own style guide does, and so does most code you will encounter in the wild. Single quotes earn their keep inside double-quoted strings: `"it's easy"`.

R uses UTF-8 as its modern string encoding, a scheme that Ken Thompson and Rob Pike designed on a placemat in a New Jersey diner in September 1992, one that is backward-compatible with ASCII, self-synchronizing, and variable-width (one to four bytes per character). If accented characters come out garbled, the file is probably Latin-1; `stringr::str_conv()` or `readr::locale(encoding = "latin1")` handles that.

Special characters use backslash escapes: `\n` (newline), `\t` (tab), `\\` (literal backslash). Compare `print()` and `cat()`:

```
print("line one\nline two")
#> [1] "line one\nline two"
cat("line one\nline two")
#> line one
#> line two
```

`print()` shows the escape sequence as literal text; `cat()` renders it. A small difference, until you spend ten minutes wondering why your output has `\n` in it instead of a line break.

Base R provides a handful of string tools:

```
nchar("penguin")
#> [1] 7
paste("Gentoo", "penguin")
#> [1] "Gentoo penguin"
paste0("Gentoo", "penguin")
#> [1] "Gentoopenguin"
sprintf("The %s weighs %d grams", "Gentoo", 5200)
#> [1] "The Gentoo weighs 5200 grams"
```

`nchar()` counts characters, `paste()` joins with a space, `paste0()` joins without, and `sprintf()` does formatted substitution borrowing syntax from C. These work, but they are inconsistent in argument order and `NA` handling, and inconsistency compounds fast once you start chaining operations. What would a consistent interface look like?

Exercises

1. What does `nchar(NA)` return? What about `nchar("")`?
2. Use `paste()` to combine `"Species"`, `":"`, and `"Adelie"` into a single string. Then do the same with `paste0()`. What is different?
3. Use `sprintf()` to produce the string `"Island: Biscoe, n = 168"`.

12.2. stringr: consistent string operations

Base R has `grep()`, `grepl()`, `sub()`, `gsub()` — they work, but the argument orders are inconsistent, the return types vary, and good luck guessing what `gsub` stands for. No convention ties them together, so you memorize each one separately.

stringr fixes this with a single rule: every function starts with `str_`, takes the string as the first argument, and takes the pattern second. Once you know the convention, you can guess function names before you look them up.

The essentials:

```
str_length("penguin")
#> [1] 7
str_sub("penguin", 1, 4)
#> [1] "peng"
str_c("Gentoo", "penguin", sep = " ")
#> [1] "Gentoo penguin"
```

`str_length()` counts characters (like `nchar()`), `str_sub()` extracts by position, and `str_c()` combines strings. But unlike `paste()`, `str_c()` propagates `NA`, which means a missing value stays missing instead of silently becoming the literal text `"NA"`:

```
paste("hello", NA)
#> [1] "hello NA"
str_c("hello", NA)
#> [1] NA
```

Case conversion and whitespace cleaning:

```
str_to_upper("gentoo")
#> [1] "GENTOO"
str_to_title("gentoo penguin")
#> [1] "Gentoo Penguin"
str_trim("  messy data  ")
#> [1] "messy data"
str_squish("  too   many   spaces  ")
#> [1] "too many spaces"
```

Pattern matching is where stringr earns its place. Where base R gives you `grepl()`, stringr gives you `str_detect()`, a name that actually describes what it does:

```
species <- penguins$species
str_detect(species, "Gentoo")[1:10]
#>  [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Extraction and replacement:

```
islands <- c("Biscoe Island", "Dream Island", "Torgersen Island")
str_extract(islands, "^[A-Za-z]+")
#> [1] "Biscoe"    "Dream"     "Torgersen"
str_replace(islands, "Island", "Isl.")
#> [1] "Biscoe Isl."    "Dream Isl."     "Torgersen Isl."
```

Splitting:

```
str_split("one-two-three", "-")
#> [[1]]
#> [1] "one"   "two"   "three"
```

`str_split()` returns a list because each input string could split into a different number of pieces, the same reason `strsplit()` does in base R, and a preview of why lists matter for irregular output.

On the penguins data:

```
str_to_upper(levels(penguins$island))
#> [1] "BISCOE"    "DREAM"     "TORGERSEN"
sum(str_detect(penguins$species, "Adelie"))
#> [1] 152
```

Exercises

1. How many penguins have species names containing the letter "e"? Use `str_detect()`.
2. Use `str_sub()` to extract the first three letters of each island name in `penguins$island`.
3. Use `str_replace_all()` to replace all spaces in `"Gentoo penguin species"` with underscores.

12.3. Regex essentials

You have seen `str_detect()` matching literal text. But what if you need to match "any string that starts with a capital letter and ends with a digit"? Literal matching cannot express that. Regular expressions can.

Regular expressions are a mini-language for describing patterns in text. The syntax is not R-specific — the same patterns work in Python, JavaScript, and `grep` on the command line. What follows covers enough to read and write simple patterns; `?regex` has the full reference.

The core building blocks:

Pattern	Matches
.	Any single character

Pattern	Matches
`^`	Start of string
`$`	End of string
`[abc]`	Any of a, b, or c
`[0-9]`	Any digit
`+`	One or more of the preceding
`*`	Zero or more of the preceding
`?`	Zero or one of the preceding
`(A|B)`	A or B (alternation)
`\\d`	Digit (same as `[0-9]`)
`\\s`	Whitespace
`\\w`	Word character (letter, digit, underscore)

The doubled backslashes (`\\d` instead of `\d`) exist because R strings process backslashes first: to get a literal `\d` to the regex engine, you write `"\\d"` in R. Regular expressions themselves come from formal language theory, where Stephen Kleene defined regular languages in 1956 using concatenation, alternation, and closure (repetition). Ken Thompson implemented them in the QED editor in 1968 and `grep` in 1973; the `*` quantifier is still called the *Kleene star* after him.

`str_view()` lets you see what a pattern matches, making it the best tool for building and debugging:

```
fruits <- c("apple", "banana", "cherry", "date", "elderberry")
str_view(fruits, "[aeiou]")
#> [1] | <a>ppl<e>
#> [2] | b<a>n<a>n<a>
#> [3] | ch<e>rry
#> [4] | d<a>t<e>
#> [5] | <e>ld<e>rb<e>rry
```

Some practical patterns:

```
# Strings that start with a capital letter
str_detect(c("Hello", "world", "R"), "^[A-Z]")
#> [1]  TRUE FALSE  TRUE

# Strings that end in a digit
str_detect(c("room101", "lobby", "floor3"), "\\d$")
#> [1]  TRUE FALSE  TRUE

# Extract numbers from text
str_extract("penguin weighs 5200 grams", "\\d+")
#> [1] "5200"
```

> **Opinion**
>
> You do not need to memorize regex. Know that it exists, know the basics from the table above, and keep the stringr cheatsheet within arm's reach. That is enough.

Exercises

1. Write a regex that matches strings starting with "G" and ending with "o". Test it on `c("Gentoo", "Galileo", "Go", "Gusto", "Goo")`.
2. Use `str_extract_all()` to pull all words (sequences of `\\w+`) from `"The quick brown fox"`.
3. Use `str_detect()` and a regex to find which island names in `penguins$island` contain two consecutive vowels.

12.4. Why factors exist

Strings handle free text. But many columns are not free text: species, island, treatment group. These columns have a fixed set of values, and the order of those values matters for plots and models. That is where factors come in.

Try plotting penguin species as a bar chart. The bars come out in alphabetical order: Adelie, Chinstrap, Gentoo. You want them ordered by count, or by body mass, or by some logic that makes sense for your story. Character vectors do not carry ordering information, so you cannot tell the plot "put Gentoo first" without something more.

A factor is that something more: a vector of integers with labels. When R stores `c("male", "female", "female")` as a factor, it is really storing `c(1, 2, 2)` with a mapping (1 = "female", 2 = "male", alphabetical by default).

```
x <- factor(c("male", "female", "female"))
x
#> [1] male   female female
#> Levels: female male
typeof(x)
#> [1] "integer"
unclass(x)
#> [1] 2 1 1
#> attr(,"levels")
#> [1] "female" "male"
```

`typeof()` returns `"integer"`. `unclass()` strips the factor shell and shows the integers underneath.

In type theory, a factor is a *sum type*: `factor(c("male", "female"))` defines a type with exactly two variants, `male | female`, and a value must be one of them. Logical vectors are the simplest case: `Bool = TRUE | FALSE`. Factors generalize it: `Species = Adelie | Chinstrap | Gentoo` is a sum type with three variants. Where a data frame is a product type (combine

fields with AND, Chapter 11), a factor is a sum type (choose one variant with OR). The distinction matters because product types grow by adding fields, while sum types grow by adding variants, and the two compose differently. Rust's `enum`, Haskell's algebraic data types, and TypeScript's union types all make this distinction explicit; R keeps it implicit in the factor machinery, but the structure is the same.

That implicit structure does real work. When you store 10,000 penguin observations with three species, a character vector holds the full string `"Adelie"` repeated thousands of times. A factor holds the integer `1` and a three-entry lookup table. The compression matters for memory, but it matters more for reasoning: the moment you write `factor(species, levels = c("Adelie", "Chinstrap", "Gentoo"))`, you are declaring a contract. These three values can appear in this column, and nothing else. That contract is what lets `lm()` generate dummy variables automatically — the full set of categories is declared up front, not discovered by scanning the data. It also means `fct_reorder()` can reorder levels without inventing ones that do not exist, and data-entry errors (`"Adelie"` vs `"adelie"`) surface at the moment of assignment rather than downstream in a model that silently treats them as two distinct categories. A character vector is an open set: anything goes. A factor is a closed set, and closed sets are easier to reason about, for you and for every function that receives them.

So why does R have this machinery at all? Statistical models need to encode categorical variables, and factors tell R "these are categories, not arbitrary text." Without factors, R would not know that `"male"` and `"female"` represent a finite set of categories rather than two arbitrary strings.

Historical baggage worth knowing: `data.frame()` used to convert all strings to factors by default, which caused years of confusion and the ubiquitous `stringsAsFactors = FALSE` incantation. R 4.0 (released 2020) changed the default to `FALSE`, ending decades of pain. If you see old code with `stringsAsFactors`, now you know why it is there.

When do factors actually help?

- Controlling the order of levels in plots (bars, legends, facets)
- Statistical modeling (`lm()`, `glm()`, and friends)
- Any time the set of possible values matters: months, Likert scales, treatment groups

When they do not: most data wrangling. If you are filtering and counting, character vectors are fine. But the moment you need a plot where the ordering tells a story, you will reach for factors, and at that point the question becomes how to wrangle them without pain.

Exercises

1. Create a factor from `c("low", "medium", "high", "low", "high")`. What are the levels? In what order?
2. Use `unclass()` to see the integer codes. Which integer corresponds to "low"?
3. What happens if you try to assign a value that is not in the levels? Try `x[1] <- "extreme"` on your factor.

12.5. forcats: taming factors

Base R's `factor()` lets you set levels manually:

```
sizes <- factor(c("small", "medium", "large"), levels = c("small", "medium",
"large"))
sizes
#> [1] small  medium large
#> Levels: small medium large
```

The `levels` argument controls the allowed values and their order. Without it, R defaults to alphabetical, which is why "high" comes before "low" and plots look wrong. But manually specifying levels for every factor in every analysis gets tedious fast, and tedium breeds errors.

forcats provides cleaner tools, all starting with `fct_`:

```
# Reorder levels manually
fct_relevel(sizes, "large", "medium", "small")
#> [1] small  medium large
#> Levels: large medium small
```

```
# Order levels by frequency in the data
fct_infreq(penguins$species) |> table()
#>
#>    Adelie    Gentoo Chinstrap
#>       152       124        68
```

```
# Collapse rare levels into "Other"
fct_lump_n(penguins$species, n = 2) |> table()
#>
#> Adelie Gentoo  Other
#>    152    124     68
```

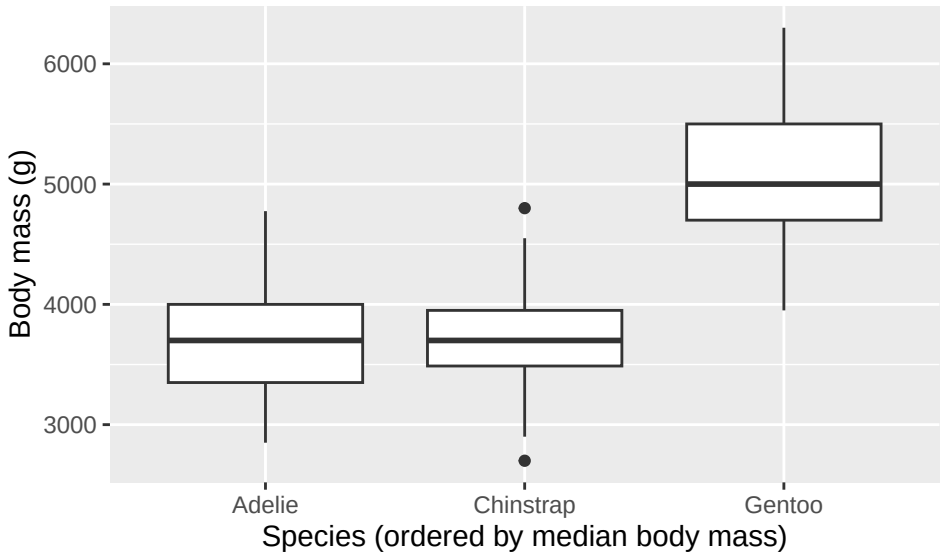
```
# Rename levels
fct_recode(penguins$species, AP = "Adelie", GP = "Gentoo", CP = "Chinstrap") |>
head()
#> [1] AP AP AP AP AP AP
#> Levels: AP CP GP
```

The most useful function is `fct_reorder()`, which reorders levels by a summary of another variable, and this is where factors stop being an annoyance and start being a plotting superpower:

```
library(ggplot2)

penguins_clean <- penguins[!is.na(penguins$body_mass_g), ]

ggplot(penguins_clean, aes(x = fct_reorder(species, body_mass_g, median), y =
body_mass_g)) +
  geom_boxplot() +
  labs(x = "Species (ordered by median body mass)", y = "Body mass (g)")
```



Without `fct_reorder()`, species appear alphabetically: Adelie, Chinstrap, Gentoo. With it, they appear in order of median body mass, and suddenly the plot tells a story instead of reciting the alphabet. This is the moment factors click: the labels reflect the data, not a sorting convention that has nothing to do with penguins.

Exercises

1. Use `fct_infreq()` on `penguins$island` to see which island has the most observations.
2. Use `fct_lump_n()` with `n = 1` on `penguins$species`. What happens?
3. Create a bar chart of `penguins$species` with bars ordered by frequency (hint: `fct_infreq()` inside `aes()`).

12.6. Dates and times

Strings are irregular text; factors are constrained categories. Dates look like strings but behave like numbers, and that dual nature is the source of most of the trouble.

Add one month to January 31. What do you get? February 28? February 31 (which does not exist)? An error? The answer depends on your tool, and the fact that reasonable tools disagree tells you something about why date arithmetic is harder than it looks. Time zones,

leap years, daylight saving transitions, and varying month lengths conspire to make even simple calculations surprisingly tricky.

R has three date-time classes:

- `Date`: date only, stored as days since 1970-01-01
- `POSIXct`: date + time, stored as seconds since 1970-01-01 (compact, use in data frames)
- `POSIXlt`: date + time as a named list of components (rarely needed)

```
today <- Sys.Date()
today
#> [1] "2026-03-23"
typeof(today)
#> [1] "double"
unclass(today)
#> [1] 20535
```

The number from `unclass()` is days since the Unix epoch. Dates in R count from 1970-01-01 because Unix measured time as seconds from that date; 32-bit signed integers can count seconds from 1970 until January 19, 2038 (the "Year 2038 problem"), and R, Python, JavaScript, and most databases all inherit this same epoch. Date arithmetic works as you would expect:

```
as.Date("2026-03-07") - as.Date("2026-01-01")
#> Time difference of 65 days
```

R returns a `difftime` object. Subtraction works, and so does addition with scalars:

```
as.Date("2026-01-01") + 30
#> [1] "2026-01-31"
```

The base R parsing function is `as.Date()`, which expects ISO 8601 format (`"YYYY-MM-DD"`) by default:

```
as.Date("2026-03-07")
#> [1] "2026-03-07"
as.Date("07/03/2026", format = "%d/%m/%Y")
#> [1] "2026-03-07"
```

The `format` argument uses `%Y` (4-digit year), `%m` (month), `%d` (day), and similar codes. These are hard to remember and easy to get wrong, especially when your data mixes formats. Is `"03/07/2026"` March 7th or July 3rd? The format string decides, and if you pick wrong, R will not complain.

Exercises

1. What day number (since 1970-01-01) is today? Use `unclass(Sys.Date())`.

2. What date is 1000 days from today? Use `Sys.Date() + 1000`.
3. How many days are between `"2024-02-28"` and `"2024-03-01"`? (2024 is a leap year.)

12.7. lubridate: dates for humans

Those `%Y/%m/%d` format codes? lubridate lets you forget them. Its parsing functions are named after the order of components, so the function name *is* the format:

```
ymd("2026-03-07")
#> [1] "2026-03-07"
dmy("07/03/2026")
#> [1] "2026-03-07"
mdy("03-07-2026")
#> [1] "2026-03-07"
```

All three produce the same date. No format strings to memorize, no ambiguity about which code means what.

For date-times:

```
ymd_hms("2026-03-07 14:30:00")
#> [1] "2026-03-07 14:30:00 UTC"
```

Extracting components:

```
d <- ymd("2026-03-07")
year(d)
#> [1] 2026
month(d)
#> [1] 3
day(d)
#> [1] 7
wday(d, label = TRUE)
#> [1] Sat
#> Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

Date arithmetic with human-readable units:

```
d + days(30)
#> [1] "2026-04-06"
d + months(1)
#> [1] "2026-04-07"
d + years(1)
#> [1] "2027-03-07"
```

lubridate handles month-length differences correctly: adding one month to January 31 gives February 28 (or 29 in a leap year), not an error. That question from the previous section now has a clear answer.

lubridate distinguishes three kinds of time spans:

- **Duration**: exact number of seconds. `ddays(1)` is always 86400 seconds.
- **Period**: human units. `days(1)` is "one day," which could be 23 or 25 hours around daylight saving transitions.
- **Interval**: an anchored span with a start and end.

For most work, periods (`days()`, `months()`, `years()`) are what you want. Durations (`ddays()`, `dmonths()`) matter when you need physical time, the kind a stopwatch measures rather than a calendar.

```
# How old is R? (first public release: 1993-08-01)
interval(ymd("1993-08-01"), Sys.Date()) %/% years(1)
#> [1] 32
```

> **Opinion**
>
> Parse with lubridate, extract with lubridate, do arithmetic with lubridate. Reach for base R date functions only when you need zero dependencies.

Exercises

1. Parse the following dates: `"15-Jan-2024"`, `"2024/06/30"`, `"December 25, 2023"`. Which lubridate function does each need?
2. What day of the week were you born? Use `ymd()` and `wday(label = TRUE)`.
3. Compute the number of days between `"2020-03-01"` and `"2026-03-01"`. Then compute the number of months using `interval()` and `%/%`.

12.8. Summary

Each of these data types has a matching tidyverse package:

Data type	Base R	Tidyverse package	Prefix
Text	`paste()`, `grep()`, `sub()`	stringr	`str_`
Categories	`factor()`, `levels()`	forcats	`fct_`
Dates	`as.Date()`, `Sys.Date()`	lubridate	`ymd()`, `year()`, …

The tidyverse packages give these operations a consistent interface and naming scheme. In some cases (like `str_detect` vs `grepl`) the difference is mostly cosmetic; in others (like lubridate's timezone handling) the package catches real bugs that base R silently lets through.

Chapter 11 covered the structure; this chapter covered the three column types that cause the most trouble. But data frames need data, so Chapter 13 deals with getting files off disk and into R. After that, Chapter 14 shows how these pieces combine: `str_detect()` inside `filter()`, `fct_reorder()` inside `ggplot()`, date arithmetic inside `mutate()`. That is where the column-level work starts saving you real time.

13 Reading and writing data

So reading is solved, or at least manageable. What about the return trip?

Try loading a CSV you downloaded six months ago from a government portal. The accented characters in city names have turned into question marks, the column that should be numeric is full of strings because row 4,012 contains the word "pending," and the file path you hard-coded starts with `C:/Users/someone_else/`. Three lines of `read.csv()` and three different failures, none of which have anything to do with your analysis. Every data project begins here, in the gap between the file on disk and the clean data frame in memory, and that gap is wider than most tutorials admit.

This chapter is about crossing it. CSV, Excel, databases, binary formats: each has its own traps, and two problems cut across all of them (file paths and character encodings) with a persistence that borders on malice.

13.1. File paths

Every R session has a working directory, and all relative paths resolve from it:

```
getwd()
```

If your data lives at `data/penguins.csv` and your working directory happens to be the project root, `read.csv("data/penguins.csv")` finds the file. Move the script into a subdirectory, or run it on a colleague's laptop where the project sits in a different folder, and the same call breaks silently.

> **Opinion**
>
> Don't use `setwd()`. It welds your script to one machine, one folder hierarchy, one moment in time. Use RStudio projects or the `here` package instead; they let the project root follow the project, not the other way around.

The `here` package builds paths relative to the project root by searching upward for `.Rproj` files, `.git` directories, or similar markers, so the same call works regardless of which subdirectory your script lives in:

```
here::here("data", "penguins.csv")
```

It works on any machine, any OS, any nesting depth. Every file path in this chapter uses `here::here()`.

Paths hide one more trap, and it is entirely cosmetic until it isn't: slash direction. Forward slashes work everywhere in R, even on Windows, so `"data/penguins.csv"` is always valid. Backslashes need escaping (`"data\\penguins.csv"`), which you will forget exactly once before switching to forward slashes permanently.

Before reading any file, verify it exists. A clear error you write yourself beats a cryptic one from deep inside `read.csv()`:

```
path <- here::here("data", "penguins.csv")
if (!file.exists(path)) stop("File not found: ", path)
```

`dir.exists()` does the same for directories. But finding the file is the easy part; what happens after you open it?

Exercises

1. Run `getwd()` in your console. What does it return?
2. Install the `here` package (`install.packages("here")`). Run `here::here()`. What directory does it point to?
3. Why is `here::here("data", "file.csv")` more portable than `"C:/Users/me/project/data/file.csv"`?

13.2. Reading CSV files

Reading a file is a function: hand it a path, get back a data frame. You could write the type signature as `read_csv :: FilePath -> DataFrame`. In a purely functional world the same path would always produce the same data frame, but I/O demolishes that guarantee because someone could edit the file between calls, or the disk could fail, or the network share could vanish. R doesn't track this in its type system; the function looks pure but isn't. Haskell solves the problem by wrapping I/O in a monad (`readFile :: FilePath -> IO String`), making the side effect visible in the type. R trusts you to know the difference.

That trust has consequences. Every call to `read_csv()` implicitly assumes: this file has not changed since I last looked, the encoding is what I think it is, and the column that was numeric in row 1 is still numeric in row 4012. When any of those assumptions break, the failure is hard to diagnose because the function *looks* deterministic — same path, same call — but the world behind the path has shifted. This is why `readr` lets you specify `col_types` explicitly: you are declaring the contract the file must satisfy, and `readr` validates it on your behalf. Without that declaration, the function infers types from the first thousand rows and hopes the rest agree. Specifying types is the functional programmer's workaround for a function that cannot be pure.

Comma-separated values date to the 1970s on IBM mainframes, and there is still no real standard. RFC 4180 (2005) tried, but Excel, R, Python, and most other tools disagree on

quoting rules, newlines inside fields, and encoding. "My CSV won't load" is perhaps the only truly universal data experience.

Base R reads CSVs with `read.csv()`:

```
df <- read.csv(here::here("data", "penguins.csv"))
```

It works, but the defaults are quirky. In older versions of R, `read.csv()` converted strings to factors automatically (`stringsAsFactors = TRUE`), the same pain point from Chapter 11. Since R 4.0 the default is `FALSE`, but you will still encounter old code and old advice that assume otherwise. The quirks, however, run deeper than `stringsAsFactors`.

`readr::read_csv()` addresses most of them:

```
library(readr)
df <- read_csv(here::here("data", "penguins.csv"))
```

`read_csv()` never converts strings to factors, it is faster on large files, it prints the column types it guessed so you can catch mistakes immediately, and it returns a tibble (Section 11.6) instead of a plain data frame. For most work, `read_csv()` is simply the better default.

Key arguments worth knowing:

- `col_types`: specify column types explicitly instead of letting readr guess.
- `na`: which strings should be treated as missing values (default: `c("", "NA")`).
- `skip`: skip header lines before reading data.
- `locale`: control decimal marks, encodings, and date formats.

```
df <- read_csv(
  here::here("data", "survey.csv"),
  col_types = cols(
    id = col_integer(),
    name = col_character(),
    score = col_double(),
    date = col_date()
  ),
  na = c("", "NA", "missing")
)
```

Opinion

Specify `col_types` explicitly in any script you will run more than once. readr guesses types from the first 1,000 rows, which works beautifully until row 1,001 contains a string in what it decided was a numeric column. Guessing is for exploration; production code deserves certainty.

For European CSVs where `;` is the delimiter and `,` the decimal mark, use `read_csv2()`. For tab-separated files, `read_tsv()`. For anything else, `read_delim()` lets you specify whatever delimiter the file actually uses:

```
read_delim(here::here("data", "pipes.txt"), delim = "|")
```

Exercises

1. Read a CSV file (any file you have, or create one with a text editor). Use `read_csv()` and inspect the column types it prints.
2. Read the same file with explicit `col_types`. What happens if you declare a character column as `col_integer()`?
3. Create a small CSV where one value is the string `"NA"` and another is genuinely missing (empty cell). Read it with `read_csv()` and check the result with `is.na()`.

13.3. Writing CSV files

`readr::write_csv()` writes a data frame to disk:

```
write_csv(df, here::here("output", "clean_data.csv"))
```

It always writes UTF-8 encoding and never adds row names. Both are good defaults.

Base R's `write.csv()`, by contrast, adds row names by default, which usually just clutters the output with an unnamed first column:

```
write.csv(df, here::here("output", "clean_data.csv"), row.names = FALSE)
```

That `row.names = FALSE` is easy to forget, and forgetting it connects back to the same problem from Chapter 11: row names cause more confusion than they solve.

CSV is the right format for small to medium data that needs to be human-readable or shared across tools. But "small to medium" has a ceiling, and CSV hits it faster than you might expect. What happens when a file is too large to open in a text editor, or when you need to preserve exact types across languages?

Exercises

1. Create a small data frame and write it with both `write_csv()` and `write.csv()`. Open both files in a text editor. What's different?
2. Write a data frame with `write.csv()` and forget `row.names = FALSE`. Read it back with `read_csv()`. What happened to the row names?

13.4. Excel files

The `readxl` package reads both `.xls` and `.xlsx` files with no external dependencies (no Java, no Perl, just C++ under the hood):

```
library(readxl)
df <- read_excel(here::here("data", "survey.xlsx"))
```

Key arguments:

- `sheet`: select a sheet by name or index.
- `range`: read a specific cell range (e.g., `"B3:D15"`).
- `col_types`, `na`, `skip`: same idea as readr.

```
df <- read_excel(
  here::here("data", "survey.xlsx"),
  sheet = "raw_data",
  range = "A1:F100",
  na = c("", "N/A")
)
```

When you don't know the structure of an Excel file (and with Excel files, you often don't), list the sheet names first:

```
excel_sheets(here::here("data", "survey.xlsx"))
```

To read all sheets into a list of data frames:

```
path <- here::here("data", "survey.xlsx")
all_sheets <- purrr::map(excel_sheets(path), \(s) read_excel(path, sheet = s))
```

This uses `purrr::map()` to apply `read_excel()` to each sheet name, returning a list. The full treatment of `map()` comes in Chapter 19.

For writing Excel files, `writexl::write_xlsx()` is lightweight and sufficient for most cases:

```
writexl::write_xlsx(df, here::here("output", "results.xlsx"))
```

Opinion

Excel is a fine data source. It is a terrible data format. Merged cells break `read_excel()` because R has no concept of a cell spanning two columns; multiple tables on one sheet force you to guess where each one starts; and color-coded values are simply invisible to any reader. Read from Excel, clean in R, save as CSV or Parquet. Do not round-trip back to `.xlsx`.

Exercises

1. If you have an Excel file with multiple sheets, use `excel_sheets()` to list them, then read one sheet by name.
2. Use the `range` argument to read only a subset of cells from a sheet.

13.5. Databases

When data lives in a database, R connects to it through the DBI package, and the pattern is always the same three steps: connect, query, disconnect.

```
library(DBI)

con <- dbConnect(RSQLite::SQLite(), here::here("data", "example.sqlite"))
result <- dbGetQuery(con, "SELECT * FROM measurements LIMIT 10")
dbDisconnect(con)
```

`dbConnect()` takes a driver (here, SQLite) and connection parameters; `dbGetQuery()` sends SQL and returns a data frame; `dbDisconnect()` closes the connection when you are done. This three-step shape works with every database R supports: PostgreSQL, MySQL, SQL Server, DuckDB, and others. Only the driver and connection string change.

SQLite is self-contained (one file, no server, no configuration) and therefore ideal for learning:

```
con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbWriteTable(con, "mtcars", mtcars)
dbGetQuery(con, "SELECT mpg, cyl FROM mtcars WHERE cyl = 6")
dbDisconnect(con)
```

DuckDB is the modern choice for local analytical work. It stores data in columns rather than rows, runs aggregations fast, and can read Parquet and CSV files directly without an import step:

```
con <- dbConnect(duckdb::duckdb())
result <- dbGetQuery(con, "SELECT * FROM read_parquet('data/big_table.parquet')")
dbDisconnect(con, shutdown = TRUE)
```

The real power, though, comes from combining databases with dplyr. Instead of writing SQL by hand you can use dplyr verbs on a database table and let R translate them into the appropriate SQL dialect behind the scenes:

```
con <- dbConnect(RSQLite::SQLite(), here::here("data", "example.sqlite"))
tbl(con, "measurements") |>
  filter(year > 2020) |>
```

```
  summarise(mean_value = mean(value))
```

dplyr generates the SQL for you. The full treatment of database-backed dplyr is in Section 13.5.

What these tools share is a common interface: you describe what you want, and the backend figures out how to get it. But databases solve the "too big" problem by moving computation to the data. What if you just want a faster file?

Exercises

1. Create an in-memory SQLite database. Write the `iris` data frame to it, then query it with SQL.
2. Use `dbListTables()` to see what tables exist in a connection.

13.6. Binary and columnar formats

CSV is human-readable, which is wonderful for debugging and terrible for performance. The bottleneck is structural: CSV is row-oriented, storing each record as a unit, so reading a single column means scanning every row and throwing away everything you don't need. Columnar formats (Parquet, Feather, DuckDB's native format) flip this arrangement by storing each column contiguously, which makes analytical queries that touch few columns but many rows dramatically faster while also compressing far better because similar values sit next to each other. This row-versus-column distinction shapes every file format decision in data engineering.

`saveRDS()` and `readRDS()` serialize any R object to disk:

```
saveRDS(df, here::here("cache", "processed.rds"))
df <- readRDS(here::here("cache", "processed.rds"))
```

RDS is fast, compact, and preserves R types exactly: factors, dates, attributes, nested lists, everything. The catch is that it is R-only. Python can't read it, Excel can't read it, and the moment a collaborator opens Jupyter instead of RStudio, you need a different format.

Parquet is that format:

```
library(arrow)
write_parquet(df, here::here("data", "measurements.parquet"))
df <- read_parquet(here::here("data", "measurements.parquet"))
```

Parquet files are columnar, compressed, and readable by R, Python, Spark, DuckDB, and dozens of other tools. They are typically a fraction of the size of equivalent CSVs and far faster to read.

Arrow's Feather format offers yet another trade-off: faster than Parquet for reading and writing, but producing larger files because it prioritizes speed over compression. Use Parquet

for storage and sharing; use Feather for speed-critical intermediate files where disk space is cheap:

```
write_feather(df, here::here("cache", "fast_cache.feather"))
df <- read_feather(here::here("cache", "fast_cache.feather"))
```

When to use what:

Format	Strengths	Weaknesses
CSV	Universal, human-readable	Slow, large, lossy types
RDS	Fast, preserves all R types	R-only
Parquet	Fast, compressed, cross-language	Not human-readable
Feather	Very fast read/write	Larger than Parquet, less adopted

If your data doesn't fit in memory at all, vectra (a package I wrote for exactly this case) can stream a CSV and run dplyr-like verbs (`filter`, `select`, `mutate`, `group_by`, `summarise`) without loading the full file into R.

> **Opinion**
>
> If your CSV takes more than a few seconds to read, switch to Parquet. You will wonder why you waited.

Exercises

1. Save the `mtcars` data frame as CSV, RDS, and (if you have the `arrow` package) Parquet. Compare the file sizes with `file.size()`.
2. Read each format back and verify the result is identical with `all.equal()`.
3. Time reading the CSV and RDS versions with `system.time()`. Which is faster?

13.7. Encodings

You have read the file, parsed the columns, and everything looks right until you notice that half the city names contain `\xe9` instead of an accented e, or that German umlauts have become pairs of nonsense characters. The data isn't corrupted. The bytes are fine. Your reader is just interpreting them with the wrong mapping.

An encoding is that mapping: a table that says "byte 0xE9 means e-with-accent" (in Latin-1) or "byte sequence 0xC3 0xA9 means e-with-accent" (in UTF-8). UTF-8 is the modern standard, capable of representing every character in every living script; Latin-1 (ISO-8859-1) is the old European standard; Windows-1252 is Microsoft's variant of Latin-1. All three agree on basic ASCII (English letters, digits, punctuation) and disagree on everything else, which is precisely why files that contain only English text never trigger encoding problems and files with accented characters break constantly.

`readr::read_csv()` assumes UTF-8. Base R's `read.csv()` assumes your system's default encoding, which on Windows is often not UTF-8. This is why the same CSV can load perfectly on a Mac and produce garbled text on a Windows machine in the next room.

When you encounter garbled text, specify the encoding explicitly:

```
df <- read_csv(
  here::here("data", "european_data.csv"),
  locale = locale(encoding = "latin1")
)
```

For base R:

```
df <- read.csv(here::here("data", "european_data.csv"), fileEncoding = "latin1")
```

The BOM (byte order mark) is another common irritant. Excel sometimes prepends three invisible bytes to UTF-8 files as a signature, and `read_csv()` strips them automatically; `read.csv()` sometimes doesn't, producing a garbled first column name (you'll see something like `X.U.FEFF.id` where `id` should be).

> **Opinion**
>
> Save everything as UTF-8. If you receive a file in a different encoding, convert it once, save the result, and never look back. Life is too short to debug the same encoding issue twice.

You can check and convert encodings with base R:

```
# Check encoding of a string
Encoding(x)

# Convert from Latin-1 to UTF-8
x_utf8 <- iconv(x, from = "latin1", to = "UTF-8")
```

The practical rule fits in one sentence: use `readr` (it assumes UTF-8), and when characters still come out wrong, pass the correct encoding explicitly. Which brings us back to where we started: the gap between the file on disk and the data frame in memory is full of invisible decisions (paths, delimiters, types, encodings) that someone made for you, often badly, and that break the moment you move the file to a different machine.

Exercises

1. Create a CSV file in a text editor with a non-ASCII character (an accented letter, an umlaut). Read it with `read_csv()`. Does it display correctly?
2. What does `Encoding("hello")` return? What about `Encoding("\u00e9")`?

14 Data transformation

So now you can keep rows and pick columns. What you cannot yet do is create something that was not there before. `mutate()` adds new columns while the originals stay untouched — the same functor pattern from Section 4.4: apply a function inside a container, get back the same container with different contents. There, the container was a vector; here it is a data frame with 344 rows. You can now keep rows, pick columns, and compute new values from old ones. One operation remains: collapsing many rows into few.

Are Gentoo penguins heavier than Adelie penguins? The `penguins` data has the answer, but getting there means filtering rows, picking columns, computing a mean, and grouping by species. Every data analysis you will ever write is some arrangement of those same moves, and the question that shapes this chapter is: what is the smallest set of operations that covers all of them?

14.1. The idea: verbs for data

A dplyr verb takes a data frame and returns a data frame. The data frame goes in first, the result comes out the same shape, and nothing else changes. Five verbs cover most of what you need:

- `filter()` keeps rows that match a condition.
- `select()` picks columns by name.
- `mutate()` adds new columns computed from existing ones.
- `summarise()` collapses rows into a summary.
- `arrange()` reorders rows.

You compose them for complex transformations (via pipes, Chapter 15), but each one does exactly one thing. A dplyr pipeline is function composition: each verb transforms a data frame and passes it to the next, the same composition from lambda calculus, $(g \quad f)(x) = g(f(x))$, applied to tables instead of numbers.

These verbs did not come from nowhere. Edgar Codd's 1970 paper defined a *relational algebra*: a small set of operations on tables where every operation takes a table in and returns a table out. Restriction (keep rows), projection (keep columns), extension (add computed columns), aggregation (collapse groups). IBM resisted — they had a competing database product (IMS) — but the composability was too useful, and by the 1980s relational databases had won.

dplyr is Codd's algebra with friendlier names. The same ideas appear in SQL, pandas, and data.table because the operations are the same; only the syntax differs. `arrange()` is the one exception — ordering has no relational equivalent, but you need it for display.

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#> The following objects are masked from 'package:base':
#>
#>     intersect, setdiff, setequal, union
```

The `penguins` data frame (loaded by `palmerpenguins`) has 344 rows and 8 columns: species, island, bill measurements, flipper length, body mass, sex, and year. Every example in this chapter uses it.

```
penguins
#> # A tibble: 344 x 8
#>    species island  bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>    <fct>   <fct>            <dbl>         <dbl>             <int>       <int>
#>  1 Adelie  Torger~           39.1          18.7               181        3750
#>  2 Adelie  Torger~           39.5          17.4               186        3800
#>  3 Adelie  Torger~           40.3          18                 195        3250
#>  4 Adelie  Torger~           NA            NA                  NA          NA
#>  5 Adelie  Torger~           36.7          19.3               193        3450
#>  6 Adelie  Torger~           39.3          20.6               190        3650
#>  7 Adelie  Torger~           38.9          17.8               181        3625
#>  8 Adelie  Torger~           39.2          19.6               195        4675
#>  9 Adelie  Torger~           34.1          18.1               193        3475
#> 10 Adelie  Torger~           42            20.2               190        4250
#> # i 334 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

14.2. `filter()`: keep rows

You have 344 penguins. You only care about the Adelie ones. How do you throw the rest away?

```
filter(penguins, species == "Adelie")
#> # A tibble: 152 x 8
#>    species island  bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>    <fct>   <fct>            <dbl>         <dbl>             <int>       <int>
#>  1 Adelie  Torger~           39.1          18.7               181        3750
#>  2 Adelie  Torger~           39.5          17.4               186        3800
#>  3 Adelie  Torger~           40.3          18                 195        3250
#>  4 Adelie  Torger~           NA            NA                  NA          NA
#>  5 Adelie  Torger~           36.7          19.3               193        3450
#>  6 Adelie  Torger~           39.3          20.6               190        3650
#>  7 Adelie  Torger~           38.9          17.8               181        3625
#>  8 Adelie  Torger~           39.2          19.6               195        4675
#>  9 Adelie  Torger~           34.1          18.1               193        3475
#> 10 Adelie  Torger~           42            20.2               190        4250
#> # i 142 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

The logical expression works the same way as in Section 8.2, but notice that `species` here refers to the column, not to anything in your global environment. dplyr evaluates the expression inside the data frame.

Multiple conditions, separated by commas, are combined with AND:

```
filter(penguins, species == "Adelie", body_mass_g > 4000)
```

This keeps only Adelie penguins heavier than 4000 grams. Comma means "and." You can also write `&` explicitly, but the comma is cleaner.

For OR, use `|`:

```
filter(penguins, species == "Adelie" | species == "Chinstrap")
```

When testing membership in a set, `%in%` is clearer than chaining `|`:

```
filter(penguins, species %in% c("Adelie", "Chinstrap"))
```

One detail worth knowing: `filter()` drops rows where the condition evaluates to `NA`. If `body_mass_g` is `NA` for a row, then `body_mass_g > 4000` is `NA`, and `filter()` excludes it. This is usually what you want, but if you need to keep `NA` rows, test for them explicitly: `filter(penguins, is.na(body_mass_g))`.

Under the hood, `filter()` is doing the same logical subsetting from Section 8.2: `penguins[penguins$species == "Adelie", ]`. The verb makes the intent explicit and, more importantly, composable with other verbs. But keeping rows is only half the story; what about columns you don't need?

Exercises

1. Filter `penguins` to keep only female penguins from Biscoe island.
2. Filter `penguins` to keep rows where `flipper_length_mm` is greater than 200 or `body_mass_g` is less than 3000.
3. How many rows have missing values in `sex`? Use `filter()` and `nrow()`.

14.3. `select()`: pick columns

Eight columns is manageable. Eighty is not. When you only need species, bill length, and body mass, carrying around five extra columns clutters every printout, slows every join, and obscures what matters. `select()` strips a data frame down to the columns you name.

```
select(penguins, species, bill_length_mm, body_mass_g)
#> # A tibble: 344 x 3
#>    species bill_length_mm body_mass_g
#>    <fct>            <dbl>       <int>
#>  1 Adelie            39.1        3750
#>  2 Adelie            39.5        3800
#>  3 Adelie            40.3        3250
#>  4 Adelie            NA            NA
#>  5 Adelie            36.7        3450
#>  6 Adelie            39.3        3650
#>  7 Adelie            38.9        3625
#>  8 Adelie            39.2        4675
#>  9 Adelie            34.1        3475
#> 10 Adelie            42          4250
#> # i 334 more rows
```

The rest are gone.

Drop columns by prefixing with `-`:

```
select(penguins, -year)
```

For more flexible selection, dplyr provides helper functions. `starts_with()`, `ends_with()`, and `contains()` match column names by pattern:

```
select(penguins, starts_with("bill"))
```

```
select(penguins, ends_with("mm"))
```

`where()` selects columns based on their content, not their name. `where(is.numeric)` keeps every numeric column:

```
select(penguins, species, where(is.numeric))
```

You can rename columns inside `select()`:

```
select(penguins, species, mass = body_mass_g)
#> # A tibble: 344 x 2
#>    species  mass
#>    <fct>   <int>
#>  1 Adelie   3750
#>  2 Adelie   3800
#>  3 Adelie   3250
#>  4 Adelie     NA
#>  5 Adelie   3450
#>  6 Adelie   3650
#>  7 Adelie   3625
#>  8 Adelie   4675
#>  9 Adelie   3475
#> 10 Adelie   4250
#> # i 334 more rows
```

This selects `species` and `body_mass_g`, renaming the latter to `mass`. But notice what happened: every other column is gone. Renaming inside `select()` is really selection-with-renaming, not renaming-with-selection. If you want a new name but need the other six columns to survive, you have to decide: do you list every column you want to keep, or do you use a different verb entirely? That trade-off between concision and preservation comes up constantly.

If you want to rename without dropping columns, use `rename()`:

```
rename(penguins, mass = body_mass_g)
```

To reorder columns (move `species` to the front, keep everything else), use `everything()`:

```
select(penguins, island, species, everything())
```

Exercises

1. Select all columns from `penguins` that contain the string `"length"`.
2. Select `species`, `island`, and all numeric columns. (Hint: combine names and `where()`.)
3. What is the difference between `select(penguins, mass = body_mass_g)` and `rename(penguins, mass = body_mass_g)`? Try both.

14.4. `mutate()`: compute new columns

Body mass is recorded in grams, but your collaborator's model expects kilograms.

```
mutate(penguins, mass_kg = body_mass_g / 1000)
#> # A tibble: 344 x 9
#>    species island  bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>    <fct>   <fct>            <dbl>         <dbl>             <int>       <int>
#>  1 Adelie  Torger~           39.1          18.7               181        3750
#>  2 Adelie  Torger~           39.5          17.4               186        3800
#>  3 Adelie  Torger~           40.3          18                 195        3250
#>  4 Adelie  Torger~           NA            NA                  NA          NA
#>  5 Adelie  Torger~           36.7          19.3               193        3450
#>  6 Adelie  Torger~           39.3          20.6               190        3650
#>  7 Adelie  Torger~           38.9          17.8               181        3625
#>  8 Adelie  Torger~           39.2          19.6               195        4675
#>  9 Adelie  Torger~           34.1          18.1               193        3475
#> 10 Adelie  Torger~           42            20.2               190        4250
#> # i 334 more rows
#> # i 3 more variables: sex <fct>, year <int>, mass_kg <dbl>
```

You can reference a column you just created in the same `mutate()` call:

```
mutate(
  penguins,
  mass_kg = body_mass_g / 1000,
  mass_lb = mass_kg * 2.205
)
```

`mass_lb` uses `mass_kg`, which was defined one line above; dplyr evaluates them in order, top to bottom. This is actually surprising. R normally evaluates function arguments independently of each other. If you called a plain function with two arguments, the second could not refer to the first. `mutate()` breaks that expectation: it processes columns sequentially, so each expression can see everything defined above it. Reverse the order and `mass_lb` would fail, because `mass_kg` would not exist yet.

Any function that works on vectors works inside `mutate()`: `log()`, `sqrt()`, `cumsum()`, `lag()`, `lead()`, `percent_rank()`. The conditional functions from Section 8.6 work here too:

```
mutate(
  penguins,
  size = if_else(body_mass_g > 4000, "large", "small")
)
```

For more than two categories, `case_when()` is the tool:

```
mutate(
  penguins,
  size = case_when(
    body_mass_g > 5000 ~ "large",
```

```
    body_mass_g > 3500 ~ "medium",
    .default = "small"
  )
)
```

If you want to keep only the new columns and drop everything else, use `.keep = "none"`:

```
mutate(
  penguins,
  mass_kg = body_mass_g / 1000,
  .keep = "none"
)
#> # A tibble: 344 x 1
#>    mass_kg
#>      <dbl>
#>  1    3.75
#>  2    3.8
#>  3    3.25
#>  4   NA
#>  5    3.45
#>  6    3.65
#>  7    3.62
#>  8    4.68
#>  9    3.48
#> 10    4.25
#> # i 334 more rows
```

Exercises

1. Add a column `bill_ratio` that divides `bill_length_mm` by `bill_depth_mm`.
2. Use `mutate()` and `case_when()` to create a column `island_code` that maps `"Biscoe"` to `"B"`, `"Dream"` to `"D"`, and `"Torgersen"` to `"T"`.
3. What happens if you use a column name that already exists in `mutate()`? Try `mutate(penguins, species = toupper(species))`.

14.5. `summarise()` and `group_by()`: aggregate

What is the average body mass of a penguin in this dataset?

```
summarise(penguins, mean_mass = mean(body_mass_g, na.rm = TRUE))
#> # A tibble: 1 x 1
#>   mean_mass
#>       <dbl>
#> 1     4202.
```

One row. One number. On its own, collapsing everything into a single summary is limited. The interesting question is not "what is the mean?" but "how does the mean differ across species?" That is where `group_by()` enters.

`group_by()` marks a data frame as grouped. It doesn't change the data; it attaches metadata that tells subsequent verbs to operate per group:

```
penguins_grouped <- group_by(penguins, species)
summarise(penguins_grouped, mean_mass = mean(body_mass_g, na.rm = TRUE))
#> # A tibble: 3 x 2
#>   species   mean_mass
#>   <fct>         <dbl>
#> 1 Adelie        3701.
#> 2 Chinstrap     3733.
#> 3 Gentoo        5076.
```

`summarise()` applied `mean()` within each group and returned one row per species.

Compare the two calls. Without `group_by()`, `summarise()` collapsed everything into a single row — one mean, one answer. With `group_by(species)`, the same verb produced three rows, one per species, without any change to the `summarise()` call itself. The column you group by acts as a partitioning function: it splits the data into subsets, and `summarise()` runs independently inside each one. SQL spells this `GROUP BY`; Spark uses explicit partitioning; in functional programming the same idea appears as `foldByKey`, where a reduce operation runs per partition rather than globally. The underlying pattern — an aggregation function paired with a partitioning structure — is the same across all of them. What makes dplyr's version distinctive is that the partition is invisible metadata attached to the data frame — `group_by()` changes nothing you can see in a printout, but it changes everything `summarise()` does. That silent metadata is powerful, and it is also the source of the most common grouping bugs.

You can group by multiple variables:

```
summarise(
  group_by(penguins, species, island),
  mean_mass = mean(body_mass_g, na.rm = TRUE),
  count = n()
)
#> `summarise()` has regrouped the output.
#> i Summaries were computed grouped by species and island.
#> i Output is grouped by species.
#> i Use `summarise(.groups = "drop_last")` to silence this message.
#> i Use `summarise(.by = c(species, island))` for per-operation grouping
#>   (`?dplyr::dplyr_by`) instead.
#> # A tibble: 5 x 4
#> # Groups:   species [3]
#>   species   island    mean_mass count
#>   <fct>     <fct>         <dbl> <int>
```

```
#> 1 Adelie    Biscoe        3710.    44
#> 2 Adelie    Dream         3688.    56
#> 3 Adelie    Torgersen     3706.    52
#> 4 Chinstrap Dream         3733.    68
#> 5 Gentoo    Biscoe        5076.   124
```

`n()` counts the number of rows in each group; `n_distinct()` counts unique values:

```
summarise(
  group_by(penguins, species),
  n_islands = n_distinct(island)
)
#> # A tibble: 3 x 2
#>   species   n_islands
#>   <fct>         <int>
#> 1 Adelie            3
#> 2 Chinstrap         1
#> 3 Gentoo            1
```

The shortcut `count()` does `group_by()` plus `summarise(n = n())` in one step:

```
count(penguins, species)
#> # A tibble: 3 x 2
#>   species       n
#>   <fct>     <int>
#> 1 Adelie      152
#> 2 Chinstrap    68
#> 3 Gentoo      124
```

Opinion

After `summarise()`, the last grouping variable is silently peeled off, which means the result may still be grouped. A `mutate()` you expect to operate on the full data frame then operates per group instead. Add `.groups = "drop"` to `summarise()`, or call `ungroup()` afterward.

Exercises

1. Compute the median `flipper_length_mm` for each species.
2. Use `group_by()` and `summarise()` to find the maximum `bill_length_mm` per species and island combination.
3. Use `count()` to find how many penguins were observed in each year.

14.6. `arrange()`: order rows

`arrange()` sorts rows. The default is ascending:

```
arrange(penguins, body_mass_g)
#> # A tibble: 344 x 8
#>    species  island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>    <fct>    <fct>           <dbl>         <dbl>             <int>       <int>
#>  1 Chinstr~ Dream            46.9          16.6               192        2700
#>  2 Adelie   Biscoe           36.5          16.6               181        2850
#>  3 Adelie   Biscoe           36.4          17.1               184        2850
#>  4 Adelie   Biscoe           34.5          18.1               187        2900
#>  5 Adelie   Dream            33.1          16.1               178        2900
#>  6 Adelie   Torge~           38.6          17                 188        2900
#>  7 Chinstr~ Dream            43.2          16.6               187        2900
#>  8 Adelie   Biscoe           37.9          18.6               193        2925
#>  9 Adelie   Dream            37.5          18.9               179        2975
#> 10 Adelie   Dream            37            16.9               185        3000
#> # i 334 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

Wrap a column in `desc()` for descending order:

```
arrange(penguins, desc(body_mass_g))
```

Multiple columns break ties; sort by species alphabetically, then by mass descending within each species:

```
arrange(penguins, species, desc(body_mass_g))
```

Unlike the other verbs, `arrange()` ignores grouping by default. If you want to sort within groups, set `.by_group = TRUE`.

`arrange()` is pure display. No information is added or removed, only the order of rows changes. But a well-sorted table can reveal patterns that a jumbled one hides, and sometimes the five core verbs are not quite enough to expose them.

14.7. More verbs

`distinct()` returns unique rows:

```
distinct(penguins, species, island)
#> # A tibble: 5 x 2
#>   species   island
#>   <fct>     <fct>
```

```
#> 1 Adelie    Torgersen
#> 2 Adelie    Biscoe
#> 3 Adelie    Dream
#> 4 Gentoo    Biscoe
#> 5 Chinstrap Dream
```

Seven combinations of species and island exist in the data.

`slice_head()` and `slice_tail()` keep the first or last `n` rows:

```
slice_head(penguins, n = 3)
#> # A tibble: 3 x 8
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>             <dbl>         <dbl>             <int>       <int>
#> 1 Adelie  Torgers~           39.1          18.7               181        3750
#> 2 Adelie  Torgers~           39.5          17.4               186        3800
#> 3 Adelie  Torgers~           40.3          18                 195        3250
#> # i 2 more variables: sex <fct>, year <int>
```

`slice_max()` and `slice_min()` keep the rows with the largest or smallest values:

```
slice_max(penguins, body_mass_g, n = 3)
#> # A tibble: 4 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>           <dbl>         <dbl>             <int>       <int>
#> 1 Gentoo  Biscoe           49.2          15.2               221        6300
#> 2 Gentoo  Biscoe           59.6          17                 230        6050
#> 3 Gentoo  Biscoe           51.1          16.3               220        6000
#> 4 Gentoo  Biscoe           48.8          16.2               222        6000
#> # i 2 more variables: sex <fct>, year <int>
```

`pull()` extracts a single column as a vector, the pipe-friendly equivalent of `$`:

```
pull(penguins, species) |> head()
#> [1] Adelie Adelie Adelie Adelie Adelie Adelie
#> Levels: Adelie Chinstrap Gentoo
```

`relocate()` moves columns without dropping any. Move `sex` and `year` to the front:

```
relocate(penguins, sex, year)
```

Finally, `across()` applies a function to multiple columns at once. Summarise all numeric columns by their mean:

```
summarise(penguins, across(where(is.numeric), \(x) mean(x, na.rm = TRUE)))
#> # A tibble: 1 x 5
```

```
#>   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  year
#>            <dbl>         <dbl>             <dbl>       <dbl> <dbl>
#> 1           43.9          17.2              201.       4202. 2008.
```

`across()` works inside `mutate()` too. Round every numeric column to one decimal place:

```
mutate(penguins, across(where(is.numeric), \(x) round(x, 1)))
```

`across()` pairs naturally with `where()`, `starts_with()`, and the other selection helpers from Section 14.3. Chapter 19 generalizes the same idea (apply a function to each element of a collection) beyond data frames. But before you build elaborate pipelines out of these verbs, it is worth seeing what they look like without dplyr at all.

Exercises

1. Use `distinct()` to find all unique values of `island`.
2. Use `slice_max()` to find the 5 penguins with the longest flippers.
3. Use `across()` inside `summarise()` to compute the standard deviation (`sd()`) of every numeric column, grouped by species.

14.8. The base R equivalents

Everything dplyr does, base R can do too. The verbs are convenient wrappers, not new capabilities. So what exactly do you gain?

`filter()` is logical subsetting:

```
# dplyr
filter(penguins, species == "Adelie", body_mass_g > 4000)

# base R
penguins[penguins$species == "Adelie" & penguins$body_mass_g > 4000, ]
```

`select()` is column indexing:

```
# dplyr
select(penguins, species, body_mass_g)

# base R
penguins[, c("species", "body_mass_g")]
```

`mutate()` is direct assignment:

```
# dplyr
mutate(penguins, mass_kg = body_mass_g / 1000)

# base R
penguins$mass_kg <- penguins$body_mass_g / 1000
```

The base R version modifies the binding; the dplyr version returns a new data frame and leaves the original untouched. This distinction matters more as pipelines grow longer.

`summarise()` plus `group_by()` maps to `aggregate()`:

```
# dplyr
summarise(group_by(penguins, species), mean_mass = mean(body_mass_g, na.rm =
TRUE))

# base R
aggregate(body_mass_g ~ species, data = penguins, FUN = mean, na.rm = TRUE)
```

`arrange()` is `order()`:

```
# dplyr
arrange(penguins, desc(body_mass_g))

# base R
penguins[order(-penguins$body_mass_g), ]
```

Base R also has `subset()`, which combines filtering and column selection:

```
subset(penguins, species == "Adelie", select = c(species, body_mass_g))
```

`subset()` is clean and readable; for simple operations, it works well. Where dplyr pulls ahead is composability: every verb takes a data frame and returns one, so you can chain them. Base R's equivalents each return something slightly different: `aggregate()` renames your columns, `order()` gives you indices instead of a data frame. So you spend time wiring them together instead of thinking about your data.

dplyr and SQL are the same language wearing different syntax. Both implement relational algebra, which is why they feel similar:

dplyr	SQL
`filter(penguins, species == "Adelie")`	`SELECT * FROM penguins WHERE species = 'Adelie'`
`group_by(species) |> summarise(n = n())`	`SELECT species, COUNT(*) AS n ... GROUP BY species`

Call `show_query()` on a database-backed tibble to see the translation directly.

The reason dplyr pipelines compose so cleanly is that every verb has the same type signature: data frame in, data frame out, column structure preserved. This is the same monoid pattern from Section 4.1: an associative operation (verb composition) with an identity element (a verb that returns the data frame unchanged). Compose any sequence of dplyr verbs and the result is still a data frame.

> **Opinion**
>
> Learn the base R equivalents. You will encounter them in other people's code, in Stack Overflow answers, in older packages. But write dplyr in your own work, because the verbs compose, the names communicate intent, and the argument order is consistent. When you read `filter(penguins, species == "Adelie")`, the operation is the function name. When you read `penguins[penguins$species == "Adelie", ]`, you have to parse the indexing to understand what's happening. That parsing cost, multiplied across every line of a script, is exactly the kind of friction Codd's algebra was designed to remove.

Now you have every verb you need to answer the question from the start of the chapter:

```
penguins |>
  filter(!is.na(body_mass_g)) |>
  group_by(species) |>
  summarise(mean_mass = mean(body_mass_g))
#> # A tibble: 3 x 2
#>   species   mean_mass
#>   <fct>         <dbl>
#> 1 Adelie        3701.
#> 2 Chinstrap     3733.
#> 3 Gentoo        5076.
```

Gentoo penguins average roughly 5,076 grams; Adelie average 3,706. The answer was always in the data. Getting to it required four verbs and five lines.

Exercises

1. Rewrite `select(penguins, species, island, body_mass_g)` using base R bracket notation.
2. Rewrite `arrange(penguins, bill_length_mm)` using `order()`.
3. Use `aggregate()` to compute the mean `flipper_length_mm` per island. Compare with the `group_by()` plus `summarise()` version.

15 Pipes and composition

Pipes give you composition. Section 19.3 asks the next question: how do you apply the same transformation to many things at once?

Consider a line of code that selects columns, filters rows, and sorts the result. Written with nested function calls, you read it inside-out: the first operation you want is buried deepest, and by the third level of parentheses you are counting closing parens by hand. The order you write the code runs backwards from the order you think about the problem.

15.1. The problem with nesting

Real analysis chains multiple operations. Take the `penguins` data, keep only Adelie penguins, select two columns, sort by body mass. Without pipes, you have three choices.

Nesting: read inside-out, right-to-left.

```
arrange(filter(select(penguins, species, body_mass_g), species == "Adelie"),
body_mass_g)
```

This works for two functions, but at three it strains, and at five you are matching parentheses like a Lisp programmer on a bad day. The first operation you want to happen (`select`) is buried deepest inside the expression.

Intermediate variables: name every step.

```
a <- select(penguins, species, body_mass_g)
b <- filter(a, species == "Adelie")
c <- arrange(b, body_mass_g)
```

Readable, but the environment fills with throwaway names that exist only to shuttle a value from one line to the next: `a`, `b`, `c`, `temp`, `df2`.

Overwriting: reuse the same name.

```
df <- select(penguins, species, body_mass_g)
df <- filter(df, species == "Adelie")
df <- arrange(df, body_mass_g)
```

Compact, but fragile. You cannot re-run a single line without re-running everything above it, because each line depends on whatever `df` was a moment ago.

All three approaches work, yet none compose well. What would it look like if each step simply fed into the next?

15.2. The pipe: `|>`

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#> The following objects are masked from 'package:base':
#>
#>     intersect, setdiff, setequal, union

penguins |>
  select(species, body_mass_g) |>
  filter(species == "Adelie") |>
  arrange(body_mass_g)
#> # A tibble: 152 x 2
#>    species body_mass_g
#>    <fct>         <int>
#>  1 Adelie         2850
#>  2 Adelie         2850
#>  3 Adelie         2900
#>  4 Adelie         2900
#>  5 Adelie         2900
#>  6 Adelie         2925
#>  7 Adelie         2975
#>  8 Adelie         3000
#>  9 Adelie         3000
#> 10 Adelie         3050
#> # i 142 more rows
```

Read it top to bottom: take `penguins`, select two columns, keep Adelie rows, sort by mass. Each line transforms the result of the previous one, and the reading order matches the execution order.

`x |> f()` is syntactic sugar for `f(x)`. R literally rewrites it before the code runs. Chain two and `x |> f() |> g()` becomes `g(f(x))`, which is (g f)(x) written left-to-right instead of inside-out. That is function composition from Section 1.2, just in friendlier notation.

This is another monoid (Chapter 14): is the operation, the identity function `\(x) x` is the identity element, and chaining any number of verbs always gives you back a data frame. The same structure shows up again with `Reduce()` in Chapter 21.

The native pipe `|>` was introduced in R 4.1 (2021) and is built into the language, requiring no packages. The idea, however, is older than R itself: Doug McIlroy proposed the pipe concept for Unix in 1964, writing "we should have some ways of coupling programs like garden hose." The `|` operator in the Unix shell chains programs, each reading from standard input and writing to standard output; R's `|>` is the same idea applied to data frames, where each dplyr verb takes a data frame and returns one.

> **Opinion**
>
> One verb per line. Put a space before `|>` and a newline after it. A pipeline crammed into a single long line throws away all the readability the pipe was designed to give you.

Exercises

1. Rewrite the following nested call as a pipeline: `head(sort(sqrt(1:20)), 5)`.
2. Take `penguins`, filter for penguins heavier than 4000g, and select only `species` and `island`. Write it as a pipeline.
3. What does `10 |> sqrt()` return? What about `10 |> log(base = 2)`? Explain why `log` receives two arguments.

15.3. Building a pipeline

The best way to build a pipeline is the way you would give directions: one step at a time, checking the view at each turn.

```
penguins |>
  filter(!is.na(body_mass_g)) |>
  group_by(species) |>
  summarise(mean_mass = mean(body_mass_g)) |>
  arrange(desc(mean_mass))
#> # A tibble: 3 x 2
#>   species   mean_mass
#>   <fct>         <dbl>
#> 1 Gentoo        5076.
#> 2 Chinstrap     3733.
#> 3 Adelie        3701.
```

"Take penguins, remove NAs, group by species, compute mean mass, sort descending." The code reads like the sentence.

Build incrementally: run the first line, check the output, add the next step, check again. You grow a pipeline one verb at a time, inspecting the intermediate data frame at each stage, and when the result looks right the pipeline is done.

```
penguins |>
  filter(!is.na(flipper_length_mm), !is.na(body_mass_g)) |>
  group_by(species, island) |>
  summarise(
    n = n(),
    mean_flipper = mean(flipper_length_mm),
    mean_mass = mean(body_mass_g),
    .groups = "drop"
  )
#> # A tibble: 5 x 5
#>   species   island        n mean_flipper mean_mass
#>   <fct>     <fct>     <int>        <dbl>     <dbl>
#> 1 Adelie    Biscoe       44         189.     3710.
#> 2 Adelie    Dream        56         190.     3688.
#> 3 Adelie    Torgersen    51         191.     3706.
#> 4 Chinstrap Dream        68         196.     3733.
#> 5 Gentoo    Biscoe      123         217.     5076.
```

Each verb does one thing: `filter` removes rows, `group_by` sets the grouping, `summarise` collapses groups to summaries, `arrange` sorts. You compose them into a pipeline the same way you compose sentences into a paragraph, one idea at a time, in order. But what happens when you want to reuse a pipeline across different data sets, or embed one inside a larger computation?

Exercises

1. Build a pipeline that counts how many penguins of each species live on each island. Start with `penguins`, group by `species` and `island`, then summarise with `n = n()`.
2. Extend your pipeline from Exercise 1: arrange the result by `n` in descending order.
3. Write a pipeline that computes the median `bill_length_mm` per species (removing NAs), then filters to keep only species where the median exceeds 40mm.

15.4. Function composition

Lambda calculus (Section 1.2) is built on applying functions to arguments. Composition is the next step: apply one function, then apply another to the result.

`x |> f() |> g()` is `g(f(x))`. In mathematical notation, that is (g . f)(x): the composition of g and f, applied to x. The pipe rewrites composition from inside-out to left-to-right. Nothing else changes.

This is why dplyr verbs all follow the same contract: take a data frame as the first argument, return a data frame. Functions that follow this contract compose via the pipe; functions that don't (first argument is a formula, a filename, a model object) require workarounds. The convention exists precisely so that verbs snap together like sections of garden hose.

Functions are values you can pass around (Chapter 7), and they are also things you compose. Passing and composing are the two fundamental operations on functions, and together they account for most of what makes R code concise.

```
clean_and_summarise <- function(df) {
  df |>
    filter(!is.na(body_mass_g)) |>
    group_by(species) |>
    summarise(mean_mass = mean(body_mass_g), .groups = "drop")
}

penguins |> clean_and_summarise()
#> # A tibble: 3 x 2
#>   species   mean_mass
#>   <fct>         <dbl>
#> 1 Adelie        3701.
#> 2 Chinstrap     3733.
#> 3 Gentoo        5076.
```

Wrapping a pipeline in a function gives the composition a name. `clean_and_summarise` is a single function built from four composed steps; you can pass it around, store it in a list, or use it inside another pipeline. The pipeline describes the transformation without naming intermediates:

```
penguins |> filter(species == "Adelie") |> nrow()
```

In Haskell, this style is called "point-free" or "tacit" programming, tracing back to combinatory logic (Schönfinkel, 1924): functions are composed without mentioning the data they act on. R's pipe enables a mild version of this. It isn't fully point-free (dplyr verbs still reference column names), but the same impulse is at work.

15.5. The placeholder _

The pipe inserts the left side as the first argument. But what happens when the data belongs somewhere else?

```
penguins |>
  filter(!is.na(body_mass_g), !is.na(bill_length_mm)) |>
  lm(body_mass_g ~ bill_length_mm, data = _)
#>
#> Call:
```

```
#> lm(formula = body_mass_g ~ bill_length_mm, data = filter(penguins,
#>     !is.na(body_mass_g), !is.na(bill_length_mm)))
#>
#> Coefficients:
#>    (Intercept)  bill_length_mm
#>         362.31           87.42
```

The _ placeholder says "put the piped value here." It must appear as a named argument (`data = _`), and it can appear only once. This syntax requires R 4.2+.

For anything more complex, use an anonymous function:

```
c(1, 4, 9, 16) |>
  (\(x) x[x > 3])()
#> [1]  4  9 16
```

The `\(x)` syntax from Section 7.2 creates a function inline, and the trailing `()` calls it immediately with the piped value. This works for any situation where _ is too restrictive.

Exercises

1. Pipe `mtcars` into `lm()` to fit `mpg ~ wt`, using the _ placeholder for the `data` argument.
2. Rewrite the same call using an anonymous function instead of _.
3. Why does `penguins |> lm(body_mass_g ~ species, _)` fail? (Hint: the placeholder must be a *named* argument.)

15.6. `|>` vs `%>%`

The magrittr pipe `%>%` came first (2014). The native pipe `|>` arrived in R 4.1 (2021). Both do the same basic thing: insert the left side as the first argument of the right side. The differences are practical.

`|>` requires parentheses on the right side: `x |> sqrt()`, not `x |> sqrt`. It uses _ as its placeholder (named argument only, once), while `%>%` uses `.` (anywhere, multiple times). The native pipe is faster because it is a syntax transformation: the parser rewrites `x |> f()` to `f(x)` before evaluation, whereas `%>%` is a function call carrying its own overhead. And `|>` needs no package, while `%>%` needs magrittr (loaded automatically by the tidyverse).

The theoretical distinction runs deeper. The base pipe `|>` is pure composition: (g . f)(x), with no implicit variable and no branching. The magrittr pipe `%>%` is reminiscent of what Haskell calls *Kleisli composition*, where you chain functions of the form a -> m b and m is some context (a list, a maybe, an I/O action). The analogy is loose — `%>%` carries no monadic structure, no `return`, no bind that satisfies the monad laws — but the shape rhymes: the `.` pronoun threads the result through, allowing you to refer to it multiple times, branch on it, or pass it to non-first arguments. This is why the following works with `%>%` but has no direct equivalent with `|>`:

```
x %>% { if (nrow(.) > 0) filter(., cond) else . }
```

The magrittr pipe echoes the shape of the monadic bind operator `>>=` more than it does simple composition — both chain computations while giving a name to the intermediate result — but the resemblance is structural, not formal. If this sounds abstract, think of it concretely: `|>` is "apply the next function," while `%>%` is "apply the next function, and here is a name for what you are working with." That extra naming power makes `%>%` more flexible but also more complex, which is precisely why the base pipe chose to omit it.

Either way, pipes make substitution visible. When you write `x |> f() |> g() |> h()`, you are watching data flow through a chain of function applications, each one receiving the previous result. The same computation written as `h(g(f(x)))` hides the substitution order inside nested parentheses — you have to read from the innermost call outward, which reverses the sequence. Lambda calculus is agnostic about direction; `(h  g  f)(x)` and `x |> f |> g |> h` denote the same reduction. But human readers are not agnostic. Left-to-right pipelines follow the order in which things actually happen, and that alignment between notation and execution is why pipes took over R code within a few years of magrittr's release. No new computational power was involved; the same reductions were always available. Pipes made them legible.

```
# These are equivalent
penguins |> nrow()
#> [1] 344

library(magrittr)
penguins %>% nrow()
#> [1] 344
```

Opinion

Use `|>`. It lives in base R, it is faster, and it is the future of the language. The only reason to reach for `%>%` is legacy code or the rare case where you need `.` in multiple positions within a single call.

15.7. When not to pipe

Pipes are for linear sequences: A then B then C. Not every problem is linear.

Multiple inputs. If two data frames need to interact, pipe one and pass the other as an argument. A join fits naturally in a pipeline; a complex merge of three data frames, each requiring its own preparation steps, does not.

Multiple outputs. If you need intermediate results for different purposes later, name them. A pipeline produces one result, so if you need both the filtered data and the summary, use two separate pipelines from a shared intermediate.

```
adelie <- penguins |> filter(species == "Adelie")

# Two different analyses from the same intermediate
adelie |> summarise(mean_mass = mean(body_mass_g, na.rm = TRUE))
#> # A tibble: 1 x 1
#>   mean_mass
#>       <dbl>
#> 1     3701.
adelie |> count(island)
#> # A tibble: 3 x 2
#>   island        n
#>   <fct>     <int>
#> 1 Biscoe       44
#> 2 Dream        56
#> 3 Torgersen    52
```

Long chains. Past eight or ten steps, break the pipeline and name the intermediate result at a meaningful boundary. A pipeline that scrolls off the screen tells a run-on story, and run-on stories lose their audience.

Side effects in the middle. `print()`, `write.csv()`, and `plot()` produce side effects; they belong at the end of a pipeline, not wedged between two transformations where they interrupt the data flow and make the pipeline harder to reason about.

Debugging. When a pipeline produces unexpected output, break it apart. Assign intermediates, inspect each one, find the step that goes wrong, fix it, and reassemble the pipeline.

> **Opinion**
>
> A pipe should tell a story. If it reads like a run-on sentence, break it into paragraphs and name the paragraph boundaries with meaningful variable names, not `temp1` and `temp2`.

The pipe gives you composition in a readable notation, but composition is only the beginning. Once you can chain transformations, the next question is how to apply them to many things at once: to every column, every group, every element of a list. That is the territory of Section 19.3.

Exercises

1. The following pipeline tries to do too much. Break it into two or three named steps with meaningful names:

```
penguins |>
  filter(!is.na(body_mass_g)) |>
  mutate(mass_kg = body_mass_g / 1000) |>
```

```
group_by(species) |>
summarise(mean_kg = mean(mass_kg)) |>
arrange(desc(mean_kg)) |>
mutate(label = paste(species, round(mean_kg, 1), "kg"))
```

2. Why is putting `write.csv()` in the middle of a pipeline (between `filter` and `summarise`) a bad idea? What could go wrong?
3. You need both a summary table (mean mass per species) and a filtered dataset (Adelie penguins only) from `penguins`. Write code that avoids repeating the `filter(!is.na(body_mass_g))` step.

16 Tidy data

You have a data frame with population figures for three countries, and the years are column names: `pop_2020`, `pop_2021`, `pop_2022`. You want to plot population over time, so you try mapping `year` to the x-axis. There is no `year` column. The information is there, hiding in plain sight, encoded in the syntax of your column headers rather than the semantics of your data. Every dplyr verb you learned in Chapter 14, every ggplot2 aesthetic you'll ever write, every model formula you'll ever fit: they all assume that each variable lives in its own column. When it doesn't, nothing works. Getting data into the right shape occupies more of a working analyst's time than any model or visualization, and yet most introductions to R treat it as an afterthought, something you do before the real work begins. This chapter treats it as the real work, because it is.

You already have the pieces: data frames from Chapter 11, verbs from Chapter 14, pipes from Chapter 15. What you're missing is the target shape those tools expect.

16.1. What tidy means

Hadley Wickham's 2014 paper "Tidy data" defines three rules:

1. Each variable is a column.
2. Each observation is a row.
3. Each type of observational unit is a table.

They sound obvious. They aren't. Most real-world data breaks at least one of them, and recognizing which rule is broken (let alone fixing it) takes practice. The rules themselves are a simplified version of something database designers have known since the 1970s, when Edgar Codd defined normal forms (1NF through 5NF) to eliminate redundancy; tidy data corresponds roughly to third normal form, with no transitive dependencies and no repeating groups. The structural problem R programmers wrestle with today is the same one Codd solved in relational algebra before R existed.

Consider a table of population data with years spread across the column names:

```
pop_wide <- data.frame(
  country    = c("Norway", "Sweden", "Denmark"),
  pop_2020   = c(5380, 10350, 5831),
  pop_2021   = c(5391, 10380, 5840),
```

```
  pop_2022   = c(5408, 10420, 5857)
)
pop_wide
#>   country pop_2020 pop_2021 pop_2022
#> 1  Norway     5380     5391     5408
#> 2  Sweden    10350    10380    10420
#> 3 Denmark     5831     5840     5857
```

What is the variable here? Year. But "year" doesn't appear as a column; it's buried inside the names `pop_2020`, `pop_2021`, `pop_2022`, three columns where there should be one. Not tidy.

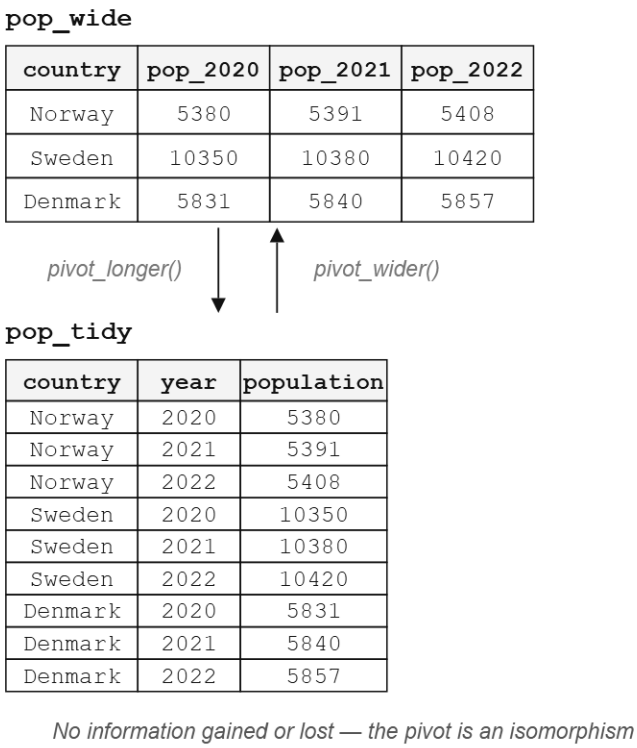


Figure 16.1.: Wide and long formats contain the same information — the pivot is an isomorphism.

The tidy version has three columns: `country`, `year`, and `population`. One variable per column, one observation per row, nine values arranged so that every tool you've learned can reach them.

```
pop_tidy <- data.frame(
  country    = rep(c("Norway", "Sweden", "Denmark"), each = 3),
  year       = rep(2020:2022, times = 3),
  population = c(5380, 5391, 5408, 10350, 10380, 10420, 5831, 5840, 5857)
)
pop_tidy
```

```
#>   country year population
#> 1  Norway 2020       5380
#> 2  Norway 2021       5391
#> 3  Norway 2022       5408
#> 4  Sweden 2020      10350
#> 5  Sweden 2021      10380
#> 6  Sweden 2022      10420
#> 7 Denmark 2020       5831
#> 8 Denmark 2021       5840
#> 9 Denmark 2022       5857
```

Now you can filter by year, group by country, plot year on the x-axis. Every dplyr verb and every ggplot2 aesthetic mapping works because each variable occupies its own column, and the tools compose because the data has a consistent shape. But who wants to rearrange nine values by hand?

Exercises

1. Look at this table. Which tidy-data rule does it break?

```
data.frame(
  student = c("Alice", "Bob"),
  math_score = c(90, 85),
  english_score = c(78, 92)
)
```

2. Sketch (on paper or in a comment) what the tidy version of that table would look like. How many rows? How many columns?
3. A data frame has columns `name`, `date`, and `temperature`. One row per city per day. Is this tidy? Why or why not?

16.2. `pivot_longer()`: columns to rows

`pivot_longer()` from the tidyr package does the reshaping in one call:

```
library(tidyr)
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#> The following objects are masked from 'package:base':
```

```
#>
#>     intersect, setdiff, setequal, union

pop_wide |>
  pivot_longer(
    cols      = pop_2020:pop_2022,
    names_to  = "year",
    values_to = "population"
  )
#> # A tibble: 9 x 3
#>   country year     population
#>   <chr>   <chr>         <dbl>
#> 1 Norway  pop_2020       5380
#> 2 Norway  pop_2021       5391
#> 3 Norway  pop_2022       5408
#> 4 Sweden  pop_2020      10350
#> 5 Sweden  pop_2021      10380
#> 6 Sweden  pop_2022      10420
#> 7 Denmark pop_2020       5831
#> 8 Denmark pop_2021       5840
#> 9 Denmark pop_2022       5857
```

Three countries, three years, nine population values: the information content is identical in both forms. `pivot_longer()` and `pivot_wider()` are inverses; applying one and then the other returns you to where you started (up to column ordering). A pair of reversible transformations like this is called an *isomorphism*: two structures that contain the same information in different arrangements. Wide and long formats are isomorphic; neither is "more correct."

The word isomorphism earns its weight here because it tells you something practical: no information is created or destroyed by pivoting. You cannot learn anything from the long format that was not already present in the wide format, and vice versa. What changes is the *alignment* between the data's shape and your next tool's expectations. A report reader wants wide, because scanning countries across years in a single row is easier than scrolling. A `ggplot()` call wants long, because each aesthetic maps to one column and each observation occupies one row. Grouped summaries need the grouping variable to be a column, not a set of column names baked into the header. Choosing a format is choosing which tool you are about to hand the data to, and pivoting is the translation between those expectations. Once that distinction is clear, the reshaping itself is mechanical.

The hard part is not the reshaping itself but deciding *what* to reshape. Given an arbitrary table with merged cells, hierarchical headers, and embedded metadata, determining the correct tidy structure requires human judgment about what constitutes a variable, an observation, and a value. Wickham's 2014 paper defines these three terms precisely to make that judgment tractable: once you answer "is year a variable or a value?", the reshaping follows mechanically.

Three arguments control the transformation:

- `cols`: which columns to pivot. Accepts tidy selection helpers like `starts_with()`, `where(is.numeric)`, or a range with `:`.
- `names_to`: what to call the new column made from column names.
- `values_to`: what to call the new column made from cell values.

The column names still carry the `pop_` prefix, and `year` is character, not integer. Two more arguments fix that:

```
pop_wide |>
  pivot_longer(
    cols            = pop_2020:pop_2022,
    names_to        = "year",
    names_prefix    = "pop_",
    names_transform = list(year = as.integer),
    values_to       = "population"
  )
#> # A tibble: 9 x 3
#>   country  year population
#>   <chr>   <int>      <dbl>
#> 1 Norway   2020       5380
#> 2 Norway   2021       5391
#> 3 Norway   2022       5408
#> 4 Sweden   2020      10350
#> 5 Sweden   2021      10380
#> 6 Sweden   2022      10420
#> 7 Denmark  2020       5831
#> 8 Denmark  2021       5840
#> 9 Denmark  2022       5857
```

`names_prefix` strips the leading text; `names_transform` applies a function to the extracted names, here converting character years to integers.

For more complex column names, `names_sep` and `names_pattern` let you split structured names into multiple new columns. If you had columns like `bill_length_mm` and `bill_depth_mm`, you could split on `_` to extract both the measurement and the unit. The tidyr pivoting vignette covers these cases in detail. But what about the opposite direction, when your data is *too* long?

Exercises

1. Create this data frame and pivot it to tidy form with columns `city`, `quarter`, and `sales`:

```
quarterly <- data.frame(
  city = c("Oslo", "Bergen"),
  Q1   = c(120, 95),
  Q2   = c(135, 110),
  Q3   = c(150, 105),
```

```
  Q4   = c(160, 130)
)
```

2. Modify your pivot to strip the `Q` prefix from the quarter column and convert it to an integer.
3. What happens if you set `values_to` to the name of an existing column? Try it.

16.3. `pivot_wider()`: rows to columns

Sometimes the problem goes the other way: distinct variables are stacked into a single column, and you need to spread them out.

```
measurements <- data.frame(
  site        = rep(c("A", "B"), each = 2),
  measurement = rep(c("temperature", "humidity"), times = 2),
  value       = c(22.1, 65, 19.8, 72)
)
measurements
#>   site measurement value
#> 1    A temperature  22.1
#> 2    A    humidity  65.0
#> 3    B temperature  19.8
#> 4    B    humidity  72.0
```

Temperature and humidity are different variables, and cramming them into a single `value` column conflates two things that should be separate. `pivot_wider()` pulls them apart:

```
measurements |>
  pivot_wider(
    names_from  = measurement,
    values_from = value
  )
#> # A tibble: 2 x 3
#>   site  temperature humidity
#>   <chr>       <dbl>    <dbl>
#> 1 A            22.1       65
#> 2 B            19.8       72
```

Two arguments:

- `names_from`: which column's values become the new column names.
- `values_from`: which column's values fill the new cells.

When some combinations don't exist in the data, `pivot_wider()` fills those cells with `NA`. The `values_fill` argument lets you choose a different default:

```
incomplete <- data.frame(
  site        = c("A", "A", "B"),
  measurement = c("temperature", "humidity", "temperature"),
  value       = c(22.1, 65, 19.8)
)

incomplete |>
  pivot_wider(
    names_from  = measurement,
    values_from = value,
    values_fill = 0
  )
#> # A tibble: 2 x 3
#>   site  temperature humidity
#>   <chr>       <dbl>    <dbl>
#> 1 A            22.1       65
#> 2 B            19.8        0
```

Opinion

If you reach for `pivot_wider()` during analysis, pause and ask whether your downstream tool really needs wide data. Usually it doesn't. `pivot_wider()` is most useful for building display tables and for feeding data into functions that expect matrix-like input. The fact that wide format *looks* tidier to human eyes doesn't mean it *is* tidier for your code.

Exercises

1. Start with `pop_tidy` (the tidy population table from Section 16.1). Use `pivot_wider()` to recreate the wide format, with one column per year.
2. What happens if the combination of id columns isn't unique before widening? Create a data frame where two rows have the same `site` and `measurement` and try `pivot_wider()`. Read the warning.

16.4. When data isn't tidy (and that's fine)

Tidy data is for analysis. Not every use case is analysis.

Wide tables are easier for humans to scan: a table with countries as rows and years as columns fits on a screen, while the same data in long form might scroll for hundreds of rows. Wide format is also what some matrix operations, correlation computations, and model interfaces expect when they want one column per predictor.

Data entry follows the same logic. If you're recording daily temperatures for three cities, a spreadsheet with cities as rows and dates as columns feels natural; nobody fills in a three-column long-form table by hand (and anyone who tells you they do is lying, or masochistic, or both).

So reshape to tidy when you need dplyr and ggplot, reshape to wide when you need a human to read the table. Do not get religious about it.

Sometimes the information you need spans multiple tables.

16.5. Joins: combining tables

You have customers and orders, species and observations, sites and measurements. The data lives in separate tables, linked by a shared key. Joins are how you bring them together.

```
bands <- data.frame(
  name  = c("John", "Paul", "George", "Ringo"),
  plays = c("guitar", "bass", "guitar", "drums")
)

albums <- data.frame(
  name  = c("John", "Paul", "George", "Mick"),
  album = c("Imagine", "Ram", "All Things Must Pass", "Some Girls")
)
```

Mutating joins

Mutating joins add columns from the second table to the first.

`left_join()` keeps all rows from the left table and attaches matching columns from the right, filling `NA` where no match exists:

```
bands |> left_join(albums, by = "name")
#>     name  plays                album
#> 1   John guitar              Imagine
#> 2   Paul   bass                  Ram
#> 3 George guitar All Things Must Pass
#> 4  Ringo  drums                 <NA>
```

Ringo has no album in the `albums` table, so his `album` comes back `NA`; Mick has no row in `bands`, so he's dropped entirely. This is the workhorse join: keep everything on the left, enrich with what you can from the right.

`inner_join()` is stricter, keeping only rows that match in both tables:

```
bands |> inner_join(albums, by = "name")
#>     name  plays                album
#> 1   John guitar              Imagine
#> 2   Paul   bass                  Ram
#> 3 George guitar All Things Must Pass
```

Ringo and Mick are both gone. Only the three names appearing in both tables survive.

When the key columns have different names, use `join_by()`:

```
instruments <- data.frame(
  musician   = c("John", "Paul", "George", "Ringo"),
  instrument = c("guitar", "bass", "guitar", "drums")
)

albums |> left_join(instruments, by = join_by(name == musician))
#>     name                album instrument
#> 1   John              Imagine     guitar
#> 2   Paul                  Ram       bass
#> 3 George All Things Must Pass     guitar
#> 4   Mick           Some Girls       <NA>
```

Filtering joins

Filtering joins don't add columns. They filter rows.

`semi_join()` keeps rows in the left table that have a match in the right:

```
bands |> semi_join(albums, by = "name")
#>     name  plays
#> 1   John guitar
#> 2   Paul   bass
#> 3 George guitar
```

Same three rows as `inner_join()`, but without the `album` column. You're using `albums` as a filter, not as a source of new data.

`anti_join()` does the opposite, keeping rows in the left table that have *no* match in the right:

```
bands |> anti_join(albums, by = "name")
#>    name plays
#> 1 Ringo drums
```

Only Ringo: the one band member with no album in the `albums` table. `anti_join()` is your best tool for finding what's missing. Which customers placed no orders? Which species have no observations? Which students didn't submit?

Opinion

Start with `left_join()`. Use `anti_join()` to debug. The others are situational. `right_join()` exists but is rarely needed: swap the table order and use `left_join()` instead. `full_join()` keeps everything from both sides, which sounds safe but usually signals that you haven't decided what your unit of analysis actually is.

Exercises

1. Create two small data frames: `students` with columns `id` and `name`, and `grades` with columns `id` and `score`. Give them overlapping but not identical `id` values. Use `left_join()` to combine them.
2. Use `anti_join()` on the same tables to find students with no grades and grades with no student. (Hint: swap which table is on the left.)
3. What is the difference between `semi_join(bands, albums, by = "name")` and `inner_join(bands, albums, by = "name") |> select(name, plays)`? Try both.

16.6. Other tidyr tools

Pivoting handles structure; joins handle combination. A handful of other tidyr functions handle the messy details that remain after those two big moves.

Separating and uniting columns

`separate_wider_delim()` splits one column into several by a delimiter:

```
dates <- data.frame(
  id   = 1:3,
  date = c("2024-03-15", "2024-07-22", "2024-11-01")
)

dates |>
  separate_wider_delim(
    cols  = date,
    delim = "-",
    names = c("year", "month", "day")
  )
#> # A tibble: 3 x 4
#>      id year  month day
#>   <int> <chr> <chr> <chr>
#> 1     1 2024  03    15
#> 2     2 2024  07    22
#> 3     3 2024  11    01
```

This replaces the older `separate()`, which relied on position-based splitting and was less predictable. For complex patterns, `separate_wider_regex()` lets you use regular expressions.

`unite()` does the inverse, pasting columns together:

```
data.frame(year = 2024, month = "03", day = 15) |>
  unite(col = "date", year, month, day, sep = "-")
#>         date
#> 1 2024-03-15
```

Completing and filling

`complete()` makes implicit missing values explicit. If your data has observations for some year/site combinations but not all, `complete()` adds rows for the gaps and fills them with `NA`:

```
obs <- data.frame(
  year = c(2020, 2020, 2021),
  site = c("A", "B", "A"),
  count = c(5, 3, 7)
)

obs |> complete(year, site)
#> # A tibble: 4 x 3
#>    year site  count
#>   <dbl> <chr> <dbl>
#> 1  2020 A         5
#> 2  2020 B         3
#> 3  2021 A         7
#> 4  2021 B        NA
```

`fill()` propagates the last non-missing value downward (or upward), which is useful for data where group labels are only written once and the rows beneath are left blank:

```
report <- data.frame(
  group = c("Control", NA, NA, "Treatment", NA),
  value = c(10, 12, 11, 20, 22)
)

report |> fill(group)
#>       group value
#> 1   Control    10
#> 2   Control    12
#> 3   Control    11
#> 4 Treatment    20
#> 5 Treatment    22
```

Nesting

`nest()` packs subsets of a data frame into list-columns, and `unnest()` unpacks them. This pattern shines when fitting models to groups, but it requires familiarity with list-columns and iteration, both covered in Section 19.3.

16.7. Summary

Tidy data is a structural contract: one variable per column, one observation per row, one observational unit per table. `pivot_longer()` turns columns into rows (the common direction), `pivot_wider()` turns rows into columns (the occasional direction), and joins combine tables by matching keys. `left_join()` enriches; `anti_join()` finds gaps; the rest control how unmatched rows are handled.

These tools compose precisely because they all expect the same shape, and the connection to their mathematical foundations runs deeper than most R programmers realize. Codd's relational algebra and the simply typed lambda calculus are both built on function application and composition over structured types, which is why the same ideas keep surfacing in R, SQL, and formal logic. In category theory, a join is a *pullback*: given two tables that each map to a shared key space, the join constructs the most general table consistent with both. Think of it as a Venn diagram for rows, where the overlap is determined by the key column; `inner_join` keeps only the overlap, while `left_join` keeps everything on the left and fills the gaps with `NA`.

But shape is only half the story. Once your data is tidy, you still need to *do* something with it, and doing the same thing to many groups, many columns, or many models at once is where iteration enters. That's next.

17 Visualization

Try building a bar chart in base R, then a scatterplot, then a histogram. Each one calls a different function with different arguments, different parameter names, different assumptions about what your data looks like. You learn three APIs instead of one idea. ggplot2 takes the opposite approach: it gives you a grammar, a small vocabulary of composable pieces that describe *any* plot, and you assemble the pieces yourself. Once the grammar clicks, you stop asking "which function draws a heatmap?" and start asking "what mapping, what geometry, what coordinate system?"

This chapter assumes you have tidy data (Chapter 16), can reshape it with dplyr (Chapter 14), and can pipe it through transformations (Chapter 15). The data is clean. Now you see it.

17.1. The grammar of graphics

In 1999, Leland Wilkinson published *The Grammar of Graphics*, and the central argument was deceptively simple: a plot is not a type. Not a bar chart, not a scatterplot, not a heatmap. A plot is a mapping from data to visual properties, rendered by geometric objects, in a coordinate system; types are consequences of choices, not starting points. The intellectual roots stretch further back to Jacques Bertin's *Semiologie Graphique* (1967), which classified visual variables (position, size, shape, value, color, orientation, texture) and ranked their effectiveness for different data types. Wilkinson built on Bertin's foundation, and when ggplot2 maps a variable to `color` vs `size` vs `shape`, it is implementing Bertin's taxonomy: position is most effective for quantitative data, color for categorical.

Hadley Wickham turned this theory into software in 2005, building ggplot2 as a layered grammar where plots emerge from composing independent components:

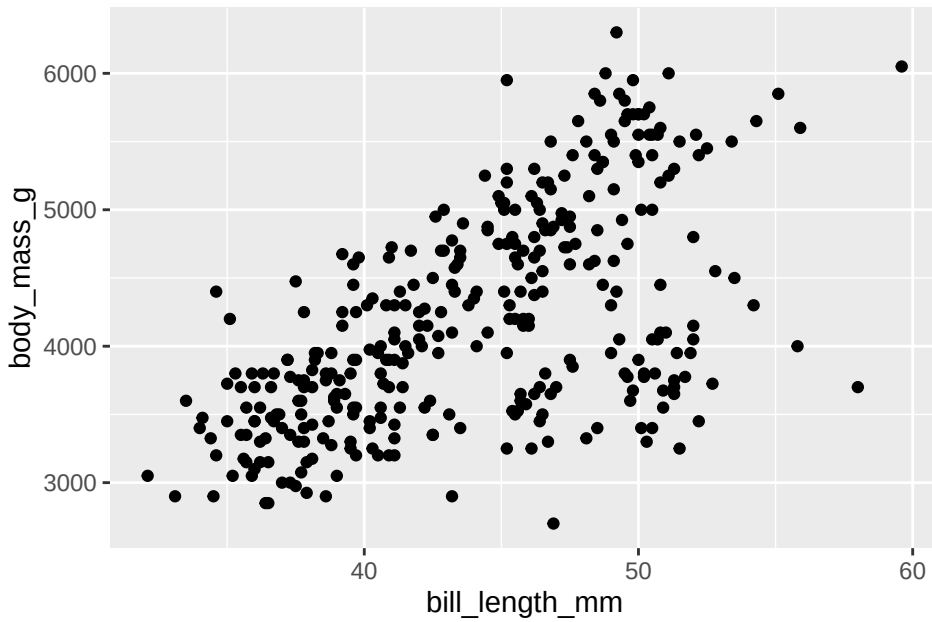
1. **Data**: a tidy data frame (Chapter 16).
2. **Aesthetics** (`aes()`): which variables map to which visual properties.
3. **Geoms**: what shapes represent the data.
4. **Stats**: statistical transformations (often implicit).
5. **Scales**: how data values translate to aesthetic values (colors, sizes, axes).
6. **Facets**: small multiples, splitting by a variable.
7. **Themes**: non-data visual styling.

Most plots need only three: data + aesthetics + geom. The rest are refinements you reach for when the defaults aren't enough.

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
library(ggplot2)
```

The simplest ggplot2 call needs three things: data, an aesthetic mapping, and a geom:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point()
```



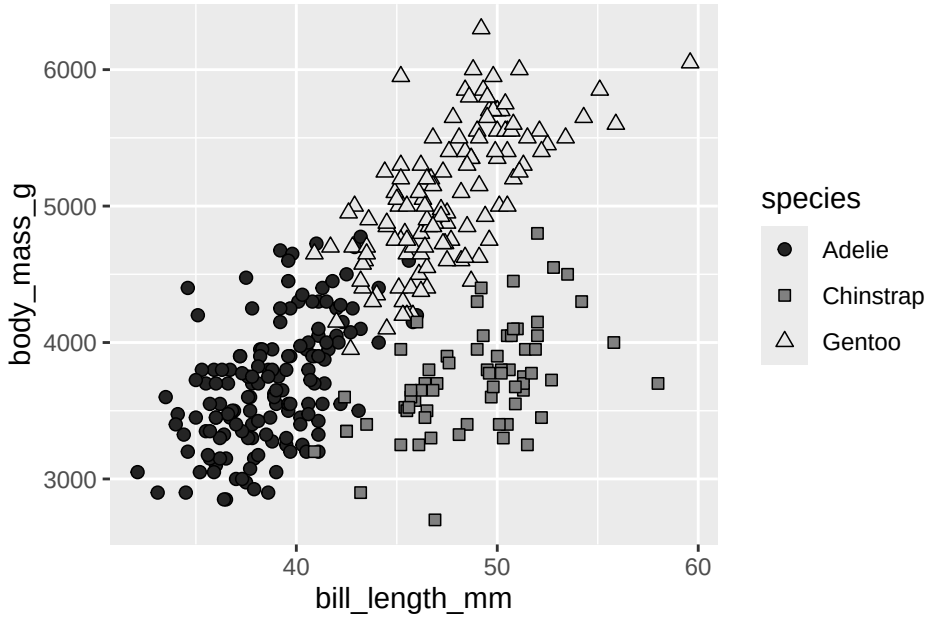
`ggplot()` sets up the coordinate system, `aes()` maps bill length to the x-axis and body mass to the y-axis, and `geom_point()` draws one point per row. The `+` adds the geom as a layer. Three lines, the entire grammar. But what happens when you want to encode a third variable?

17.2. Aesthetics: data to visuals

`aes()` creates a mapping from columns to visual properties:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, fill = species, shape =
species)) +
  geom_point(stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
```

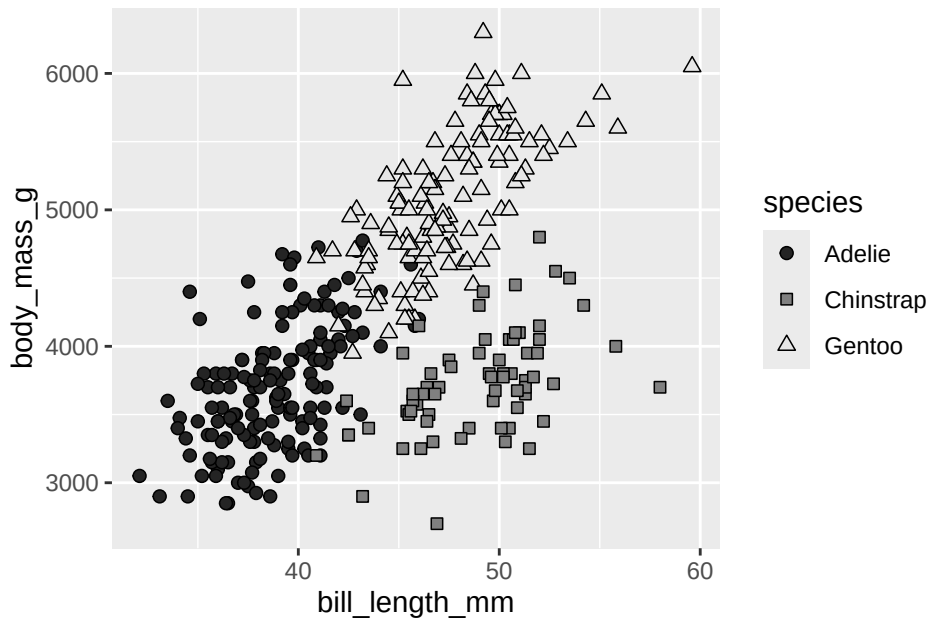
```
  scale_shape_manual(values = c(21, 22, 24))
```



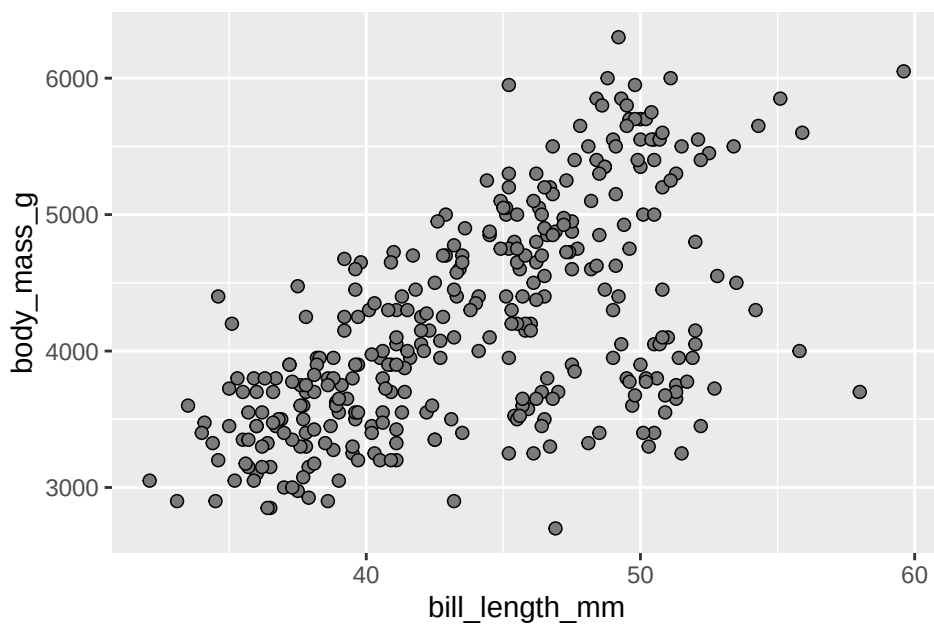
Writing `color = species` maps each species to a different color; ggplot2 picks the palette automatically and adds a legend. Common aesthetics include `x`, `y`, `color` (or `colour`), `fill`, `shape`, `size`, `alpha`, `linetype`, and `group`.

There is one distinction that trips up everyone, usually within the first hour. Inside `aes()` means *mapped to a variable*. Outside `aes()` means *fixed for all observations*.

```
# Color varies by species
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point(aes(fill = species, shape = species), stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24))
```




```
# All points are steel blue
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point(fill = "steelblue", shape = 21, stroke = 0.4, size = 2)
```



> **Opinion**
>
> If someone's plot has a legend they didn't ask for, they probably put a variable outside `aes()`. If all their points turned the same color when they shouldn't have, they put a constant inside `aes()`. Getting this one distinction right fixes roughly half of all ggplot2 questions on Stack Overflow.

Aesthetics placed in `ggplot()` are inherited by every layer; aesthetics placed in a specific `geom_*()` apply only to that layer. When multiple geoms share the same mapping, put it in `ggplot()`. When a mapping belongs to one layer only, put it in the geom. This inheritance rule keeps your code DRY, but it also means that a misplaced aesthetic can quietly propagate to layers you didn't intend. So what shapes can those layers draw?

Exercises

1. Create a scatterplot of `flipper_length_mm` vs `body_mass_g` from `penguins`. Map `species` to color.
2. What happens if you put `color = "blue"` inside `aes()`? Try it and explain the result.
3. Map `island` to the `shape` aesthetic in a scatterplot. What does the legend show?

17.3. Geoms: shapes for data

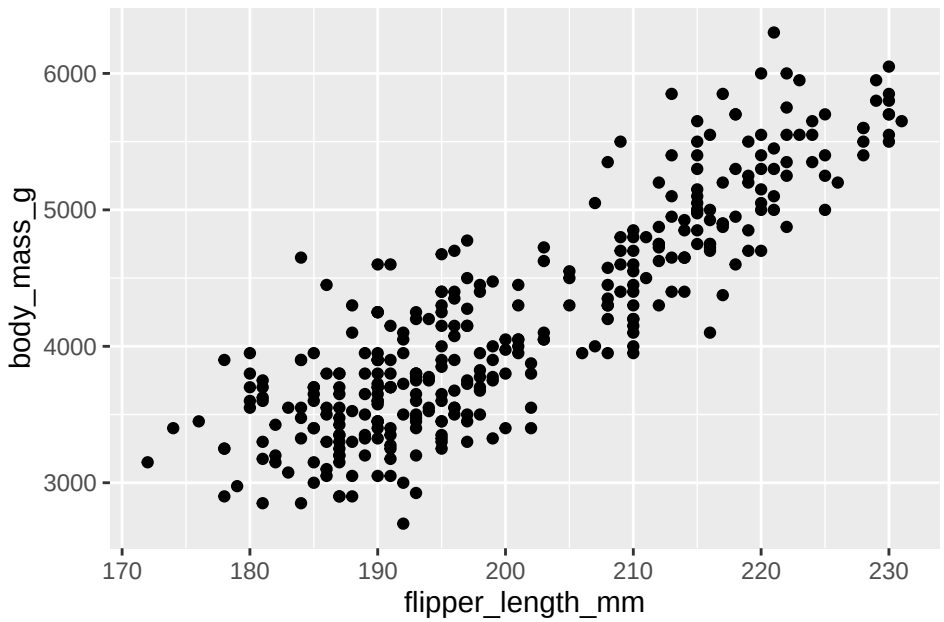
Each geom renders data differently. Same aesthetics, different geom, different plot. The question is which geom fits the question you are asking. A rough guide:

Question	Geom	Data shape
How are two continuous variables related?	`geom_point()`, `geom_smooth()`	x continuous, y continuous
How is one variable distributed?	`geom_histogram()`, `geom_density()`	x continuous
How do distributions compare across groups?	`geom_boxplot()`, `geom_violin()`	x categorical, y continuous
How does a quantity change over time?	`geom_line()`	x ordered (time), y continuous
How do counts or totals compare across categories?	`geom_bar()` (counts rows), `geom_col()` (uses a y value)	x categorical

Start from the question, pick the geom, then refine. The rest of this section shows each one in turn.

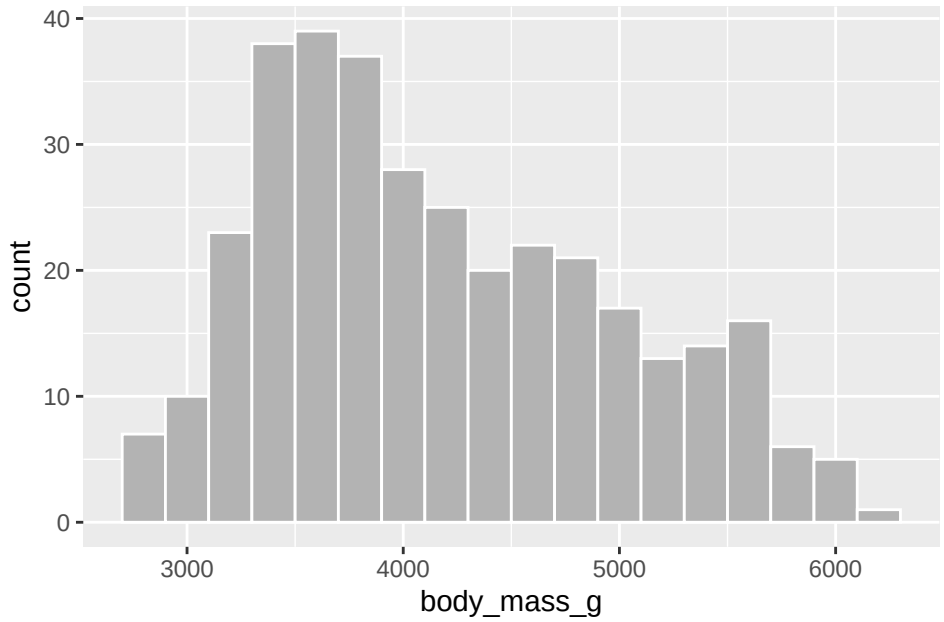
`geom_point()` draws a scatterplot, the natural choice for two continuous variables:

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```



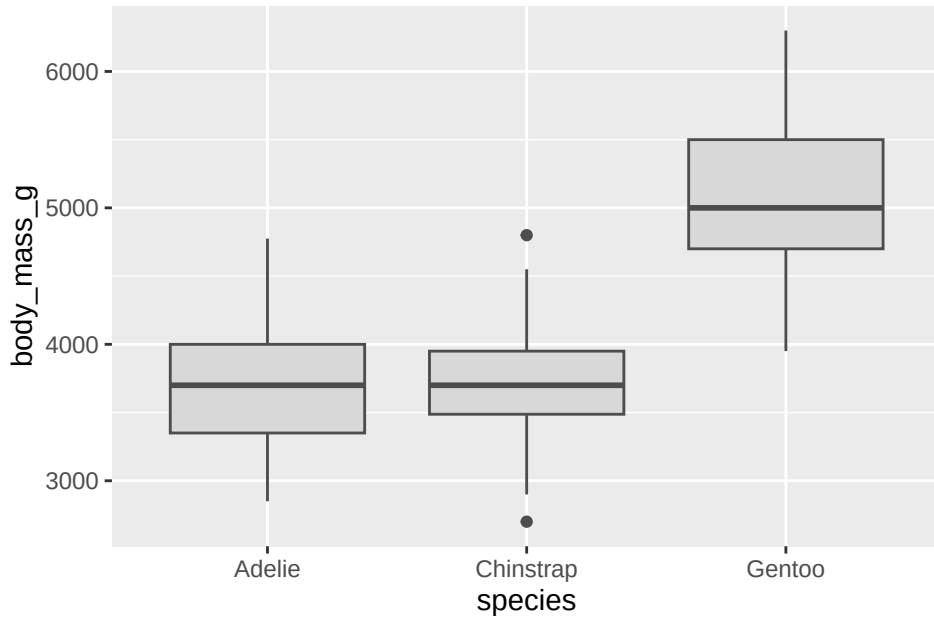
`geom_histogram()` shows the distribution of one continuous variable, and you control the resolution with `bins` or `binwidth`:

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 200, fill = "grey70", color = "white")
```



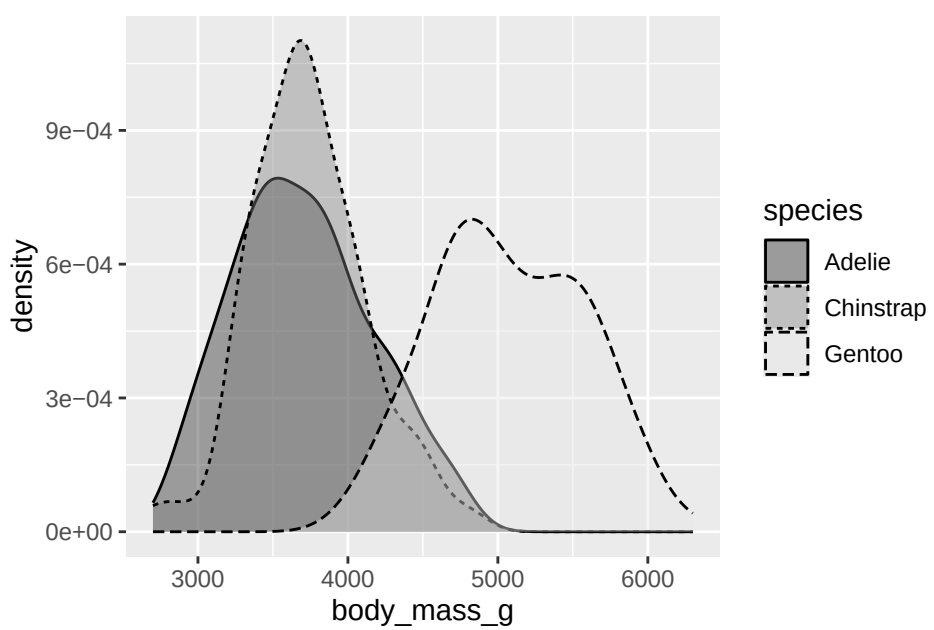
`geom_boxplot()` summarizes distributions by group (median, quartiles, outliers):

```
ggplot(penguins, aes(x = species, y = body_mass_g)) +
  geom_boxplot(fill = "grey85", color = "grey30")
```



`geom_density()` is a smooth alternative to the histogram that estimates probability density, making it easier to overlay and compare distributions:

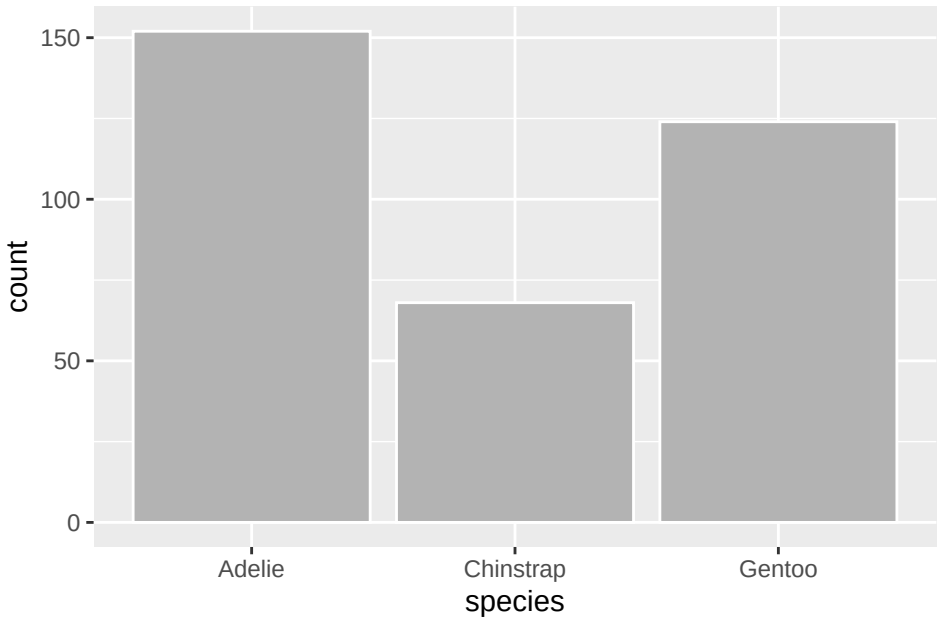
```
ggplot(penguins, aes(x = body_mass_g, fill = species, linetype = species)) +
  geom_density(alpha = 0.4) +
  scale_fill_viridis_d()
```



`geom_line()` connects points from left to right, the standard geom for time series and trends. Order matters: if the data isn't sorted by x, the lines will zigzag into nonsense.

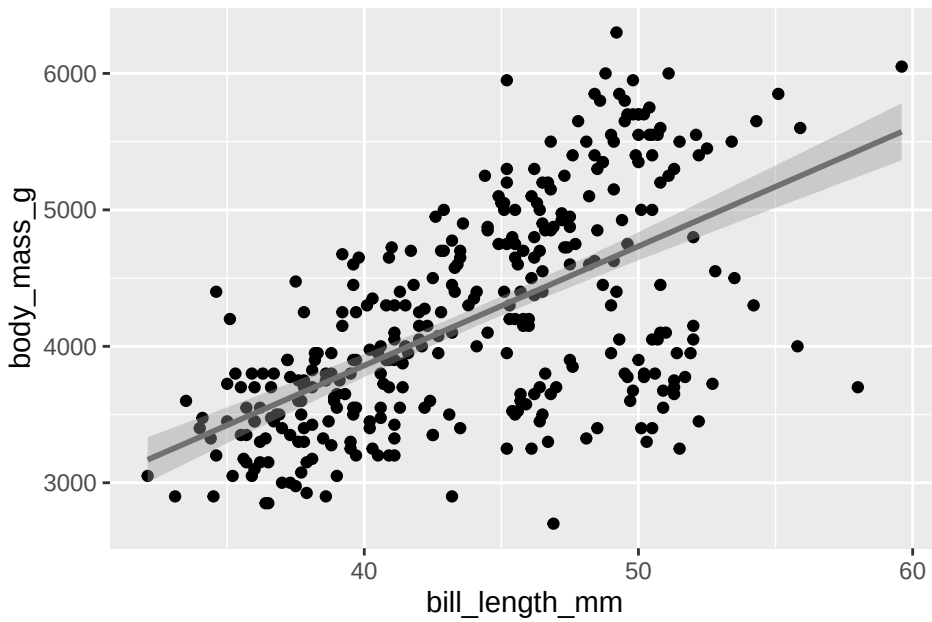
`geom_col()` draws bars with heights taken directly from the data, while `geom_bar()` counts rows for you. The difference is the stat: `geom_bar()` applies `stat = "count"` internally, so you only map `x`; `geom_col()` uses `stat = "identity"`, so you map both `x` and `y`.

```
ggplot(penguins, aes(x = species)) +
  geom_bar(fill = "grey70", color = "white")
```



`geom_smooth()` adds a fitted line or curve, with `method = "lm"` for linear and `method = "loess"` for smooth:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point() +
  geom_smooth(method = "lm")
```



Layers compose with `+`. The plot above has two layers, points and a linear fit, and each `+` adds a component to the same plot object. But adding layers only changes the geometry. What about controlling *how* data values map to visual properties?

Exercises

1. Create a scatterplot of `bill_length_mm` vs `bill_depth_mm`. Add a smooth line with `method = "loess"`.
2. Replace `geom_point()` with `geom_density2d()` in the same plot. What changes?
3. Make a boxplot of `flipper_length_mm` by `island`. Add `geom_jitter(width = 0.2, alpha = 0.3)` as a second layer.

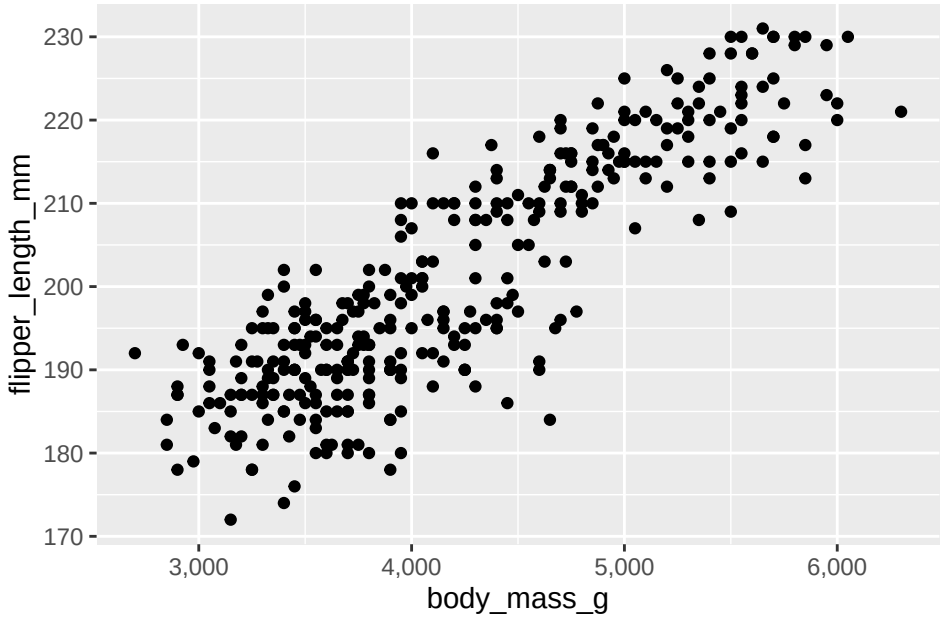
17.4. Scales: controlling the mapping

Every aesthetic has a scale, whether you set one or not. Scales control how data values become visual values: which colors, which axis limits, which labels.

When you write `aes(x = bill_length_mm)`, ggplot2 quietly creates a default `scale_x_continuous()` behind the scenes, and you only need to override it when the default behavior falls short: when you want to change axis limits, transform the axis, or pick specific colors.

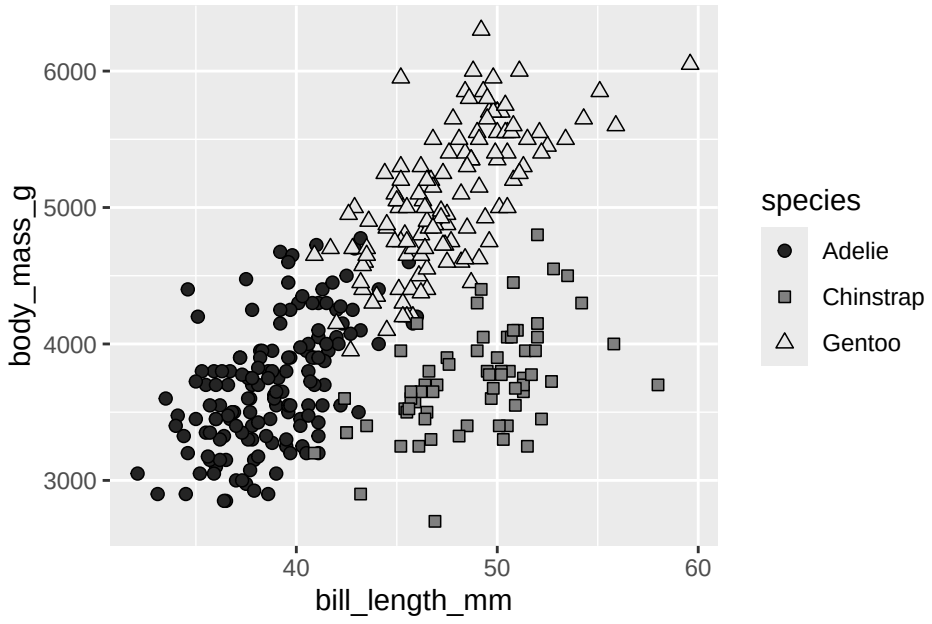
Position scales control axes:

```
ggplot(penguins, aes(x = body_mass_g, y = flipper_length_mm)) +
  geom_point() +
  scale_x_continuous(labels = scales::comma)
```



Color scales control how values map to colors. For discrete variables, `scale_color_viridis_d()` is a strong default (colorblind-friendly, perceptually uniform):

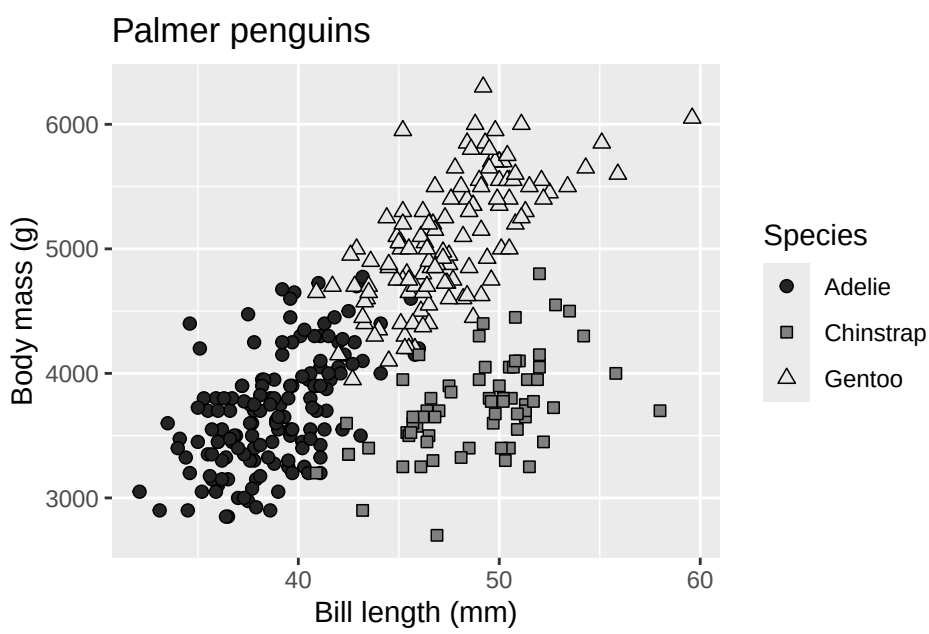
```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, fill = species, shape =
species)) +
  geom_point(stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24))
```



`labs()` controls titles and labels. Always label your axes with units, because `bill_length_mm`

is a column name, not a label:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, fill = species, shape =
species)) +
  geom_point(stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24)) +
  labs(
    x = "Bill length (mm)",
    y = "Body mass (g)",
    fill = "Species",
    shape = "Species",
    title = "Palmer penguins"
  )
```



Opinion

Always label your axes with units. A plot with `bill_length_mm` on the axis is a working draft; a plot with "Bill length (mm)" is communication. The difference is thirty seconds of typing and the entirety of your audience's comprehension.

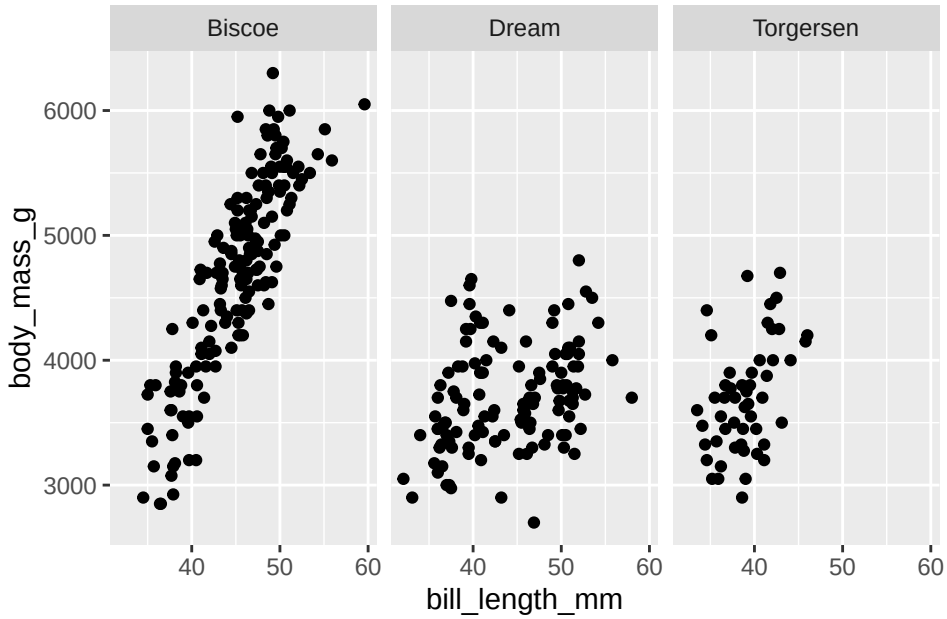
Exercises

1. Create a scatterplot of `bill_length_mm` vs `body_mass_g`, colored by `species`. Use `scale_color_brewer(palette = "Set2")` instead of viridis.
2. Add a `labs()` call with a title, subtitle, and proper axis labels.
3. Use `scale_y_log10()` on a plot of `body_mass_g`. When might a log scale be appropriate?

17.5. Facets: small multiples

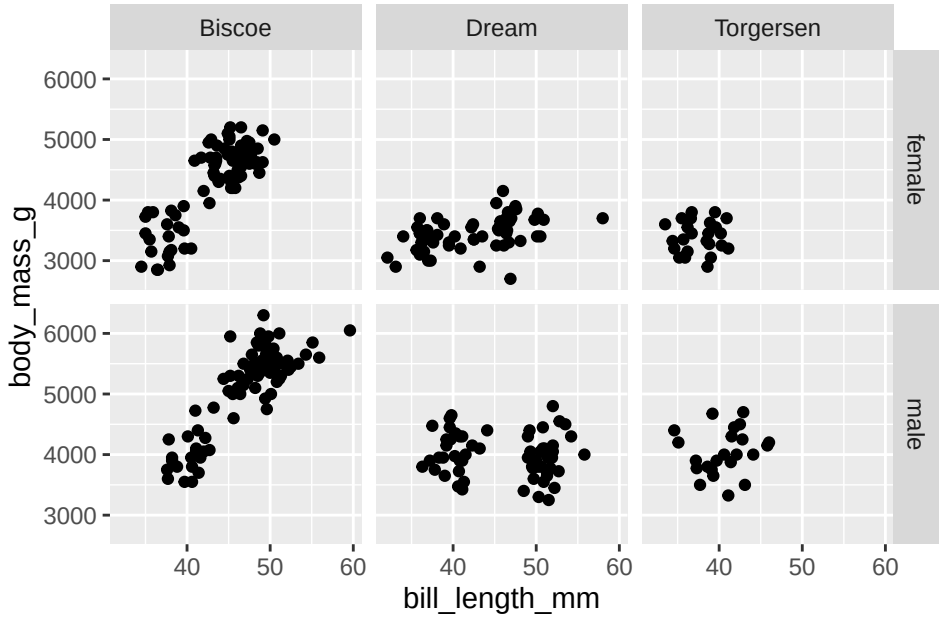
Faceting splits one plot into multiple panels, one per level of a variable. `facet_wrap()` wraps panels into rows:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point() +
  facet_wrap(~ island)
```



`facet_grid()` creates a grid with rows and columns:

```
ggplot(penguins |> dplyr::filter(!is.na(sex)),
       aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point() +
  facet_grid(sex ~ island)
```



By default, all panels share the same axis scales. You can free them with `scales = "free_y"` or `scales = "free"`, but use this cautiously: free scales make comparison across panels harder, which is the whole point of small multiples in the first place.

When should you facet instead of coloring? Facet when the groups would overlap too much for color to separate them, or when you want each group's pattern to stand on its own without visual interference. Color works when groups are few, visually separable, and you want to see them in the same coordinate space.

Either way, faceting replaces the temptation to build the same plot three times with a loop or copy-paste. One `facet_wrap()` call produces all the panels, with shared axes for honest comparison, and the grammar handles the layout while you focus on the question. But what about the visual details that have nothing to do with data?

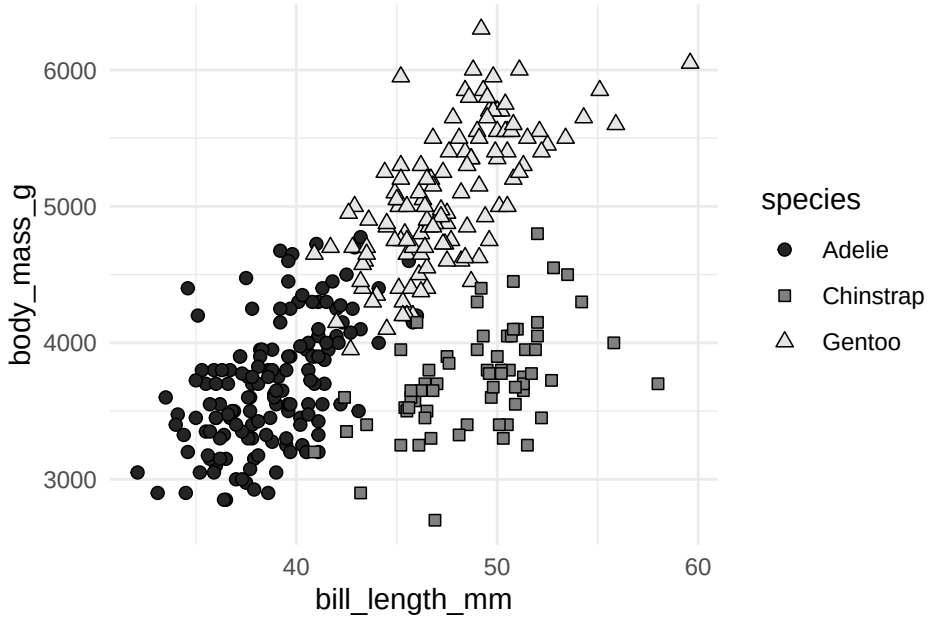
Exercises

1. Facet the penguins scatterplot (bill length vs body mass) by `species` using `facet_wrap()`.
2. Use `facet_grid(species ~ island)` on the same plot. Which cells are empty, and why?
3. Add `scales = "free"` to your faceted plot. What changes? Is the comparison easier or harder?

17.6. Themes: non-data styling

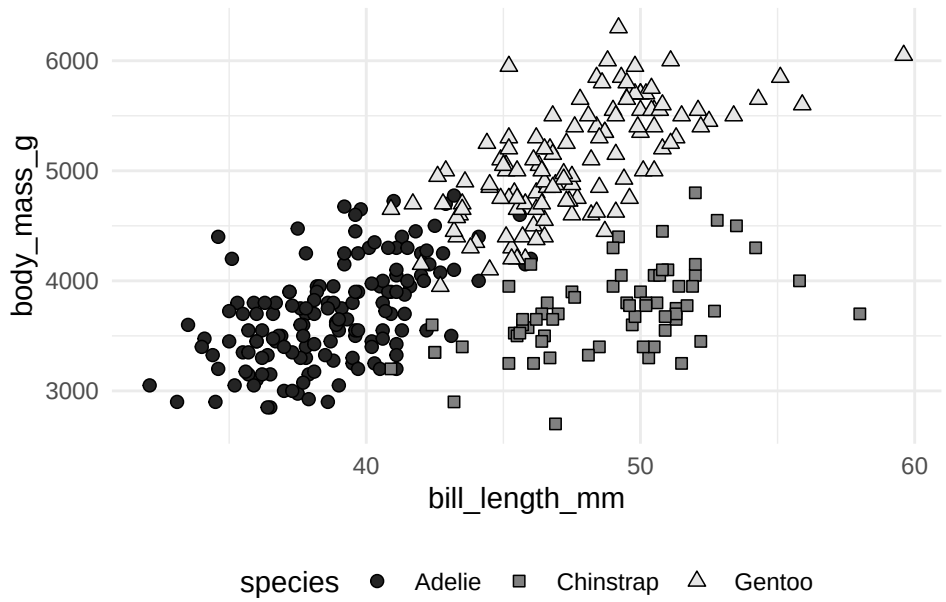
Themes control everything on a plot that isn't data: background, grid lines, fonts, legend position. ggplot2 ships several built-in options:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, fill = species, shape =
species)) +
  geom_point(stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24)) +
  theme_minimal()
```



Other useful defaults include `theme_classic()` (white background, no grid) and `theme_bw()` (white background, light grid). For fine-grained control, `theme()` adjusts individual elements:

```
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, fill = species, shape =
species)) +
  geom_point(stroke = 0.4, size = 2) +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24)) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold")
  )
```



To set a default theme for your entire session, use `theme_set()`:

```
theme_set(theme_minimal())
```

Themes are cosmetic. Spend your time on clear mappings and good labels before worrying about fonts.

That said, pick one default early (`theme_minimal()` or `theme_bw()` are the usual choices) and use it everywhere. Three plots in the same report with the same theme are preferable.

Exercises

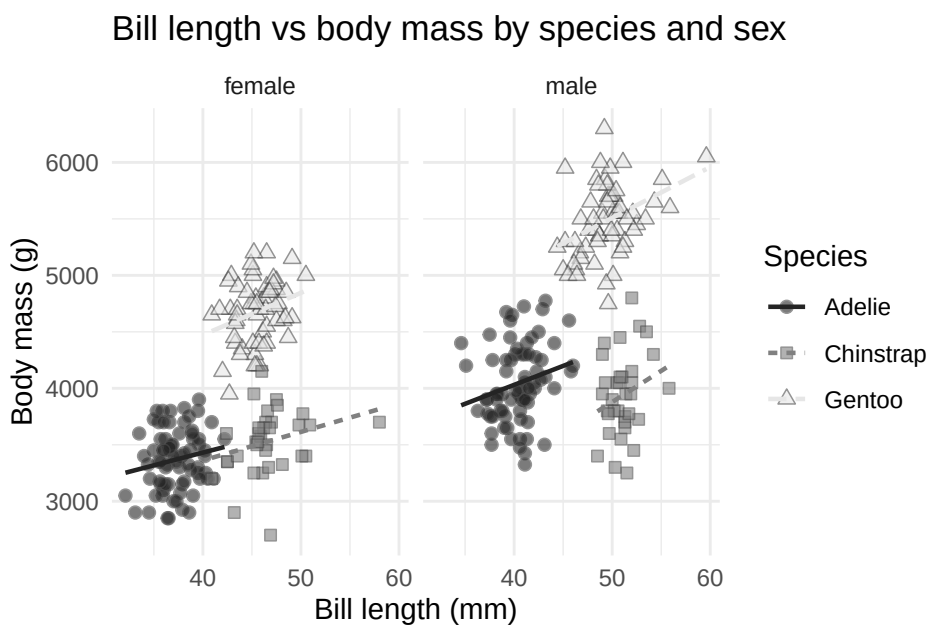
1. Apply `theme_classic()` to any scatterplot from this chapter. How does it differ from `theme_minimal()`?
2. Use `theme(axis.text.x = element_text(angle = 45, hjust = 1))` to rotate x-axis labels. When is this useful?

17.7. Putting it together

Read this example like a sentence: take penguins, remove missing sex, map bill length to x and mass to y and species to color, draw points, add linear fits, split by sex, use viridis colors, label everything, apply a minimal theme.

```
penguins |>
  dplyr::filter(!is.na(sex)) |>
  ggplot(aes(x = bill_length_mm, y = body_mass_g, color = species, fill = species,
  shape = species, linetype = species)) +
```

```
  geom_point(alpha = 0.6, stroke = 0.4, size = 2, color = "grey30") +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  facet_wrap(~ sex) +
  scale_color_viridis_d() +
  scale_fill_viridis_d() +
  scale_shape_manual(values = c(21, 22, 24)) +
  labs(
    x = "Bill length (mm)",
    y = "Body mass (g)",
    color = "Species",
    fill = "Species",
    shape = "Species",
    linetype = "Species",
    title = "Bill length vs body mass by species and sex"
  ) +
  theme_minimal()
```



The pipe feeds data into `ggplot()`, and after that, `+` composes the layers. Each line adds one component, so you can read the full specification top to bottom, like a recipe where every ingredient appears in order.

The pattern for most visualizations follows the same shape:

1. Start with the data.
2. Pipe it through any needed transformations (filtering, summarizing).
3. Hand it to `ggplot()` with your aesthetic mappings.
4. Add geoms.
5. Refine with scales, facets, labels, and a theme.

Each step is independent: swap `geom_point()` for `geom_jitter()` and nothing else in the specification changes. `+` is doing more work than it looks like.

17.8. ggplot2 as lambda calculus

A ggplot specification is a program. Not metaphorically: it is a tree of expressions that gets evaluated to produce output, just like any other R expression, and the connection to the ideas from Chapter 7 runs deep.

`aes()` is a function that returns unevaluated expressions. When you write `aes(x = bill_length_mm, y = body_mass_g)`, R does not look up `bill_length_mm` in your environment; it captures the expression and stores it for later evaluation inside the data frame. This is the same quoting mechanism you will encounter in Chapter 26: `aes()` is non-standard evaluation applied to visualization. The mapping is a *description* of a computation, not the computation itself, exactly like a lambda expression describes a function without executing it.

The `+` operator on ggplot layers is monoidal composition. Each layer is an independent unit, combining two with `+` produces a new plot object, and the empty plot `ggplot()` acts as an identity element (adding it changes nothing). An associative binary operation with an identity element is a monoid, one of the simplest structures in algebra. String concatenation with `""` as identity is a monoid; list concatenation with `list()` is a monoid; function composition with the identity function is a monoid. ggplot layers are another instance of the same pattern. We will see in Chapter 21 that `Reduce()` works precisely because it folds over a monoid, and in Section 30.8 how the pattern generalizes further.

The rendering pipeline itself is function composition. When ggplot2 draws a plot, it runs a sequence of transformations: data, then stat (statistical transformation), then scale (map values to aesthetic space), then coord (project into pixel coordinates), then render (draw shapes). Each step takes a data structure and returns a data structure, making the pipeline `render coord scale stat`, pure composition. A graphics engine, at its core, is a chain of morphisms between categories: data space to statistical space to aesthetic space to physical space (pixels). Each `geom_*()` specifies what the final rendering step should draw; everything before it is a data transformation pipeline.

This is why the grammar works so well: a small set of composable functions rather than a grab bag of plot types. You have already seen this principle at work: small functions that compose are more powerful than large functions that don't (Chapter 15). ggplot2 applies that principle to visualization.

Compare this to what came before. Base R's `plot()`, `barplot()`, and `hist()` are self-contained functions, each with its own parameter names and its own assumptions about data shape. Adding a fitted line to a scatterplot means calling `abline()` after `plot()`, a second function that depends on the first having already drawn to the graphics device. There is no plot object to inspect or modify; the drawing has already happened. If you want a variant (a scatterplot with a smoothed trend, faceted by species, with a custom color scale), you are writing procedural drawing code. The grammar inverts that relationship. Each layer is an independent object, and combining two with `+` produces a new plot object you can store, modify, and pass around before anything touches the screen. The layers compose because

they are values, not side effects, following the same principle that makes pipe chains and `map()` calls work. Graphics is where composition becomes visible, because you can see the result on the screen and trace each layer back to the line of code that produced it.

What happens when you apply this principle to your own domain?

Exercises

1. Build a plot from scratch: filter `penguins` to only Adelie penguins, then create a scatterplot of flipper length vs body mass, colored by island. Add proper labels and a theme.
2. Create a faceted histogram of `body_mass_g` by `species`, with `binwidth = 100`. Use `fill = species` and set `alpha = 0.7`.
3. Start from the full example above and modify it: change the geom to `geom_density2d()`, remove the faceting, and switch to `theme_classic()`. What does the plot reveal?

17.9. Common mistakes

Putting `aes()` in the wrong place. A variable inside `aes()` creates a mapping; a value outside `aes()` sets a constant. If your legend looks unexpected, check which side of `aes()` your arguments are on (Section 17.2).

Using `|>` instead of `+` after `ggplot()`. The pipe feeds data into `ggplot()`, but after that first call, every addition uses `+`. This trips up every beginner exactly once. If you get an error like "could not find function `geom_point`", you probably wrote `|>` where you needed `+`. The reason ggplot2 uses `+` rather than `|>` is partly historical (ggplot2 predates both magrittr and the native pipe by a decade) and partly conceptual: the pipe transforms data sequentially, while `+` accumulates structure, and these are different composition models (pipeline vs builder).

```
# Wrong
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) |>
  geom_point()

# Right
ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g)) +
  geom_point()
```

Making multiple layers when the data needs reshaping. If you find yourself writing `geom_line(aes(y = col_a))` and then `geom_line(aes(y = col_b))` for different columns, that is a signal to use `pivot_longer()` first (Chapter 16) and map the new column to an aesthetic.

Complex calculations inside `aes()`. Move them to `mutate()` first (Chapter 14). Something like `aes(x = log(value + 1) / max(value))` is hard to read and harder to debug; create the column, then map it.

Overloading a single plot. Too many colors, too many geoms, too much data crammed into one frame. If a plot is hard to read, split it into facets or separate plots. A simpler plot almost always communicates better.

Most ggplot2 errors aren't really about syntax. They are about structure: the data is in the wrong shape, the mapping is in the wrong place, or the wrong composition operator connects the layers. When a plot doesn't look right, check the data and the mappings before adjusting visual parameters, because the grammar tells you where the problem is if you listen to it.

Once the plot is right, you need to get it out of R.

17.10. Saving plots

A ggplot object is a deferred computation: it describes a plot but does not produce pixels until you print or save it. Printing sends the result to the screen; `ggsave()` sends it to a file:

```
ggsave("penguins_scatter.png", width = 8, height = 5, dpi = 300)
```

The file format is inferred from the extension: `.png`, `.pdf`, `.svg`, `.jpg`. For publication, PDF or SVG gives you vector graphics that scale cleanly at any size. For slides and web, PNG at 300 DPI is standard.

You can also pass a stored plot object explicitly:

```
p <- ggplot(penguins, aes(x = bill_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  theme_minimal()

ggsave("penguins_scatter.pdf", plot = p, width = 8, height = 5)
```

The `width` and `height` arguments control the output dimensions in inches (the default unit), and getting these right matters more than any theme adjustment: a plot squeezed into half its natural width produces unreadable axis labels, while a plot stretched too wide scatters the data into a sparse, floating emptiness. Experiment with dimensions before finalizing. It's the last step, but it determines whether anyone can actually read what you built.

> **Opinion**
>
> Always save with `ggsave()`, never with right-click or the RStudio export button. `ggsave()` is reproducible: the same code produces the same file tomorrow. The export button is a one-off action with no record of the dimensions or resolution you chose, and six months from now, when a reviewer asks you to regenerate Figure 3 at higher resolution, you will understand why that matters.

Part III

Part III: Thinking functionally

18 Closures and scope

Call this function three times and watch what happens:

```
f <- function() {
  n <- 0
  n <- n + 1
  n
}

f()
#> [1] 1
f()
#> [1] 1
f()
#> [1] 1
```

Always 1. The counter never advances, because every call starts from scratch: every local variable is born and dies within a single invocation. You could call it a thousand times and nothing would accumulate. So how do you build a function that *remembers*? The answer runs through environments, scoping rules, and a mechanism with a name borrowed from mathematical logic: the closure.

18.1. Environments

An environment maps names to values. Each environment has a parent, forming a chain: when R encounters a name, it checks the current environment first, then walks up the chain until it finds a binding or hits the empty environment at the top. Functions remember things by holding a reference to the environment where they were created, which is why scope matters here.

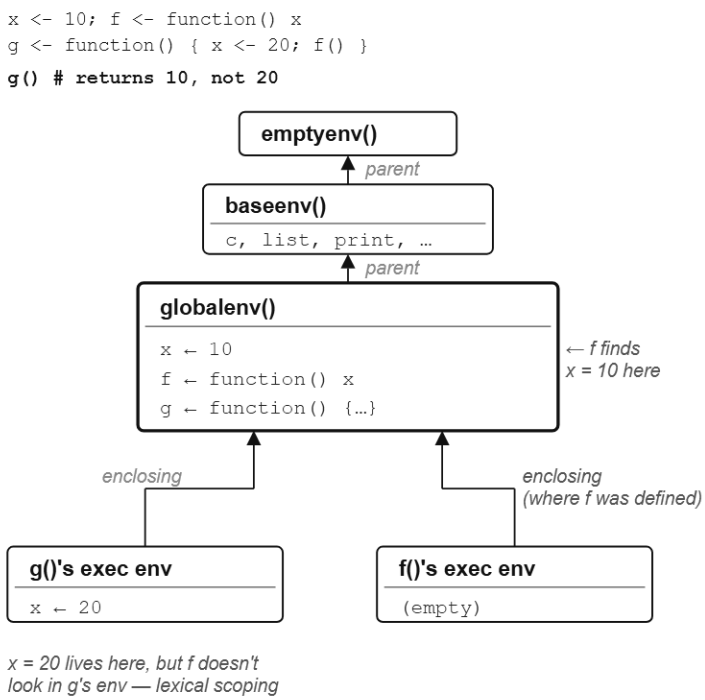


Figure 18.1.: The environment chain: `f` finds `x = 10` in the global environment where it was defined, not `x = 20` in `g`'s execution environment.

You can inspect the current environment with `environment()`, list its contents with `ls()`, and find its parent with `parent.env()`:

```
x <- 42
y <- "hello"
ls()
#> [1] "f" "x" "y"
```

```
environment()
#> <environment: R_GlobalEnv>
```

18.1.1. The global environment

The global environment (`.GlobalEnv`, or equivalently `globalenv()`) is where your interactive work lives. Every time you type `x <- 42` at the console, you create a binding here, at the bottom of the search path, the starting point for name resolution, the default enclosing environment for any function you define interactively. But that default status has consequences.

```
identical(environment(), globalenv())
#> [1] TRUE
```

Three properties make it special. It never gets garbage-collected; it persists for the entire R session. Its parent is not fixed: it shifts whenever you attach a new package with `library()`. And it is the only environment you routinely modify by hand, because functions create and destroy their own execution environments automatically (Section 18.3), while the global environment simply accumulates bindings as you work.

That accumulation lets you explore interactively, building data objects step by step. It also means any function defined in the global environment can see and silently depend on those objects. A function that works in your current session may break in a fresh one because it relied on a global variable you forgot to pass as an argument. In lambda calculus, that function is an *open term*: its result depends on bindings outside its own body. Pass all dependencies as arguments and it becomes a *closed term*, portable and self-contained. But what happens when a name *isn't* passed as an argument?

18.1.2. The search path

When you type `x` at the console, R checks the global environment first; if no match turns up, it moves to the parent, then the parent's parent, walking up the chain until it either finds the name or runs out of environments. This chain is the *search path*:

```
search()
#> [1] ".GlobalEnv"        "package:stats"     "package:graphics"
#> [4] "package:grDevices" "package:utils"     "package:datasets"
#> [7] "package:methods"   "Autoloads"         "package:base"
```

The global environment sits at the bottom, attached packages stack above it, and `package:base` lives near the top. That is why you can call `mean()` without writing `base::mean()`: R walks the chain, finds `mean` in the base package, and uses it. But the search path only governs lookups that start from the global environment. What governs lookups inside a function?

Exercises

1. Run `ls()` in a fresh R session. What do you see? Now assign `a <- 1` and run `ls()` again.
2. Run `search()` and count how many environments are in the chain. Load a package with `library(tools)` and run `search()` again. Where did the new package appear?
3. What does `parent.env(globalenv())` return? What about `parent.env(baseenv())`?

18.2. Lexical scoping

Consider this:

```
x <- 10
f <- function() x
g <- function() { x <- 20; f() }
g()
#> [1] 10
```

The result is 10, not 20. `f` was defined in the global environment, so it looks for `x` there, ignoring the `x <- 20` inside `g` where it was merely *called*. This is *lexical scoping*: a function resolves names where it was defined, not where it was called. The alternative, *dynamic scoping*, searches the call stack instead; the same function would return different results depending on who calls it. R got lexical scoping from Scheme (1975), not from S, which originally used dynamic scoping (Section 2.3). Without it, closures (the subject of this chapter) would not work.

Four rules govern how R resolves names:

1. **Name masking.** Local names shadow names in parent environments. If a function defines `x`, that `x` hides any `x` in the global environment.
2. **Functions vs variables.** R distinguishes function lookups from variable lookups. If you call `f(3)`, R searches for `f` but skips non-function values. This means you can have a variable `c <- 10` and still call `c(1, 2, 3)` to create a vector, because R knows you want the function.
3. **Fresh start.** Every function call gets a fresh execution environment. Variables from a previous call don't carry over.
4. **Dynamic lookup.** R looks up names when the function *runs*, not when it's defined. If a function uses a free variable, its value can change between calls:

```
multiplier <- 2
scale <- function(x) x * multiplier
scale(5)
#> [1] 10
```

```
multiplier <- 10
scale(5)
#> [1] 50
```

`scale` never snapshots the value of `multiplier` at definition time; it reaches into the enclosing environment fresh on every call, which means the function adapts automatically if `multiplier` changes. Occasionally useful. More often, a source of bugs: someone modifies a global variable, and a seemingly unrelated function starts returning different results, with nothing at the call site to explain why.

Exercises

1. Predict the output before running:

```
x <- 1
f <- function() {
  x <- 2
  g <- function() x
  g()
}
f()
```

2. Predict the output:

```
x <- 1
f <- function() {
  g <- function() x
  x <- 2
  g()
}
f()
```

   Why is the result different from what you might expect? (Hint: rule 4.)

3. Can you have a variable named `mean` and still call the function `mean()`? Try it.

18.3. Execution environments

Every time you call a function, R creates a new environment (the *execution environment*) with the function's arguments and local variables as bindings. That is why the counter at the top of this chapter never advances:

```
f <- function() {
  n <- 0
  n <- n + 1
  n
}

f()
#> [1] 1
f()
#> [1] 1
f()
#> [1] 1
```

Call it ten times, a hundred times; you always get 1, because each call gets its own execution environment with its own `n`, and the previous call's `n` vanishes the moment that call returns (rule 3).

The parent of this fresh execution environment is not the environment where the function was *called*; it is the environment where the function was *defined* (the enclosing environment). This is what makes scoping lexical: the parent chain is determined by the structure of the source code, not by which function happened to call which at runtime.

```
x <- "global"

outer <- function() {
  x <- "outer"
  inner <- function() x
  inner()
}

outer()
#> [1] "outer"
```

`inner` was defined inside `outer`, so its enclosing environment is `outer`'s execution environment, and when `inner` looks for `x`, it finds `"outer"`, not `"global"`. But `outer`'s execution environment is temporary. Normally, local variables live and die with the call: when the function returns, its execution environment gets garbage-collected and everything inside it disappears. Unless something keeps it alive. What if `outer` returned `inner` instead of calling it? What if a function escaped the environment where it was born, carrying a reference that prevented the garbage collector from reclaiming it?

Exercises

1. Write a function `fresh()` that creates a local variable `n <- 0`, increments it, and returns it. Call it three times and verify you always get 1.
2. Predict the output:

```
make <- function() {
  a <- 1
  function() a
}
h <- make()
a <- 99
h()
```

18.4. Closures

All R functions are technically closures (they all carry an enclosing environment), but the term earns its keep when a function captures variables from a parent that isn't the global environment. Here is a counter that actually counts:

```
make_counter <- function() {
  n <- 0
  function() {
    n <<- n + 1
    n
  }
}

count <- make_counter()
count()
#> [1] 1
count()
#> [1] 2
count()
#> [1] 3
```

When you call `make_counter()`, R creates an execution environment with `n <- 0`. The inner function is defined in that environment, making it the inner function's enclosing environment, and when `make_counter` returns this inner function, the execution environment would normally be garbage-collected, except the returned function still points to it, so it survives. Each subsequent call to `count()` creates its own fresh execution environment for its local work, but the enclosing environment (the one from `make_counter` that holds `n`) is shared across all calls, persistent as long as `count` exists. That is how `count` remembers its state between invocations.

But notice the `<<-`. That choice is not cosmetic.

`<<-` is the super-assignment operator. Instead of creating a local binding, it searches parent environments for an existing binding named `n` and modifies it in place. Without `<<-`, writing `n <- n + 1` would create a *local* `n` in the execution environment, shadowing the captured one, and the counter would always return 1 (the same broken non-counter from the top of the chapter).

Two counters are independent, because each call to `make_counter()` creates a separate execution environment with its own `n`:

```
a <- make_counter()
b <- make_counter()
a()
#> [1] 1
a()
#> [1] 2
```

```
a()
#> [1] 3
b()
#> [1] 1
```

`a` has counted to 3; `b` has counted to 1. They share no state; each factory call minted a private universe. But the mechanism that keeps these universes alive is the same one that can leak state into the global environment if `<<-` is used carelessly.

Connection to Section 7.4: `make_adder` was a preview. Now you understand *why* the returned function remembers `n`: an environment stays alive because something still points to it.

In lambda calculus, the same thing happens through substitution. In `make_adder(5)`, the returned function `\(x) x + n` has `n` as a *free variable*: a name that is used but not defined inside the function. The closure captures `n = 5`, binding that free variable. In lambda notation, `make_adder` is `( n. x. x + n)`. Applying it to 5 gives `x. x + 5` by substitution (beta reduction). The free variable `n` gets replaced by a concrete value. Closures are the runtime mechanism that makes this substitution real. The name itself comes from this idea: a closure *closes over* its free variables, turning an open term (one with unresolved names) into a closed one (where every variable is accounted for).

Every closure you create in R is a beta reduction frozen mid-step. `make_adder(5)` reduces `( n. x. x + n)` to `x. x + 5`, but the returned function doesn't evaluate further until you give it an `x`. The closure holds the partially reduced term, waiting. When you finally call `add5(3)`, another beta reduction fires: `( x. x + 5)(3)` becomes `3 + 5`, and R evaluates it to `8`. Two reductions, two calls, one answer. R does exactly this internally when it resolves the captured binding `n = 5` in the enclosing environment. But `<<-` gave the closure something lambda calculus never had to worry about: the ability to mutate its captured bindings. That power needs guardrails.

Exercises

1. Create a counter with `make_counter`. Call it five times. Then inspect the captured `n` with `environment(count)$n`.
2. Modify `make_counter` to accept a starting value: `make_counter <- function(start = 0) { ... }`. Verify that `make_counter(10)` starts counting from 11.
3. Write `make_countdown(n)` that counts *down* from `n`. Each call returns the next value. What happens when it reaches 0?

18.5. `<<-` and mutable state

`<<-` is the only way to create mutable state in R's functional world, because it searches parent environments for an existing binding and modifies it there. If it doesn't find the variable in any parent, it creates one in the global environment. That is almost always a mistake.

```
rm(oops, envir = globalenv())  # clean slate
#> Warning in rm(oops, envir = globalenv()): object 'oops' not found
f <- function() {
  oops <<- "surprise"
}
f()
oops
#> [1] "surprise"
```

`oops` was never defined anywhere, so `<<-` walked all the way up the parent chain, found nothing, and quietly created it in the global environment. A side effect invisible at the call site, exactly the kind of hidden dependency that makes code hard to debug.

> **Opinion**
>
> Use `<<-` inside closures, never in top-level scripts. If you find yourself using `<<-` to modify a global variable, you are writing a bug you haven't found yet. The legitimate use case is closures that encapsulate state: counters, caches, accumulators, where the modified variable lives in a private environment invisible to the outside world.

Inside a closure, `<<-` is safe precisely because the variable it modifies lives in the closure's private environment, not in the global environment. Nobody outside the closure can see it or change it (unless they deliberately reach into the environment with `environment(f)$n`, which is an explicit, conscious choice, not an accident).

But safe is not the same as free. When you use `<<-`, the function gains memory — each call can depend on every previous call — and every reader of that function must now track two things: what happens inside the function body, and what state the enclosing environment carries from prior calls. Lambda calculus never had this problem because substitution is the only operation: `(n. x. x + n)(5)` reduces to `x. x + 5`, and the `5` is baked in permanently, not stored in a mutable cell that some future call might overwrite. R's closures give you the substitution *and* the mutable cell. That combination is what makes the counter work — each call to `count()` modifies `n` in the enclosing environment, and the next call sees the updated value. It is also what makes `<<-` bugs invisible from the call site. If someone writes `n <- n + 1` instead of `n <<- n + 1` inside `make_counter`, the counter returns 1 on every call, silently, with no error message, because `<-` creates a local `n` that shadows the captured one. The bug is one character wide and produces no warning.

```
rm(oops, envir = globalenv())
```

Exercises

1. Write a closure `make_accumulator(start)` that returns a function. Each call adds its argument to a running total and returns the new total. Test: `acc <- make_accumulator(0); acc(5); acc(3); acc(10)` should return 5, 8, 18.

2. What happens if you use `<-` instead of `<<-` inside the closure? Try it with the counter example.

18.6. Closures as portable state

A closure bundles a function with its data: no global variables, no side effects visible from outside. Consider a function that must remember every value it has ever seen, a running mean that updates with each new observation:

```
make_running_mean <- function() {
  total <- 0
  count <- 0
  function(x) {
    total <<- total + x
    count <<- count + 1
    total / count
  }
}

avg <- make_running_mean()
avg(10)
#> [1] 10
avg(20)
#> [1] 15
avg(6)
#> [1] 12
```

A factory function creates an environment, returns an inner function that captures it, and `<<-` lets the inner function modify the captured state. The state lives in a private environment that travels with the function: no outside code can see or tamper with it, and it persists across calls.

The global-variable approach does none of this:

```
# Don't do this
total <- 0
count <- 0

running_mean <- function(x) {
  total <<- total + x
  count <<- count + 1
  total / count
}
```

This version pollutes the global environment with `total` and `count`, which any other code can read or modify; you cannot have two independent running means; and if you forget to

reset them, your next analysis starts with stale state. The closure version has none of these problems.

Practical uses for closures reach well beyond counters and accumulators:

- **Function factories** (Chapter 20): parameterized families of functions. `make_adder`, `make_multiplier`, and their real-world cousins in ggplot2 themes, statistical tests, and data transformations.
- **Memoization:** cache expensive results so they're computed only once.
- **Callbacks and event handlers:** carry context without globals, common in Shiny applications.
- **Encapsulated modules:** a list of closures sharing a private environment.

That last pattern deserves a concrete example, because a list of closures sharing an environment is functionally equivalent to an object with methods and private fields:

```
make_bank_account <- function(balance = 0) {
  list(
    deposit  = function(amount) {
      balance <<- balance + amount
      invisible(balance)
    },
    withdraw = function(amount) {
      balance <<- balance - amount
      invisible(balance)
    },
    check    = function() balance
  )
}

acct <- make_bank_account(100)
acct$deposit(50)
acct$withdraw(30)
acct$check()
#> [1] 120
```

Three closures share a single environment containing one private variable, `balance`. From the outside, `acct` behaves like an object with methods; from the inside, it is just functions closing over a shared environment. Peter Norvig observed that "closures are poor man's objects, and objects are poor man's closures." R6 works this way internally: each R6 object is an environment, and its methods are functions whose enclosing environment is that same environment. Chapter 24 explores where that equivalence leads.

Exercises

1. Build a `make_running_mean` closure. Feed it the values 4, 8, 12. Verify the running mean is 4, 6, 8.

2. Create two independent running means, `avg1` and `avg2`. Feed different values to each and verify they don't interfere.
3. Extend `make_bank_account` with a `statement` function that returns a character vector of all transactions (deposit or withdrawal). You'll need a `history` variable in the enclosing environment.

18.7. Inspecting environments

Closures are easier to understand when you can look inside them. `environment(f)` returns the enclosing environment of a function, `ls()` lists what's in it, and you can access captured variables directly with `$`:

```
count <- make_counter()
count()
#> [1] 1
count()
#> [1] 2

environment(count)
#> <environment: 0x000001e576b26740>
ls(environment(count))
#> [1] "n"
environment(count)$n
#> [1] 2
```

The counter has been called twice, so `n` is 2. Watch it change:

```
count()
#> [1] 3
environment(count)$n
#> [1] 3
```

Now `n` is 3, incremented by the call.

For a richer view, `rlang::env_print()` shows the environment's contents, parent, and memory address:

```
rlang::env_print(environment(count))
```

These tools are not for production code (reaching into a closure's environment breaks its encapsulation), but for learning they are the fastest way to see what a closure actually holds. Make one, inspect its environment, call it, inspect again. A closure that holds state and a list of closures that share an environment start to look a lot like an object with methods, which is exactly the connection Section 24.2 explores.

Exercises

1. Create a running mean with `make_running_mean`. Feed it three values. Then inspect `environment(avg)$total` and `environment(avg)$count` to verify the internal state.
2. Create two counters. Inspect their environments and confirm they have different `n` values after calling them different numbers of times.
3. What does `environment(mean)` return? Why is it different from `environment(count)`?

19 Iteration without loops

You have a list of three data frames and you want to compute the mean of a column in each one. Six lines of `for` loop, or one line of `map()`. The `map()` version is shorter, but the real difference is structural.

Functions are values (Chapter 7) and you can pass them to other functions (Section 7.1). A *functional* builds on that idea: it is a higher-order function that handles the iteration for you. You give it a function and a collection; it applies the function to each element, manages the indexing and pre-allocation, and returns the results.

This chapter uses three packages. Load them now if you are following along:

```
library(palmerpenguins)
library(purrr)
library(dplyr)
```

We will work with the penguins data split by species throughout:

```
penguins_split <- split(penguins, penguins$species)
names(penguins_split)
#> [1] "Adelie"    "Chinstrap" "Gentoo"
```

Three data frames, one per species. Almost everything in this chapter amounts to doing something to each of them, and the question is always the same: how much of the work should you spell out yourself, and how much should the language handle?

19.1. The problem with `for` loops

Suppose you want the mean body mass for each species. Here is the loop version:

```
results <- vector("list", length(penguins_split))
for (i in seq_along(penguins_split)) {
  results[[i]] <- mean(penguins_split[[i]]$body_mass_g, na.rm = TRUE)
}
names(results) <- names(penguins_split)
results
```

```
#> $Adelie
#> [1] 3700.662
#>
#> $Chinstrap
#> [1] 3733.088
#>
#> $Gentoo
#> [1] 5076.016
```

Six lines to say "compute the mean mass for each group." Look at how much of that is bookkeeping: pre-allocate a container, set up an index, store results at the right position, copy names over at the end. The actual computation, `mean(df$body_mass_g, na.rm = TRUE)`, sits buried in the middle, surrounded by scaffolding that has nothing to do with the question you asked.

A functional says the same thing in one line:

```
map_dbl(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE))
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

Same answer. No index variable, no pre-allocation, no name copying. You said what to compute; `map_dbl` handled the iteration.

The problem with `for` loops is not performance; in modern R, a well-written loop is fast. The problem is that loops mix the *mechanism* of iteration (allocate, index, store) with the *content* of the computation (compute a mean). Functionals separate the two. The mechanism disappears inside the functional; the content lives in the function you pass to it. And once those two concerns are separated, each becomes easier to read, test, and reuse.

> **Opinion**
>
> For loops are not bad R. They are verbose R. If you find yourself writing the same loop scaffolding three or four times in a project, a functional is waiting to collapse all that boilerplate into a single expressive line.

Exercises

1. Write a `for` loop that computes the number of rows in each element of `penguins_split`. Then rewrite it using `map_int()` and `nrow`.
2. What happens if you forget to pre-allocate `results` and instead grow it with `results <- c(results, value)` inside a loop? Why is this slower?

19.2. `lapply()` and `sapply()`: base R functionals

The idea of handing a function to something that applies it to every element in a collection is older than R, older than S, older than most programming languages people still use. John McCarthy's 1960 paper described `mapcar`, a Lisp function that takes a function and a list and returns a new list with the function applied to each element. The name `map` became standard in later languages (Scheme, ML, Haskell), and R inherited the concept twice: once through `lapply()` in base R, and again through `purrr::map()` in the tidyverse. The mechanism is always the same. You separate *what to do* from *how to iterate*, and let the language handle the second part. `lapply()` is R's oldest version of that idea:

```
lapply(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE))
#> $Adelie
#> [1] 3700.662
#>
#> $Chinstrap
#> [1] 3733.088
#>
#> $Gentoo
#> [1] 5076.016
```

`lapply(x, f)` applies `f` to each element of `x` and returns a list. Always a list. That predictability is its strength.

`sapply()` does the same thing, but tries to "simplify" the result:

```
sapply(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE))
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

When every call to `f` returns a single value, `sapply()` hands you a named vector. But if `f` returns different lengths for different elements, `sapply()` falls back to a list, silently. You cannot predict the return type without knowing the output of `f` for every element of `x`. Fine at the console; unreliable inside a function that other code depends on.

`vapply()` solves this by making you declare the expected type:

```
vapply(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE), numeric(1))
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

The third argument, `numeric(1)`, says "I expect each call to return a single numeric value." If any call returns something else, you get an error instead of a silent type change.

Two other base R functionals are worth knowing. `tapply(x, group, f)` applies `f` to subsets of `x` defined by `group`:

```
tapply(penguins$body_mass_g, penguins$species, mean, na.rm = TRUE)
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

And `mapply(f, x, y)` iterates over multiple inputs in parallel:

```
mapply(paste, c("one", "two", "three"), c("fish", "cat", "bird"))
#>          one          two        three
#>   "one fish"    "two cat" "three bird"
```

> **Opinion**
>
> Use `lapply()` when you want a list. Use `vapply()` when you want a vector and want safety. Never use `sapply()` in non-interactive code: its return type depends on the data, and that is a bug hiding in your pipeline, waiting for the one input that changes everything.

Exercises

1. Use `lapply()` and `nrow` to get the number of rows in each element of `penguins_split`.
2. Use `vapply()` to compute the median flipper length for each species. Declare the expected output type.
3. What does `sapply(list(), mean)` return? What about `vapply(list(), mean, numeric(1))`? Which is safer?

19.3. `purrr::map()`: the tidyverse functional

The purrr package provides `map()`, which does the same thing as `lapply()`:

```
map(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE))
#> $Adelie
#> [1] 3700.662
#>
#> $Chinstrap
#> [1] 3733.088
#>
#> $Gentoo
#> [1] 5076.016
```

Apply a function to each element, return a list. So why bother with a new package?

Three reasons. First, typed variants: `map_dbl()` returns a double vector, `map_chr()` returns a character vector, and if the function returns the wrong type, you get an error. No more guessing, no more silent simplification.

```
map_dbl(penguins_split, \(df) mean(df$body_mass_g, na.rm = TRUE))
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

Second, consistency with the pipe. `map()` and its variants are designed to sit naturally in a pipeline:

```
penguins_split |>
  map_dbl(\(df) mean(df$body_mass_g, na.rm = TRUE))
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

Third, you may encounter the formula shorthand in older code: `~ mean(.x$body_mass_g, na.rm = TRUE)`. This was purrr's anonymous function syntax before R 4.1 introduced `\(x)`. You will see it in existing codebases. Prefer `\(x)` in new code.

`map()` returns a list. Always. `map_dbl()` returns a double vector. Always. This type stability is the core advantage over `sapply()`, and it matters more than it sounds: in a long pipeline, knowing the exact shape of every intermediate result is the difference between debugging for five seconds and debugging for an hour.

> **Functors and the shape of iteration**
>
> `map()` embodies a pattern from category theory: the *functor*. `map(xs, f)` applies `f` to each element of `xs`, preserving the container structure (a list of three elements in, a list of three elements out). A functor lifts a function between values (`f :: A -> B`) to a function between containers (`map(f) :: List(A) -> List(B)`). You do not need the theory to use `map()`, but recognizing the pattern explains why `map()` feels the same whether you apply it to a list, a vector, or a set of data frame columns with `across()`: it is the same structural idea in different containers.

A `for` loop iterates by maintaining a counter variable: `i` goes from 1 to `n`, and the body runs once per increment. You declare the counter, increment it, read it. With `map()`, the counter disappears. The list has three elements, so the function runs three times; the structure itself determines how many applications occur. You stop managing the iteration and let the data's shape manage it for you.

The typed variants enforce something even stricter.

Exercises

1. Use `map_dbl()` to compute the standard deviation of `body_mass_g` for each species in `penguins_split`.
2. Use `map_chr()` to extract the first value of `species` from each element of `penguins_split`. (Hint: `\(df) as.character(df$species[1])`.)
3. Rewrite the `for` loop from Section 19.1 using `map_dbl()`. Compare the two versions for readability.

19.4. Typed variants

The typed variants of `map()` enforce both the type and the length of each result:

- `map_dbl(.x, .f)`: each call to `.f` must return a single double. Result is a numeric vector.
- `map_chr(.x, .f)`: each call must return a single string. Result is a character vector.
- `map_lgl(.x, .f)`: each call must return a single logical. Result is a logical vector.
- `map_int(.x, .f)`: each call must return a single integer. Result is an integer vector.

```
map_int(penguins_split, nrow)
#>    Adelie Chinstrap    Gentoo
#>       152        68       124
```

```
map_lgl(penguins_split, \(df) any(is.na(df$body_mass_g)))
#>    Adelie Chinstrap    Gentoo
#>      TRUE     FALSE      TRUE
```

If `.f` returns the wrong type or a vector of length other than one, you get an error:

```
map_dbl(penguins_split, \(df) df$body_mass_g)
#> Error in `map_dbl()`:
#> i In index: 1.
#> i With name: Adelie.
#> Caused by error:
#> ! Result must be length 1, not 152.
```

When the output does not match your expectation, you find out immediately, not three functions downstream when something produces `NULL` instead of a number.

For results that are data frames, the older `map_dfr()` is deprecated. The current pattern is `map()` followed by `list_rbind()`:

```
penguins_split |>
  map(\(df) tibble(
    species = df$species[1],
    mean_mass = mean(df$body_mass_g, na.rm = TRUE),
    n = nrow(df)
  )) |>
  list_rbind()
#> # A tibble: 3 x 3
#>   species   mean_mass     n
#>   <fct>         <dbl> <int>
#> 1 Adelie        3701.   152
#> 2 Chinstrap     3733.    68
#> 3 Gentoo        5076.   124
```

`map()` returns a list of data frames, and `list_rbind()` stacks them into one. But what about iterating over more than one thing at a time?

Exercises

1. Use `map_int()` and `nrow` to count the rows in each element of `penguins_split`.
2. Use `map_lgl()` to check which elements of `penguins_split` have more than 100 rows.
3. Use `map()` and `list_rbind()` to build a summary data frame with one row per species, containing the species name, median bill length, and median bill depth.

19.5. Iterating over multiple inputs

Sometimes a single list is not enough. You need to walk over two inputs in lockstep, pulling one element from each on every step. `map2()` handles this:

```
species_names <- names(penguins_split)
species_counts <- map_int(penguins_split, nrow)

map2_chr(species_names, species_counts, \(name, n) paste(name, ":", n,
"penguins"))
#> [1] "Adelie : 152 penguins"   "Chinstrap : 68 penguins"
#> [3] "Gentoo : 124 penguins"
```

`map2(.x, .y, .f)` calls `.f(x[[1]], y[[1]])`, then `.f(x[[2]], y[[2]])`, and so on. The two inputs are consumed in parallel, not in a nested grid.

For three or more inputs, use `pmap()`. You supply a list of vectors (or a data frame, since a data frame is a list of columns):

```
params <- list(
  mean = c(0, 5, -3),
  sd = c(1, 2, 0.5),
  n = c(3, 3, 3)
)

set.seed(42)
pmap(params, \(mean, sd, n) rnorm(n, mean, sd))
#> [[1]]
#> [1]  1.3709584 -0.5646982  0.3631284
#>
#> [[2]]
#> [1] 6.265725 5.808537 4.787751
#>
#> [[3]]
#> [1] -2.244239 -3.047330 -1.990788
```

Each row of parameters becomes one call. `pmap()` is the general case; `map2()` is the special case for exactly two inputs.

The base R equivalents are `Map(f, x, y)` (which returns a list, like `map2()`) and `mapply(f, x, y)` (which simplifies, like `sapply()`). The same warning applies: `mapply` is unpredictable in return type.

Where `pmap()` really shines is in configuration-driven workflows, places where you want to fit different models to different subsets with different formulas, all controlled by a single table of parameters. Suppose you want to regress body mass on flipper length for each species, but with a different formula for each:

```
configs <- tibble(
  data = penguins_split,
  formula = list(
    body_mass_g ~ flipper_length_mm,
    body_mass_g ~ flipper_length_mm + bill_length_mm,
    body_mass_g ~ flipper_length_mm * bill_length_mm
  )
)

models <- pmap(configs, \(data, formula) lm(formula, data = data))
map_dbl(models, \(m) summary(m)$r.squared)
#>    Adelie Chinstrap    Gentoo
#> 0.2192128 0.4688941 0.5923485
```

`pmap()` iterates over the rows of the configuration table, fitting one model per species. The entire experiment lives in one tibble, and adding a fourth species or a fourth formula means adding a row, not rewriting the loop.

Exercises

1. Use `map2_chr()` to paste together the vectors `c("one", "two", "three")` and `c("fish", "cat", "bird")` with a space in between.
2. Create a list of three means and three standard deviations. Use `map2_dbl()` to generate one random normal value for each pair. (Hint: `\(m, s) rnorm(1, m, s)`.)
3. A data frame is a list of columns. What does `pmap_dbl(data.frame(x = 1:3, y = 4:6), \(x, y) x + y)` return?

19.6. Side effects: `walk()`

Everything so far has been about functions that *compute* something: they take input and produce output. But some functions exist for what they *do*, not what they *return*: printing a message, writing a file, drawing a plot. For these, use `walk()`.

```
walk(names(penguins_split), \(name) cat(name, "\n"))
#> Adelie
#> Chinstrap
#> Gentoo
```

`walk(.x, .f)` applies `.f` to each element of `.x`, but instead of collecting results into a list, it returns `.x` invisibly. The point is the side effect, not the return value.

The two-input version, `walk2()`, is especially useful for writing files:

```
file_paths <- paste0("data/", names(penguins_split), ".csv")
walk2(penguins_split, file_paths, \(df, path) write.csv(df, path, row.names =
FALSE))
```

Each data frame gets written to its own file. `walk2()` iterates over the data frames and paths in parallel, calling `write.csv()` for each pair.

Use `walk()` whenever `.f` does something rather than computes something. If you use `map()` for side effects, you end up with a list of `NULL` values cluttering your console, which is the language telling you that you reached for the wrong tool.

Because `walk()` returns its input invisibly, it slots cleanly into the middle of a pipeline:

```
penguins_split |>
  walk(\(df) cat("Species:", as.character(df$species[1]), "- n =", nrow(df),
  "\n")) |>
  map_dbl(\(df) mean(df$body_mass_g, na.rm = TRUE))
#> Species: Adelie - n = 152
#> Species: Chinstrap - n = 68
#> Species: Gentoo - n = 124
#>    Adelie Chinstrap    Gentoo
#>  3700.662  3733.088  5076.016
```

The `walk()` prints a line for each species, then passes `penguins_split` through unchanged to `map_dbl()`. Side effects and computation in one pipeline, each handled by the right tool. But there is a category of iteration that no functional – not `map()`, not `walk()` – can handle cleanly.

Exercises

1. Use `walk()` to print the column names of each element of `penguins_split`. (They should all be the same.)
2. Use `walk2()` and `cat()` to print lines like `"Adelie has 152 rows"` for each species.

19.7. When loops are fine

Not everything should be a functional. Loops earn their place in three situations.

When iterations depend on each other. If the result of iteration `i` feeds into iteration `i + 1`, a functional cannot help you, because functionals assume independence between elements. Simulation chains, iterative algorithms, and accumulating state all need explicit loops. (purrr's `reduce()` and `accumulate()` handle some of these sequential cases, and you will meet them in Chapter 21. In the language of category theory, a fold that tears down a list one element at a time is a *catamorphism*, the most general recursion pattern: any loop that starts with an accumulator, walks a list, and returns the accumulator is a catamorphism in disguise.)

```
# Random walk: each step depends on the previous
x <- numeric(20)
x[1] <- 0
for (i in 2:20) {
  x[i] <- x[i - 1] + rnorm(1)
}
x
#>  [1]  0.0000000 -0.0627141  1.2421556  3.5288009  2.1399402  1.8611515
#>  [7]  1.7278301  2.3637805  2.0795276 -0.5769278 -3.0173947 -1.6972814
#> [13] -2.0039200 -3.7852284 -3.9571458 -2.7424711 -0.8472776 -1.2777467
#> [19] -1.5350161 -3.2981792
```

When the loop body is complex. If you need `next` to skip elements, `break` to stop early, or multiple conditional branches that depend on runtime state, a loop is often clearer than contorting the logic into a function passed to `map()`.

When you are still learning. A loop you understand is better than a functional you do not. Use `map()` when the pattern clicks; until then, write the loop, get the right answer, and refactor later when the shape of the problem becomes obvious.

> **Opinion**
>
> Replace a loop with a functional when the pattern is "do this to each thing independently." Keep a loop when iterations talk to each other.

19.8. `across()` revisited

In Section 21.7, you saw `across()` for applying a function to multiple columns inside a dplyr verb. So what is `across()`, exactly? It is `map()` thinking applied inside a data frame pipeline.

```
penguins |>
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)))
```

```
#> # A tibble: 1 x 5
#>   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  year
#>            <dbl>         <dbl>             <dbl>       <dbl> <dbl>
#> 1           43.9          17.2              201.       4202. 2008.
```

`across(.cols, .fns)` takes a column selection and a function (or list of functions) and applies the function to each selected column. It is iteration over columns, the same structural idea as `map()` iterating over list elements, but specialized for the column-wise case inside dplyr verbs.

Multiple functions at once:

```
penguins |>
  summarise(across(
    where(is.numeric),
    list(mean = \(x) mean(x, na.rm = TRUE), sd = \(x) sd(x, na.rm = TRUE))
  ))
#> # A tibble: 1 x 10
#>   bill_length_mm_mean bill_length_mm_sd bill_depth_mm_mean bill_depth_mm_sd
#>                 <dbl>             <dbl>              <dbl>            <dbl>
#> 1                43.9              5.46               17.2             1.97
#> # i 6 more variables: flipper_length_mm_mean <dbl>,
#> #   flipper_length_mm_sd <dbl>, body_mass_g_mean <dbl>,
#> #   body_mass_g_sd <dbl>, year_mean <dbl>, year_sd <dbl>
```

Combined with `group_by()`:

```
penguins |>
  group_by(species) |>
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)))
#> # A tibble: 3 x 6
#>   species   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  year
#>   <fct>              <dbl>         <dbl>             <dbl>       <dbl> <dbl>
#> 1 Adelie              38.8          18.3              190.       3701. 2008.
#> 2 Chinstrap           48.8          18.4              196.       3733. 2008.
#> 3 Gentoo              47.5          15.0              217.       5076. 2008.
```

`across()` replaces the older `summarise_at()`, `summarise_if()`, and `summarise_all()`, which are now superseded. If you encounter them in older code, they do the same thing with less flexible syntax.

Under the hood, `across()` iterates over the selected columns, applies the function to each one, and assembles the results. That is the same thing `map()` does over list entries and `sapply()` does over vector values. The `for` loop at the start of this chapter, `lapply()`, `map_dbl()`, `across()`: each separates *what to do* from *how to iterate*, and each walks a different container with the same logic. The `for` loop, `lapply()`, `map_dbl()`, and `across()` all apply a function to each element of a collection; they differ only in which container they walk and what they return.

Exercises

1. Use `across()` inside `summarise()` to compute the `median()` of every numeric column in `penguins`, removing `NA` values.
2. Use `across()` with a list of two functions to compute both the `min()` and `max()` of every numeric column, grouped by species.
3. What does `across(where(is.character), n_distinct)` compute when used inside `summarise()`? Try it on `penguins`.

20 Function factories

Suppose you need twelve formatting functions: one for dollars, one for euros, one for percentages, one for each of nine other currencies your client cares about. You could write twelve functions by hand, each identical except for a prefix string. Or you could write one function that manufactures the other twelve. In Section 7.4, you saw `make_adder`, a function that returns a function; in Chapter 18, you learned why it works, because the returned function is a closure that captures its creation environment. This chapter puts both ideas to serious use, building entire families of functions from a single template.

20.1. The pattern

A function factory is a function that returns a function:

```
power <- function(exponent) {
  function(x) x ^ exponent
}

square <- power(2)
cube <- power(3)
```

```
square(5)
#> [1] 25
cube(5)
#> [1] 125
```

When you call `power(2)`, R creates an execution environment where `exponent` is 2, then returns an anonymous function that closes over that environment. `square` remembers `exponent = 2` forever; `cube` remembers `exponent = 3`. Same logic, different parameters. The factory produces a *family* of related functions, and each member of that family carries its own private copy of the parameter that distinguishes it from its siblings.

This is exactly `make_adder` from Section 7.4, with exponentiation instead of addition. That section introduced the idea without naming it. The name is *function factory*, and the functions it produces are *closures*.

Function factories are *partial application* made explicit: `power(2)` fixes one argument of a two-argument operation and returns a function of the remaining argument. In lambda calculus,

`power` is `e. x. x^e`; applying it to 2 gives `x. x^2`. This decomposition of multi-argument functions into chains of single-argument functions is called *currying* (Schonfinkel 1924, Curry 1958). In Haskell, all functions are curried by default: `add 5 3` is really `(add 5) 3`, where `add 5` returns a function. R's function factories are manual currying. In combinatory logic, the simplest factory is the K combinator (`K = x. y. x`): it takes a value and returns a function that always produces that value, ignoring its argument. `make_formatter("$")` is K applied: it captures `"$"` and returns a function that ignores everything except its argument.

Exercises

1. Write a function factory `make_multiplier` that takes a `factor` and returns a function that multiplies its argument by `factor`. Create `double` and `triple` and test them on the number 7.
2. What does `power(1)` return? Is it the identity function? Test it.
3. Write a factory `make_greeter` that takes a greeting string and returns a function that takes a name and produces the greeting. `make_greeter("Hello")("Alice")` should return `"Hello, Alice"`.

20.2. The lazy evaluation trap

Here is a bug that will cost you an afternoon if you have never seen it before. R evaluates arguments lazily, meaning they are not computed until they are actually used, and this seemingly innocent optimization has consequences for factories that are anything but innocent (the *lazy evaluation trap*):

```
exp <- 2
sq <- power(exp)
exp <- 3
sq(5)
#> [1] 125
```

That returns 125, not 25. Why? Because the variable `exponent` inside the factory is a *promise* pointing to `exp` in the calling environment. When `sq(5)` finally forces the evaluation of `exponent`, `exp` has already been changed to 3. The factory captured a promise, not a value.

The loop version is worse:

```
fns <- list()
for (i in 1:3) {
  fns[[i]] <- function(x) x ^ i
}

fns[[1]](2)
#> [1] 8
fns[[2]](2)
```

```
#> [1] 8
fns[[3]](2)
#> [1] 8
```

All three return 8. By the time any of them is called, `i` is 3.

The fix is `force()`:

```
power <- function(exponent) {
  force(exponent)
  function(x) x ^ exponent
}
```

`force(exponent)` evaluates the argument immediately, capturing the value rather than the promise, and now the factory works correctly:

```
exp <- 2
sq <- power(exp)
exp <- 3
sq(5)
#> [1] 25
```

25, as expected. The loop version needs `force()` too, but the idiomatic solution is `lapply`, which sidesteps the problem entirely:

```
fns <- lapply(1:3, \(i) function(x) x ^ i)

fns[[1]](2)
#> [1] 2
fns[[2]](2)
#> [1] 4
fns[[3]](2)
#> [1] 8
```

Why does this work when the `for` loop didn't? Because the anonymous function `\(i)` creates a new scope for each iteration. When `lapply` calls `\(i)` with the value `1`, that call gets its own execution environment where `i` is `1`, and the inner `function(x) x ^ i` closes over that environment. The next call gets a fresh environment where `i` is `2`, and so on. Each manufactured function captures a different `i` in a different environment, whereas in the `for` loop there is only one `i` in one environment and every function points to the same variable. The `\(i)` wrapper is doing what `force()` does, but structurally: by binding `i` as a function argument, it forces evaluation at the moment of the call.

> **Opinion**
>
> Every factory should `force()` every argument that the manufactured function uses. Make this a habit, not a debugging exercise. The lazy evaluation trap is one of the most common sources of subtle bugs in R, and `force()` costs nothing.

Exercises

1. Predict the output of the following code, then run it to check:

```
val <- 10
make_adder <- function(n) function(x) x + n
add_val <- make_adder(val)
val <- 20
add_val(1)
```

2. Fix the `make_adder` factory above with `force()`. Verify that changing `val` after creating `add_val` no longer affects the result.
3. Why does `lapply(1:3, \(i) function(x) x ^ i)` work without an explicit `force()` call?

20.3. Practical factories

The pattern shows up wherever you find yourself repeating the same logic with different parameters: formatting, filtering, statistical transformations, anywhere a family of behaviors differs by a handful of constants.

Parameterized formatters. A factory can bake a prefix and suffix into a formatting function:

```
make_formatter <- function(prefix, suffix = "") {
  force(prefix)
  force(suffix)
  function(x) paste0(prefix, x, suffix)
}

usd <- make_formatter("$")
pct <- make_formatter("", "%")
```

```
usd(42)
#> [1] "$42"
pct(0.15)
#> [1] "0.15%"
```

Threshold filters. A factory can produce filter functions for different cutoffs:

```
above <- function(threshold) {
  force(threshold)
  function(x) x[x > threshold]
}

above_zero <- above(0)
above_zero(c(-2, 0, 3, -1, 5))
#> [1] 3 5
```

Statistical function families. The Box-Cox transformation is parameterized by a single value, lambda, which makes it a natural fit for a factory:

```
box_cox <- function(lambda) {
  force(lambda)
  if (lambda == 0) {
    \(x) log(x)
  } else {
    \(x) (x ^ lambda - 1) / lambda
  }
}

bc1 <- box_cox(1)
bc0 <- box_cox(0)

bc1(c(1, 2, 4))
#> [1] 0 1 3
bc0(c(1, 2, 4))
#> [1] 0.0000000 0.6931472 1.3862944
```

Notice something subtle about that `if` statement: it runs once, when the factory is called, not every time the produced function is called. The factory picks the right formula at creation time and returns a function that does no branching at all.

This is staged computation: you separate the cost of *choosing* behavior from the cost of *running* behavior. The `if` branch evaluates once, during construction; the produced function evaluates many times, during use, and pays nothing for the choice that was already made. The same pattern drives much of ggplot2. When you call `scale_color_brewer(palette = "Set1")`, the factory runs once at plot-build time, baking the palette into a function that maps data values to colors. If it re-evaluated the palette choice every time it colored a point, the plot would slow to a crawl on large datasets. ggplot2 gains its speed from that separation: the decision moves from the data path (runs per point) to the construction path (runs once). Every factory you write makes a version of the same trade. The returned function carries less weight because the decision has already been made, baked into the closure's environment at creation time, and never revisited.

The next section starts from the other direction: instead of building a function from

parameters, you take a function that already exists and modify its behavior.

Exercises

1. Write a factory `between` that takes `low` and `high` and returns a function that keeps only elements of a vector that fall in the range `(low, high)`. Test it on `1:20` with bounds 5 and 15.
2. Write a factory `make_counter` that returns a function with no arguments. Each time the returned function is called, it should return the next integer (1, 2, 3, …). Hint: use `<<-` to modify a variable in the enclosing environment.

20.4. Memoization

Sometimes you already have the right function, and the only problem is speed. Memoization wraps a function so it caches its results: the first call computes normally, but subsequent calls with the same arguments return the cached value instantly, skipping the computation entirely.

```
library(memoise)

slow_square <- function(x) {
  Sys.sleep(1)
  x ^ 2
}

fast_square <- memoise(slow_square)
```

```
system.time(fast_square(10))
#>    user  system elapsed
#>    0.01    0.00    1.03
system.time(fast_square(10))
#>    user  system elapsed
#>       0       0       0
```

The first call takes about a second. The second is instant, because `memoise` recognizes the argument and returns the cached result without re-executing the function body.

`forget()` clears the cache:

```
forget(fast_square)
#> [1] TRUE
```

When to memoize: expensive computations with repeated inputs, API calls, simulations, recursive algorithms like Fibonacci, file parsing.

When not to: functions with side effects (plotting, writing files), functions whose inputs are large or unique every time (the cache grows without bound), functions that depend on external state (database contents, system time). Memoization trades memory for speed, and it only pays off when inputs recur.

Exercises

1. Write a function `slow_sum` that takes a vector, sleeps for 1 second, and returns the sum. Memoize it. Call it twice with the same input and verify the second call is fast.
2. What happens if you memoize `rnorm`? Try `memo_rnorm <- memoise(rnorm); memo_rnorm(5); memo_rnorm(5)`. Is this useful?

20.5. Function operators

What do you do when a function might fail on some inputs, but you need to apply it to hundreds of values and cannot afford to let one error kill the whole pipeline? You could wrap every call in `tryCatch`. Or you could wrap the function itself once, producing a new function that handles errors gracefully everywhere it is called.

A function operator takes a function as input and returns a modified function as output. Like a factory, but the raw material is a function rather than a number or string. This pattern is called a "decorator" in Python (where `@decorator` syntax makes it first-class) and appeared in the Gang of Four design patterns book (1994) as the Decorator Pattern. R has no `@`-syntax, but `safely()`, `memoise()`, and `with_logging()` are all decorators: they modify behavior by wrapping, without changing the original.

`safely()` wraps a function so it never errors, returning instead a list with `$result` and `$error`:

```
library(purrr)

safe_log <- safely(log)
safe_log(10)
#> $result
#> [1] 2.302585
#>
#> $error
#> NULL
safe_log("a")
#> $result
#> NULL
#>
#> $error
#> <simpleError in .f(...): non-numeric argument to mathematical function>
```

`possibly()` is simpler: it returns a default value on error:

```
careful_log <- possibly(log, otherwise = NA)
careful_log(10)
#> [1] 2.302585
careful_log("a")
#> [1] NA
```

`quietly()` captures messages, warnings, and output as list components instead of printing them:

```
quiet_log <- quietly(log)
quiet_log(-1)
#> $result
#> [1] NaN
#>
#> $output
#> [1] ""
#>
#> $warnings
#> [1] "NaNs produced"
#>
#> $messages
#> character(0)
```

You can write your own operators just as easily:

```
with_logging <- function(f) {
  force(f)
  function(...) {
    cat("Calling function with", length(list(...)), "argument(s)\n")
    f(...)
  }
}

logged_mean <- with_logging(mean)
logged_mean(1:10)
#> Calling function with 1 argument(s)
#> [1] 5.5
```

Notice the `force(f)` call. Function operators are factories whose input happens to be a function, so the same lazy evaluation trap from Section 20.2 applies: without `force()`, `f` is a promise, and if the variable it points to changes before the operator's result is called, you get the wrong function.

Exercises

1. Use `safely()` and `map()` to apply `log()` to the list `list(1, -1, "a", 10)`. Extract the results and the errors separately.
2. Write a function operator `with_timer` that wraps a function so it prints the elapsed time each time it is called. Test it with `Sys.sleep`.
3. What does `possibly(possibly(log, NA), NA)` do? Is double-wrapping useful?

20.6. Composing factories and operators

Each of these patterns (factories, operators, functionals like `map()`) does one small thing. The real power emerges when you compose them:

```
safe_log <- safely(log)
results <- map(list(1, -1, "a", 10), safe_log)
#> Warning in .f(...): NaNs produced

str(results)
#> List of 4
#>  $ :List of 2
#>   ..$ result: num 0
#>   ..$ error : NULL
#>  $ :List of 2
#>   ..$ result: num NaN
#>   ..$ error : NULL
#>  $ :List of 2
#>   ..$ result: NULL
#>   ..$ error :List of 2
#>   .. ..$ message: chr "non-numeric argument to mathematical function"
#>   .. ..$ call   : language .f(...)
#>   .. ..- attr(*, "class")= chr [1:3] "simpleError" "error" "condition"
#>  $ :List of 2
#>   ..$ result: num 2.3
#>   ..$ error : NULL
```

`safely(log)` is a function operator applied to `log`; the result is passed to `map()`, a functional from Section 7.1. Three concepts from three chapters, working together in a single pipeline. What if you need both safety *and* logging on the same function?

```
safe_logged_log <- with_logging(safely(log))
safe_logged_log(10)
#> Calling function with 1 argument(s)
#> $result
#> [1] 2.302585
#>
```

```
#> $error
#> NULL
safe_logged_log("a")
#> Calling function with 1 argument(s)
#> $result
#> NULL
#>
#> $error
#> <simpleError in .f(...): non-numeric argument to mathematical function>
```

The inner operator (`safely`) handles errors; the outer one (`with_logging`) adds logging. Each layer does one thing, and you combine them to get the behavior you want.

A factory builds specialized functions, an operator modifies them, and `map()` applies them across inputs.

```
formatters <- map(c("$", "EUR ", "GBP "), make_formatter)

map(formatters, \(f) f(100))
#> [[1]]
#> [1] "$100"
#>
#> [[2]]
#> [1] "EUR 100"
#>
#> [[3]]
#> [1] "GBP 100"
```

`map()` over a vector of prefixes produces a list of formatters, then `map()` over the formatters applies each one. A list of functions is data you can iterate over, filter, compose, and pass around just like a list of numbers. Chapter 21 uses that idea to collapse entire sequences into single values.

21 Reduce and accumulate

You have a list of data frames from an API and you need to merge them all into one. `map()` will not help: it treats every element in isolation and never looks at what came before. What you need is a way to take two things, combine them, feed the result into the next combination, and keep going until one thing remains. That operation is `Reduce()`.

`map()` (Chapter 19) replaces loops that apply a function element by element. `Reduce()` replaces the other kind: loops that carry state from one pass to the next, like a running total or a chain of merges that folds data frames together one by one.

21.1. Left fold

`Reduce()` takes a binary function and a list, then combines the elements pairwise from left to right:

```
Reduce(`+`, 1:5)
#> [1] 15
```

That computes `((((1 + 2) + 3) + 4) + 5)`. The first call is `f(1, 2)`, which gives 3; then `f(3, 3)`, giving 6; then `f(6, 4)`, giving 10; then `f(10, 5)`, giving 15. Each intermediate result becomes the left argument of the next call, which is why this is called a *left fold*: the parentheses nest to the left.

The name comes from Haskell's `foldl`. APL had the same operation in 1962, spelled `+/1 2 3 4 5`.

With a named function the folding structure becomes visible:

```
Reduce(paste, c("the", "cat", "sat", "on", "the", "mat"))
#> [1] "the cat sat on the mat"
```

Each step takes the accumulated string and pastes the next word onto it. The accumulator grows; the list shrinks. When nothing remains, the accumulator is the answer.

21.2. The init argument

What does `Reduce(f, x)` do when `x` has one element? It returns that element without ever calling `f`. Reasonable enough. What about zero elements?

```
Reduce(`+`, list())
#> NULL
```

An error, because there is nothing to return and no identity to fall back on. The `init` argument fixes this:

```
Reduce(`+`, list(), init = 0)
#> [1] 0
```

With `init`, the fold starts by calling `f(init, x[[1]])` instead of `f(x[[1]], x[[2]])`, which means the fold always has something to work with, even when the list is empty. The value you pass as `init` should be the identity element for the operation: 0 for addition, 1 for multiplication, `""` for `paste0`, `NULL` for `c()`. For `intersect` you would want the universal set, though R has no such thing, so you typically peel off the first element yourself.

This connection between an operation, its identity element, and associativity has a name in algebra: a *monoid*. Integers under addition with identity 0 form a monoid. Strings under concatenation with identity `""` form a monoid. Lists under `c()` with identity `NULL` form a monoid. The pattern shows up everywhere, and recognizing it tells you something immediately practical: if your operation is associative, the fold can be split across chunks, parallelized, and recombined without changing the result. Google's MapReduce (Dean and Ghemawat, 2004) is literally `map()` followed by a monoid fold, scaled to warehouses full of machines.

`Reduce()` is the operation that collapses a monoid. Given a list and an associative binary function, it produces a single value. That description sounds specific to lists, but the same structure appears whenever you compose things sequentially: piping functions is folding over a list of functions with composition as the binary operation and the identity function as `init`; building a ggplot layer by layer is folding over a list of layers with `+` as the operation and `ggplot()` as `init`. Tallying, concatenating, composing, intersecting: all of them are folds over a monoid.

> **Opinion**
>
> Always pass `init` to `Reduce()`. Without it, your code breaks on empty input and behaves subtly differently on length-1 input. The identity element makes the fold total: it works for every list, including the empty one, and it still works six months from now when some upstream function returns zero results.

21.3. Right fold

`Reduce()` folds from the left by default. Setting `right = TRUE` reverses the nesting:

```
Reduce(`-`, 1:4)
#> [1] -8
Reduce(`-`, 1:4, right = TRUE)
#> [1] -2
```

The left fold computes `(((1 - 2) - 3) - 4) = -8`. The right fold computes `(1 - (2 - (3 - 4))) = -2`. For associative operations (addition, multiplication, `paste0`, `c()`, `intersect`, `union`) the direction does not matter; you get the same answer either way, and left fold is the natural default. The difference only surfaces for non-associative operations: subtraction, division, exponentiation. These give different results depending on which way you nest.

In Haskell, `foldl` and `foldr` are separate functions, and choosing between them is a design decision with performance implications (lazy evaluation makes right folds cheaper in some cases). In R, `right = TRUE` is rarely needed. If you find yourself reaching for a right fold, ask first whether your binary function simply has its arguments in the wrong order.

Exercises

1. Use `Reduce()` to compute the product of the numbers 1 through 6 (i.e., 6 factorial). Check your answer against `factorial(6)`.
2. What is `Reduce(intersect, list(c(1,2,3,4), c(2,3,4,5), c(3,4,5,6)))`? Work through the steps by hand, then verify.
3. Write a `Reduce()` call that merges three data frames by a shared `"id"` column. (Hint: `merge` is a binary function.)
4. Explain why `Reduce(/, c(120, 2, 3, 4, 5))` gives 1 with a left fold and something different with a right fold.

21.4. accumulate: keeping the intermediates

Sometimes the final result is not enough. You want every intermediate step: the running total after each deposit, the growing sentence after each word, the high-water mark after each measurement. Base R handles this by adding a single argument:

```
Reduce(`+`, 1:5, accumulate = TRUE)
#> [1]  1  3  6 10 15
```

Running totals: `1, 3, 6, 10, 15`. This is the *scan* operation (Haskell's `scanl`, APL's \). Where fold collapses a list to a single value, scan collapses it to a list of all the values the fold passed through on its way to the answer. Running maxima, cumulative sums, progressive string construction: they all fall out of the same pattern.

```
Reduce(max, c(3, 1, 4, 1, 5, 9, 2, 6), accumulate = TRUE)
#> [1] 3 3 4 4 5 9 9 9
```

A running maximum, where each value is the largest seen so far. The 1s and the 2 cannot drag the maximum back down; it can only rise or hold steady.

```
Reduce(paste, c("one", "two", "three", "four"), accumulate = TRUE)
#> [1] "one"                "one two"            "one two three"
#> [4] "one two three four"
```

A sentence assembling itself one word at a time.

21.5. purrr::reduce() and purrr::accumulate()

The purrr package provides `reduce()` and `accumulate()` as tidyverse-flavored alternatives:

```
reduce(1:5, `+`)
#> [1] 15
accumulate(1:5, `+`)
#> [1]  1  3  6 10 15
```

Same results. The differences from base `Reduce()` are syntactic: `.x` and `.f` instead of positional arguments, `.init` instead of `init`, and `.dir = "backward"` instead of `right = TRUE`. The purrr versions also accept lambda-style formulas, though anonymous functions with `\()` are clearer.

Where purrr adds genuine power is `reduce2()`, which folds over two lists in parallel, handing an extra argument from the second list at each step:

```
reduce2(
  c("a", "b", "c"),
  c("-", "+"),
  \(acc, val, sep) paste0(acc, sep, val)
)
#> [1] "a-b+c"
```

The binary function receives three arguments: the accumulator, the current element from the first list, and the corresponding element from the second list. The second list must be one element shorter than the first (or the same length if `.init` is provided), because the first element of the first list seeds the accumulator.

21.6. Practical patterns

Combining data frames. When `split()` or repeated API calls hand you a list of data frames, `Reduce(rbind, dfs)` stacks them. For large lists, `do.call(rbind, dfs)` is faster because it avoids the $O(n^2)$ copying that comes from binding two at a time, growing the result frame with every step.

```
dfs <- list(
  data.frame(x = 1:2, y = c("a", "b")),
  data.frame(x = 3:4, y = c("c", "d")),
  data.frame(x = 5:6, y = c("e", "f"))
)
Reduce(rbind, dfs)
#>   x y
#> 1 1 a
#> 2 2 b
#> 3 3 c
#> 4 4 d
#> 5 5 e
#> 6 6 f
```

Nested list extraction. Walking into a deeply nested structure one key at a time:

```
nested <- list(a = list(b = list(c = 42)))
Reduce(\(x, i) x[[i]], c("a", "b", "c"), init = nested)
#> [1] 42
```

Each step indexes one level deeper. The fold replaces a chain of `[[` calls. (purrr's `pluck()` does the same thing with better error messages.)

Set operations across many sets:

```
sets <- list(c(1,2,3,4), c(2,3,4,5), c(3,4,5,6))
Reduce(intersect, sets)
#> [1] 3 4
Reduce(union, sets)
#> [1] 1 2 3 4 5 6
```

State machines with accumulate. An `accumulate()` call can model any sequential process where each step depends on the previous state:

```
# Simple bank account: deposits and withdrawals
transactions <- c(100, -20, -30, 50, -10)
accumulate(transactions, `+`)
#> [1] 100  80  50 100  90
```

The running balance after each transaction. For more complex state, where the accumulator needs to carry multiple fields rather than a single number, use a list as the accumulator and a function that unpacks, updates, and repacks it.

Exercises

1. Given `words <- c("R", "is", "a", "functional", "language")`, use `accumulate()` to produce the vector `c("R", "R is", "R is a", "R is a functional", "R is a functional language")`.
2. Write a `Reduce()` call that finds the intersection of the columns present in a list of data frames. (Hint: use `names()` and `intersect`.)
3. Use `accumulate()` to compute a running product of `c(2, 3, 5, 7)`. Verify that the last element equals `prod(c(2, 3, 5, 7))`.

21.7. Row-wise folds with c_across()

`Reduce()` folds across the elements of a list. In Section 19.8, you saw `across()` apply a function across the columns of a data frame. But what about folding across columns *within a single row*? That is a row-wise reduction, and `dplyr::c_across()` handles it.

The setup is `rowwise()`, which tells dplyr to treat each row as a group of one:

```
penguins <- palmerpenguins::penguins

penguins |>
  rowwise() |>
  mutate(bill_ratio = bill_length_mm / bill_depth_mm) |>
  ungroup() |>
  head(3)
#> # A tibble: 3 x 9
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>             <dbl>         <dbl>             <int>       <int>
#> 1 Adelie  Torgers~           39.1          18.7               181        3750
#> 2 Adelie  Torgers~           39.5          17.4               186        3800
#> 3 Adelie  Torgers~           40.3          18                 195        3250
#> # i 3 more variables: sex <fct>, year <int>, bill_ratio <dbl>
```

`c_across()` gathers the specified columns into a vector for each row, so you can pass that vector to any summary function:

```
df <- tibble(a = 1:3, b = 4:6, c = 7:9)
df |>
  rowwise() |>
  mutate(total = sum(c_across(a:c))) |>
  ungroup()
```

```
#> # A tibble: 3 x 4
#>       a     b     c total
#>   <int> <int> <int> <int>
#> 1     1     4     7    12
#> 2     2     5     8    15
#> 3     3     6     9    18
```

Each row's `a`, `b`, and `c` values are collected into a vector, summed, and stored in `total`. Same fold as `Reduce()`, same monoid structure (Section 21.2), just across columns instead of down a list.

`rowwise()` + `c_across()` processes one row at a time in R, so it scales poorly. On 100,000 rows, `rowSums()` finishes in milliseconds because it is vectorized in C; the `rowwise()` version takes seconds. Use `c_across()` when the operation is genuinely complex or when you need tidyselect helpers to pick columns. Otherwise use `rowSums()` or `pmax()`.

Exercises

1. Using `rowwise()` and `c_across()`, compute the maximum of columns `a`, `b`, and `c` for each row of `df <- tibble(a = c(3,1,4), b = c(1,5,9), c = c(2,6,5))`.
2. Rewrite the `c_across()` sum example using `rowSums()` instead. Which is clearer? Which is faster? (Try `bench::mark()` on a larger data frame.)
3. Use `rowwise()` and `c_across(where(is.numeric))` to compute the range (max minus min) of every numeric column for each row of `penguins`. Why would this be hard to do without `c_across()`?

21.8. When to fold and when not to

`Reduce()` is the right tool when you have a binary operation and a list of things to combine: merging data frames, chaining set operations, collapsing nested structures. It is the wrong tool when a vectorized function already exists. `sum(x)` is faster and clearer than `Reduce(+, x)`. `paste(x, collapse = " ")` beats `Reduce(paste, x)`. The base R cumulative functions (`cumsum`, `cumprod`, `cummax`, `cummin`) are compiled C and will always outperform `accumulate(+, x)`.

The rule is simple: if a specialized function exists, use it. Reach for `Reduce()` when your binary operation has no built-in vectorized cousin.

`Reduce()` also becomes awkward when the accumulator needs complex state. If your fold carries a list with multiple fields and each step updates several of them conditionally, you are no longer folding; you are writing a state machine with the fold syntax as a straitjacket. That is better expressed as an explicit loop or a recursive function (Chapter 22), where the state updates are visible and the control flow is honest.

Graham Hutton proved in 1999 that fold can express *any* function on finite lists. This is a theoretical universality result, not practical advice. Just because you *can* write something as a fold does not mean it will be readable, or maintainable, or that the person who inherits your code (including future you) will thank you for it.

Opinion

Fold is the most underused functional in R. Most programmers who discover `map()` never learn `Reduce()`, and they end up writing loops for exactly the problems fold solves in a single line. If you take one thing from this chapter: next time you write a loop that carries an accumulator variable, growing it with each pass, stop and try `Reduce()` first. You might not need that loop at all.

22 Recursion and fixed points

Suppose you need to process a tree of unknown depth, a structure that nests inside itself with no predetermined bottom. A `for` loop can't express this because you don't know how many levels to iterate; you would need a loop inside a loop inside a loop, turtles all the way down. The alternative is a function that calls itself, peeling away one layer of nesting each time until it reaches something solid. That function is recursive, and recursion predates loops by decades in the theory of computation.

A recursive function calls itself. The sentence sounds circular because circularity is the point: you solve a problem by breaking it into smaller copies of the same problem, bottoming out when the copies are small enough to answer directly. You saw `Reduce()` in Chapter 21 collapse a list into a single value by repeatedly applying a binary function, but `Reduce()` is just one specific recursive pattern wearing a loop's clothing. Recursion itself can express anything a loop can, and several things loops express awkwardly. What happens, though, when the chain of self-calls grows too long for R to handle?

22.1. Recursive functions

The simplest case is the one every textbook reaches for: the factorial. n! = n * (n-1) * … * 1, with 0! = 1:

```
factorial_r <- function(n) {
  if (n == 0) return(1)
  n * factorial_r(n - 1)
}

factorial_r(6)
#> [1] 720
```

Two parts make this work. A *base case* stops the recursion (`n == 0` here), and a *recursive case* makes progress toward it (since `n - 1` is closer to 0 than `n` was). Forget the base case and you get infinite recursion, which in R means a stack overflow error, the computational equivalent of a sentence that never finds its period.

Fibonacci numbers are the other classic:

```
fib <- function(n) {
  if (n <= 1) return(n)
  fib(n - 1) + fib(n - 2)
}

sapply(0:10, fib)
#>  [1]  0  1  1  2  3  5  8 13 21 34 55
```

This works but is spectacularly inefficient: each call spawns two more calls, ballooning to O(2^n) time complexity (see Chapter 9 for Big-O), so `fib(30)` alone triggers over a million calls. Memoization (Section 20.4) or an accumulator (later in this chapter) fixes the performance problem, but the naive version illustrates a real danger of recursion: elegant code can hide terrible performance. Performance aside, there is a more basic question. Does a recursive function even need a name?

`Recall()` calls the current function without naming it:

```
(function(n) {
  if (n == 0) return(1)
  n * Recall(n - 1)
})(6)
#> [1] 720
```

It looks up the function on the call stack using `sys.function()`, so it works even inside an anonymous function. In practice, named functions are clearer, but `Recall()` occasionally appears in one-off recursive helpers passed to higher-order functions.

Exercises

1. Write a recursive function `sum_to(n)` that computes 1 + 2 + … + n. What is the base case?
2. Write a recursive function `power_r(x, n)` that computes x^n using repeated multiplication. Handle the case n = 0.
3. The naive `fib()` recomputes the same values many times. Use `memoise::memoise()` to create `fib_memo` and compare `system.time(fib(30))` with `system.time(fib_memo(30))`.

22.2. The call stack

Every time R calls a function, it creates a new *frame* on the call stack: a record of the function, its arguments, and its local variables (each frame corresponds to the execution environment from Section 18.3). When the function returns, its frame is popped. Recursive calls pile up frames like plates on a spring-loaded stack, and you can watch it happen:

```
factorial_traced <- function(n) {
  cat("Entering n =", n, "at depth", sys.nframe(), "\n")
  if (n == 0) return(1)
  result <- n * factorial_traced(n - 1)
  cat("Returning", result, "for n =", n, "\n")
  result
}

factorial_traced(4)
#> Entering n = 4 at depth 31
#> Entering n = 3 at depth 32
#> Entering n = 2 at depth 33
#> Entering n = 1 at depth 34
#> Entering n = 0 at depth 35
#> Returning 1 for n = 1
#> Returning 2 for n = 2
#> Returning 6 for n = 3
#> Returning 24 for n = 4
#> [1] 24
```

`sys.nframe()` returns the current depth of the call stack, and each recursive call adds one frame. For `factorial_traced(4)`, the stack grows to depth 4 (plus whatever depth the calling context contributes), then unwinds as each call returns its result back up the chain.

R imposes a limit on this depth. The default sits around 5,000 frames, controlled by `options(expressions = ...)`, and if you exceed it you get one of two messages:

```
Error: C stack usage <number> is too close to the limit
```

or:

```
Error: evaluation nested too deeply
```

This is not a soft limit you should raise. It exists to protect against infinite recursion crashing your R session entirely, and if your algorithm needs more than a few thousand levels, the right response is to restructure it, not to bump a number in your options.

```
# This will error:
count_down <- function(n) {
  if (n == 0) return(0)
  count_down(n - 1)
}
count_down(10000)
```

Exercises

1. Call `factorial_traced(5)` and note the depth values. Then call `factorial_traced(5)` from inside another function. How do the depths change?
2. What is the default value of `getOption("expressions")`? Try setting it to 100 and calling `count_down(200)`. What error do you get?

22.3. Tail recursion and accumulators

Look at `factorial_r` one more time:

```
factorial_r <- function(n) {
  if (n == 0) return(1)
  n * factorial_r(n - 1)
}
```

After the recursive call returns, there is still work left: multiply the result by `n`. That pending multiplication means R cannot discard the current frame while waiting for the recursion to bottom out; every single frame in the chain must stay alive, holding onto its local `n`, until the deepest call finally returns 1 and the whole tower of deferred multiplications collapses upward.

A *tail-recursive* version eliminates that pending work by threading the partial answer through an accumulator argument, so the recursive call becomes the very last operation the function performs:

```
factorial_tail <- function(n, acc = 1) {
  if (n == 0) return(acc)
  factorial_tail(n - 1, acc * n)
}

factorial_tail(6)
#> [1] 720
```

When the recursive call sits in *tail position* (nothing happens after it), a smart compiler can reuse the current frame instead of allocating a new one, converting O(n) stack usage to O(1). This is *tail-call optimization* (TCO), and Scheme guarantees it. Haskell effectively provides it. R does not. So `factorial_tail(10000)` will still overflow the stack, even though the function is logically a loop in disguise.

The accumulator pattern still pays its rent, though. It clarifies intent (the accumulator carries the "answer so far"), it is the prerequisite for trampolining (next section), and it can convert exponential algorithms into linear ones. Here is Fibonacci rewritten with two accumulators, running in linear time instead of exponential:

```
fib_tail <- function(n, a = 0, b = 1) {
  if (n == 0) return(a)
  fib_tail(n - 1, b, a + b)
}

fib_tail(30)
#> [1] 832040
```

Where the naive `fib()` needed millions of calls for `fib(30)`, this version makes exactly n. But it still can't survive `fib_tail(10000)` on R's stack. What if we could get the O(1) stack usage ourselves, without waiting for a language feature R will never add?

Exercises

1. Write a tail-recursive `sum_to_tail(n, acc = 0)` that computes 1 + 2 + … + n.
2. Write a tail-recursive `reverse_list(x, acc = list())` that reverses a list.
3. Verify that `factorial_tail(5000)` still crashes. Why can R not optimize this?

22.4. Trampolining

Since R refuses to optimize tail calls, you can do it by hand. The idea: instead of making a recursive call, return a *thunk* (a zero-argument function that represents the call you *would* have made). Then a loop, the *trampoline*, keeps calling thunks until it gets a non-function result:

```
trampoline <- function(f, ...) {
  result <- f(...)
  while (is.function(result)) {
    result <- result()
  }
  result
}
```

Now rewrite `factorial_tail` to return thunks instead of calling itself:

```
factorial_thunk <- function(n, acc = 1) {
  if (n == 0) return(acc)
  function() factorial_thunk(n - 1, acc * n)
}

trampoline(factorial_thunk, 10)
#> [1] 3628800
trampoline(factorial_thunk, 1000)
#> [1] Inf
```

That second call churns through 1,000 iterations without growing the stack at all; each "recursive" step returns a closure, and the `while` loop in `trampoline` calls it, bouncing like a ball on a trampoline (hence the name). Stack depth stays at O(1) no matter how deep the logical recursion goes.

The pattern works for any tail-recursive function: rewrite the tail call as a thunk and let the trampoline do the looping. The cost is one closure allocation per step, which is cheap. The CRAN package `trampoline` provides a polished implementation with better error handling, but the core idea is exactly this `while` loop. The trampoline frees recursion from the stack, but notice that `factorial_thunk` still refers to itself by name inside the returned closure. What if you wanted recursion without even that?

> **Opinion**
>
> Trampolining is clever, but if you catch yourself trampolining in production R code, pause and ask whether a `for` loop or `Reduce()` would be simpler. The trampoline rescues recursive algorithms from the stack limit; it does not replace straightforward iteration. Save it for cases where you genuinely need recursion (tree traversal of unknown depth, for instance) but the depth could be large.

Exercises

1. Convert `fib_tail` into a trampolined version. Compute `fib(100000)` without a stack overflow. (The number will be huge; just verify it completes.)
2. The `trampoline` function above has a subtle limitation: it cannot return a function as a final result (since functions trigger another iteration). How would you fix this? Hint: wrap the final result in a special marker.

22.5. Divide and conquer

Some problems split cleanly in half, and each half looks just like the original problem, only smaller. Split, solve each piece recursively, combine. This is the *divide and conquer* strategy, and recursion expresses it so naturally that the code almost reads like the definition.

Merge sort is the cleanest example: split the vector in half, sort each half, merge the sorted halves back together:

```
merge_sort <- function(x) {
  if (length(x) <= 1) return(x)

  mid <- length(x) %/% 2
  left  <- merge_sort(x[1:mid])
  right <- merge_sort(x[(mid + 1):length(x)])

  merge_sorted(left, right)
```

```
}

merge_sorted <- function(a, b) {
  result <- numeric(length(a) + length(b))
  i <- j <- k <- 1
  while (i <= length(a) && j <= length(b)) {
    if (a[i] <= b[j]) {
      result[k] <- a[i]; i <- i + 1
    } else {
      result[k] <- b[j]; j <- j + 1
    }
    k <- k + 1
  }
  while (i <= length(a)) { result[k] <- a[i]; i <- i + 1; k <- k + 1 }
  while (j <= length(b)) { result[k] <- b[j]; j <- j + 1; k <- k + 1 }
  result
}

merge_sort(c(5, 2, 8, 1, 9, 3))
#> [1] 1 2 3 5 8 9
```

Each level of recursion halves the problem, giving O(log n) levels, and each level does O(n) work merging, so the total comes to O(n log n) as discussed in Chapter 9. The recursion depth is only log2(n), which means even a million-element vector needs just 20 levels. Stack overflow is not a concern for balanced divide-and-conquer algorithms; the branching that makes them powerful also keeps them shallow.

Binary search follows the same pattern but only recurses into one half:

```
binary_search <- function(x, target, lo = 1, hi = length(x)) {
  if (lo > hi) return(NA)
  mid <- (lo + hi) %/% 2
  if (x[mid] == target) return(mid)
  if (x[mid] < target) binary_search(x, target, mid + 1, hi)
  else binary_search(x, target, lo, mid - 1)
}

binary_search(1:100, 73)
#> [1] 73
```

Exercises

1. Trace through `merge_sort(c(4, 1, 3, 2))` on paper. How many recursive calls are made?
2. Write a recursive function `tree_depth` that takes a nested list and returns its maximum depth. `tree_depth(list(1, list(2, list(3))))` should return 3.

22.6. Recursive list processing

Nested lists are R's tree structure, and recursion is the natural way to walk them. You might think `unlist()` already handles this, but `unlist()` flattens blindly: it discards the nesting, coerces everything to a common type, and gives you no control over which elements to keep or transform. When the tree has mixed types or you need to preserve structure while operating on selected leaves, hand-written recursion recovers what `unlist()` throws away.

Suppose you need to extract all numeric values buried somewhere inside an arbitrarily nested list, but you don't know how deep the nesting goes or where the numbers hide:

```
find_numbers <- function(x) {
  if (is.numeric(x)) return(x)
  if (!is.list(x)) return(NULL)
  unlist(lapply(x, find_numbers))
}

nested <- list("a", list(1, list("b", 2, list(3, "c"))))
find_numbers(nested)
#> [1] 1 2 3
```

The function dispatches on type: numbers come back directly, non-list non-numbers get discarded, and lists trigger a recursive `lapply` that peels off one layer at a time. This pattern of "check the base cases, recurse on the structure" is the backbone of all tree-processing code; once you see it, you see it everywhere.

Base R provides `rapply()` for recursive application over lists:

```
rapply(nested, sqrt, classes = "numeric", how = "unlist")
#> [1] 1.000000 1.414214 1.732051
```

`rapply()` walks the nested structure, applies `sqrt` to every numeric element it encounters, and returns the results. The `classes` argument filters which elements get processed, and `how` controls the output shape ("unlist", "replace", or "list").

```
rapply(nested, toupper, classes = "character", how = "replace")
#> [[1]]
#> [1] "A"
#>
#> [[2]]
#> [[2]][[1]]
#> [1] 1
#>
#> [[2]][[2]]
#> [[2]][[2]][[1]]
#> [1] "B"
#>
```

```
#> [[2]][[2]][[2]]
#> [1] 2
#>
#> [[2]][[2]][[3]]
#> [[2]][[2]][[3]][[1]]
#> [1] 3
#>
#> [[2]][[2]][[3]][[2]]
#> [1] "C"
```

With `how = "replace"`, the nesting is preserved: character elements are uppercased in place while everything else stays untouched. `rapply()` is surprisingly powerful but rarely seen in practice, partly because deeply nested lists are uncommon in day-to-day R work, and partly because fewer people know about it than about `lapply` or `map`. If your data is a tree, though, it deserves a look.

Exercises

1. Write a recursive function `flatten` that takes a nested list and returns a flat list of its leaf elements. Compare with `rapply(x, identity, how = "unlist")`.
2. Use `rapply()` with `how = "replace"` to double every numeric element in `list(1, list("a", 2, list(3, "b")))`.
3. Write a recursive function that counts the total number of leaf elements in a nested list.

22.7. Mutual recursion

Two functions can call each other, passing control back and forth. Mutual recursion appears naturally in recursive descent parsers, where each grammar rule becomes a function that may call functions for other rules, and in state machines, where each state is a function that transitions by calling another state's function. The pattern is easier to see, though, in a stripped-down example:

```
is_even <- function(n) {
  if (n == 0) return(TRUE)
  is_odd(n - 1)
}

is_odd <- function(n) {
  if (n == 0) return(FALSE)
  is_even(n - 1)
}

is_even(4)
```

```
#> [1] TRUE
is_odd(7)
#> [1] TRUE
```

Each function delegates to the other, peeling off one layer at a time. Nobody would check parity this way, but the structure is identical to what a parser does when an expression rule calls a term rule that calls an expression rule again.

The stack cost is the same as ordinary recursion, doubled: each round trip consumes two frames. For deep mutual recursion, trampolining works identically; just have both functions return thunks instead of calling each other directly. But mutual recursion raises a subtler question than stack depth: both `is_even` and `is_odd` refer to each other by name. The trampoline removed the stack constraint; can anything remove the naming constraint?

22.8. Fixed-point combinators

Here is an odd question. Can you write a recursive function without ever giving it a name? Named functions call themselves by name, so recursion seems to require naming. But what if you only have anonymous functions, nothing else?

A *fixed point* of a function f is a value x where f(x) = x. The number 0 is a fixed point of `function(x) x^2`, because 0^2 = 0. A *fixed-point combinator* extends this idea to functions that return functions: it takes a "template" for recursion and threads the self-reference through automatically, producing a working recursive function from anonymous parts alone.

In lazy languages (Haskell, untyped lambda calculus), the Y combinator does this:

```
Y = f. ( x. f(x x))( x. f(x x))
```

R is strict, though: it evaluates arguments before passing them, and that eagerness is fatal here. If you translate Y directly into R, the self-application `x(x)` evaluates immediately, which triggers another `x(x)`, which triggers another, and the whole thing diverges before `f` ever gets a chance to run. Step by step: `Y(f)` calls `(function(x) f(x(x)))` with itself as argument, so `x` becomes `function(x) f(x(x))`. To evaluate `f(x(x))`, R must first evaluate `x(x)`, which means evaluating `f(x(x))` again, forever:

```
# This diverges in R:
Y <- function(f) (function(x) f(x(x)))(function(x) f(x(x)))
Y(function(f) function(n) if (n == 0) 1 else n * f(n - 1))
# Error: C stack usage is too close to the limit
```

The fix is the Z combinator, which wraps each `x(x)` in a lambda, `function(v) x(x)(v)`, to delay evaluation. That extra wrapper acts as a thunk: `x(x)` is not evaluated when the combinator is constructed, only when the inner function is actually called with an argument `v`:

```
Z <- function(f) {
  (function(x) f(function(v) x(x)(v)))(
    function(x) f(function(v) x(x)(v)))
}
```

The structure is identical to Y, except each `x(x)` wears a `function(v) ...` coat. That single change is the difference between divergence and termination in a strict language.

Now define factorial without any self-reference at all:

```
fact <- Z(function(f) function(n) {
  if (n == 0) 1 else n * f(n - 1)
})

fact(6)
#> [1] 720
```

The argument `f` inside the inner function comes from the Z combinator, which threads in the self-reference from thin air — the function never names itself. The function never names itself, never uses `Recall()`, yet it recurses.

This is not a party trick. In 1936, Church proved that recursion can be derived from lambda calculus, a system with only three constructs: variables, abstraction (function definition), and application (function calling). No built-in recursion, no loops, no assignment. The Y/Z combinator is the mechanism that makes it work, and Chapter 30 explores the full implications, along with Church numerals and the deeper connections between lambda calculus and computation.

You will never use the Z combinator in production R code. But understanding that recursion is not primitive, that it emerges from function application alone, changes how you think about what functions are capable of. If anonymous functions can bootstrap recursion without any help from the language, what else might be hiding in plain sight?

Exercises

1. Use the Z combinator to define a recursive Fibonacci function without self-reference.
2. Apply `Z` to `function(f) function(xs) if (length(xs) == 0) 0 else xs[[1]] + f(xs[-1])` and test it on `list(1, 2, 3, 4, 5)`. What does this compute?
3. Why does the Y combinator (without the extra `function(v)` wrapper) diverge in R? What happens if you try `Y <- function(f) (function(x) f(x(x)))(function(x) f(x(x)))`?

22.9. When not to recurse

R is not Scheme. It does not optimize tail calls, its stack is limited, and function call overhead is high compared to vectorized operations. The shape of the data tells you whether

recursion is the right tool or a trap.

Tree-shaped problems, where the structure branches and nests (nested lists, file system traversal, parsing hierarchical data, divide-and-conquer algorithms with O(log n) depth), stay well within stack limits and express naturally as recursive functions. These are recursion's home territory.

Linear problems are a different story. Summing a vector, computing a running total, filtering elements: these are loops, `Reduce()` calls, or vectorized operations, and writing them recursively wastes n stack frames on something that `sum()` handles in a single C call.

> **Opinion**
>
> If your recursive function processes a flat sequence from left to right, it is a fold; use `Reduce()` or `purrr::reduce()` from Chapter 21. If it processes elements independently, it is a map; use `lapply()` or `purrr::map()` from Section 7.1. Reserve recursion for problems where the structure of the data is itself recursive. When the data branches, your code should too.

The table below summarizes when each approach fits:

Problem shape	Preferred approach	Why
Element-wise transformation	`lapply()` / `map()` / vectorized	No overhead, parallel-friendly
Linear fold (sum, concat, merge)	`Reduce()` / `reduce()`	Constant stack, clear intent
Tree traversal, nested structures	Recursion / `rapply()`	Matches the data shape
Divide and conquer (sort, search)	Recursion	O(log n) depth, natural decomposition
Deep linear recursion (>1000)	Trampoline or convert to loop	Stack limit workaround

22.10. Historical notes

By the early twentieth century, mathematicians had built formal systems powerful enough to express most of mathematics. The natural next question was whether those systems could verify themselves: could a formal system prove that every true statement within it is provable? If yes, mathematics would be self-certifying, a closed loop with no loose ends. The systems were expressive enough to state any mathematical truth, so surely they were expressive enough to prove any mathematical truth. Hilbert had made exactly this bet (the same program that led to the Entscheidungsproblem in Section 9.6.1), challenging mathematicians to prove their formal systems complete and consistent, but then in 1931 a twenty-five-year-old logician in Vienna found a way to make a formal system reason about its own proofs.

He encoded logical statements as numbers using primitive recursive functions, turning each proof step into arithmetic. Once proofs were numbers, he could construct a statement inside the system that said, in effect, "this statement has no proof in this system." If the system could prove it, the statement would be false, and the system would be proving a falsehood. If the system could not prove it, the statement would be true, and there would be a true statement the system could never reach. Either way, the closed loop was shattered. Mathematics would always have blind spots: true statements that no amount of cleverness, no added axioms, no better proof strategies could ever capture. It became known as Godel's incompleteness theorem.

Five years later, Church showed that recursion did not need to be a primitive at all: the fixed-point combinator could derive it from pure function application. Turing's machines provided the alternative model, and Church and Turing proved the two equivalent. But recursion stayed in mathematics until McCarthy gave it a programming language. LISP (1960) was the first language where recursive function calls served as the primary control structure; `car`, `cdr`, and `cons` made recursive list processing practical. Recursion was no longer a proof technique restricted to mathematics; it was how you wrote programs.

The question that remained was efficiency. Landin's SECD machine (1964) showed how to mechanically evaluate expressions with closures and recursive calls. A decade later, Steele and Sussman proved something that surprised the programming world: tail-recursive calls and `goto` are the same thing. Their "Lambda the Ultimate" papers (1975-1980) showed that lambda expressions can replace every control structure, and that a sufficiently smart compiler can make functional code as fast as imperative code. Scheme guaranteed tail-call optimization as a consequence.

R chose not to. S was designed for interactive statistical computing, where vectorized operations do the heavy lifting, not deep recursion. The absence of TCO reflects that priority: the language optimized for the common case (vectorized numerical work) at the cost of the uncommon one (recursive descent past a thousand frames). But R did make another choice about evaluation that most languages avoided: when you pass an argument to a function, R does not evaluate it immediately.

23 Lazy evaluation

You write `filter(penguins, species == "Adelie")` and it works. But `species` isn't a variable in your environment; it's a column buried inside a data frame, invisible to ordinary evaluation rules. So how does R find it?

The answer begins with something R does not do. When you pass an argument to a function, R does not evaluate it. Not yet. Instead, R wraps the argument in an invisible object called a *promise*, a small bundle containing the expression you wrote and the environment where that expression should eventually be evaluated, and then it waits, holding the unevaluated expression in suspension until the function actually needs the result. This single design choice, the decision to procrastinate, ripples outward into default arguments that reference other arguments, functions that inspect your code before running it, and the entire tidyverse convention of writing column names as if they were variables.

The idea has a lineage. Early programming languages split over a simple question: when should arguments be evaluated? Fortran and C chose eagerness, computing each argument before handing it to the function (call-by-value). Algol 60 tried the opposite, passing the raw expression and re-evaluating it every time the function touched it (call-by-name). If the expression had side effects, they fired on every access; if the expression was expensive, the cost multiplied by the number of times the function touched the argument. The compromise came in 1976: pass the expression unevaluated, but cache the result after the first evaluation. Evaluate at most once. The technique was called *lazy evaluation*. Haskell made laziness the default for everything when it launched in 1990; R took a narrower path, inheriting through Scheme and S a design where function arguments are lazy but everything else is eager. That middle ground gives you expression capture without pervasive laziness, but it also means that every function argument in R carries a hidden payload most programmers never notice.

The mechanism behind all of this is promises. Once you see how they work, non-standard evaluation, data masking, and `{ }` follow as consequences. Full metaprogramming lives in Chapter 26; here, the goal is understanding the machinery well enough to use it, and well enough to know when it's working against you.

23.1. Promises

When you call `f(x + 1)`, R does not compute `x + 1` and pass the result. It wraps `x + 1` in a *promise*: a bundle containing the expression (`x + 1`) and the environment where that

expression should be evaluated (your calling environment). The promise travels into `f` unevaluated.

The first time `f` actually touches that argument, R cracks the promise open: it takes the stored expression, evaluates it in the stored environment, and caches the result. If the argument is never used, the promise is never evaluated, which means side effects inside it never happen. If the argument is used twice, the second access returns the cached value rather than re-evaluating the expression. One evaluation, at most. This sounds like a harmless optimization, but it changes what it means for a function to "receive" an argument.

A promise has three components:

1. **The expression**: what was written (`x + 1`).
2. **The environment**: where to evaluate it (the caller's environment).
3. **The cached value**: filled in after the first evaluation, empty before.

Promises are invisible by design: R has no user-facing "promise" type. You cannot create one manually, inspect one with `str()`, or test for one with `is.promise()`. The invisibility is deliberate, because the act of inspecting a promise would force its evaluation, changing program behavior. The abstraction only works because you cannot look behind it.

In the terminology of lambda calculus, this is *call-by-need* evaluation: the argument is passed unevaluated (like call-by-name) but the result is cached after the first evaluation (unlike call-by-name, which would re-evaluate each time). Lambda calculus distinguishes three evaluation strategies: call-by-value (strict, as in C and Python), call-by-name (non-strict, as in Algol 60), and call-by-need (lazy, as in Haskell). R uses call-by-need for function arguments through promises, though most arguments are forced immediately in practice. The Church-Rosser theorem guarantees that if a reduction terminates, all strategies reach the same normal form, but they may differ in whether they terminate at all.

That difference matters more than it sounds. In an eager language, every argument is fully evaluated before the function sees it — the computation tree grows left to right, arguments resolved before calls. In R, arguments arrive as suspended expressions, unevaluated until the function actually needs them. The tree hangs in place, branches dormant until touched. This is what makes non-standard evaluation possible at all: `aes(x = bill_length_mm)` works because `bill_length_mm` stays unevaluated in your environment, captured as a promise, carried into the plotting function, and evaluated later inside the data frame. Data masking, quasiquotation, and formula interfaces all depend on the fact that arguments are not evaluated the moment you pass them. Without lazy evaluation, none of these techniques would work, because the expression would have already been reduced to a value before the function had a chance to inspect it.

What does laziness buy you in practice? Default arguments that would be impossible under eager evaluation:

```
f <- function(x, y = x * 2) y
f(5)
#> [1] 10
```

The default for `y` is a promise containing the expression `x * 2`. When R needs `y`'s value, it evaluates `x * 2` inside the function body, where `x` is already bound to 5, yielding 10. Supply `y` explicitly and the default promise is never touched:

```
f(5, 99)
#> [1] 99
```

Defaults can depend on computations that happen inside the function itself, which gets stranger:

```
g <- function(x, n = length(x)) n
g(c(1, 2, 3))
#> [1] 3
```

The default `n = length(x)` is an expression, not a precomputed value; R evaluates it when `n` is first accessed, by which time `x` is already bound. If R evaluated defaults at the moment of the call, before the function body had a chance to run, none of this would work. But what happens when laziness interacts with side effects?

Exercises

1. Predict the output of this code, then run it:

   ```
   h <- function(x) {
     cat("about to use x\n")
     x
   }
   h(cat("evaluating argument\n"))
   ```

   Which `cat()` call prints first? Why?

2. Write a function `f(x, y = x + 1)` and call `f(10)`. What is `y`? Now call `f(10, 50)`. What changed?

3. What happens if you call a function that never uses its argument?

   ```
   ignore <- function(x) 42
   ignore(stop("this should error"))
   ```

   Does it error? Why or why not?

23.2. Consequences of laziness

R chose a specific point in the design space: function arguments are lazy, everything else is eager. Haskell sits at the other extreme, where *everything* is lazy (`let xs = [1..]` creates an infinite list without computing all elements), enabling infinite streams and on-demand

computation at the cost of making performance harder to reason about. R's middle ground gives you NSE and flexible defaults without the mental overhead of tracking which values exist yet and which are still suspended.

Side effects in arguments may never happen. If a function ignores an argument, the promise wrapping it is never forced, which means any side effects buried in the expression (printing, writing files, raising errors) simply vanish:

```
quiet <- function(x) "I ignore my argument"
quiet(print("you will never see this"))
#> [1] "I ignore my argument"
```

No output from `print()`. The promise sat there, inert, and was eventually garbage-collected without ever running. This is not a bug; it's the logical consequence of evaluate-only-when-needed. But it does mean you cannot rely on argument expressions to produce side effects unless the function actually touches them.

`missing()` **can detect unsupplied arguments.** Because arguments start life as unevaluated promises, R can distinguish between "the caller supplied this argument" and "the caller didn't," a distinction that would be impossible if all arguments were evaluated before the function saw them:

```
report <- function(x) {
  if (missing(x)) "not supplied" else "supplied"
}
report()
#> [1] "not supplied"
report(42)
#> [1] "supplied"
```

This only works in the window before evaluation. Once an argument is used, `missing()` loses its ability to answer the question.

Default expressions, not default values. A function like `function(x, n = length(x))` does not store a default *value* for `n`. It stores a default *expression*, evaluated lazily inside the function at the moment `n` is first accessed. Defaults can therefore depend on other arguments, on computations that happen earlier in the function body, or on variables in the function's enclosing environment. The entire default-argument system is built on promises.

The lazy evaluation trap in function factories. If you read Chapter 20, you saw that promises can cause trouble when a factory captures a loop variable:

```
funs <- list()
for (i in 1:3) {
  funs[[i]] <- function() i
}
funs[[1]]()
#> [1] 3
```

```
funs[[2]]()
#> [1] 3
```

All three functions return 3. Why? Because `i` was captured as a promise pointing to the loop variable, and by the time any function is called, the loop has finished and `i` is 3. The fix requires two tools working together: `force()`, which evaluates a promise immediately, and `local()`, which creates a fresh environment for each iteration so that each function captures its own copy of `i` rather than sharing the loop variable:

```
funs <- list()
for (i in 1:3) {
  local({
    force(i)
    funs[[i]] <<- function() i
  })
}
funs[[1]]()
#> [1] 3
funs[[2]]()
#> [1] 3
```

`local()` evaluates its body in a new, temporary environment, giving each iteration its own `i` binding. `force(i)` inside that environment evaluates the promise immediately, locking in the current loop value before the next iteration overwrites it. Without `local()`, all three functions share the same binding. Without `force()`, the promise might linger unevaluated until after the loop finishes. You need both: `local()` for isolation, `force()` for immediacy.

What is `force(x)` actually doing? Nothing clever. Its definition is just `x`, which accesses the argument and thereby triggers evaluation and caching. The name exists purely to communicate intent: "evaluate this now, don't wait." Which raises a broader question: if laziness can cause subtle bugs in loops and closures, why does R use it at all?

Exercises

1. Predict: does `quiet(log(-1))` produce a warning? Why or why not? (Use the `quiet` function defined above.)
2. Write a function `greet(name = "stranger")` that returns `paste("Hello,", name)`. Call it with and without an argument. Explain which default mechanism makes this work.

23.3. What is non-standard evaluation?

The previous section ended with a question: if laziness can cause subtle bugs, why does R bother with it? Because arguments travel as unevaluated promises, a function can intercept the expression before R reduces it to a value, and that capability is the foundation of non-standard evaluation.

Standard evaluation is straightforward: R evaluates an expression, produces a value, and hands the value to a function. When you write `mean(c(1, 2, 3))`, R computes the vector `[1] 1 2 3` first, then passes it to `mean`, and `mean` never sees the expression `c(1, 2, 3)` at all.

Now try `subset(df, x > 3)`. If R evaluated `x > 3` in your environment before passing it to `subset`, the call would fail: there is no `x` in your workspace. Something different is happening here, something that depends on the laziness you've been studying. Instead of evaluating the argument, `subset` reaches into the promise, extracts the *expression* before evaluation, and then decides for itself where that expression should run:

```
df <- data.frame(x = 1:5, y = c(10, 20, 30, 40, 50))
subset(df, x > 3)
#>   x  y
#> 4 4 40
#> 5 5 50
```

`x` here is a column of `df`, not a variable in your global environment. `subset` never evaluates `x > 3` in your environment. It captures the expression with `substitute()`, then evaluates it inside `df` with `eval()`:

```
# Simplified subset internals
subset.data.frame <- function(x, subset, ...) {
  expr <- substitute(subset)
  row_mask <- eval(expr, x, parent.frame())
  x[row_mask & !is.na(row_mask), , drop = FALSE]
}
```

`substitute()` grabs the unevaluated promise. `eval()` evaluates it in an environment where the columns of `x` are visible as variables. That's why `x > 3` finds the column instead of failing with "object 'x' not found."

This is NSE in its simplest form: intercept the expression, choose where to evaluate it. Compare the two approaches side by side:

```
# Standard evaluation: explicit, verbose
df[df$x > 3, ]

# Non-standard evaluation: concise, readable
subset(df, x > 3)
```

NSE is the reason the tidyverse feels like a domain-specific language for data analysis. `filter(penguins, species == "Adelie")` reads almost like English because you write column names directly, without `$` or quotes. The cost is that the rules become less transparent: the meaning of `species` depends on which function you're inside, not just on what's in your environment. That tension between conciseness and predictability runs through everything that follows.

Exercises

1. Run `subset(df, x > 3)` with the data frame above, then try `df[df$x > 3, ]`. Verify they produce the same result. Which is easier to read?
2. What happens if you define `x <- 100` in your global environment and then run `subset(df, x > 3)` again? Does it use the column or the variable? Why?

23.4. Data masking

Data masking is the tidyverse's name for NSE applied to data frames, and it follows a specific lookup order. When you write:

```
library(dplyr)
filter(penguins, species == "Adelie")
```

R looks for `species` first in `penguins`, then in your calling environment. The data *masks* the environment: if the data frame has a column called `species`, that column wins over any variable named `species` in your workspace. Same principle applies to `aes(x = bill_length_mm)` in ggplot2, which captures the expression and evaluates it against the data at plot time, treating columns as if they were ordinary variables.

The benefit for interactive analysis is enormous. You type column names hundreds of times in a session; not having to write `penguins$species` or `penguins[["species"]]` each time keeps your code compact and readable, closer to how you think about the data than to how the computer stores it.

The cost appears the moment you try to program with data-masked functions. Suppose you want to write a function that filters by a column whose name is stored in a variable:

```
col <- "species"
filter(penguins, col == "Adelie")
```

This doesn't do what you want. `filter` looks for a column named `col` in `penguins`, doesn't find one, then finds `col` in your environment (the string `"species"`), and compares that string to `"Adelie"`. No error, just wrong results, the most dangerous kind of failure.

To see the mechanism concretely, here is a simplified `filter()` you could write yourself in about ten lines:

```
my_filter <- function(.data, expr) {
  e <- substitute(expr)                    # capture the caller's expression
  env <- list2env(.data, parent = parent.frame())  # build an environment from the
data frame
  mask <- eval(e, envir = env)             # evaluate the expression in that
environment
  .data[mask & !is.na(mask), , drop = FALSE]
```

```
}

df <- data.frame(x = 1:5, y = c(10, 20, 30, 40, 50))
my_filter(df, x > 3)
#>   x  y
#> 4 4 40
#> 5 5 50
```

`substitute(expr)` captures the caller's expression (`x > 3`) without evaluating it, exactly as a promise holds an unevaluated expression (Section 23.1). `list2env(.data, parent = parent.frame())` creates a new environment whose bindings are the columns of the data frame, with the caller's environment as the parent so that variables not in the data frame are still found. `eval(e, envir = env)` forces the expression in that constructed environment, where `x` resolves to the column rather than to anything in the global environment. This works when you write `my_filter(df, x > 3)` directly. Try wrapping it in another function, and the expression gets trapped in the wrong scope. This is what `dplyr::filter()` does, with more machinery for tidy evaluation, error handling, and grouped data frames layered on top.

Data masking is optimized for the common case (interactive analysis) at the expense of the less common case (writing reusable functions). The moment you wrap a data-masked function inside another function, the column name is trapped in your wrapper's scope, and the caller has no way to pass it through. That's the problem tidy evaluation exists to solve.

Exercises

1. Define `x <- 1000` in your global environment. Create a data frame `df <- data.frame(x = 1:5)`. What does `dplyr::filter(df, x > 3)` return? Does it use the column or the variable?
2. Explain in one sentence why `aes(x = bill_length_mm)` doesn't need quotes around `bill_length_mm`.

23.5. Tidy evaluation basics

Data masking works because `substitute()` captures the promise from the immediate caller. But when you wrap a data-masked function inside your own function, `substitute()` captures your wrapper's argument name, not the expression the original caller wrote. Tidy evaluation exists to thread promises through that extra layer.

The embrace operator `{ }` (called "curly-curly") solves the most common programming-with-NSE problem: passing a column name through your function to a dplyr verb without it getting lost along the way.

```
library(dplyr)
```

```
my_summary <- function(data, var) {
  data |>
    summarise(mean = mean({{ var }}, na.rm = TRUE))
}
```

`{ var }` says: "take whatever the caller passed as `var`, capture it as a data-masked expression, and inject it here." The caller writes column names unquoted, exactly as they would with dplyr directly:

```
my_summary(penguins, body_mass_g)
```

Without `{ }`, you'd have to teach your users a different syntax for your wrapper than they use for dplyr itself, which defeats the purpose of wrapping. With it, the wrapper is transparent.

For tidy selection (used in `select()`, `across()`, and similar), `{ }` works the same way:

```
my_select <- function(data, cols) {
  data |> select({{ cols }})
}
```

```
my_select(penguins, starts_with("bill"))
```

The caller passes tidy-select expressions exactly as they would to `select()` directly. Your function just forwards them.

When `{ }` isn't enough, there are more tools. `.data[["col_name"]]` lets you use string column names inside data-masked functions. `!!` (bang-bang) and `enquo()` give finer control over quoting and unquoting. These are part of the full tidy evaluation framework in the rlang package, and they matter when you're building complex programmatic interfaces. But they belong in Chapter 26, not here.

> **Opinion**
>
> For most dplyr wrapper functions, `{ }` is sufficient. The full tidy evaluation system (quosures, `enquo()`, `!!`, `!!!`) exists for cases where you need to build expressions programmatically, splice multiple variables, or mix quoted and unquoted inputs.

Exercises

1. Write a function `my_count(data, group_var)` that uses `{ }` to count rows per group. Test it with any data frame.
2. Write a function `my_arrange(data, sort_var)` that arranges a data frame by a column passed by the caller. Use `{ }`.

3. What happens if you forget `{ }` and write `summarise(data, mean = mean(var, na.rm = TRUE))` inside a wrapper function? What error do you get?

23.6. The trade-offs of NSE

Promises enable NSE, NSE enables data masking, and `{ }` makes data masking programmable. Each link adds a layer of indirection.

Convenience vs. predictability. `filter(df, x > 5)` is concise and readable, but if a variable `x` exists in your environment *and* a column `x` exists in your data frame, the column wins silently. When that's wrong, nothing tells you.

Interactive vs. programmatic. NSE is optimized for the console, where you type `select(df, name, age)` and it just works. Writing a function that wraps `select` requires `{ }`, an extra concept that sits between you and the dplyr you already know. The interactive user pays nothing; the programmer pays a tax, and the tax scales with the complexity of the interface you're building.

Debugging. "Object 'species' not found" could be a typo, missing data, or a masking scope problem. The error message is the same in all three cases.

This isn't a tidyverse invention. Base R uses NSE in `subset()`, `with()`, `transform()`, and the formula interface (`lm(y ~ x, data = df)`). The tidyverse applies it more systematically and more pervasively, but the idea is as old as S.

> **Opinion**
>
> NSE is the right trade-off for data analysis. You type column names hundreds of times a day, and quoting each one would be painful. The cost falls on the programmer writing reusable functions, not on the analyst exploring data interactively, and that's a reasonable place for the cost to land since most R users spend more time analyzing than abstracting. Accept the trade-off, learn `{ }` for when you cross the line into programming, and spend your energy on the next question: when you call `print(x)`, how does R decide which `print` to use?

Part IV

Part IV: The type system

24 S3, S4, and S7

You call `print(x)` and it does the right thing whether `x` is a number, a data frame, or a linear model. How? R looks at the class of `x` and picks the right `print` function. That is dispatch.

24.1. What dispatch means

`print(x)` isn't one function. It's a *generic*: a function that inspects its input and delegates to a specialized *method*. `print.data.frame`, `print.Date`, `print.lm`: each is a method. The generic `print` finds the right one based on `class(x)`.

```
x <- Sys.Date()
class(x)
#> [1] "Date"
```

When you call `print(x)`, R sees that `class(x)` is `"Date"`, looks for a function named `print.Date`, and calls it. If no such function exists, R falls back to `print.default`.

`summary()`, `plot()`, `mean()`, `[`: all generics. R is full of them. You can list the methods for any generic:

```
head(methods(print), 10)
#>  [1] "print.acf"               "print.activeConcordance"
#>  [3] "print.AES"               "print.anova"
#>  [5] "print.aov"               "print.aovlist"
#>  [7] "print.ar"                "print.Arima"
#>  [9] "print.arima0"            "print.AsIs"
```

This is *functional OOP*: methods belong to generics, not to classes. You call `print(x)`, not `x.print()`. In Java, Python, or C++, methods belong to objects and you write `object.method()`. R dispatches on the first argument. Anyone can add a method for any class to any generic without modifying the class. Package A defines a class, package B defines a method for it, and neither needs to know the other exists.

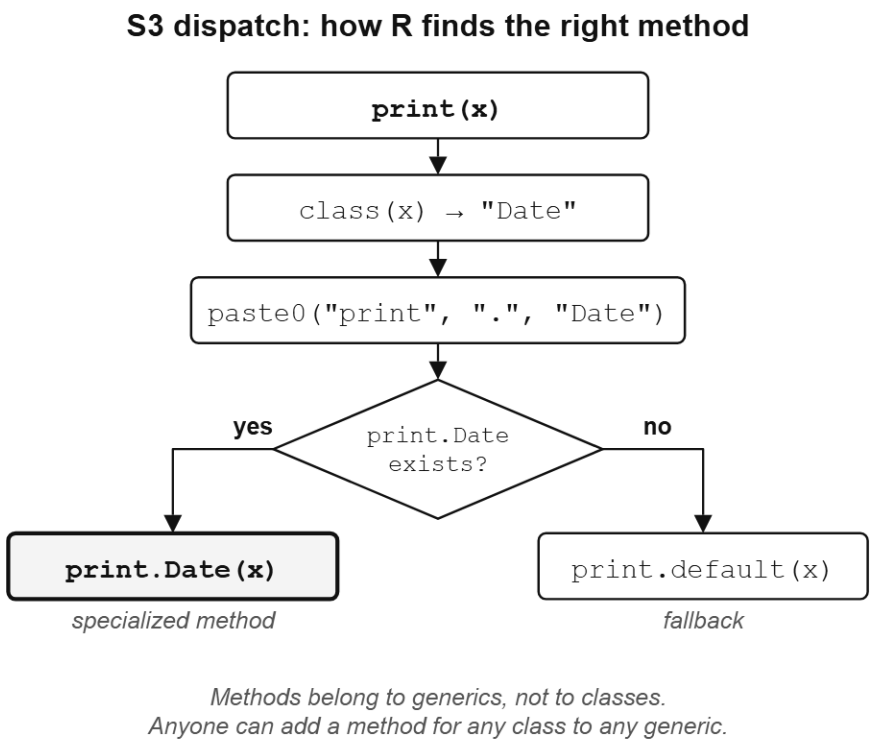


Figure 24.1.: S3 dispatch: R constructs a method name from the generic and the class, then calls it if it exists.

Dispatch is R picking *which* function to call, the same principle as "everything that happens is a function call" (Section 7.5).

The type-theory connection

In type theory, the type of an expression determines which reduction rules apply. S3 dispatch is the same idea: the class attribute determines which method runs, and the generic is a polymorphic term whose behavior is resolved at runtime by inspecting the type of its argument. This is *ad-hoc polymorphism* (different code for different types), as opposed to the *parametric polymorphism* of Hindley-Milner type systems, where the same code works for all types. R does not have parametric polymorphism in the formal sense, but functions like `length()` and `c()` behave parametrically: they work on any vector regardless of element type.

S3 dispatch also connects to sum types. When you give an object a class, you are tagging it as one variant of a type. `class(x) = "penguin"` says "this object is a penguin, not a dog, not a book." The generic function then *pattern matches* on the tag, dispatching to the method for that variant, exactly how sum types work in Haskell (`case x of`) or Rust (`match x`): inspect the tag, run the corresponding branch. R's mechanism is informal (a string attribute instead of a compiler-checked type), but the algebraic structure is the same sum type you saw with `TRUE | FALSE` (Section 8.1) and factor levels (Section 12.4). The class vector is the tag; `UseMethod()` is the match.

When you call `print(x)` and R inspects `class(x)` to choose a method, the class attribute is playing the same role that a type annotation plays in a statically typed language: it determines which code runs. In Haskell, the compiler verifies at compile time that every case is handled. In R, you find out at runtime that no method exists for your class, and `print.default` picks up the slack. That informality gives you freedom — attaching a class string to any object without declaring a type hierarchy first is useful in interactive work, where data structures evolve faster than formal definitions. But the freedom costs you exhaustiveness: every missing method is a bug that only surfaces when someone passes an object you did not anticipate.

24.2. S3 basics

S3 is informal. A class is just a string attached to an object:

```
x <- structure(list(name = "Adelie", mass = 3750), class = "penguin")
class(x)
#> [1] "penguin"
```

That's all it takes: a list (or vector) with a `class` attribute attached to it.

A generic is a function that calls `UseMethod()`:

```
greet <- function(x, ...) UseMethod("greet")
```

A method is a function named `generic.class`:

```
greet.penguin <- function(x, ...) paste("Hello,", x$name)
greet(x)
#> [1] "Hello, Adelie"
```

The dispatch path: `greet(x)` checks `class(x)`, finds `"penguin"`, looks for `greet.penguin`, calls it. If no method is found, R falls back to `greet.default` (if you defined one).

You can watch this happen with `sloop::s3_dispatch()`:

```
sloop::s3_dispatch(greet(x))
#> => greet.penguin
#>    greet.default
```

The `=>` arrow marks the method that was actually called. The `*` marks methods that exist but weren't called. This is useful for debugging inheritance chains (more on that in Section 24.4).

Exercises

1. Create an S3 class `"dog"` with fields `name` and `breed` (use `structure()` and a list). Write a `print.dog` method that prints something like `"Rex (Labrador)"`. Verify that `print(your_dog)` calls your method.
2. Write a `greet.default` method that returns `"I don't know how to greet this"`. Test it on a plain list without a class.
3. Use `sloop::s3_dispatch()` to see the dispatch path for `print(Sys.Date())`. Which method gets called?

24.3. Writing S3 classes properly

Setting a class with `structure()` works, but it's fragile. Nothing stops someone from creating a `"penguin"` with a numeric name or negative mass. The established best practice is three functions: a constructor, a validator, and a helper.

The **constructor** (`new_penguin`) creates the object with minimal checks. It is for developers:

```
new_penguin <- function(name, mass) {
  structure(list(name = name, mass = mass), class = "penguin")
}
```

The **validator** (`validate_penguin`) checks invariants. It ensures the object makes sense:

```
validate_penguin <- function(x) {
  if (!is.character(x$name) || length(x$name) != 1) {
    stop("`name` must be a single character string")
  }
  if (!is.numeric(x$mass) || length(x$mass) != 1 || x$mass <= 0) {
    stop("`mass` must be a single positive number")
  }
  x
}
```

The **helper** (`penguin`) is user-facing. It calls the constructor, then the validator:

```
penguin <- function(name, mass) {
  validate_penguin(new_penguin(name, mass))
}
```

Now compare:

```
# Direct construction: no safety net
bad <- structure(list(name = 42, mass = -100), class = "penguin")
```

```
# Helper: catches the problem
penguin(42, 3750)
#> Error in `validate_penguin()`:
#> ! `name` must be a single character string
```

The constructor is fast and trusting. The validator is thorough. The helper is what users call. This separation of concerns comes from Advanced R: construction is cheap, validation is careful, and the user interface is clean.

If you write S3 classes, write all three functions.

Exercises

1. Write the constructor/validator/helper trio for a `"book"` class with fields `title` (character), `pages` (positive integer), and `year` (four-digit number). Test the validator by trying to create a book with -50 pages.
2. Add a `print.book` method that prints something like `"Title (Year, N pages)"`.

24.4. Inheritance

S3 supports single inheritance via class vectors:

```
emperor <- structure(
  list(name = "Emperor", mass = 23000, dive_depth = 500),
  class = c("diving_penguin", "penguin")
)
class(emperor)
#> [1] "diving_penguin" "penguin"
```

When you call `greet(emperor)`, R looks for `greet.diving_penguin` first, then `greet.penguin`, then `greet.default`. The class vector is a priority list.

```
sloop::s3_dispatch(greet(emperor))
#>    greet.diving_penguin
#> => greet.penguin
#>    greet.default
```

No `greet.diving_penguin` exists, so R falls through to `greet.penguin`. You can define a specialized method and use `NextMethod()` to delegate to the parent:

```
greet.diving_penguin <- function(x, ...) {
  parent_greeting <- NextMethod()
  paste(parent_greeting, "(dives to", x$dive_depth, "m)")
}
```

```
greet(emperor)
#> [1] "Hello, Emperor (dives to 500 m)"
```

`NextMethod()` calls the next method in the chain, like `super()` in Python or Java. The diving penguin method adds dive information to whatever the penguin method already produces.

Keep inheritance shallow. One or two levels is typical in R. Deep hierarchies (five classes deep, overriding methods at every level) are rare and usually a sign you want a different design. If you find yourself building a tree of classes, step back and ask whether composition (storing one object inside another) would be simpler.

Exercises

1. Create a `"fiction_book"` class that inherits from `"book"` (from the previous exercise) and adds a `genre` field. Write a `print.fiction_book` method that uses `NextMethod()` and appends the genre.
2. Use `sloop::s3_dispatch()` on `print(your_fiction_book)` to see the dispatch chain.

24.5. Why S3 works

S3 is R's dominant OOP system, and the name tells you where it came from. When John Chambers designed the S language at Bell Labs in the early 1980s, he needed a way to make `print()` and `summary()` do different things for different kinds of objects. The solution was deliberately informal: attach a class string to any object, and let the generic function look up the right method by pasting the generic name and the class name together. No class registry, no formal type hierarchy, no compiler checks. Just a naming convention. This was S version 3, so the system became known as S3.

It was supposed to be temporary. By 1998, Chambers had designed S4, a formal OOP system with typed slots, multiple dispatch, and a class registry enforced by `setClass()` and `setGeneric()`. S4 was adopted by the Bioconductor project for its genomics infrastructure, but the rest of the R community kept using S3. S3 let package authors add methods to existing generics without coordination, which is exactly what a decentralized ecosystem of 20,000+ CRAN packages needs.

Three decades later, S7 arrived. A joint working group from R-Core, Bioconductor, and the tidyverse designed it to combine S3's simplicity with S4's safety: typed properties, built-in validation, clean syntax, and full backward compatibility with S3 dispatch.

S3 requires no registration, no formal definitions, and no boilerplate. You can add methods to existing generics from any package: write `print.my_class` in your package and it just works. You can prototype quickly: slap a class on a list and start writing methods.

The downside is obvious: no enforcement. Nothing stops you from creating a `"penguin"` with a numeric name or negative mass, unless you write the validator yourself. S3 trusts the programmer.

Opinion

S3 works because R programmers are disciplined. Follow the constructor/validator/helper pattern and you get most of the safety of a formal system with none of the ceremony. Skip the pattern and you get bugs that hide until production.

24.6. S4 in brief

S4 adds formality. `setClass()` defines a class with typed *slots*, `setGeneric()` creates a generic, and `setMethod()` registers a method:

```
setClass("Animal",
  slots = list(
    name = "character",
    mass = "numeric"
  )
)

setGeneric("describe", function(x, ...) standardGeneric("describe"))
setMethod("describe", "Animal", function(x, ...) {
  paste(x@name, "weighs", x@mass, "g")
})

a <- new("Animal", name = "Cat", mass = 4500)
a@name
describe(a)
```

Slots are accessed with `@` instead of `$`. Validation can be built in via the `validity` argument to `setClass()`.

S4's distinctive feature is *multiple dispatch*: `setMethod("combine", c("Matrix", "vector"), ...)` chooses a method based on the types of *both* arguments, not just the first. S3 dispatches on one argument only and cannot naturally express "when type A meets type B, do C." Julia (2012) made multiple dispatch its central paradigm. The trade-off is real: single dispatch is simpler, multiple dispatch is more expressive but harder to reason about.

S4 is used extensively in Bioconductor and some base R infrastructure (the Matrix package, the methods package). This book does not teach S4 in depth. If you need it, you will know: Bioconductor work, formal class hierarchies, or extending packages that already use S4. For most R programming, S3 or S7 is the right choice.

24.7. S7

S7 is available on CRAN. In May 2024, R-Core merged four patches into base R (version 4.4) that generalize the internal dispatch machinery to support systems like S7. Those patches

did not add S7 to base R, but they changed the foundation underneath it: `UseMethod()` and the method lookup tables now accommodate external OOP systems that follow S7's functional dispatch model. The long-term plan, stated by the R Consortium OOP Working Group, is to incorporate S7 into base R itself.

S3 attached a string to an object and hoped someone would write the right method. S4 replaced hope with a registry, typed slots, and `setClass()` declarations. `new_class()` returns a constructor function. `new_generic()` returns a function. You register methods by assigning to `method()`. The entire OOP system is made of functions you call, compose, and pass around — the same primitive this book has been building on since Chapter 5. The objects have typed properties and built-in validation, but the machinery that defines classes, creates generics, and dispatches methods is itself just functions.

Classes are created with `new_class()`:

```
library(S7)

penguin <- new_class("penguin",
  properties = list(
    name = class_character,
    mass = class_double
  ),
  validator = function(self) {
    if (self@mass <= 0) "mass must be positive"
  }
)

# penguin is now a constructor function:
p <- penguin(name = "Adelie", mass = 3750)
p
#> <penguin>
#>  @ name: chr "Adelie"
#>  @ mass: num 3750
```

Properties are typed and validated. You access them with `@`, like S4 slots, but the syntax is cleaner. The validator returns an error message string (or `NULL` if valid), rather than calling `stop()`.

Generics and methods use `new_generic()` and `method()`:

```
greet <- new_generic("greet", "x")
method(greet, penguin) <- function(x, ...) {
  paste("Hello,", x@name)
}

p <- penguin(name = "Adelie", mass = 3750)
p@name
#> [1] "Adelie"
```

```
greet(p)
#> [1] "Hello, Adelie"
```

Notice what you get for free: the class constructor (`penguin(...)`) is generated automatically with named arguments matching the properties. The validator runs on construction. Properties are type-checked. Compare this to the three-function pattern you had to build by hand in Section 24.3.

S7 over S3: properties have declared types (`class_character`, `class_double`, `class_integer`). Validation is built in, not bolted on. The syntax for generics and methods is explicit and clean. And S7 is fully compatible with S3: S7 methods can dispatch on S3 classes, and S3 methods can dispatch on S7 classes.

S7 over S4: simpler syntax. No `setClass`/`setGeneric`/`setMethod` ceremony. Builds on S3 dispatch, which everything already uses. Designed for the modern R ecosystem rather than being a parallel system.

Inheritance uses the `parent` argument:

```
diving_penguin <- new_class("diving_penguin",
  parent = penguin,
  properties = list(
    dive_depth = class_double
  )
)

emperor <- diving_penguin(name = "Emperor", mass = 23000, dive_depth = 500)
emperor
#> <diving_penguin>
#>  @ name      : chr "Emperor"
#>  @ mass      : num 23000
#>  @ dive_depth: num 500

greet(emperor)
#> [1] "Hello, Emperor"
# inherited from the penguin method
```

The child class inherits the parent's properties and methods. You can override methods for the child class the same way: `method(greet, diving_penguin) <- function(x, ...) ....`

Opinion

For new code, use S7 if you want validated properties and formal structure. Use S3 if you want simplicity and zero dependencies. Both are good choices. S4 is for existing codebases that already use it. Do not start new projects with S4 unless you are writing Bioconductor packages.

Exercises

1. Rewrite the `"book"` class from the earlier exercise as an S7 class with properties `title` (character), `pages` (integer), and `year` (integer). Add a validator that checks `pages > 0` and `year > 0`. (You will need `library(S7)` installed.)
2. Create an S7 generic `describe` and a method for your `book` class that returns `"Title (Year, N pages)"`.
3. Create a child class `"ebook"` with an additional `format` property (character, e.g. `"epub"` or `"pdf"`). Verify that the `describe` method is inherited.

25 Contracts and defensive code

You have written a function that computes a rate from two columns of a data frame. Someone passes it a data frame where one column is character instead of numeric. R does not crash; `mean()` on a character vector returns `NA` with a warning, and your function divides `NA` by a number and returns `NA`. The warning scrolls past in a long pipeline. The `NA` propagates through `mutate()`, `summarise()`, `left_join()`, and into a CSV that feeds a report. Three weeks later a collaborator asks why half the rates in the report are missing. You trace it back to a column that was read as character because one cell contained the string `"N/A"`.

Every line of code between the bad input and the discovery was wasted work. The function that produced the `NA` could have stopped the pipeline in the first line, with a message that said exactly what went wrong. It didn't, because nobody wrote that check. This was the problem a young man faced in the 1980s while working at Électricité de France, where a wrong number in a pipeline could mean a miscalculated load on a reactor.

After nine years of watching functions accept garbage and hand it downstream, he left and decided to build a programming language where that could not happen. His name was Bertrand Meyer; the language he was building was called Eiffel, which was defined by one central idea he called *Design by Contract*. It was simple: every function states a precondition (what must be true before it runs), a postcondition (what it guarantees after it runs), and an invariant (what stays true throughout). If a precondition fails, the caller broke the deal. If a postcondition fails, the function broke it. Either way, execution stops with a message that names the broken obligation.

R has no contract syntax. But `stop()` at the top of a function is a precondition. `stopifnot()` before the return value is a postcondition. The validator in a constructor/validator/helper pattern (Section 24.3) is a class invariant. The tools are manual; the discipline is the same. State your assumptions, halt the moment one is violated, and make the error message say what went wrong.

25.1. The case for validation

R is dynamically typed, which means nothing stops you from passing a character vector to a function that expects a number. Without validation, the error surfaces deep inside the function body, far from the actual mistake:

```
add_tax <- function(price, rate) {
  price * (1 + rate)
}

add_tax("42", 0.2)
#> Error in `price * (1 + rate)`:
#> ! non-numeric argument to binary operator
```

The error message says `non-numeric argument to binary operator`. If you already know what went wrong, that is enough; if you don't, it is useless. The real problem is that someone passed `"42"` instead of `42`, but R doesn't tell you that. It tells you about a multiplication that failed three levels deep.

With validation, the error surfaces immediately:

```
add_tax <- function(price, rate) {
  if (!is.numeric(price)) {
    stop(sprintf("price must be numeric, got %s", class(price)[1]))
  }
  price * (1 + rate)
}

add_tax("42", 0.2)
#> Error in `add_tax()`:
#> ! price must be numeric, got character
```

Check inputs at the boundary, where data enters your function, not in the guts where it is consumed. The principle is short: *fail fast, fail loud, fail clear*.

R has no static types, no compiler, no ahead-of-time guarantees. Every precondition you want enforced must be enforced by code you write. That sounds like a burden, but it has an upside: every assumption is visible in the function body when you revisit it six months later. `if (!is.numeric(price)) stop(...)` is verbose, but it is also impossible to miss.

The Ariane 5 rocket self-destructed 37 seconds after launch in 1996 because a 64-bit float was silently converted to a 16-bit integer, overflowing without a check. A single `stop()` equivalent would have saved a $370 million rocket. A 2003 NASA study found that 40% of aerospace software failures came from *missing* code: checks the programmers never wrote. When your function returns `NA` instead of erroring on bad input, you are in the same category of silent failure. `stop()` with a clear message forces the caller to deal with the problem immediately, before the bad value propagates through fifty lines of downstream code.

> **The type theory connection**
>
> A `stop()` that fires on wrong input is a runtime proof by contradiction: "if this value were numeric, execution would continue; it isn't, so we halt." The Curry-Howard correspondence formalizes this: types are propositions, programs are proofs, and a function with signature `A -> B` is a proof that "if A then B." Dependently typed languages (Idris, Agda) encode preconditions in the type system itself, making runtime `stop()` calls unnecessary. R sits at the opposite end of that spectrum. The question both approaches answer is the same: "what must hold for this computation to be sound?"

(This is about functions you write for yourself and for others, not about validating user input in a Shiny app.)

25.2. `stop()`, `warning()`, and `message()`

R gives you three signalling functions, each pitched at a different severity. The differences matter more than you might think.

`stop("msg")` halts execution and signals an error. Use it when the function cannot produce a correct result.

```
divide <- function(x, y) {
  if (y == 0) stop("y must not be zero")
  x / y
}

divide(10, 0)
#> Error in `divide()`:
#> ! y must not be zero
```

`warning("msg")` continues execution but signals a warning, which makes it appropriate when the function can proceed but the result might surprise the caller.

```
safe_log <- function(x) {
  if (any(x <= 0)) warning("non-positive values replaced with NA")
  ifelse(x > 0, log(x), NA)
}

safe_log(c(1, -2, 3))
#> Warning in safe_log(c(1, -2, 3)): non-positive values replaced with NA
#> Warning in log(x): NaNs produced
#> [1] 0.000000       NA 1.098612
```

`message("msg")` prints informational output: progress updates, status reports, diagnostics. `suppressMessages()` silences these, which is why `message()` is better than `cat()` for status

output. It respects the caller's wishes.

```
load_data <- function(path) {
  message(sprintf("Reading %s...", path))
  # ... actual loading code ...
}
```

The decision rule: if the result would be wrong, `stop()`. If the result is correct but surprising, `warning()`. If you are just providing status, `message()`. But where exactly is the line between "surprising" and "wrong"?

> **Opinion**
>
> Never use `warning()` when you should use `stop()`. A warning that scrolls past unread is worse than no warning at all; it gives you the illusion of safety while the bad value keeps propagating. If you are not confident the function produced a correct result, don't return one.

25.3. Writing informative errors

The difference between a helpful error and a frustrating one is the difference between ten seconds of debugging and ten minutes. Most of the gap comes down to one thing: does the message tell you what was expected and what was received?

```
# Bad
stop("invalid input")

# Good
stop(sprintf("x must be numeric, got %s", class(x)[1]))

# Better
stop(sprintf(
  "x must be a single positive number, got %s of length %d",
  class(x)[1], length(x)
))
```

Say what was *expected*, say what was *received*. The caller should be able to fix the problem without reading your source code.

`stopifnot()` is base R's assertion function, terse and good for internal checks where you trust the caller to interpret the output:

```
normalize <- function(x) {
  stopifnot(is.numeric(x), length(x) > 0)
  x / sum(x)
```

```
}

normalize("abc")
#> Error in `normalize()`:
#> ! is.numeric(x) is not TRUE
```

The auto-generated error message is functional but cryptic. Since R 4.0, you can name the assertions to control the message:

```
normalize <- function(x) {
  stopifnot(
    "x must be numeric" = is.numeric(x),
    "x must not be empty" = length(x) > 0
  )
  x / sum(x)
}

normalize("abc")
#> Error in `normalize()`:
#> ! x must be numeric
```

Named `stopifnot()` gives you the conciseness of assertions with messages you control. For functions that other people will call, this is the minimum standard. But what happens when you have ten arguments to validate, each with its own type, range, and length constraint?

Exercises

1. The following function has a bad error message. Rewrite it so the error says what was expected and what was received.

```
compute_bmi <- function(weight_kg, height_m) {
  if (!is.numeric(weight_kg) || !is.numeric(height_m)) {
    stop("bad input")
  }
  weight_kg / height_m^2
}
```

2. Rewrite the validation using named `stopifnot()`. Does the error message improve?
3. What error message does `stopifnot(is.numeric("hello"))` produce? What about `stopifnot("x must be numeric" = is.numeric("hello"))`?

25.4. Common validation patterns

A function that expects a numeric scalar but receives a character vector of length three will fail somewhere. The question is where. These patterns catch the problem at the door.

- **Type checks**: `is.numeric(x)`, `is.character(x)`, `is.logical(x)`, `is.data.frame(x)`, `is.list(x)`.
- **Length checks**: `length(x) == 1` for scalars, `length(x) > 0` for non-empty inputs.
- **Range checks**: `x > 0` for positive, `x >= 0 && x <= 1` for probabilities.
- **NULL and NA handling**: `is.null(x)` and `anyNA(x)`.

```
if (is.null(x)) stop("x must not be NULL")
if (anyNA(x)) stop("x must not contain NA values")
```

File existence. `file.exists(path)`.

```
if (!file.exists(path)) stop(sprintf("file not found: %s", path))
```

Set membership. This pattern deserves special attention: `match.arg()`.

```
my_summary <- function(x, method = c("mean", "median", "trimmed")) {
  method <- match.arg(method)
  switch(method,
    mean    = mean(x),
    median  = median(x),
    trimmed = mean(x, trim = 0.1)
  )
}
```

`match.arg()` does three things at once. It validates that the argument is one of the allowed choices. It supports partial matching, so `my_summary(1:10, "med")` works. And it picks the first option as the default, so calling `my_summary(1:10)` uses `"mean"`. All of this comes from the function signature, which means the valid choices are visible in the documentation and in `?my_summary`.

```
my_summary(1:10)
#> [1] 5.5
my_summary(1:10, "median")
#> [1] 5.5
my_summary(1:10, "tri")
#> [1] 5.5
```

```
my_summary(1:10, "xyz")
#> Error in `match.arg()`:
#> ! 'arg' should be one of "mean", "median", "trimmed"
```

The error message from `match.arg()` is clear and lists the valid options, which is why it is preferred over manual `%in%` checks for string arguments with fixed options. But all of these patterns share a problem: writing them out by hand, over and over, for every function in a package.

Exercises

1. Write a function `rescale()` that takes a numeric vector `x` and a `method` argument accepting `"minmax"` or `"zscore"`. Use `match.arg()` for validation. Test that partial matching works.
2. Add validation to the following function using `stopifnot()` or `if/stop()`. Check that `data` is a data frame, `n` is a single positive integer, and `col` is a string that exists as a column name.

```
top_rows <- function(data, col, n = 5) {
  data[order(data[[col]], decreasing = TRUE), ][seq_len(n), ]
}
```

25.5. The checkmate package

Writing the same `if (!is.numeric(x)) stop(...)` pattern for the fifteenth time in a package gets old fast, and each repetition is another chance to misspell a class name or forget a length check. The `checkmate` package provides concise, fast assertion functions that handle the boilerplate:

```
library(checkmate)

process <- function(data, method = "fast", threshold = 0.5) {
  assert_data_frame(data, min.rows = 1)
  assert_choice(method, choices = c("fast", "accurate"))
  assert_number(threshold, lower = 0, upper = 1)
  # ...
}
```

Each `assert_*` function throws an informative error if the check fails. No `if/stop` boilerplate, no `sprintf` formatting. The function names are self-documenting: `assert_numeric`, `assert_character`, `assert_file_exists`, `assert_flag` (for single TRUE/FALSE), and dozens more.

`checkmate` also provides two softer variants. `test_*` functions return `TRUE` or `FALSE` for use in conditional logic; `check_*` functions return the error message as a string (or `TRUE` on success) for custom handling.

```
# Conditional logic
if (test_numeric(x)) {
  mean(x)
} else {
  length(x)
}

# Custom error message
msg <- check_numeric(x, lower = 0)
if (!isTRUE(msg)) stop("Custom prefix: ", msg)
```

Opinion

For package code with many validated arguments, `checkmate` pays for itself in readability and consistency. For scripts and one-off functions, `stopifnot()` and `match.arg()` are enough. Don't add a dependency for two assertions.

25.6. `tryCatch()` and error handling

Everything so far has been about signalling your own errors. But sometimes the error comes from somewhere else: a file that doesn't exist, a web API that times out, a parsing function that chokes on malformed input. You need to *catch* those errors and decide what to do about them.

`tryCatch()` catches an error, warning, or message and runs a handler:

```
result <- tryCatch(
  log("a"),
  error = function(e) NA
)

result
#> [1] NA
```

Instead of crashing, `log("a")` returns `NA`. The error handler receives the condition object `e` and returns a fallback value. This is *recovery*: you know an error might happen, and you have a plan for it.

`try()` is a simpler wrapper that returns the result on success, or an object of class `"try-error"` on failure:

```
result <- try(log("a"), silent = TRUE)
class(result)
#> [1] "try-error"
```

`try()` is convenient for quick scripts, but `tryCatch()` gives you more control.

For functional programming, `purrr::safely()` and `purrr::possibly()` wrap a function to make it error-tolerant. Together with `tryCatch()`, these form R's condition handling system. `safely()` returns a list with `$result` and `$error`, while `possibly()` returns a default value on failure. Both are designed for use with `map()` when applying a function that might fail on some elements:

```
safe_log <- purrr::safely(log)
safe_log(10)
#> $result
#> [1] 2.302585
#>
#> $error
#> NULL
safe_log("a")
#> $result
#> NULL
#>
#> $error
#> <simpleError in .f(...): non-numeric argument to mathematical function>
```

Erlang (1986, Ericsson) takes this philosophy to its extreme: instead of preventing crashes with validation, Erlang designs *supervisors* that detect and restart failed processes. Ericsson's telecom switches achieved 99.9999999% uptime using this "let it crash" philosophy, and R's `tryCatch()` is a mild version of the same idea. Both philosophies agree on the essential point: silent failures are the real enemy. But what does a disciplined version of "catch and recover" look like as a data structure?

In functional programming, the answer is the *Either* type (or *Result* in Rust): a computation either succeeds with a value or fails with an error. `tryCatch` runs the expression; if it succeeds, you get the result; if it throws, you get the error handler's output. This is the same "value or absence" logic as `NA` propagation (Section 4.6), but for errors instead of missing data. `NA` implements Maybe (present or absent); `tryCatch` implements Either (success or failure). Both thread the possibility of failure through a computation without crashing. Haskell and Rust make these patterns explicit with monadic types; R keeps them implicit in `NA` semantics and `tryCatch` mechanics, but the underlying structure is identical.

> **Opinion**
>
> Use `tryCatch()` when you have a recovery plan. Don't use it to silently swallow errors. An error you hide is a bug you invite.

Exercises

1. Write a function `safe_read` that takes a file path and returns the file contents (via `readLines()`), or `NULL` if the file doesn't exist. Use `tryCatch()`.
2. Use `purrr::map()` and `purrr::possibly()` to apply `as.numeric()` to the list `list("1", "two", "3", "four")`. Use `NA` as the default.

25.7. `rlang::abort()` and cli

`rlang::abort()` is a modern replacement for `stop()` that automatically saves a backtrace (the chain of function calls that led to the error) and supports *condition classes* for selective catching:

```
rlang::abort(
  message = "x must be positive",
  class = "validation_error",
  x = x
)
```

A caller can now catch `validation_error` specifically, while letting other errors propagate:

```
tryCatch(
  my_function(x),
  validation_error = function(e) {
    message("Validation failed: ", e$message)
  }
)
```

The `cli` package takes this further. `cli::cli_abort()` produces structured, formatted error messages with bullets and inline markup:

```
cli::cli_abort(c(
  "x" = "{.arg x} must be a positive number.",
  "i" = "You supplied {.cls {class(x)}} of length {length(x)}."
))
```

The `"x"` prefix marks the main error; `"i"` marks informational context. `{.arg x}` and `{.cls ...}` are cli's inline formatting: they render the argument name and class with special styling in terminals that support it.

If you have ever wondered why tidyverse error messages look so polished, with colored bullets and formatted argument names, `cli::cli_abort()` is the answer. You don't need cli-formatted errors in scripts, but knowing they exist helps you read the error messages that tidyverse packages produce.

`rlang` also provides `rlang::arg_match()`, a stricter alternative to `match.arg()` that produces tidyverse-styled error messages and does *not* support partial matching, which makes it safer for package APIs where partial matching can cause subtle bugs.

Exercises

1. Rewrite one of your `stop()` calls from the exercises above using `rlang::abort()` with a custom class (e.g., `"input_error"`). Then write a `tryCatch()` that catches only that class.

2. Use `cli::cli_abort()` to produce an error message with a bullet point showing the expected type and the actual type. Compare the output with a plain `stop()` message.

25.8. Designing function interfaces

Validation is downstream of design. A well-designed function is hard to misuse in the first place, which means much of your defensive code never needs to exist.

Required arguments have no defaults. Optional arguments have sensible defaults. If a function needs a file path to work, don't give `path` a default of `NULL` and then check for it inside the body. Make the caller provide it.

Use `match.arg()` for string options. The valid choices are visible in the function signature and in the help page, so the caller doesn't need to read the source code to know what's allowed.

```
# Good: choices are visible
my_plot <- function(data, type = c("scatter", "line", "bar")) {
  type <- match.arg(type)
  # ...
}

# Bad: choices are hidden inside the body
my_plot <- function(data, type = "scatter") {
  if (!type %in% c("scatter", "line", "bar")) stop("invalid type")
  # ...
}
```

Put the data first. This makes the function pipe-friendly (`data |> my_func()`). Modifier arguments come after.

Use `...` sparingly. If you use it, document what it accepts and where it gets forwarded. An undocumented `...` is a trap: the caller passes a misspelled argument, `...` absorbs it silently, and no error is raised. (This is one of R's most common sources of invisible bugs.)

Don't overload argument types. An argument `x` that can be a vector, a data frame, or a formula is hard to validate and hard to use, because each accepted type needs its own validation branch and its own documentation. If you find yourself writing `if (is.data.frame(x)) ... else if (is.numeric(x)) ...`, consider splitting the function or using S3 dispatch.

This connects to Chapter 24: S7 properties with type declarations are validation built into the class definition. When you declare that a property must be a positive numeric scalar, invalid objects cannot exist. Validation at construction time, enforced by the type system, is the strongest form of defensive programming.

Opinion

The best validation is the kind you don't have to write. A function with a clear signature, named options via `match.arg()`, and typed S7 properties catches most mistakes before any `if/stop` check runs. Design the interface right, and defensive code becomes a safety net instead of a load-bearing wall.

25.9. Summary

Silent failures are the worst kind of bug: the function returns an answer, and the answer is wrong, and nothing tells you. Validate at the boundary. Write error messages that say what was expected and what was received. Catch errors only when you have a recovery plan. And design interfaces that make misuse difficult in the first place, so the validation you write covers less ground.

26 Metaprogramming

You have been writing `lm(y ~ x)` since Chapter 17, and `filter(penguins, species == "Adelie")` since Chapter 14. Both do something that might seem impossible: `y ~ x` is not evaluated before `lm()` sees it, and `species` is found inside a data frame, not in your environment. Normally, `1 + 1` evaluates to `2` and the expression disappears. R can keep it. Code in R is a data structure you can hold, examine, rearrange, and run whenever you choose.

That difference has a name: metaprogramming. Code that operates on code. You have been using it from your first formula, your first call to `aes()`, your first dplyr pipeline, all of it built on the machinery this chapter describes. This chapter shows you the mechanism.

26.1. Code is data

`quote()` captures an expression without evaluating it:

```
quote(x + 1)
#> x + 1
```

No result. What you got back is the expression itself: `x + 1` was not computed but returned as a data structure (a *call object*) that you can store, inspect, and eventually evaluate. If that sounds strange, think of it through lambda calculus: `(x. x + 1)(3)` reduces to 4, but if you "quote" the term you get the syntactic object `(x. x + 1)(3)`, something you can pull apart and study without triggering the reduction. `quote(1 + 1)` in R gives you the expression tree, not the value 2.

```
e <- quote(x + 1)
typeof(e)
#> [1] "language"
class(e)
#> [1] "call"
```

The rlang package provides `expr()`, which does the same thing:

```
rlang::expr(x + 1)
#> x + 1
```

To run a captured expression, use `eval()`:

```
x <- 10
eval(e)
#> [1] 11
```

`eval()` takes the frozen expression and evaluates it in the current environment, where `x` is 10, so the result is 11. The cycle is always the same: capture code, possibly modify it, then evaluate it somewhere. But why would you want to?

In Section 1.2, Church's lambda calculus treated functions as data that could be passed around and applied. R extends that principle to *all code*, not only functions. An expression is a value, exactly like a number or a string; you can assign it, put it in a list, pass it to a function, transform it, then run the transformed version. In Section 7.5, you saw that `+` is a function and `2 + 3` is a function call. That function call is also a data structure you can capture and manipulate before it ever executes.

R inherited this idea from Lisp, where `(quote (+ 1 2))` returns the list `(+ 1 2)` instead of computing 3. A language whose code has the same structure as its data is called *homoiconic*: Lisp code is lists, Lisp data is lists; R code is call objects (trees), and R data includes call objects. Python, Java, and C++ are not homoiconic — their code is text, and metaprogramming means parsing strings, which is fragile. R's homoiconicity means it gets this for free.

Python's `inspect.getsource()` returns a string you then have to parse; Java's reflection is a separate API with its own rules and sharp edges. R skips both steps. `quote(x + 1)` gives you the syntax tree directly — a call object you can subset with `[[`, modify with replacement, and evaluate in any environment you choose. That tree is an ordinary R object. It lives in memory, responds to R's standard operations, and you can pass it to functions like any vector or list. Tidy evaluation, formula interfaces, and ggplot2's `aes()` all rest on that foundation.

26.2. Abstract syntax trees

Every R expression has a tree structure called an *abstract syntax tree* (AST). The `lobstr` package makes them visible:

```
lobstr::ast(x + y * 2)
#>  `+`
#>  x
#>   `*`
#>    y
#>    2
```

What looks like a flat sequence of tokens on the page is actually a tree where `+` sits at the root, `x` hangs off one branch, and `* (y, 2)` hangs off the other. The tree encodes precedence: `*` binds tighter than `+`, so `y * 2` forms a subtree nested inside the `+` node, and R knows to evaluate it first without needing parentheses from you.

Three kinds of nodes make up every AST:

- **Constants**: `1`, `"hello"`, `TRUE`. These are leaves. They have no children.
- **Symbols** (names): `x`, `y`, `mean`. Also leaves. They represent name lookups: when evaluated, R searches for the value bound to that name.
- **Calls**: function applications. These are branches. The first child is the function, the rest are arguments.

That is the entire vocabulary. `x + 1` is a call to `` `+` `` with arguments `x` and `1`. `if (x > 0) "yes" else "no"` is a call to `` `if` `` with three arguments. There is no special syntax that escapes the tree; even control flow is just function calls wearing different clothes.

```
lobstr::ast(if (x > 0) "yes" else "no")
#>  `if`
#>   `>`
#>    x
#>    0
#>  "yes"
#>  "no"
```

You can take a call object apart with standard list operations, where the first element is the function and the rest are the arguments:

```
e <- quote(mean(x, na.rm = TRUE))
e[[1]]
#> mean
e[[2]]
#> x
e[[3]]
#> [1] TRUE
```

`as.list()` converts the whole call into a list, which makes the structure easy to see at a glance:

```
as.list(e)
#> [[1]]
#> mean
#>
#> [[2]]
#> x
#>
#> $na.rm
#> [1] TRUE
```

You can convert between text and expressions with `parse()` and `deparse()`:

```
deparse(quote(x + y * 2))
#> [1] "x + y * 2"
parse(text = "x + y * 2")[[1]]
#> x + y * 2
```

The conversion is not perfectly symmetric; comments and whitespace vanish in the round trip. But the tree structure survives, and the tree is what matters. So what happens when you need to capture not your own expression but someone else's?

Exercises

1. Draw the AST for `f(a, g(b, c))` on paper. Then check your answer with `lobstr::ast()`.
2. What does `lobstr::ast(1 + 2 + 3)` look like? Is `+` left-associative or right-associative?
3. Use `lobstr::ast()` to visualize `x[1]`. What function is at the root?

26.3. Capturing and evaluating expressions

Capturing *your own* expression and capturing *the caller's* expression require different tools, and confusing the two is a reliable source of bugs.

`quote()` and `expr()` capture what you type directly:

```
quote(a + b)
#> a + b
rlang::expr(a + b)
#> a + b
```

`substitute()` captures what the caller passed:

```
f <- function(x) substitute(x)
f(a + b)
#> a + b
```

You called `f(a + b)`, and `substitute(x)` reached back across the function boundary to the call site, grabbing `a + b`, the expression the caller actually wrote. The same mechanism powers `dplyr::filter()`: it sees `species == "Adelie"` as an expression rather than immediately evaluating it and getting an error about a missing variable.

The rlang equivalents are `enexpr()` (captures an expression) and `enquo()` (captures an expression plus its environment, forming a *quosure*). The decision rule is simple: use `substitute()` in base R code, use `enexpr()`/`enquo()` in tidyverse/rlang code. If your function will be called by dplyr or other tidy-evaluation-aware code, reach for the rlang tools because they integrate with `!!`, `{ }`, and `eval_tidy()`. If you are writing standalone base R, `substitute()` and `eval()` are sufficient and carry zero dependencies.

```
g <- function(x) rlang::enexpr(x)
g(a + b)
#> a + b
```

Once you have an expression, you evaluate it with `eval()`. The second argument controls *where*:

```
e <- quote(x + 1)
eval(e, list(x = 10))
#> [1] 11
eval(e, list(x = 100))
#> [1] 101
```

The same frozen expression, evaluated in different environments, gives different results. Data masking is exactly that trick: `eval_tidy()` from rlang evaluates an expression against a data frame, so `filter(penguins, species == "Adelie")` looks up `species` in the data rather than in the global environment.

```
library(rlang)
df <- data.frame(x = c(1, 2, 3), y = c(10, 20, 30))
eval_tidy(expr(x + y), data = df)
#> [1] 11 22 33
```

Tidy evaluation calls this pattern defuse-and-inject: capture, optionally modify, then evaluate in the right context.

One more base R tool worth knowing: `match.call()`. Inside a function, it returns the entire call as the user typed it, with arguments matched by name:

```
my_lm <- function(formula, data, subset = NULL) {
  match.call()
}
my_lm(y ~ x, data = mtcars)
#> my_lm(formula = y ~ x, data = mtcars)
```

Many modeling functions use `match.call()` to record the call for reproducibility. When you print a fitted model and see `Call: lm(formula = y ~ x, data = mtcars)`, that string came from `match.call()` stashing the original invocation.

Exercises

1. Write a function `show_code` that takes an argument and prints the expression the caller passed (use `substitute()` and `deparse()`). Test: `show_code(mean(x, na.rm = TRUE))` should print `"mean(x, na.rm = TRUE)"`.
2. Evaluate `quote(x * 2)` in an environment where `x = 7`. Then evaluate it where `x = -3`.

3. Write a function that uses `match.call()` to return its own call. Call it with several arguments and observe the output.

26.4. Building expressions programmatically

What if the function name or the variable is not known until runtime, when your code has to decide at the last moment what expression to build? You need to construct the expression from parts.

`rlang::call2()` constructs a call object:

```
rlang::call2("+", 1, 2)
#> 1 + 2
eval(rlang::call2("+", 1, 2))
#> [1] 3
```

The next step is *quasiquotation*: building an expression template with holes that get filled in at construction time. The `!!` (bang-bang) operator injects a value into an expression:

```
my_var <- rlang::expr(body_mass_g)
rlang::expr(mean(!!my_var))
#> mean(body_mass_g)
```

`!!` replaced `my_var` with its value (`body_mass_g`), producing the expression `mean(body_mass_g)`. Without `!!`, you would get `mean(my_var)`, which is an entirely different expression and not the one you wanted.

`!!!` (triple bang, or splice) injects a list of expressions as separate arguments:

```
vars <- rlang::exprs(species, island)
rlang::expr(group_by(penguins, !!!vars))
#> group_by(penguins, species, island)
```

This is what `{ }` (embrace) from tidy evaluation does under the hood. When you write a function like:

```
my_summary <- function(data, var) {
  data |> dplyr::summarise(mean = mean({{ var }}, na.rm = TRUE))
}

my_summary(palmerpenguins::penguins, body_mass_g)
#> # A tibble: 1 x 1
#>    mean
#>   <dbl>
#> 1 4202.
my_summary(palmerpenguins::penguins, flipper_length_mm)
```

```
#> # A tibble: 1 x 1
#>    mean
#>   <dbl>
#> 1  201.
```

the embrace operator defuses `var` with `enquo()` and injects it with `!!`, which means it is syntactic sugar for the defuse-and-inject pattern. The caller writes bare column names, exactly as they would with dplyr directly, and your function forwards them transparently, no quoted strings, no special syntax at the call site.

Base R has `bquote()` for quasiquotation, using `.()` instead of `!!`:

```
my_var <- quote(body_mass_g)
bquote(mean(.(my_var)))
#> mean(body_mass_g)
```

It works, but `bquote()` is less common in practice and does not support splicing. All of these tools, `!!`, `bquote()`, `expr()`, solve the same problem that Lisp macros have addressed since the 1960s: how to write code that writes code, safely, without collapsing into string manipulation. When Wickham designed tidy evaluation, he was adapting that lineage for R, and the result is that you can generate expressions with the same structural guarantees that `quote()` gives you by hand.

Exercises

1. Use `rlang::call2()` to build the expression `sqrt(16)`, then evaluate it.
2. Create a variable `col <- rlang::expr(bill_length_mm)`. Use `!!` to build the expression `mean(bill_length_mm, na.rm = TRUE)`.
3. Given `fns <- rlang::exprs(mean, sd, median)`, use `lapply()` and `call2()` to build three expressions: `mean(x)`, `sd(x)`, `median(x)`.

26.5. Formulas as expressions

Before anyone used the word "metaprogramming" in an R context, there were formulas. When you write:

```
lm(body_mass_g ~ bill_length_mm, data = palmerpenguins::penguins)
#>
#> Call:
#> lm(formula = body_mass_g ~ bill_length_mm, data = palmerpenguins::penguins)
#>
#> Coefficients:
#>    (Intercept)  bill_length_mm
#>         362.31           87.42
```

the expression `body_mass_g ~ bill_length_mm` is not evaluated in the ordinary sense. R captures it as a *formula object*: two expressions (the left-hand side and the right-hand side) bundled together with the environment where the formula was created.

```
f <- y ~ x + z
typeof(f)
#> [1] "language"
length(f)
#> [1] 3
f[[2]]
#> y
f[[3]]
#> x + z
```

A formula stores its terms as call objects. `f[[2]]` is the left-hand side (`y`), `f[[3]]` is the right-hand side (`x + z`). The formula also carries an environment attribute:

```
environment(f)
#> <environment: R_GlobalEnv>
```

Formulas work across function boundaries because the formula remembers where its variables should be looked up. Quosures in tidy evaluation solve the same problem: an expression bundled with its environment. Wickham borrowed the concept directly from formulas, and their 30-year track record in R's modeling ecosystem is what made the design credible.

The formula language (`+`, `*`, `:`, `-`, `I()`) is a domain-specific language for specifying models. `y ~ x1 * x2` does not mean "multiply x1 by x2." It means "include x1, x2, and their interaction." `model.matrix()` interprets these operators to build the design matrix:

```
model.matrix(~ species + island, data = palmerpenguins::penguins) |> head()
#>   (Intercept) speciesChinstrap speciesGentoo islandDream islandTorgersen
#> 1           1                0             0           0               1
#> 2           1                0             0           0               1
#> 3           1                0             0           0               1
#> 4           1                0             0           0               1
#> 5           1                0             0           0               1
#> 6           1                0             0           0               1
```

You can also build formulas dynamically, which becomes useful when the set of predictors is not known in advance:

```
predictors <- c("bill_length_mm", "flipper_length_mm")
f <- as.formula(paste("body_mass_g ~", paste(predictors, collapse = " + ")))
f
#> body_mass_g ~ bill_length_mm + flipper_length_mm
```

Constructing code from data, then running it: that is metaprogramming at its most practical.

The formula is a piece of code assembled from strings, about to be interpreted by `lm()` as a model specification, and neither the user nor the modeling function needs to know it was built programmatically.

The connection to Chapter 23 is direct. Formulas are non-standard evaluation: the variable names are unquoted, `body_mass_g` is looked up in `penguins` rather than in the calling environment. Dplyr uses the same trick, just with newer machinery.

Exercises

1. Create a formula `y ~ x1 + x2` and extract its right-hand side.
2. Build a formula programmatically: given `response <- "mpg"` and `predictors <- c("wt", "hp")`, construct the formula `mpg ~ wt + hp` using `as.formula()` and `paste()`. Pass it to `lm()` with the `mtcars` dataset.

26.6. When to use metaprogramming

Metaprogramming lets you generate code, build DSLs, and eliminate boilerplate. That power comes with a cost.

Good uses:

- **Domain-specific languages.** Model formulas, ggplot2 aesthetics, dplyr pipelines. These are all DSLs built on metaprogramming. If you are building a package with an interactive interface that benefits from concise, expressive syntax, metaprogramming is the right tool.
- **Code generation.** Building model specifications programmatically, creating batches of test cases, generating reports from templates.
- **Debugging and inspection.** `substitute()` to see what was passed, `match.call()` to record the exact call for reproducibility.

Bad uses:

- **Things that work with regular functions.** If `f(x)` solves the problem, do not complicate it with `eval(substitute(...))`.
- **Performance.** Metaprogramming is not faster. It is more flexible, which is a different axis entirely.
- **Showing off.** Code that manipulates code is hard to read and hard to debug. Use it when the benefit (a concise user interface) outweighs the cost (a complex implementation), not before.

Opinion

Most R users consume metaprogramming (by using dplyr, ggplot2, formulas) and very few need to produce it. Package authors building interactive interfaces may need these tools; analysts writing data pipelines almost certainly do not. The test: does your user-facing API become meaningfully better with non-standard evaluation? If yes, the complexity is worth absorbing. Saving yourself a quoted string is not enough reason.

26.7. The metaprogramming toolkit

For further study:

- **Tidy evaluation** (rlang): `expr()`, `enquo()`, `eval_tidy()`, `!!`, `!!!`, `{ }`. The system behind dplyr, tidyr, and ggplot2. Covered in depth in Advanced R, chapters 17 through 20.
- **Base R tools**: `quote()`, `substitute()`, `eval()`, `match.call()`, `sys.call()`, `bquote()`. These predate rlang and still power much of R's infrastructure. Every R programmer should know `substitute()` and `eval()`.
- **Inspection packages**: `lobstr` for AST visualization, `pryr` for probing environments and function internals.
- **Further reading**: Wickham's *Advanced R* (2nd edition) devotes four chapters to metaprogramming. Mailund's *Metaprogramming in R* (Springer, 2017) is a book-length treatment covering domain-specific languages and code generation.

Exercises

1. Look at the source of `dplyr::filter` (type `dplyr::filter.data.frame` at the console). Can you spot where it captures the user's expressions?
2. Compare `quote()`, `rlang::expr()`, `substitute()`, and `rlang::enexpr()`. Write one sentence describing when you would use each.

27 Building a DSL

Suppose you write `y ~ x1 + x2` and R does not add anything. The plus sign, which in every other context means addition, now means "include this predictor in the model." The tilde computes nothing; it freezes two expressions into an object for later interpretation. You have been using this notation since your first linear regression, probably without asking the obvious question: how does R let ordinary operators mean something completely different depending on context? The answer runs through everything you have learned so far: non-standard evaluation, operator overloading, and S3 dispatch combine to let you rewrite what R's syntax means.

You already have the pieces. `quote()` captures code as data (Section 26.1), `substitute()` captures the caller's code (Section 26.3), S3 dispatch selects behavior by type (Section 24.1), and operators are just functions (Section 7.5). This chapter puts them together: you will dissect DSLs you already use, then build one from scratch.

A domain-specific language, or DSL, is a small language designed for one task. SQL queries data. Regular expressions match text. HTML structures documents. These are *external* DSLs with their own parsers, their own syntax, their own tooling. R excels at something different: *internal* DSLs, mini-languages embedded inside R itself, using R's own syntax but giving it new meaning. The formula interface, ggplot2's grammar of graphics, dplyr's verb chains, data.table's `[i, j, by]` notation: each lives inside R and plays by R's rules (mostly). What makes R unusually good at this is non-standard evaluation and operator overloading: the ability to capture user expressions without evaluating them, reinterpret what operators mean, and dispatch behavior through S3.

By the mid-1960s, programmers were drowning in general-purpose syntax that forced every domain into the same mold. Peter Landin argued in 1966 that they should be able to build "a language appropriate to the problem." Three decades later, Guy Steele's keynote "Growing a Language" made the same case from the other side: a good language is one that users can extend until it fits the domain. R's formula interface, which dates to S in the 1980s, predates both papers, but the instinct is the same: capture code, give it new meaning.

27.1. R's existing DSLs

The test of a DSL is not cleverness of syntax but consistency of mapping: every domain concept corresponds to a language construct, and the correspondence holds without exceptions

that the user must memorize. R's existing DSLs each take a different approach to achieving this.

Formulas. `y ~ x1 + x2` is R's oldest DSL. The `~` operator creates a formula interface object storing two expressions (the left- and right-hand sides) plus the environment where it was created; it computes nothing. Inside the formula, `+` does not mean addition but "include this term," `*` means "include main effects and their interaction," and `:` means "interaction only." This is code that means something completely different from its usual meaning, interpreted not by R's evaluator but by `model.matrix()`. You explored this in Section 26.5. But formulas freeze structure at definition time. What if you want to build structure incrementally, adding pieces one at a time?

ggplot2. The `+` operator for ggplot objects does not add numbers. It layers graphical components:

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm")
```

`aes()` captures column names as expressions without evaluating them, while `+.gg` (the S3 method for `+` on ggplot objects) takes the existing plot and attaches the new layer. This is the `+` operator overloading pattern in ggplot2. The plot builds incrementally, one `+` at a time, accumulating a list of layers, scales, and coordinates. This is an expression builder pattern: each `+` returns a new (or modified) plot object, and the final object is rendered only when printed. Incremental building solves the formula problem, but it introduces a new one: every `+` is a standalone layer, with no awareness of what the other layers are doing. What if the verbs themselves need to know about the data?

dplyr. `filter(penguins, species == "Adelie")` captures `species == "Adelie"` as an expression and evaluates it against the data frame using data masking (Section 26.3). The pipe operator chains verbs together, each doing one thing: filter rows, select columns, create new columns, summarize groups. Because the verbs are S3 generics, they dispatch on the data source. `filter.data.frame` works on data frames; `filter.tbl_lazy` generates SQL. Same syntax, different backend. That generality comes at a cost, though: each verb is a separate function call, and the pipe threading them together is syntactic, not semantic. Could you collapse the whole pipeline into a single expression?

data.table. `DT[i, j, by]` overloads the `[` operator so that all three arguments are captured by `substitute()` and evaluated in the data.table's scope. No pipe, no chain of function calls. Everything happens inside `[`, which is both concise and dense.

Each DSL makes a different trade-off: formulas redefine arithmetic operators inside a special object, ggplot2 overloads `+` on a class, dplyr uses data masking and generic dispatch, data.table overloads `[`. The technique varies, but the principle does not. Capture code, give it new meaning. What happens when you try this yourself?

Exercises

1. Type `ggplot2::aes` at the console (no parentheses) and read the source. Where does it capture the user's expressions? What function does it use?
2. Look at the source of `ggplot2`'s internal `"+.gg"` function (use the triple-colon operator). What does `+` actually do to a ggplot object?
3. Compare `filter(df, x > 5)` to `df[df$x > 5, ]`. What does dplyr's version gain from NSE? What does it cost?

27.2. How `aes()` works

`aes()` deserves a close look because it demonstrates the full pattern in miniature: capture expressions, store them, evaluate them later in the right context.

```
mapping <- ggplot2::aes(x = wt, y = mpg, color = cyl)
mapping
#> Aesthetic mapping:
#> * `x`      -> `wt`
#> * `y`      -> `mpg`
#> * `colour` -> `cyl`
```

What comes back is a list of quosures, each pairing an expression (`wt`, `mpg`, `cyl`) with the environment where it was written. The column names were never evaluated; they were captured and stored for later. But capturing is only half the problem. The expression still has to find the right data at the right time, and the quosure's environment is the user's environment, not the data frame.

```
mapping$x
#> <quosure>
#> expr: ^wt
#> env:  global
class(mapping$x)
#> [1] "quosure" "formula"
rlang::get_expr(mapping$x)
#> wt
```

When ggplot2 eventually builds the plot, it evaluates these quosures against the data frame, so `wt` is looked up in `mtcars`, not in the global environment. This separation of capture and evaluation is the mechanism that makes the entire system work: you write code that looks like it refers to columns directly, and the library quietly intercepts those references and redirects them to the data.

The `+.gg` method, meanwhile, simply takes the existing plot object, attaches the new component (a layer, a scale, a theme), and returns the modified plot. The plot is a list that grows with each `+`. Rendering happens only when you print it.

This two-phase design (build a description, then execute it) appears in SQL query builders, in compiler intermediate representations, and in ggplot2. The user describes *what* they want; the system decides *how* to produce it. But you do not need ggplot2's sophistication to exploit the same pattern.

27.3. Building a unit-aware DSL

Here is a concrete problem: unit arithmetic is error-prone because adding meters to seconds is meaningless, yet nothing in base R stops you from doing it. A small DSL can enforce dimensional correctness at the language level, so that `meters(5) + meters(3)` returns 8 meters, `meters(5) + seconds(3)` throws an error, and `meters(100) |> to("km")` returns 0.1 km. The DSL is deliberately small, but it exercises every technique from the last three chapters: S3 classes, operator overloading, constructors, validation, and dispatch.

Start with the class. A unit value is a number paired with a unit string, using an S3 class:

```
new_unit <- function(value, unit) {
  structure(list(value = value, unit = unit), class = "unit_val")
}

print.unit_val <- function(x, ...) {
  cat(x$value, x$unit, "\n")
  invisible(x)
}
```

Then add constructor functions for common units:

```
meters  <- function(x) new_unit(x, "m")
seconds <- function(x) new_unit(x, "s")
kg      <- function(x) new_unit(x, "kg")
```

```
meters(5)
#> 5 m
seconds(3)
#> 3 s
```

These constructors are the DSL's vocabulary, the words users will actually write. Each one creates a `unit_val` object, and the user never calls `new_unit()` directly. Vocabulary in place, what about grammar?

27.3.1. Operator overloading

Overload `+` so that it works on `unit_val` objects, but only when the units match:

```
"+.unit_val" <- function(a, b) {
  if (a$unit != b$unit) {
    stop(sprintf("cannot add %s and %s", a$unit, b$unit))
  }
  new_unit(a$value + b$value, a$unit)
}
```

```
meters(5) + meters(3)
#> 8 m
```

```
meters(5) + seconds(3)
#> Error in `+.unit_val`:
#> ! cannot add m and s
```

The error is immediate and readable: no silent coercion, no mysterious `NA`. The same pattern works for `-`, `*`, and `/`, though multiplication and division need to combine units (meters times seconds gives meter-seconds, for instance). For brevity, handle just addition and subtraction here:

```
"-.unit_val" <- function(a, b) {
  if (missing(b)) return(new_unit(-a$value, a$unit))  # unary negation
  if (a$unit != b$unit) {
    stop(sprintf("cannot subtract %s from %s", b$unit, a$unit))
  }
  new_unit(a$value - b$value, a$unit)
}
```

Note the `missing(b)` check. R uses `-` both as a binary operator (`a - b`) and a unary operator (`-a`), dispatching to the same method in both cases but leaving `b` missing when there is only one argument. Without this check, `-meters(5)` would blow up trying to access `b$unit` on something that does not exist.

```
meters(10) - meters(3)
#> 7 m
-meters(5)
#> -5 m
```

You now have a language for unit-safe arithmetic. `meters(5) + meters(3)` reads like a sentence and does the right thing; `meters(5) + seconds(3)` fails clearly and immediately. This is what a DSL buys you: correct code that is easy to write and incorrect code that is hard to ignore. But arithmetic on its own only gets you so far.

27.3.2. Unit conversion

You also need to move between scales. A lookup table maps unit pairs to conversion factors:

```
conversions <- list(
  "m_to_km"  = 0.001,
  "km_to_m"  = 1000,
  "s_to_min" = 1 / 60,
  "min_to_s" = 60,
  "kg_to_g"  = 1000,
  "g_to_kg"  = 0.001
)

to <- function(x, target) {
  if (!inherits(x, "unit_val")) {
    stop(sprintf("x must be a unit_val, got %s", class(x)[1]))
  }
  if (!is.character(target) || length(target) != 1) {
    stop("target must be a single character string")
  }
  # The DSL's operators maintain the unit_val invariant internally,
  # but to() is a boundary function: it accepts a string from the
  # user, so we validate here to keep invalid states out of the system.
  key <- paste0(x$unit, "_to_", target)
  factor <- conversions[[key]]
  if (is.null(factor)) {
    stop(sprintf("no conversion from %s to %s", x$unit, target))
  }
  new_unit(x$value * factor, target)
}
```

```
meters(1500) |> to("km")
#> 1.5 km
```

```
meters(5) |> to("min")
#> Error in `to()`:
#> ! no conversion from m to min
```

The pipe (`|>`) makes conversion read naturally: "take 1500 meters, convert to km." And because every operation returns a `unit_val`, the DSL is composable; you can chain arithmetic and conversion without any glue code:

```
(meters(500) + meters(1000)) |> to("km")
#> 1.5 km
```

27.3.3. Comparison operators

Overload comparison so that units are checked there too:

```
">.unit_val" <- function(a, b) {
  if (a$unit != b$unit) stop(sprintf("cannot compare %s and %s", a$unit, b$unit))
  a$value > b$value
}

"==.unit_val" <- function(a, b) {
  if (a$unit != b$unit) stop(sprintf("cannot compare %s and %s", a$unit, b$unit))
  a$value == b$value
}
```

```
meters(10) > meters(5)
#> [1] TRUE
meters(5) == meters(5)
#> [1] TRUE
```

Every operator enforces the same invariant: you cannot mix units. The type system (S3 classes) and operator overloading work together to make invalid states unrepresentable, which is the whole point of designing a DSL rather than writing bare functions. But the unit system is small, almost toy-like. What techniques would you reach for when the domain gets more complex?

Exercises

1. Add a `*.unit_val` method that combines units. `meters(5) * seconds(2)` should return `10 m*s`. (Hint: paste the unit strings together with `*` as a separator.)
2. Add a `format.unit_val` method so that `paste("Distance:", meters(42))` produces `"Distance: 42 m"`.
3. Add `celsius()` and `fahrenheit()` constructors plus a `to()` conversion between them. This one is not a simple multiplication; you need an offset. How does that change the design of the `conversions` table?

27.4. Techniques for DSL construction

The unit DSL used three techniques: S3 classes, operator overloading, and constructor functions. More ambitious DSLs draw on a larger toolkit.

Operator overloading. R lets you define methods for `+`, `-`, `*`, `/`, `[`, `[[`, `<`, `>`, `==`, `|`, `&`, and more. You can also create custom infix operators with the `%op%` syntax:

```
"%to%" <- function(x, target) to(x, target)
meters(1500) %to% "km"
#> 1.5 km
```

Custom infixes are useful when the built-in operators do not express the right meaning. The `%>%` pipe from magrittr is the most famous example; `%in%` is a base R one.

NSE for expression capture. If your DSL needs to capture column names, formulas, or unevaluated expressions, use `substitute()` (base R) or `enquo()` (rlang). The unit DSL did not need NSE because its inputs are plain values, but a query-builder DSL would:

```
where <- function(.data, expr) {
  e <- substitute(expr)
  rows <- eval(e, .data, parent.frame())
  .data[rows, ]
}
where(mtcars, mpg > 30)
```

That is a miniature version of what `dplyr::filter()` does: capture the expression, evaluate it against the data frame.

Formula interfaces. Use formulas when your DSL needs two-sided specifications. The `~` already prevents evaluation, and the formula carries its environment:

```
specify <- function(formula, data) {
  lhs <- all.vars(formula[[2]])
  rhs <- all.vars(formula[[3]])
  list(response = lhs, predictors = rhs)
}
specify(mpg ~ wt + hp, mtcars)
```

S3 dispatch as polymorphism. If your DSL should work on multiple data backends (data frames, databases, remote APIs), define your verbs as S3 generics so each backend gets its own method. This is exactly how dplyr supports data frames and SQL databases with the same syntax.

Expression builders. The ggplot2 approach: each operation returns an object, and the objects compose. The user builds up a description while execution is deferred until the end, separating specification from computation and making the DSL composable. When should you *not* use these techniques?

27.5. `eval(parse(text = ...))`: the anti-pattern

You will encounter code like this:

```
col_name <- "mpg"
eval(parse(text = paste0("mtcars$", col_name)))
```

It constructs R code as a string, parses it into an expression, and evaluates it. It works. It is also almost always wrong.

The problems are concrete. First, injection: if `col_name` comes from user input, a string like `"mpg; system('rm -rf /')"` gets parsed and executed. Second, debugging: when the generated code errors, the traceback points at the parsed text, not your source file. Third, tooling: static analysis, linting, and IDE autocompletion cannot see inside a string.

The alternative is to work with expressions as data structures, not strings. Everything you learned in Chapter 26 avoids `eval(parse(text = ...))`:

```
col_name <- "mpg"
mtcars[[col_name]]
#>  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2
#> [15] 10.4 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4
#> [29] 15.8 19.7 15.0 21.4
```

For more complex cases, `rlang::sym()` converts a string to a symbol, and `!!` injects it:

```
col <- rlang::sym("mpg")
dplyr::summarise(mtcars, mean_val = mean(!!col))
```

Opinion

If you find yourself writing `eval(parse(text = ...))`, stop and ask whether the problem can be solved with `[[`, `rlang::sym()`, `call2()`, or `do.call()`. In nearly every case, it can. The string-based approach is a last resort for code generation tasks where you are producing R scripts to be run later, not for interactive computation.

Exercises

1. Rewrite `eval(parse(text = paste0("mean(mtcars$", col, ")")))` without `eval(parse(...))`. Use `[[` or `rlang::sym()`.
2. Write a function `safe_select(data, col_name)` that takes a string column name and returns that column. Do it without `eval(parse(text = ...))`.

27.6. Design principles

A DSL is an interface, and the same mistakes that make function APIs hard to use make DSLs worse because the user has fewer escape hatches when things go sideways.

Keep the surface small. A DSL with fifty special operators is a burden, not a language. The formula interface has about six operators (`~`, `+`, `*`, `:`, `-`, `I()`). ggplot2 has a handful of geom types and a `+`. Resist adding syntax, because each new operator is a concept the user must internalize before they can read anyone else's code.

Make errors readable. When units do not match, the error says "cannot add m and s." When a conversion does not exist, it says "no conversion from m to min." Good error messages name the things the user wrote, not the implementation details, which is Chapter 25 applied to DSL design.

Make it composable. Every expression in the DSL should return something usable as input to the next expression. `meters(5) + meters(3)` returns a `unit_val` that you can pass to `to()` or combine with more values; ggplot2's `+` returns a plot object ready for more layers. Composability comes from consistent types: operations on `unit_val` always return `unit_val`. Notice the tension with "keep the surface small": real composability often demands more operators, not fewer. If you can add unit values but not multiply them, the DSL is small but users will hit a wall the moment they need to compute velocity from distance and time. You have to decide which wall is worse: the cognitive load of a larger surface, or the frustration of an expressive dead end. There is no universal answer; the domain decides.

Do not surprise. If your DSL overloads `+`, it should still be associative: `(a + b) + c` should equal `a + (b + c)`. If it redefines `*`, users will expect it to distribute over `+`. Violating mathematical expectations creates confusion even when the behavior is internally consistent.

Document the grammar. A DSL has a grammar: what expressions are valid, what they mean, how they compose. Write it down, even informally. "A `unit_val` can be combined with `+` or `-` if units match. Use `to()` to convert between units. Constructors: `meters()`, `seconds()`, `kg()`." Five sentences. That is the complete grammar, and a user who reads it can write any valid expression in the DSL without further instruction. Whether five sentences will still be enough after your DSL has lived in production for a year is a different question.

27.7. Putting it all together

The chapter's DSL used five ingredients:

1. **S3 class** (`unit_val`) to represent domain objects.
2. **Constructor functions** (`meters()`, `seconds()`, `kg()`) to provide vocabulary.
3. **Operator overloading** (`+.unit_val`, `-.unit_val`) to give arithmetic domain-specific meaning.
4. **Validation** (unit mismatch errors) to enforce domain rules.
5. **Composability** (every operation returns a `unit_val`) to make expressions chainable.

A more ambitious DSL would add NSE to capture column names or expressions, use S3 generics to dispatch across backends, and build expression objects that defer evaluation. ggplot2 uses all of these. The progression from "simple class with overloaded operators" to "full expression-builder DSL with lazy evaluation and multiple backends" is a continuum, and you do not need the full machinery for every problem. Start with the simplest technique

that makes your interface read naturally; add complexity only when the domain demands it.

The formula interface from the opening of this chapter, where `+` stopped meaning addition and started meaning "include this predictor" — you now know exactly how that trick works: an S3 class, a method for `+`, and a function that interprets the captured expression in a new context. R gave the formula system no special privileges. The same tools are available to you. When you find yourself writing repetitive, mechanical code, consider whether a small DSL, even just a class with a few overloaded operators, would let you express the same ideas more directly.

Exercises

1. Extend the unit DSL with a `summary.unit_val` method that, given a list of `unit_val` objects (all with the same unit), returns the min, max, and mean. Validate that all units match.
2. Design (on paper, no code required) a DSL for describing file-processing pipelines: read a CSV, filter rows, rename columns, write output. What would the verbs be? What class would they operate on? How would they compose?
3. Read the source of `htmltools::tag()`. How does it build HTML from R function calls? Which of the techniques from this chapter does it use?
4. Pick a repetitive task from your own work. Sketch a five-function DSL that would make it more concise. What class would you define? What operators would you overload?

Part V

Part V: Building on R

28 Performance

So here is where you stand: you can find bottlenecks, eliminate unnecessary copies, hand tight loops to compiled code, and split independent work across cores. But the optimization that actually matters most is still the one at the top of the list. Measure first. Your profiler will catch things your intuition misses every time.

Your code is too slow. You know this because you ran it, waited, checked your phone, looked back at the screen, and it was still running. The question is: which part?

Most R code finishes before you notice it started. When something takes too long, the question is where the seconds go, and the answer is almost never where you expect. Profile first. After that, the fix depends on what the profiler shows: sometimes it is a vectorization you missed, sometimes a copy you did not know was happening, sometimes a loop that belongs in C++. The tools in this chapter cover all of those, and **?@sec-connecting-to-other-languages** covers the compiled-code path in detail.

28.1. Profile before you optimize

The function you suspect is slow is, more often than not, innocent. Some other line, one you barely glanced at while writing, runs a million times while the suspect runs once. Optimization without measurement is guesswork dressed up as engineering. You have to measure.

Quick timing with `system.time()`:

```
system.time({
  x <- rnorm(1e6)
  y <- cumsum(x)
})
#>    user  system elapsed
#>    0.03    0.00    0.03
```

The first number (`user`) is CPU time. The third (`elapsed`) is wall-clock time. Good enough for ballpark estimates, but ballpark estimates deceive you when differences are small, when the thing you are comparing takes three milliseconds and the noise takes five.

Precise benchmarking with `bench::mark()`:

```
bench::mark(
  sqrt = sqrt(x),
  power = x^0.5,
  check = TRUE
)
```

`bench::mark()` runs each expression multiple times, reports median time and memory allocation, and checks that all expressions return the same result. Use it for head-to-head comparisons. `microbenchmark::microbenchmark()` is the older alternative: sub-millisecond accurate, runs expressions 100 times by default, reports summary statistics. Both work well; `bench::mark` is newer and checks for equal results automatically.

Profiling with `profvis::profvis()`:

```
profvis::profvis({
  data <- read.csv("large_file.csv")
  cleaned <- dplyr::filter(data, !is.na(value))
  model <- lm(value ~ group, data = cleaned)
  summary(model)
})
```

`profvis` produces an interactive flame graph showing which lines eat the most time and allocate the most memory. Use it when you do not know where the bottleneck is, which, if you are honest with yourself, is most of the time.

Here is a concrete example. The function `slow_analysis` generates data, fits a model, copies residuals in a loop (using `c()` to grow a vector), and computes a summary:

```
slow_analysis <- function(n = 5e4) {
  x <- rnorm(n)
  y <- 2 * x + rnorm(n, sd = 0.5)
  df <- data.frame(x = x, y = y)
  fit <- lm(y ~ x, data = df)

  # The bottleneck: growing a vector one element at a time
  residuals_copy <- c()
  for (i in seq_len(n)) {
    residuals_copy <- c(residuals_copy, fit$residuals[i])
  }
  summary(fit)
}

profvis::profvis(slow_analysis(5e4))
```

The profile output shows where time actually goes:

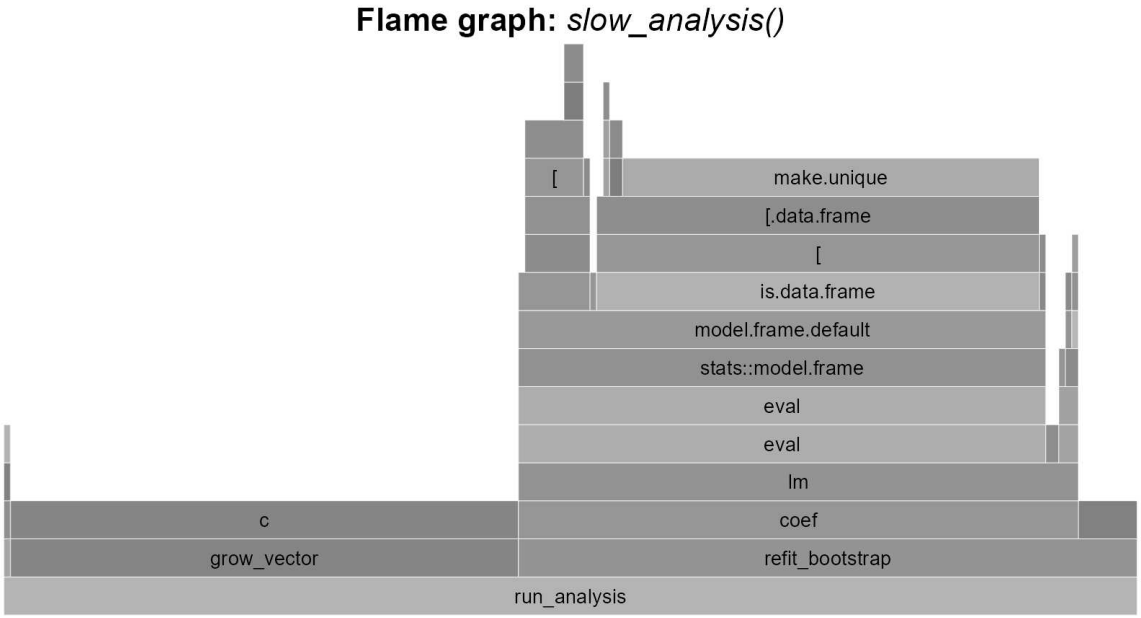


Figure 28.1.: Flame graph of `slow_analysis()`: width is time. The `c()` tower on the left (the residual-copying loop) and the `lm` call stack on the right are the two hot spots. The `c()` loop takes roughly half the total time.

Read from bottom to top. `slow_analysis` calls `lm` on the right and runs the residual-copying loop on the left; inside the loop, nearly all time goes to `c()`, the vector-growing bottleneck, while inside `lm` the cost distributes across its internals (`model.frame`, `eval`, `lm.fit`). Without this graph, you might guess model fitting is the problem. The graph says otherwise.

The workflow is simple: run `profvis` on realistic input, find the hot spot, fix that one thing, re-profile. What do you do once you have found the bottleneck?

> **Opinion**
>
> If you have not profiled, you do not have a performance problem. You have a feeling.

Exercises

1. Use `system.time()` to compare `sort(x)` and `x[order(x)]` for `x <- rnorm(1e6)`.
2. Use `bench::mark()` to compare `sum(x)` and `Reduce("+", x)` for `x <- rnorm(1e4)`. Which is faster, and by how much?

28.2. Vectorization

Consider a vector of a million numbers. You want to double each one. You could write a loop that visits each element, multiplies it by two, stores the result. Or you could write `x *`

`2` and let R handle the iteration. The second version runs 10 to 100 times faster. Why?

The answer traces back to 1962, when Kenneth Iverson at Harvard published *A Notation as a Tool of Thought*, describing APL (A Programming Language). APL's central insight was that mathematical operations should apply to entire arrays, not individual elements: you do not write a loop to add two vectors; you write `A + B` and the language handles the rest. Iverson won the Turing Award for this work in 1979, and the idea traveled through S (where Chambers adopted it for statisticians who think in columns, not cells) and into R, where `x * 2` multiplies every element of `x` without a loop, an index variable, or any mention of how many elements `x` contains.

When you call a vectorized function, R dispatches to optimized C code underneath. The loop still happens, but it happens in compiled machine code rather than stepping through R's interpreter one element at a time. Think of it as the difference between reading a book aloud word by word and handing the entire manuscript to a speed-reader: the same work gets done, but the overhead per unit collapses.

```
x <- rnorm(1e5)

# Vectorized: fast
system.time(y <- x * 2)
#>    user  system elapsed
#>       0       0       0

# Loop: slow
system.time({
  y <- numeric(length(x))
  for (i in seq_along(x)) y[i] <- x[i] * 2
})
#>    user  system elapsed
#>       0       0       0
```

The difference grows with input size, and part of the speedup has nothing to do with R at all. Vectorized operations work on contiguous memory with sequential access patterns that the CPU prefetcher predicts correctly; `sum(x)` on a contiguous vector can be 100x faster than summing the same values scattered in a list, because L1 cache is roughly 27x faster than main memory. Martin Thompson, a high-performance computing advocate who borrowed the term from racing driver Jackie Stewart, calls this *mechanical sympathy*: writing software that works with the hardware rather than against it.

Common vectorization patterns:

Replace `for` + `if` with `ifelse()` or `dplyr::case_when()`:

```
# Slow
result <- character(length(x))
for (i in seq_along(x)) {
  if (x[i] > 0) result[i] <- "pos" else result[i] <- "neg"
}
```

```
# Fast
result <- ifelse(x > 0, "pos", "neg")
```

Replace `for` + accumulate with `cumsum()`, `cumprod()`, `cummax()`:

```
# Slow
result <- numeric(length(x))
result[1] <- x[1]
for (i in 2:length(x)) result[i] <- result[i-1] + x[i]

# Fast
result <- cumsum(x)
```

Replace loop-based row/column operations with `rowSums()`, `colMeans()`, matrix operations:

```
# Slow
row_totals <- numeric(nrow(mat))
for (i in 1:nrow(mat)) row_totals[i] <- sum(mat[i, ])

# Fast
row_totals <- rowSums(mat)
```

Not everything yields to vectorization. Iterations where each step depends on the previous result (Markov chains, recursive algorithms, some simulations) genuinely need loops, and for those Section 28.6 offers an escape hatch. But before reaching for compiled code, make sure you are not confusing convenience wrappers with the real thing.

`sapply()` and `lapply()` look like vectorization. They are not. They are convenient wrappers around loops that still call your R function once per element, no faster than a well-written `for` loop. True vectorization means the loop runs in C: `sum()`, `cumsum()`, `ifelse()`, `pmin()`, `rowSums()`. If the loop is still in R, the speed is still in R.

The distinction traces back to R's inheritance. S was designed for interactive data analysis, not for writing loops. When John Chambers built S at Bell Labs, and when R's creators reimplemented it at Auckland, the assumption was that heavy computation would happen in compiled Fortran or C routines, and the user-facing language would orchestrate those routines at a high level. Functions like `sum()` and `rowSums()` exist because they *are* the compiled routines, exposed directly. When you call `sum(x)`, a single trip into C walks the memory block in a tight loop and returns the result. `sapply(x, f)` never leaves the interpreter; it calls your R function once per element, paying the full cost of argument matching, environment creation, and promise evaluation on every iteration. The speed gap between these two paths reflects a design choice that predates R itself: the language was built to orchestrate compiled routines, and code that stays in the interpreter is working against that architecture rather than with it.

So what happens when you have a loop that genuinely cannot be vectorized, but it still needs to be fast?

Exercises

1. Write a loop that computes the absolute value of each element in a vector. Then write the vectorized version using `abs()`. Benchmark both with `bench::mark()`.
2. Replace this loop with a single vectorized expression: `for (i in 1:length(x)) if (x[i] < 0) x[i] <- 0`. (Hint: `pmax()`.)

28.3. Memory: pre-allocate and avoid copies

A loop might be slow not because loops are inherently slow in R, but because R is copying your entire result vector on every iteration.

```
# Slow: O(n^2) because each c() copies the entire vector
slow_squares <- function(n) {
  result <- c()
  for (i in 1:n) result <- c(result, i^2)
  result
}

# Fast: O(n) with pre-allocation
fast_squares <- function(n) {
  result <- numeric(n)
  for (i in 1:n) result[i] <- i^2
  result
}

# Fastest: vectorized
vec_squares <- function(n) (1:n)^2
```

```
bench::mark(
  growing  = slow_squares(1000),
  prealloc = fast_squares(1000),
  vector   = vec_squares(1000),
  check = FALSE
)
#> # A tibble: 3 x 6
#>   expression      min   median `itr/sec` mem_alloc `gc/sec`
#>   <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
#> 1 growing     407.3us  469.3us     2002.    3.88MB   162.
#> 2 prealloc     15.3us   16.9us    58226.   25.78KB     5.82
#> 3 vector        900ns    1.3us   686346.   11.81KB   137.
```

Each call to `c(result, value)` allocates a new vector of size `length(result) + 1` and copies every existing element into it. For `n` iterations, that totals $1 + 2 + 3 + \cdots + n = O(n^2)$ copies, which means doubling your input size quadruples your runtime. Pre-allocation sidesteps the entire problem: you tell R the final size upfront with `numeric(n)`, `character(n)`, `logical(n)`, or `vector("list", n)`, and each assignment writes directly into the slot without copying anything.

But pre-allocation only solves the allocation problem. There is a second performance trap hiding in how R shares memory.

Copy-on-modify: R copies an object when you modify it *and* another name points to it:

```
x <- 1:1e6
y <- x        # y and x share the same memory
y[1] <- 0L    # now R copies, because modifying y would change x
```

You can track copies with `tracemem()`:

```
x <- 1:5
tracemem(x)
#> [1] "<0000027013A31460>"
y <- x
y[1] <- 0L    # triggers a copy
#> tracemem[0x0000027013a31460 -> 0x0000027014f0bea8]: eval eval withVisible
withCallingHandlers eval eval with_handlers doWithOneRestart withOneRestart
withRestartList doWithOneRestart withOneRestart withRestartList withRestarts
<Anonymous> evaluate in_dir in_input_dir eng_r block_exec call_block process_group
withCallingHandlers with_options <Anonymous> process_file <Anonymous> <Anonymous>
execute .main
untracemem(x)
```

When only one name references an object, R modifies it in place, which is why pre-allocated loops are fast: the result vector has a single reference, so `result[i] <- value` writes directly without triggering a copy. The subtlety is that functions create references too. When you pass `x` to a function, the function parameter and the caller's variable both point to the same object; if the function modifies its copy, R silently allocates a new one. This means helper functions called inside tight loops can trigger copies you never intended. `tracemem()` is your diagnostic tool here.

Exercises

1. Write a function that grows a character vector by appending one element at a time in a loop. Time it for `n = 10000`. Then rewrite with pre-allocation. What is the speedup?
2. Use `tracemem()` to observe when R copies a vector. Create `x <- 1:5`, then `y <- x`, then modify `y[1] <- 99L`. How many copies happen?

28.4. data.table

You have vectorized your operations, pre-allocated your loops, and the code is still too slow. The bottleneck is the data frame engine itself. `dplyr` is readable and expressive, but on millions of rows its abstractions carry a cost: intermediate copies, grouped operations that could be fused, filters that scan more data than necessary.

`data.table` strips away those abstractions:

```
library(data.table)

dt <- fread("large_file.csv")  # much faster than read.csv()

# Filter, compute, group in one expression
dt[age > 30, .(mean_income = mean(income)), by = region]
```

The `dt[i, j, by]` syntax puts row filtering (`i`), column operations (`j`), and grouping (`by`) in a single expression, which lets data.table optimize the entire operation as a single pass. It modifies columns in place (no copy-on-modify overhead), automatically parallelizes grouped operations, uses less memory than equivalent dplyr pipelines, and its `fread()` reads CSV files much faster than `read.csv()` and often faster than `readr::read_csv()`.

The trade-off is a steeper learning curve; the `dt[i, j, by]` idiom reads differently from dplyr pipes. If you do not want to learn the syntax, `dtplyr` bridges the gap: write dplyr verbs, get data.table speed. It translates your pipeline behind the scenes.

A quick comparison:

```
# dplyr
library(dplyr)
sales |>
  filter(year == 2024) |>
  group_by(region) |>
  summarise(total = sum(revenue))

# data.table
library(data.table)
dt <- as.data.table(sales)
dt[year == 2024, .(total = sum(revenue)), by = region]
```

On small data, the difference is negligible. On millions of rows with many groups, the gap opens up:

```
n <- 5e6
sales <- data.frame(
  region = sample(letters, n, replace = TRUE),
  revenue = rnorm(n, 1000, 200)
)
```

```
dt <- data.table::as.data.table(sales)

bench::mark(
  dplyr = sales |>
    dplyr::group_by(region) |>
    dplyr::summarise(total = sum(revenue)),
  data.table = dt[, .(total = sum(revenue)), by = region],
  check = FALSE
)
# data.table is typically 3-10x faster on grouped aggregations at this scale.
# The gap widens with more groups and more rows.
```

Reach for data.table when your data has millions of rows, when you are grouping over many categories, or when the same pipeline runs repeatedly in a loop or a Shiny app. If your data fits comfortably in memory and dplyr finishes in seconds, switching buys you nothing but a syntax change. Both dplyr and data.table operate entirely in memory. data.table still loads the entire dataset into RAM before it touches a single row.

Exercises

1. Install `data.table` and convert `mtcars` to a data.table with `as.data.table()`. Compute the mean `mpg` grouped by `cyl` using the `dt[i, j, by]` syntax. Verify your result matches `mtcars |> group_by(cyl) |> summarise(mean(mpg))`.
2. Use `fread()` to read a CSV file (any file you have, or write one with `fwrite(mtcars, "test.csv")` first). Compare its speed against `read.csv()` using `bench::mark()`.

28.5. Columnar engines: Arrow and DuckDB

What happens when your data no longer fits in memory, or when it does fit but you want analytics that finish before your coffee cools?

You have probably already hit the wall. A grouped `summarise()` on 50 million rows takes minutes and turns your laptop fan into a jet engine. `read.csv()` on a 10 GB file eats all available RAM and crashes the session without producing a single result. The data is not even big by industry standards; R just was not designed to hold it all at once and churn through it.

Arrow (`arrow` package) reads Parquet and Feather files and processes them with dplyr syntax. Operations are lazy: they build a query plan, then execute it in a single pass, loading only the columns and rows you actually need:

```
library(arrow)

open_dataset("data/large_parquet/") |>
  dplyr::filter(year == 2024) |>
```

```
  dplyr::group_by(region) |>
  dplyr::summarise(total = sum(revenue)) |>
  dplyr::collect()  # only now does data enter R
```

DuckDB (`duckdb` package) is an embedded analytical database that reads Parquet, CSV, and data frames, handles data larger than RAM, and supports both SQL and dplyr syntax (via `dbplyr`):

```
library(duckdb)
con <- dbConnect(duckdb())
duckdb_register(con, "sales", sales_df)

dbGetQuery(con, "SELECT region, SUM(revenue) FROM sales GROUP BY region")
dbDisconnect(con)
```

duckplyr removes the SQL entirely: it is a drop-in replacement for dplyr that routes computation through DuckDB's engine. Same syntax, faster execution, near-zero transition cost. In June 2025, duckplyr formally joined the tidyverse. Existing dplyr code runs unchanged; duckplyr intercepts the verbs, routes what it can through DuckDB, and falls back to dplyr for anything DuckDB does not support.

The stack looks like this: Parquet files on disk, Arrow or DuckDB as the engine, dplyr verbs as the interface, results collected into R. You write the same code you already know; the machinery underneath is faster. Much of that speed comes from *query fusion*, combining multiple operations into a single pass over the data and eliminating intermediate allocations. In functional programming, this technique is called *deforestation* (Wadler, 1988): `map f . map g` creates an intermediate list, but fusion rewrites it as `map (f . g)`, traversing the data once. data.table and Arrow apply the same principle at scale.

The dplyr verbs from Chapter 14 were designed as a functional interface: data frame in, data frame out, no mutation of the input. data.table, Arrow, DuckDB, and Polars are all different engines, written in different languages (C, C++, Rust), with different memory layouts and different optimization strategies. The dplyr verbs sit on top of all of them unchanged. `filter()`, `mutate()`, `summarise()` work on a local data frame, on a Parquet file through Arrow, on a DuckDB table through duckplyr. Four engines have appeared since dplyr was designed, and the interface has not changed once.

> **Opinion**
>
> If your data fits in memory and dplyr is fast enough, stop. If it does not fit, or if grouped aggregations drag, try DuckDB. With duckplyr the transition cost is nearly zero.

Exercises

1. Install `duckdb` and `DBI`. Create an in-memory DuckDB connection, register `mtcars` as a table, and run a SQL query to compute mean `mpg` grouped by `cyl`. Compare the result

with your data.table answer from the previous section.

2. If you have a Parquet file (or create one with `arrow::write_parquet(mtcars, "test.parquet")`), open it with `arrow::open_dataset()`, filter, and `collect()`. Verify the result matches the equivalent dplyr operation on the in-memory data frame.

28.6. When and how to call compiled code

Sometimes R is simply the wrong language for the inner loop. You have profiled, vectorized, pre-allocated, tried data.table, and the bottleneck is a tight loop where each iteration depends on the last, where no vectorized function exists, where the work is purely computational and measured in millions of iterations. At that point, the right move is to write the hot loop in a compiled language and call it from R. Three options: C, C++ (via Rcpp), and Rust (via extendr).

C via `.Call()`: R's native foreign function interface. You write C functions that take and return `SEXP` objects (R's internal type), managing memory yourself with `PROTECT`/`UNPROTECT`:

```
// sum_c.c
#include <R.h>
#include <Rinternals.h>

SEXP sum_c(SEXP x) {
    double total = 0;
    for (int i = 0; i < length(x); i++) {
        total += REAL(x)[i];
    }
    return ScalarReal(total);
}
```

Low-level, no dependencies, maximum control. Used extensively in base R and older packages.

C++ via Rcpp: the most popular choice, and for good reason. It handles type conversion and memory management automatically, so you write code that looks almost like normal C++:

```
// sum_cpp.cpp
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double sum_cpp(NumericVector x) {
    double total = 0;
    for (int i = 0; i < x.size(); i++) {
        total += x[i];
    }
```

```
    return total;
}
```

`Rcpp::sourceCpp("sum_cpp.cpp")` compiles and loads the function in one step. Immediate gratification.

Rust via extendr: the newest option. Rust provides memory safety without garbage collection; segfaults are, by design, impossible in safe Rust:

```
// sum_rust.rs
use extendr_api::prelude::*;

#[extendr]
fn sum_rust(x: &[f64]) -> f64 {
    x.iter().sum()
}
```

`rextendr::rust_function()` for interactive use, `rextendr::use_extendr()` to set up a Rust-powered package. The ecosystem is young but growing fast.

When to reach for compiled code:

- Loops where each iteration depends on the previous one.
- Recursive algorithms (tree traversal, dynamic programming).
- Operations that process each element individually (no vectorization possible).
- Hot loops identified by profiling.

When not to: when vectorization solves the problem, when the bottleneck is I/O (more C++ will not make your disk spin faster), when correctness matters more than speed and the compiled code would be complex enough to harbor bugs.

> **Opinion**
>
> The three options are not interchangeable items on a menu; they sit at different points on a genuine trade-off. Rcpp gives you the deepest ecosystem and the smoothest on-ramp, but nothing stops you from writing a buffer overrun that corrupts memory silently and crashes R ten minutes later. Rust catches those errors at compile time, before your code ever runs, but its R integration is younger and the community smaller, so you will find fewer examples and hit more rough edges. Raw C gives you total control and zero dependencies at the cost of managing every allocation yourself.
> Start with Rcpp. It has the largest community, the most examples, and the gentlest on-ramp. Consider Rust if you want compile-time safety guarantees. Consider raw C only if you need zero dependencies.

28.7. The optimization checklist

When code is too slow, work through this list in order:

1. **Profile**: find the bottleneck. Do not guess.
2. **Vectorize**: replace element-wise loops with vectorized operations.
3. **Pre-allocate**: if you must loop, pre-allocate the result.
4. **Algorithm**: sometimes the problem is $O(n^2)$ and the fix is $O(n \log n)$. Better algorithms beat faster languages.
5. **data.table or DuckDB**: for large data, switch the engine.
6. **Compiled code** (Rcpp, extendr, `.Call()`): for tight loops that cannot be vectorized.
7. **Parallelize**: `future.apply`, `furrr`, `parallel`. Adds concurrency complexity; worth it when the bottleneck is embarrassingly parallel.

Profiling comes first because Amdahl's law (1967) puts a hard ceiling on what any optimization can gain. If 5% of your code takes 95% of the time, making the other 95% infinitely fast yields only a 5% speedup. The same limit applies to parallelization: if 10% of the work is sequential, you can never exceed a 10x speedup no matter how many cores you throw at it. You need to know which 5% to fix before choosing how to fix it.

R offers several frameworks for parallel execution. `parallel::mclapply()` works on Unix but not Windows. `future.apply::future_lapply()` works everywhere and swaps in for `lapply()` with minimal changes. `furrr::future_map()` is the parallel version of `purrr::map()`. All split work across cores, and all carry the same caveat: parallelization only helps when the work is CPU-bound and divisible. If the bottleneck is reading from disk, more cores will not help. If iterations depend on each other, you cannot split them. And the overhead of spawning workers and collecting results means parallelization only pays off when each unit of work is substantial (milliseconds, not microseconds).

```
library(future.apply)
plan(multisession, workers = 4)

# Parallel version of sapply
results <- future_sapply(1:100, function(i) {
  # some expensive computation
  Sys.sleep(0.1)
  i^2
})
```

29 R internals

What, exactly, is a vector?

You have written thousands of them by now, filtered them, mapped functions over them, watched them copy themselves at surprising moments. But the object itself, the thing sitting in memory when you type `x <- c(1, 2, 3)`, has remained invisible. This chapter opens it up. We are going to look at the C structs that R allocates on the heap, the pointer arithmetic that makes vectorized operations fast, the reference counter that decides when copies happen, and the garbage collector that reclaims everything you abandon. None of this is necessary for writing good R. But it makes the design choices you have seen throughout this book (copy-on-modify, lazy evaluation, lexical scoping) feel less like arbitrary rules and more like consequences of a single engineering blueprint.

R's C implementation descends from S, which John Chambers created at Bell Labs in the 1970s. When Ross Ihaka and Robert Gentleman reimplemented it at the University of Auckland in the early 1990s, they borrowed the evaluation model from Scheme and dressed it in S-compatible syntax. The linked-list environments come from Lisp; the vectorized operations come from S; the garbage collector is a textbook generational mark-and-sweep. What holds all of it together is a single data structure.

29.1. Stack and heap

A running program has two regions of memory. The *stack* is small, fast, and automatic: when you call a function, its local variables go on the stack, and when the function returns, they vanish. The *heap* is large, slow, and manual: objects allocated on the heap survive until something explicitly frees them (in C) or until a garbage collector determines that nothing points to them anymore (in R, Python, Java). Every R object lives on the heap. When you write `x <- c(1, 2, 3)`, R allocates a block of memory on the heap, fills it with your three numbers, and stores a pointer to that block in the binding for `x`. When `x` goes out of scope and nothing else references it, R's garbage collector reclaims the memory. The stack holds the bookkeeping (which function called which, where to return); the heap holds the data.

29.2. Every object is a SEXP

In R's C source code, every R object, without exception, is represented as a `SEXP`: a pointer to a `SEXPREC` struct. The name stands for "S expression," inherited from Lisp. When you write

`x <- c(1, 2, 3)`, R allocates a `SEXPREC` on the heap, fills it with the data for a three-element numeric vector, and stores the pointer in the binding for `x`.

The `SEXPREC` struct has two parts. The header is the same for every object, and it contains:

- A **type tag** (SEXPTYPE): an integer identifying what kind of object this is (integer vector, closure, environment, etc.).
- **GC information**: flags the garbage collector uses to track whether the object is reachable.
- **Reference count**: how many names point to this object (used to decide whether copy-on-modify needs to copy).
- **Attributes**: a pointer to a pairlist of attributes (names, class, dim, etc.).

The payload varies by type. For a numeric vector, it is a contiguous block of C `double` values. For a closure, it is three pointers (formals, body, environment). For an environment, it is a hash table plus a pointer to the parent environment. So every R object, from a humble integer to a deeply nested list of data frames, wears the same uniform; only the payload inside differs.

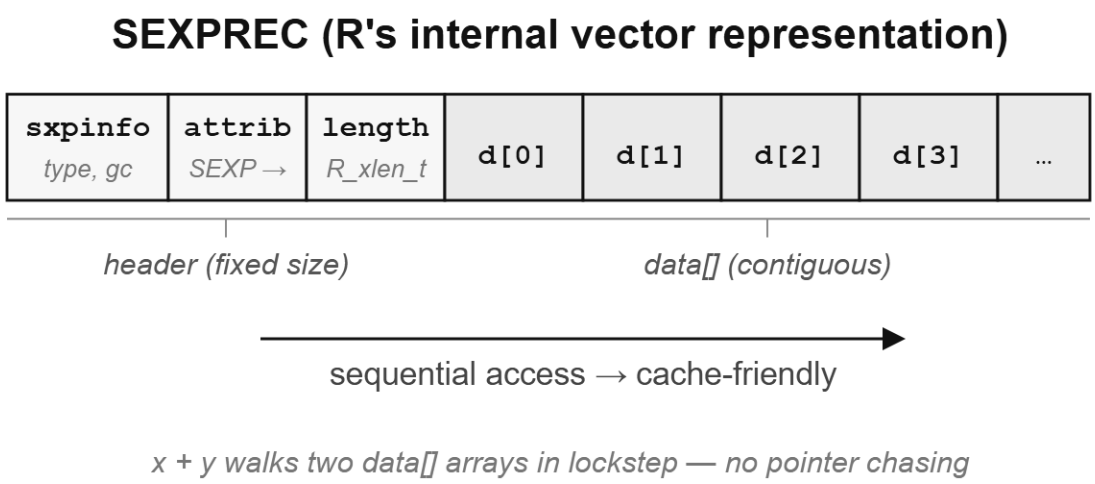


Figure 29.1.: The SEXPREC struct: a fixed-size header followed by a contiguous data array. `x + y` walks two arrays in lockstep with no pointer chasing.

You can see the type tag from R with `typeof()`:

```
typeof(1:5)
#> [1] "integer"
```

```
typeof(c(1.0, 2.0))
#> [1] "double"
typeof("hello")
#> [1] "character"
typeof(list(1, 2))
#> [1] "list"
typeof(sum)
#> [1] "builtin"
typeof(mean)
#> [1] "closure"
```

`typeof()` returns the SEXPTYPE name as a string. The mapping between R concepts and SEXPTYPEs is direct:

R concept	SEXPTYPE	C constant
Integer vector	`integer`	`INTSXP`
Double vector	`double`	`REALSXP`
Character vector	`character`	`STRSXP`
Logical vector	`logical`	`LGLSXP`
List	`list`	`VECSXP`
Function (closure)	`closure`	`CLOSXP`
Built-in function	`builtin`	`BUILTINSXP`
Special function	`special`	`SPECIALSXP`
Environment	`environment`	`ENVSXP`
Promise	`promise`	`PROMSXP`
Language object	`language`	`LANGSXP`
Symbol (name)	`symbol`	`SYMSXP`

Some of these are familiar. `CLOSXP` is the closure from Section 18.4. `ENVSXP` is the environment from Section 18.1. `PROMSXP` is the promise from Chapter 23. The new ones, LANGSXP and SYMSXP, are the building blocks of R's metaprogramming system (Chapter 26), and we will meet them properly there.

Notice that `sum` and `mean` have different types. `sum` is a `builtin`, implemented directly in C, evaluating all its arguments before being called. `mean` is a `closure`, a regular R function with formals, a body, and an environment. The distinction matters at the C level but rarely at the R level. What matters at both levels is being able to see inside these objects, so how do you look?

29.3. Inspecting objects

R provides a hidden function, `.Internal(inspect())`, that dumps the raw C-level representation of any object:

```
x <- c(1.5, 2.5, 3.5)
.Internal(inspect(x))
#> @0x0000010ff262f868 14 REALSXP g0c3 [REF(2)] (len=3, tl=0) 1.5,2.5,3.5
```

The output shows the memory address, the SEXPTYPE (`REALSXP`), the reference count, and the actual data values. Terse, but informative.

The `lobstr` package provides friendlier inspection tools:

```
library(lobstr)

obj_addr(x)
#> [1] "0x10ff262f868"
obj_size(x)
#> 80 B
```

`obj_addr()` returns the memory address as a string. `obj_size()` reports total memory consumption, including the header, so for a three-element numeric vector you get the SEXPREC header (approximately 64 bytes, implementation-dependent) plus three times 8 bytes for the doubles, plus some alignment padding.

Where things get interesting is `ref()`, which shows whether two names point to the same underlying object:

```
y <- x
ref(x, y)
#> [1:0x10ff262f868] <dbl>
#>
#> [1:0x10ff262f868]
```

Both `x` and `y` point to the same memory address. No copy has been made. This is the copy-on-modify mechanism you saw in Section 9.4, now visible at the pointer level.

```
y[1] <- 99
ref(x, y)
#> [1:0x10ff262f868] <dbl>
#>
#> [2:0x10ff067f058] <dbl>
```

After modification, `y` points to a different address. R copied the vector when `y` was modified, because `x` still needed the original. The copy decision depends on how many names point to the same data, which raises the obvious question: how does R keep track?

Exercises

1. Use `typeof()` to check the type of: `TRUE`, `1L`, `1.0`, `1+2i`, `raw(1)`, `quote(x + 1)`, `as.name("x")`. Which ones surprise you?

2. Run `.Internal(inspect(list(1, "a", TRUE)))`. How many SEXPs do you see? Why more than one?
3. Create `a <- 1:1e6` and `b <- a`. Check with `lobstr::ref()` that they share the same address. Now do `b[1] <- 0L`. Do they still share? What does `lobstr::obj_size(a, b)` report?

29.4. Memory layout of vectors

A numeric vector in R is stored as a SEXPREC header followed by a contiguous block of C `double` values, and "contiguous" is the word that matters here: the doubles sit next to each other in memory, with no gaps or pointers between them, exactly like a C array or a NumPy array.

```
/* Simplified layout of a REALSXP (not actual R source) */
struct SEXPREC {
    /* header: type, gc flags, refcount, attributes, ... */
    sxpinfo_struct sxpinfo;
    SEXP attrib;
    SEXP gengc_next_node;
    SEXP gengc_prev_node;
    /* payload for vectors: */
    R_xlen_t length;
    R_xlen_t truelength;
    double data[];  /* flexible array member: the actual numbers */
};
```

The contiguous layout is why vectorized operations are fast. When R computes `x + y`, the C code walks two arrays of doubles in lockstep, reading from sequential memory addresses; modern CPUs are optimized for exactly this access pattern, with hardware prefetchers loading the next cache line before you need it and SIMD instructions adding multiple doubles in a single clock cycle.

An integer vector (`INTSXP`) has the same layout but with `int` instead of `double`. A logical vector (`LGLSXP`) also uses `int` (not `char`), which is why logicals take 4 bytes per element, not 1.

Character vectors are a different beast entirely. A `STRSXP` is a vector of *pointers to* `CHARSXP` objects, where each `CHARSXP` holds an immutable C string. R interns (deduplicates) these strings globally, so two identical strings share the same `CHARSXP`:

```
a <- "hello"
b <- "hello"
.Internal(inspect(a))
#> @0x0000010ff09c3a80 16 STRSXP g0c1 [REF(5)] (len=1, tl=0)
#>   @0x0000010ff049c120 09 CHARSXP g0c1 [MARK,REF(4),gp=0x60] [ASCII] [cached]
"hello"
```

```
.Internal(inspect(b))
#> @0x0000010ff09c31c0 16 STRSXP g0c1 [REF(5)] (len=1, tl=0)
#>   @0x0000010ff049c120 09 CHARSXP g0c1 [MARK,REF(4),gp=0x60] [ASCII] [cached]
"hello"
```

The outer `STRSXP` addresses differ (they are separate character vectors), but the inner `CHARSXP` they point to is the same object. This interning saves memory when the same strings appear repeatedly, as in a factor or a character column with many repeated levels.

A list (`VECSXP`) is a vector of SEXP pointers. Each element can point to any R object of any type, which is why lists are heterogeneous: the list itself is just an array of pointers, and the pointed-to objects can be anything. This pointer-based layout has consequences for how R handles tabular data.

Data frames and cache friendliness. A data frame is a list of column vectors, and each column is a contiguous array. Operations that scan down a column (summing, filtering, grouping) touch sequential memory and benefit from cache prefetching. Operations that scan across rows jump between columns, touching non-sequential addresses and thrashing the cache. This is why column-wise operations in R are generally faster than row-wise ones, and why `apply(df, 1, f)` (row-wise) is slower than `lapply(df, f)` (column-wise). But memory layout only tells half the story; the other half is what happens when two names share the same layout.

Exercises

1. Use `lobstr::obj_size()` to compare the size of `integer(1000)` and `double(1000)`. Is the ratio exactly 1:2? Why or why not?
2. Create two character vectors: `x <- rep("abcdef", 1000)` and `y <- paste0("abcdef", seq_len(1000))`. Compare their sizes with `obj_size()`. Why is `x` much smaller?
3. Why does `object.size(data.frame(a = 1:1e6))` report less memory than `object.size(data.frame(a = as.double(1:1e6)))`?

29.5. Reference counting and copy-on-modify

In Section 9.4, you learned that R copies objects only when they are modified and shared. But how does R decide?

Every SEXPREC header contains a reference count: the number of names (bindings) currently pointing to this object. When you assign `y <- x`, R increments the reference count on the underlying object instead of copying it. When you modify `y`, R checks the count: if it is 1 (only `y` points to the object), R can modify in place; if it is greater than 1, R must copy first.

```
x <- c(1, 2, 3)
.Internal(inspect(x))
#> @0x0000010ff1b05e58 14 REALSXP g0c3 [REF(2)] (len=3, tl=0) 1,2,3
```

The `[MARK,NAM(X)]` in the inspect output shows the reference count. (The exact format varies by R version.)

```
y <- x
.Internal(inspect(x))
#> @0x0000010ff1b05e58 14 REALSXP g0c3 [REF(5)] (len=3, tl=0) 1,2,3
```

After `y <- x`, the reference count increases but the address stays the same. Both names share the same data.

Historical note: the NAMED mechanism. Before R 4.0, R used a cruder system called NAMED, with only three states: 0 (no references), 1 (one reference), and 2 (multiple references, or "we have lost count"). The problem was that NAMED could never decrease; once an object reached NAMED=2, R always copied it on modification, even if one reference had long since disappeared. The current reference counting system, introduced by Luke Tierney, tracks actual counts and can decrease them when bindings are removed, eliminating a whole class of unnecessary copies.

You can observe the practical effect. Modifying a vector inside a function that receives it as an argument triggers a copy, because the caller's binding and the function's parameter both point to the object (reference count of at least 2):

```
f <- function(v) {
  v[1] <- 0
  v
}

x <- c(1, 2, 3)
y <- f(x)
ref(x, y)
#> [1:0x10ff2991868] <dbl>
#>
#> [2:0x10ff29904b8] <dbl>
```

`x` and `y` are different objects. The copy happened inside `f` when `v[1] <- 0` was executed, because `v` shared its data with `x`.

Opinion

Reference counting is why you should not worry too much about "R copies everything." R copies only when it must: when an object is shared and you modify it. Patterns that look wasteful, like passing large data frames into functions, are often free, because the function receives a pointer, not a copy. The copy happens only if the function modifies the data, which idiomatic functional code rarely does.

Exercises

1. Predict whether a copy occurs in each case, then verify with `lobstr::ref()`:

```
a <- 1:1e6
b <- a           # copy?
b[1] <- 0L       # copy?
c <- a            # copy?
rm(a)
c[1] <- 0L       # copy now?
```

2. Write a function that takes a vector, does *not* modify it, and returns its `obj_addr()`. Call it with a large vector. Is the address the same inside and outside the function?

29.6. Garbage collection

Objects get created; reference counts rise and fall; eventually some objects have no references at all. What happens to them?

R uses a tracing garbage collector with a mark-and-sweep algorithm. When R runs low on memory, the collector pauses execution, traces all reachable objects by following pointers from the known roots (the global environment, the call stack, the symbol table), marks them, and sweeps away everything unmarked. If a SEXP has no path back to any root, it ceases to exist.

The collector is *generational*, dividing objects into three generations based on how long they have survived. New objects are generation 0; objects that survive one collection get promoted to generation 1, then to generation 2. The insight behind this design is that most objects die young (temporary vectors in a loop, intermediate results in a pipeline), so collecting generation 0 frequently and generation 2 rarely is efficient. It is the same principle behind most modern garbage collectors, from Java's G1 to Go's concurrent collector.

You can trigger collection manually and see the results:

```
gc()
#>           used (Mb) gc trigger (Mb) max used (Mb)
#> Ncells  684140 36.6    1419408 75.9  1419408 75.9
```

```
#> Vcells 1258195  9.6    8388608 64.0  1941835 14.9
```

The output shows memory usage in Ncells (cons cells, used for pairlists and language objects) and Vcells (vector cells, used for vector data). The "used" column is current consumption; "max used" is the peak since the last `gc(reset = TRUE)`.

`gcinfo(TRUE)` tells R to print a message every time the garbage collector runs, which is noisy but revealing when you want to understand how much allocation a piece of code is doing:

```
gcinfo(TRUE)
x <- lapply(1:1000, function(i) rnorm(1000))
gcinfo(FALSE)
```

PROTECT and UNPROTECT at C level. If you write C code that creates R objects (via `.Call()`), you must tell the garbage collector about them. The collector can run at any allocation point, and if your newly created SEXP is not reachable from any root, it will be swept away while you are still using it. The `PROTECT()` macro adds an object to a protection stack; `UNPROTECT(n)` removes the last `n` entries. Forgetting to PROTECT is the single most common source of bugs in R's C extensions: intermittent segfaults that depend on the exact moment the GC happens to run, which means they almost never reproduce on the developer's machine and almost always reproduce in production.

```
/* Example: a C function callable from R via .Call() */
SEXP add_one(SEXP x) {
    SEXP result = PROTECT(allocVector(REALSXP, length(x)));
    double *px = REAL(x);
    double *pr = REAL(result);
    for (R_xlen_t i = 0; i < length(x); i++) {
        pr[i] = px[i] + 1.0;
    }
    UNPROTECT(1);
    return result;
}
```

The `PROTECT(allocVector(...))` call allocates a new vector *and* protects it in a single line. The `UNPROTECT(1)` at the end removes it from the protection stack just before returning. Between those two points, the GC knows not to collect `result`. Get the count wrong, and you get either a segfault or a stack overflow, which is exactly why higher-level interfaces like Rcpp exist.

Exercises

1. Run `gc()` and note the "used" Vcells. Then create `x <- rnorm(1e7)`, run `gc()` again, and note the change. Now `rm(x)` and `gc()` one more time. Did the Vcells return to roughly the original level?
2. What does `gc(full = TRUE)` do differently from `gc()`?

3. In the C function `add_one` above, what would happen if you removed the `PROTECT()` call? Would the bug appear every time, or only sometimes?

29.7. The evaluator

R is an interpreted language, which means something very specific: when you type an expression at the console, R parses it into an internal tree structure (a LANGSXP), then walks that tree in a function called `eval()` in the file `eval.c`. There is no compilation step, no bytecode (well, there is a bytecode compiler now, but the default path is interpretation), just a C function recursively walking a tree of SEXPs.

The core of `eval()` is a large switch statement on the SEXPTYPE of the expression:

```
/* Simplified sketch of eval.c (not actual code) */
SEXP eval(SEXP e, SEXP rho) {
    switch (TYPEOF(e)) {
    case SYMSXP:    /* symbol: look up in environment */
        return findVar(e, rho);
    case LANGSXP:   /* function call: evaluate function and args, then apply */
        return applyClosure(e, rho);
    case PROMSXP:   /* promise: force it */
        return forcePromise(e);
    case REALSXP:
    case INTSXP:
    case STRSXP:    /* self-evaluating literals */
        return e;
    /* ... many more cases ... */
    }
}
```

When the expression is a symbol (`SYMSXP`), the evaluator looks it up in the environment `rho` by walking the chain of parent environments. When it is a function call (`LANGSXP`), the evaluator finds the function, evaluates the arguments (wrapping them in promises for closures), and calls it. When it is a literal (a number, a string), it returns the value unchanged.

This is why R is slower than compiled languages for tight loops: every iteration goes through this switch statement, every variable access is an environment lookup, and every function call involves promise creation and argument matching. Vectorized code avoids most of this overhead by dropping into C for the inner loop, where none of these costs exist. But the environments that the evaluator searches through are themselves data structures worth understanding.

Environments as linked frames. An environment in R is a hash table of bindings (name-to-SEXP mappings) plus a pointer to a parent environment. Variable lookup walks the chain: check the current environment's hash table, then the parent's, then the grandparent's, up to the global environment and then the search path of attached packages. The hash table uses a simple chaining scheme, and small environments (fewer than about 5 bindings) skip

the hash table entirely, using linear search through a pairlist instead, because the overhead of hashing is not worth it for tiny frames.

29.8. Promises at C level

In Chapter 23, you learned that function arguments are wrapped in promises. Here is what a promise actually looks like in memory.

At the C level, a promise (`PROMSXP`) is a struct with three fields:

1. **Expression** (`PRCODE`): the unevaluated expression, stored as a LANGSXP or SYMSXP.
2. **Environment** (`PRENV`): the environment where the expression should be evaluated.
3. **Value** (`PRVALUE`): initially `R_UnboundValue` (a sentinel). After the promise is forced, this holds the cached result.

When the evaluator encounters a PROMSXP, it checks `PRVALUE`. If it is still `R_UnboundValue`, it evaluates `PRCODE` in `PRENV`, stores the result in `PRVALUE`, and sets `PRENV` to `R_NilValue` (allowing the original environment to be garbage collected). If `PRVALUE` is already set, it returns the cached value immediately. Evaluate once, cache forever.

This three-field structure explains several behaviors you have seen:

- **Default arguments that refer to other arguments work** because the promise's environment is the function's execution environment, where earlier parameters are already bound.
- `substitute()` **can extract the unevaluated expression** because it reads `PRCODE` without forcing the promise.
- **The lazy evaluation trap in function factories** (Section 20.2) happens because the promise captures the expression and the environment at call time, but evaluates them later, when the environment may have changed.

You cannot inspect promises from R without forcing them — a fundamental property of the design, not an oversight:

```
f <- function(x) {
  # This LOOKS like it checks the type before forcing, but typeof(x)
  # itself forces the promise. By the time cat() prints, x is already
  # evaluated. There is no way around this in R.
  cat("typeof x:", typeof(x), "\n")
  x
}
f(1 + 1)
#> typeof x: double
#> [1] 2
```

If R provided a way to get the type without forcing, you would see `"promise"`. But calling `typeof(x)` is itself an access of `x`, which forces the promise, so by the time the result is printed, `x` is already `"double"`. The promise is deliberately invisible from R's perspective:

any inspection forces evaluation, which is precisely why `substitute()` exists as a separate mechanism to read the expression without triggering it. Closures, the other major abstraction from earlier chapters, have their own three-field structure at the C level.

29.9. Closures at C level

A closure (`CLOSXP`) is a struct with three pointers:

1. **Formals** (`FORMALS`): a pairlist of argument names and default values.
2. **Body** (`BODY`): the parsed body of the function, stored as a LANGSXP.
3. **Environment** (`CLOENV`): the environment where the function was defined.

```
adder <- function(n) function(x) x + n

add5 <- adder(5)

formals(add5)
#> $x
body(add5)
#> x + n
environment(add5)
#> <environment: 0x0000010ff295fa50>
```

`formals()`, `body()`, and `environment()` are R-level accessors to the three fields of the CLOSXP. The environment of `add5` is the execution environment created when `adder(5)` was called, and it contains the binding `n = 5`. This is lexical scoping made concrete: the closure carries a pointer to the environment where it was born, and that environment stays alive as long as the closure exists, because the GC traces the pointer and keeps the environment reachable.

Every user-defined function in R is a CLOSXP. Even a bare `function(x) x + 1` carries all three fields. The distinction between "function" and "closure" that some languages make does not exist in R's implementation: all functions are closures. But not all callable things in R are closures, and the mechanisms for reaching compiled code differ in important ways.

Exercises

1. Use `environment()`, `formals()`, and `body()` to inspect `stats::lm`. What environment does it carry?
2. Create a function factory `make_power(exp)` that returns `function(x) x^exp`. Create `square <- make_power(2)` and `cube <- make_power(3)`. Use `ls(environment(square))` and `ls(environment(cube))` to confirm each closure has its own environment.

29.10. Three roads to C

R provides three mechanisms for calling compiled code. They differ in age, safety, and flexibility.

`.Primitive()` is the oldest and most restrictive. Primitive functions are built into the R interpreter itself; you cannot write new ones, because they are defined in a table in `names.c` in the R source. Examples include `+`, `[`, `if`, `for`, `c()`, and `sum()`. Primitives skip normal argument matching (some evaluate arguments before the call, some do not), which is why they are fast but also why their behavior sometimes surprises you.

```
sum
#> function (..., na.rm = FALSE)  .Primitive("sum")
`+`
#> function (e1, e2)  .Primitive("+")
```

The `.Primitive("...")` form shows that these functions are entry points into the C code of the interpreter itself.

`.Internal()` calls C functions that are registered in R's internal table but not exposed as primitives. They go through normal argument matching first, so many base R functions are thin R wrappers around `.Internal()` calls:

```
# Not run: just showing the pattern
body(paste)
# function (..., sep = " ", collapse = NULL, recycle0 = FALSE)
#   .Internal(paste(list(...), sep, collapse, recycle0))
```

The R-level function handles argument matching, default values, and error messages. The `.Internal()` call does the actual work in C.

`.Call()` is the modern interface for R extensions. It passes R objects (SEXPs) directly to a C or C++ function in a shared library (a `.so` or `.dll` file), where the C function receives SEXPs, manipulates them using R's C API, and returns a SEXP. This is what packages like data.table, Rcpp-based packages, and the tidyverse use for performance-critical code. Section 31.1 in Chapter 31 covers the `.Call()` interface in full detail with complete working examples; this section focuses on how it fits into R's internal architecture.

```
/* A .Call function signature */
SEXP my_function(SEXP x, SEXP y) {
    /* work with x and y using R's C API */
    return result;
}
```

From R, you call it as `.Call("my_function", x, y)` or, more commonly, through a wrapper function generated by Rcpp or the package's registration mechanism.

There is also `.C()` and `.Fortran()`, older interfaces that pass raw C arrays (not SEXPs). They are still used in some legacy packages, but `.Call()` is preferred for new code because it avoids unnecessary copying and gives full access to R object metadata.

Exercises

1. Check `typeof(sum)` and `typeof(mean)`. One is `"builtin"`, the other is `"closure"`. What does this tell you about how each is implemented?
2. Look at the source of `base::nchar` (just type `nchar` at the console). Can you find the `.Internal()` call?
3. Why can't you write a new `.Primitive()` function in a package?

29.11. Reading R's source code

R's source code is available at https://svn.r-project.org/R/ and mirrored on GitHub at https://github.com/wch/r-source. The key directories:

- `src/main/`: the interpreter. `eval.c` (the evaluator), `memory.c` (allocation and GC), `envir.c` (environment operations), `names.c` (the table of primitive and internal functions), `arithmetic.c` (vectorized arithmetic).
- `src/main/gram.y`: the parser grammar (yacc format). This defines R's syntax.
- `src/include/Rinternals.h`: the public C API. All the SEXP macros, type constants, and accessor functions.
- `src/library/base/R/`: the R-level code for base functions.

A useful technique for finding where something is implemented: search `names.c` for the function name. That file maps R names to C function pointers, so searching for `"cumsum"` will show you it is implemented by `do_cum` in `cum.c`.

```
# You can also find which functions are .Internal vs .Primitive
# by checking their type:
typeof(`if`)       # "special"  - primitive, doesn't evaluate all args
typeof(`+`)        # "builtin"  - primitive, evaluates all args
typeof(mean)       # "closure"  - R function, may use .Internal inside
```

The distinction between `builtin` and `special` is about argument evaluation. Builtins evaluate all arguments before calling the C function (like normal function calls). Specials handle argument evaluation themselves, which is how `if` can avoid evaluating the branch not taken and `&&` can short-circuit. With all these pieces in view, the full lifecycle of an R object comes into focus.

29.12. Putting it together

When you type `x <- c(1, 2, 3)` at the console, here is everything that happens:

1. The **parser** (gram.y) converts the text into a LANGSXP: a tree with `<-` at the root, `x` (a SYMSXP) on the left, and a call to `c` (another LANGSXP) on the right.
2. The **evaluator** (eval.c) processes the LANGSXP. It sees `<-` (a SPECIALSXP), evaluates the right-hand side first. The call to `c` is a BUILTINSXP, so all arguments are evaluated (they are literals, so they evaluate to themselves), and the C function `do_c` builds a REALSXP: a SEXPREC with a header and three contiguous doubles.
3. The evaluator binds the name `x` to the new SEXP in the current environment's hash table. The reference count is set to 1.
4. Later, if you write `y <- x`, the evaluator adds a new binding in the hash table pointing to the same SEXP and increments the reference count to 2. No copy.
5. If you then write `y[1] <- 99`, the evaluator sees that the reference count is 2, so it copies the REALSXP, modifies the copy, and points `y` at the copy. `x` still points at the original. Reference count on the original drops to 1; reference count on the copy is 1.
6. If `x` goes out of scope or is removed with `rm()`, its reference count drops to 0. The next time the **garbage collector** runs (triggered when R needs more memory), it finds the original SEXP unreachable and reclaims its memory.

This is the full lifecycle: allocation, binding, sharing, copying (if needed), and collection. Every R program, no matter how complex, is just many of these cycles interleaved. And every design decision you have encountered in this book, from copy-on-modify to lexical scoping to lazy evaluation, exists because of how these cycles work. The abstractions are not arbitrary; they are consequences. What remains is to see how R's expression trees become the raw material for metaprogramming, and that is where we turn next.

Exercises

1. Trace the lifecycle of `f <- function(x) x + 1` using the concepts from this chapter. What SEXPTYPE is created? What are its three fields? Where is it stored?
2. Consider `df <- data.frame(a = 1:3, b = c("x", "y", "z"))`. How many SEXPs are involved? (Think about the data frame itself, each column, each string, the names attribute.)
3. Look at the R source for a base function you use frequently (e.g., `rev`, `which`, `paste`). Find the `.Internal()` or `.Primitive()` call. Then search `names.c` in the R source mirror to find the corresponding C function name.
4. Ross Ihaka's 2009 talk "R: Past and Future History" discusses design decisions he would change. Find it online and identify one regret related to the topics in this chapter.

30 R as mathematics

Take R and remove everything except `function()` and the ability to call functions. No numbers. No booleans. No `if`. No `list()`. No `<-` for naming recursive calls. Starting from that barren landscape, try to build integers, conditionals, pairs, lists, and recursion.

Surprisingly, it works. Every one of those things can be reconstructed from `function()` alone. R's `function()` keyword is Church's , its function application is beta reduction, and its closures are the scoping discipline that Scheme borrowed from lambda calculus and passed on to R (Chapter 2). The `double <- function(x) x * 2` from Chapter 5, the recursive descents in Chapter 22, the closures that captured variables and carried them somewhere new – all of them are expressions in a formal system that predates computers by decades.

This chapter runs the experiment.

30.1. Lambda calculus in one page

The lambda calculus, invented by Alonzo Church in 1936, has exactly three things:

1. **Variables**: `x`
2. **Abstraction** (making a function): `x. body`
3. **Application** (calling a function): `(f a)`

That is the entire language. No numbers, no booleans, no `if`, no loops. Church showed that this is enough to express any computation (the same class of functions Turing's machine can compute; see Section 9.6).

In R, abstraction is `function(x) body` and application is `f(a)`. The translation is mechanical:

Lambda calculus	R
`x. x`	`function(x) x`
`x. x + 1`	`function(x) x + 1`
`( x. x + 1)(5)`	`(function(x) x + 1)(5)`
`x.  y. x + y`	`function(x) function(y) x + y`

```
(function(x) x + 1)(5)
#> [1] 6
```

```
(function(x) function(y) x + y)(3)(4)
#> [1] 7
```

The second example is *currying*: a two-argument function rewritten as a function that returns a function. `function(x) function(y) x + y` takes `x`, then returns a new function that takes `y` and adds them. This is the same pattern from Section 7.4 and Section 18.4. It has a name and a history.

R can do this because Ihaka and Gentleman gave it Scheme's lexical scoping (Section 2.5). When the inner function `function(y) x + y` executes, it looks up `x` in the environment where it was *created*, not where it was called, which is precisely what lambda calculus requires: a variable refers to the nearest enclosing abstraction that binds it. Java added lambda expressions in 2014, C++ in 2011, and Python's `lambda` is restricted to a single expression. R has had full, unrestricted lambdas since 1993 – not as an added feature, but as the foundation.

So what can you build on that foundation, using nothing else?

Exercises

1. Write the lambda calculus expression `f. x. f(f(x))` in R and apply it to `sqrt` and `256`. What do you get?
2. Write a curried `power` function: `power <- function(n) function(x) x^n`. Create `square <- power(2)` and `cube <- power(3)`. Test them.
3. What happens if you call `(function(x) function(y) x)(1)(2)`? Why? Think about which variable the inner function captures.

30.2. Church booleans

Suppose all you have is `function()`. How do you represent `TRUE` and `FALSE`?

Church's answer is elegant to the point of disbelief: a boolean is a function that *chooses*. Give it two arguments; `TRUE` picks the first, `FALSE` picks the second.

```
ch_true  <- function(a) function(b) a
ch_false <- function(a) function(b) b
```

These are curried two-argument functions. `ch_true` takes `a`, returns a function that takes `b` and ignores it, handing back `a`. `ch_false` does the opposite: it ignores the first argument and returns the second.

```
ch_true("yes")("no")
#> [1] "yes"
ch_false("yes")("no")
#> [1] "no"
```

That is already `if/else`. A Church boolean *is* a conditional: call it with the "then" branch and the "else" branch, and it selects one. No dispatch mechanism, no keyword, no syntax. The boolean itself does the selecting.

```
ch_if <- function(cond) function(then_val) function(else_val) {
  cond(then_val)(else_val)
}

ch_if(ch_true)("heads")("tails")
#> [1] "heads"
ch_if(ch_false)("heads")("tails")
#> [1] "tails"
```

`ch_if` does almost nothing; it just applies the boolean to the two branches. The boolean does the work. In a sense, there is no `if` in Church encoding. There are only functions that choose.

Now the logical connectives:

```
ch_and <- function(p) function(q) p(q)(p)
ch_or  <- function(p) function(q) p(p)(q)
ch_not <- function(p) function(a) function(b) p(b)(a)
```

Watch `ch_and(p)(q)`: if `p` is true, it picks its first argument, which is `q` (so the result depends on whether `q` is also true); if `p` is false, it picks its second argument, which is `p` itself (false). The logic is embedded in the selection behavior of the booleans themselves, not in any external machinery.

```
# TRUE AND FALSE -> FALSE
ch_and(ch_true)(ch_false)("yes")("no")
#> [1] "no"

# TRUE OR FALSE -> TRUE
ch_or(ch_true)(ch_false)("yes")("no")
#> [1] "yes"

# NOT TRUE -> FALSE
ch_not(ch_true)("yes")("no")
#> [1] "no"
```

Opinion

Church booleans make a quiet point about R: `if/else` is redundant. Conditional logic can be built from functions alone, because R's built-in `if` is syntactic sugar for something that `function()` can already do. A language whose core primitive is powerful enough to subsume its own control flow has a different character from one that added lambdas later.

Exercises

1. Implement `ch_xor` (exclusive or) using only Church booleans. Test it on all four input combinations.
2. Implement `ch_eq` that checks whether two Church booleans are equal. (Hint: `xor` followed by `not` works, or find a direct encoding.)
3. Write a function `to_r_bool` that converts a Church boolean back to R's `TRUE`/`FALSE`.

30.3. Church numerals

Numbers are harder to believe. A Church numeral `n` is a function that takes a function `f` and returns `f` composed with itself `n` times: zero applies `f` zero times (returns `x` unchanged), one applies `f` once, two applies `f` twice, and so on, each number literally counting the layers of function composition that define it.

```
zero  <- function(f) function(x) x
one   <- function(f) function(x) f(x)
two   <- function(f) function(x) f(f(x))
three <- function(f) function(x) f(f(f(x)))
```

To see what a Church numeral "means," pass it a concrete function and a starting value:

```
three(function(x) x + 1)(0)
#> [1] 3
```

Three applications of "add one" to zero gives three. That is the conversion from Church numeral to R integer:

```
to_int <- function(n) n(function(x) x + 1)(0)

to_int(zero)
#> [1] 0
to_int(two)
#> [1] 2
to_int(three)
#> [1] 3
```

The successor function adds one more application of `f`:

```
succ <- function(n) function(f) function(x) f(n(f)(x))
```

Read it this way: `succ(n)` returns a new numeral that, given `f` and `x`, first applies `f` `n` times (that is `n(f)(x)`), then applies `f` one more time on top.

```
four <- succ(three)
to_int(four)
#> [1] 4
```

Build higher numbers:

```
five <- succ(four)
six  <- succ(five)
to_int(six)
#> [1] 6
```

Successor is straightforward. You might expect addition to follow the same logic, just doing `succ` repeatedly. But look at how it actually works:

```
add <- function(m) function(n) function(f) function(x) m(f)(n(f)(x))

to_int(add(two)(three))
#> [1] 5
```

Multiplication: `mult(m)(n)` means "apply `f` `n` times, repeated `m` times." Composing `n` copies of `f` gives a function that applies `f` `n` times; applying that composition `m` times gives `m * n`.

```
mult <- function(m) function(n) function(f) m(n(f))

to_int(mult(two)(three))
#> [1] 6
to_int(mult(three)(three))
#> [1] 9
```

Exponentiation falls out naturally:

```
power <- function(m) function(n) n(m)

to_int(power(two)(three))   # 2^3 = 8
#> [1] 8
to_int(power(three)(two))   # 3^2 = 9
#> [1] 9
```

`power(m)(n)` applies `m` (itself a function-repeater) `n` times. Two applied three times: `2^3`. Sit with this one. The operation that felt most abstract – exponentiation – has the shortest definition, because "applying a function-repeater repeatedly" is what exponentiation *means*. But building numbers up is only half the story. Can we inspect them? Can we ask whether a numeral is zero?

The test is short:

```
is_zero <- function(n) n(function(x) ch_false)(ch_true)

is_zero(zero)("yes")("no")
#> [1] "yes"
is_zero(three)("yes")("no")
#> [1] "no"
```

`zero` ignores `f` and returns `x`, so it returns `ch_true`. Any nonzero numeral applies `f` at least once, and since `f` always returns `ch_false`, a single application replaces `ch_true` with `ch_false`.

We can also convert R integers to Church numerals:

```
from_int <- function(n) {
  result <- zero
  for (i in seq_len(n)) result <- succ(result)
  result
}

to_int(from_int(5))
# [1] 5
to_int(add(from_int(3))(from_int(4)))
# [1] 7
```

Exercises

1. Compute `mult(from_int(4))(from_int(5))` and convert back to an integer.
2. What does `power(from_int(2))(from_int(10))` give? (This is 2^10 built from nothing but nested function calls.)
3. The predecessor function (subtracting one) is famously tricky in Church encoding. Look it up, implement it in R, and test it. This was hard enough that Church himself struggled with it; his student Kleene figured it out, reportedly while getting anesthesia at the dentist.

30.4. Church pairs and lists

A pair is a container that holds two values. In Church encoding, a pair is a function that takes a selector and applies it to both values:

```
pair <- function(a) function(b) function(f) f(a)(b)
fst  <- function(p) p(function(a) function(b) a)
snd  <- function(p) p(function(a) function(b) b)
```

`pair(a)(b)` stores `a` and `b` inside a closure, and `fst` and `snd` extract them by passing the appropriate selector (which, if you look closely, is just `ch_true` and `ch_false` again):

```
p <- pair("hello")("world")
fst(p)
#> [1] "hello"
snd(p)
#> [1] "world"
```

From pairs, you can build linked lists. A list is either empty (nil) or a pair of a head and a tail:

```
nil     <- function(f) function(x) x   # same shape as zero
is_nil  <- function(l) l(function(h) function(t) function(x) ch_false)(ch_true)
cons    <- function(h) function(t) pair(h)(t)
head_of <- function(l) fst(l)
tail_of <- function(l) snd(l)
```

How does `is_nil` work? Recall that a Church-encoded pair `pair(a)(b)` takes a selector `f` and returns `f(a)(b)`. We pass a selector that ignores both arguments and returns `ch_false`, then pass `ch_true` as the default. If `l` is `nil`, it behaves like `zero`: ignores the selector and returns the default (`ch_true`). If `l` is a `cons` cell, the selector runs, ignoring head and tail, returning `ch_false`. The structure of the data does the dispatching.

Build a list and walk it:

```
my_list <- cons("a")(cons("b")(cons("c")(nil)))

head_of(my_list)
# [1] "a"
head_of(tail_of(my_list))
# [1] "b"
head_of(tail_of(tail_of(my_list)))
# [1] "c"
```

Three nested function calls, and you have a linked list. No vectors, no `list()`, no memory allocation beyond closures. But one thing is still missing: none of these structures can refer to themselves.

Exercises

1. Write a `length_of` function that counts the elements in a Church-encoded list. Use `to_int` and Church numerals, or use R's integers for simplicity.
2. Write `map_list` that applies a function to every element of a Church list and returns a new Church list.
3. Implement `reverse_list`. (Hint: fold from the left, consing onto an accumulator that starts as `nil`.)

30.5. The Z combinator

In Chapter 22, we previewed fixed-point combinators. Now the full construction.

A recursive function refers to itself by name, but in lambda calculus, there are no names. There is no `<-`. So how do you define factorial? You need a way to express recursion using only anonymous functions.

The trick is the *fixed-point combinator*. A fixed point of a function `g` is a value `x` such that `g(x) = x`. If `g` is a "template" for a recursive function (a function that takes *itself* as an argument and returns the recursive body), then finding its fixed point gives you the recursive function you wanted.

The Y combinator does this in lazy languages like Haskell:

```
Y = f. ( x. f(x x))( x. f(x x))
```

But R is strict: it evaluates arguments before passing them, so `x(x)` would loop forever. The Z combinator wraps the self-application in a lambda to defer it:

```
Z <- function(f) {
  (function(x) f(function(v) x(x)(v)))(
    function(x) f(function(v) x(x)(v))
  )
}
```

Read it like this: `Z(f)` creates a function `x` that, when called, applies `f` to a delayed version of the recursive call. The `function(v) x(x)(v)` wrapper means "don't call `x(x)` yet; wait until `v` is provided." That delay breaks the infinite loop.

Use it to define factorial without ever naming the function:

```
fact <- Z(function(self) function(n) {
  if (n == 0) 1 else n * self(n - 1)
})

fact(5)
#> [1] 120
fact(10)
#> [1] 3628800
```

`self` is not a name the function gives itself. It is a parameter, injected by `Z`. The function never refers to `fact`; it only refers to `self`, which `Z` arranges to be the function itself.

Fibonacci:

```
fib <- Z(function(self) function(n) {
  if (n <= 1) n else self(n - 1) + self(n - 2)
})
```

```
sapply(0:10, fib)
#>  [1]  0  1  1  2  3  5  8 13 21 34 55
```

A recursive list-sum, using nothing but Church-encoded lists and the Z combinator:

```
church_sum <- Z(function(self) function(lst) {
  is_nil(lst)(
    function(dummy) 0
  )(
    function(dummy) head_of(lst) + self(tail_of(lst))
  )("go")
})

nums <- cons(10)(cons(20)(cons(30)(nil)))
church_sum(nums)
# [1] 60
```

The `function(dummy)` wrappers need explaining. `is_nil(lst)` returns a Church boolean, which selects between two arguments, but if we passed `0` and `head_of(lst) + self(tail_of(lst))` directly, R would evaluate *both* branches eagerly (it is a strict language), and the recursive branch would run even on the empty list, looping forever. Wrapping each branch in `function(dummy)` defers evaluation: the Church boolean selects one wrapper, and the `("go")` call at the end forces the selected branch to execute. The `"go"` argument itself is unused; it is just the trigger that makes the chosen `function(dummy)` run. This is the same thunking trick that distinguishes the Z combinator from the Y combinator: in a strict language, you wrap computations in lambdas to delay them until they are actually needed.

No `for`, no `while`, no `Recall()`, no named recursion. Just `function()`, application, and the Z combinator. The pieces are starting to accumulate.

> **Opinion**
>
> The Z combinator is not practical R code. Its value is conceptual: it proves that recursion is not a primitive operation. Self-reference can be *derived* from anonymous functions and application alone. That R can express this in working, runnable code – not as a thought experiment but as something you can paste into a console and run – is a direct consequence of its Scheme heritage.

Exercises

1. Define a recursive `length` function using `Z` that counts the elements of a Church list (without converting to R types).
2. Use `Z` to write a recursive `map` over Church lists.
3. Why does the Y combinator not work in R? Write it out and try calling `Y(function(self) function(n) if (n == 0) 1 else n * self(n - 1))(5)`. What happens?

30.6. The punchline

Here is the inventory. Starting from nothing but `function()` and function application:

- **Booleans**: `ch_true`, `ch_false`, `ch_and`, `ch_or`, `ch_not`
- **Conditionals**: `ch_if` (which turned out to be trivial, because booleans *are* conditionals)
- **Natural numbers**: `zero`, `succ`, and arithmetic (`add`, `mult`, `power`)
- **Pairs and lists**: `pair`, `fst`, `snd`, `cons`, `nil`
- **Recursion**: the Z combinator

No R integers, no R booleans, no `if`, no `list()`, no named recursive functions. Every one of those features reconstructed from `function()` alone.

This is Church's result from 1936. When Chapter 1 said that R descends from Church's model of computation, this is what that means concretely: `function()` is a universal computational primitive, and everything else is convenience built on top. The requirements are specific: first-class functions, closures, and unrestricted anonymous functions. R has all three, inherited from Scheme. Python's `lambda` is restricted to a single expression, which makes multi-line Church encodings awkward; Java's lambdas cannot close over mutable local variables; C has function pointers but no closures.

The connection between R and mathematics extends further, into the type system itself.

30.7. Types as propositions

There is a correspondence between logic and programming that is not merely analogy. In 1969, William Howard noticed that the rules of constructive logic map exactly onto the rules of typed lambda calculus:

Logic	Programming
Proposition	Type
Proof	Program
A implies B	Function type A -> B
A and B	Pair type (A, B)
A or B	Sum type (either A or B)
True	Unit type (a type with one value)
False	Empty type (a type with no values)

When you write a function with signature `A -> B`, you are constructing a proof that if you have an `A`, you can produce a `B`. A function `integer -> character` proves that integers can be converted to characters. If no function of type `A -> B` can be written, the proposition "A implies B" has no proof.

R's type system is too loose to enforce this correspondence strictly (every R function can error, which is like a proof that cheats). Languages like Haskell, Agda, and Coq take it seriously enough to use programs as machine-checked proofs of mathematical theorems. But the connection is there in R. Every function you write is, in a formal sense, a little proof.

The Curry-Howard correspondence (named for Haskell Curry, who noticed the connection earlier, and Howard, who formalized it) reframes types and functions entirely. A type is not a tag that says "this is a number" – it is a proposition about what values are possible. A function is not a block of code – it is evidence that a transformation exists. What happens when you start looking at the *structure* of those propositions and transformations?

30.8. Functors, monoids, and friends

Three patterns have appeared repeatedly in this book, and they all have names that sound intimidating. The names are beside the point. What matters is that these patterns are not coincidences. Category theory provides a vocabulary for patterns that recur across different mathematical structures, and once you see the vocabulary, you start noticing that R's functional design has been using these structures all along.

Functors. A functor is a mapping between categories that preserves structure. In programming terms: if you have a container and a function, a functor lets you apply the function to the contents without changing the container's shape.

```
x <- list(1, 4, 9, 16)
lapply(x, sqrt)
#> [[1]]
#> [1] 1
#>
#> [[2]]
#> [1] 2
#>
#> [[3]]
#> [1] 3
#>
#> [[4]]
#> [1] 4
```

`lapply()` applied `sqrt` to each element and gave back a list of the same length. The container's shape did not change; only the values inside it did. Strictly, the functor is the list construction itself – it sends a type `T` to "list of `T`" and sends a function `f` to "apply `f` to each element." `lapply()` (and purrr's `map()`) are the *fmap* of that functor: the part that lifts a function into the container.

What makes a functor trustworthy is that it obeys two laws. The *identity law* says mapping a do-nothing function changes nothing:

```
identical(lapply(x, identity), x)
#> [1] TRUE
```

The *composition law* says mapping two functions in sequence gives the same result as mapping their composition in one step:

```
identical(
  lapply(lapply(x, sqrt), log),    # map sqrt, then map log
  lapply(x, \(v) log(sqrt(v)))     # map the composition directly
)
#> [1] TRUE
```

A function that mapped elements but scrambled their relationships would break pipelines. `lapply |> lapply |> lapply` chains work reliably because both laws hold. `mutate()` looks similar on the surface, but it does not satisfy these laws in general – column expressions can reference other columns, introduce side effects, and break composition. It is a useful tool, not a functor.

Monoids. A monoid is a set with an associative binary operation and an identity element. Addition on numbers: `(a + b) + c = a + (b + c)`, identity is 0. String concatenation: identity is `""`. `Reduce()` works on any monoid (Chapter 21), and that is not an accident; `Reduce()` *is* the monoid pattern.

```
Reduce(paste0, c("a", "b", "c"))
#> [1] "abc"
Reduce(`+`, 1:5)
#> [1] 15
```

It needs associativity so the result is the same regardless of how intermediate computations are grouped, and it needs an identity element to handle the empty case. But not every binary operation qualifies. Subtraction is not associative:

```
(5 - 3) - 1
#> [1] 1
5 - (3 - 1)
#> [1] 3
```

Different answers. The operations that fold cleanly are precisely the monoids, and the operations `Reduce()` fails on are the ones that violate the monoid laws. If you find yourself fighting with `Reduce()`, check whether your operation is actually a monoid. If it is not, the folding direction will matter and the results will surprise you.

Natural transformations. A natural transformation converts one functor into another while preserving the relationships between elements. The defining property is the *naturality condition*: if you apply a function to the contents and then transform the container, you get the same result as transforming the container first and then applying the function. The transformation commutes with the functor's action.

`as.list()` on an atomic vector is a natural transformation from the vector functor to the list functor:

```
x <- c(1, 4, 9)
```

```
# map then convert
a <- as.list(sqrt(x))

# convert then map
b <- lapply(as.list(x), sqrt)

identical(a, b)
#> [1] TRUE
```

The order does not matter. That commutativity is the naturality condition, and it is what makes `as.list()` well-behaved: it changes the container without disturbing the relationship between the function and the data inside.

`pivot_longer()` and `pivot_wider()` (Section 16.2, Section 16.3) go further: they are invertible, which makes them natural *isomorphisms*. Reversibility is a stronger property than naturality alone; it means no information is lost in the reshaping. You can pivot long and then pivot wide and arrive back where you started, because the transformation preserves everything.

None of this requires learning category theory. But these patterns keep showing up because they reflect mathematical structure, and R's functional design makes them visible. Church encodings prove this at the foundational level. For day-to-day work, R has practical tools that put mathematical computation within reach.

Exercises

1. `paste0` with `Reduce`: `Reduce(paste0, c("a", "b", "c"))` gives `"abc"`. What is the identity element for `paste0` as a monoid? Verify by adding it as the `init` argument: `Reduce(paste0, character(0), accumulate = FALSE)`.
2. Why is subtraction not a monoid? (Hint: check associativity.) What does this imply about using `Reduce` with `-`?
3. `lapply(list(1:3, 4:6, 7:9), rev)` applies `rev` inside each list element. Verify both functor laws (identity and composition) for this example.

30.9. Practical mathematics in R

Church encodings prove R can do mathematics from nothing. But can it do mathematics that matters? Symbolic algebra, arbitrary precision, number theory: each requires capabilities that most R users never discover, because the packages that provide them sit outside the usual statistical workflow.

Symbolic mathematics. The `Ryacas` package connects R to the Yacas computer algebra system:

```
library(Ryacas)

# Symbolic differentiation
```

```
yac_str("D(x) x^3 + 2*x")  # "3*x^2+2"

# Symbolic integration
yac_str("Integrate(x) x^2")  # "x^3/3"

# Simplification
yac_str("Simplify((x^2-1)/(x-1))")  # "x+1"
```

Where R's floating-point arithmetic gives you `0.1 + 0.2 != 0.3` (Section 6.2), symbolic computation works with exact representations. There is no rounding because there are no decimals; expressions stay symbolic until you ask for a numerical result.

Arbitrary precision. The `gmp` package provides big integers and big rationals:

```
library(gmp)

# Big integers
factorial(as.bigz(100))  # all 158 digits, exact

# Big rationals: 1/3 stays 1/3, no floating-point error
as.bigq(1, 3) + as.bigq(1, 6)  # 1/2, exactly
```

Number theory. The `numbers` package has prime factorization, GCD, modular arithmetic, and related tools:

```
library(numbers)

primeFactors(2310)  # 2, 3, 5, 7, 11
GCD(48, 36)         # 12
isPrime(104729)     # TRUE
```

Project Euler. If you want to sharpen your R skills on mathematical puzzles, Project Euler is an excellent source. The first few problems are approachable with base R:

```
# Euler problem 1: sum of multiples of 3 or 5 below 1000
x <- 1:999
sum(x[x %% 3 == 0 | x %% 5 == 0])
#> [1] 233168
```

```
# Euler problem 6: difference between sum-of-squares and square-of-sum
n <- 1:100
sum(n)^2 - sum(n^2)
#> [1] 25164150
```

Both solutions are one-liners because R's vectorized operations (Chapter 1) make arithmetic on sequences natural. The expression-based style that R inherited from lambda calculus

maps directly onto mathematical notation. That inheritance has a history worth tracing.

Exercises

1. Solve Project Euler problem 2: find the sum of even Fibonacci numbers below four million. (Hint: generate Fibonacci numbers with a while loop or `Reduce`, filter, sum.)
2. Install `gmp` and compute `factorial(as.bigz(200))`. How many digits does the result have? (Use `nchar(as.character(...))`.)
3. `1/3 + 1/3 + 1/3 == 1` returns `TRUE` in R, but `0.1 + 0.1 + 0.1 == 0.3` returns `FALSE`. Why? (Review Section 6.2 if needed.)

30.10. The language is still moving

Base R is still absorbing functional idioms, and each addition shrinks the gap between what the language offers and what the lambda calculus requires.

R 4.1 (2021) introduced two changes. The first was `\(x) x + 1` as shorthand for `function(x) x + 1`. The backslash is not decoration. Church wrote ; R now writes \, and the one-character syntax makes anonymous functions cheap enough to use anywhere: inside `lapply()`, inside `Reduce()`, as arguments to `Map()`. Before R 4.1, the nine characters of `function(x)` were a tax on every lambda expression. The shorthand removed it.

The second was the native pipe `|>`. Before R 4.1, the pipe came from the magrittr package (`%>%`), which means it was a function that manipulated the call, not a language primitive. The native pipe is syntax: `x |> f()` is parsed by R's interpreter as `f(x)` before evaluation begins. Function composition is now part of R's grammar, not a library feature.

R 4.4 (2024) moved `%||%` from rlang into base R. `x %||% y` returns `x` if `x` is not `NULL`, otherwise `y`. This is the null-coalescing operator that C# has had since 2003, Swift since 2014, and Kotlin since 2016. In functional terms, it is a default combinator: given a value that might be absent, provide a fallback. It replaces the four-line `if (is.null(x)) y else x` with a single infix operator.

Anonymous functions, composition, and null handling — three operations that appear in every functional language — moved into R's core syntax between 2021 and 2024. The language is still compressing toward its functional core.

30.11. The historical thread

Each idea in this chapter passed through specific hands. The chain is worth tracing because each link solved a problem that the previous link could not.

Could you reduce all of reasoning to calculation? In 1679, a man who had already co-invented calculus thought you could. Gottfried Wilhelm Leibniz imagined a *calculus ratiocinator*, a formal system that would make thinking mechanical. The dream was premature by two and a half centuries. By the 1870s, mathematicians had spent decades making their own foundations rigorous — formalizing limits, infinity, the number line. A German mathematician named

Gottlob Frege saw that logic could be next. His *Begriffsschrift* (1879) introduced quantifiers, variables, and functions as logical primitives, the first formal notation for logic itself. Modern mathematical logic begins here, but Frege's system was too heavy. Church stripped it to three constructs (variable, abstraction, application) and showed in 1936 that everything could be built back from those alone. The Church numerals and Church booleans in this chapter are his constructions.

That was the theory. The question was whether anyone could make it run. McCarthy answered it in 1960 with LISP, the first programming language where `(lambda (x) (+ x 1))` was a direct encoding of Church's `x. x + 1`. But LISP had dynamic scoping, which meant closures did not work the way the theory said they should. Steele and Sussman fixed that in 1975 with Scheme: lexical scoping, first-class closures, and the "Lambda the Ultimate" papers that showed lambda expressions could replace `goto`, assignment, and most control structures.

In the 1970s, a statistician at Bell Labs who wanted to run a regression still had to write a Fortran program, compile it, and wait for output. Chambers designed S (1976) to change that: a language for statistical computing where you could type an expression and see the result immediately. It had functions and expressions at its core but borrowed its computational style from Fortran. The statistical vocabulary (data frames, formulas, vectorized operations) was new. When Ihaka and Gentleman reimplemented S at the University of Auckland in 1993, they made one decision that changed everything: they replaced S's scoping rules with Scheme's lexical scoping. That single choice is why closures work in R, why function factories work, why every Church encoding in this chapter runs without modification.

So the experiment worked. Starting from `function()` alone, we built integers, booleans, pairs, lists, and recursion. No primitive needed adding; each one assembled from the previous. Church proved it was possible in 1936. McCarthy made it run in 1960. Ihaka and Gentleman, by choosing Scheme's scoping rules, made it run in R without anyone asking them to. The barren landscape from the opening turned out to have everything buried in it already.

30.12. References and sources

Lambda calculus in R:

- Alonzo Church, "An Unsolvable Problem of Elementary Number Theory" (1936). The paper that started it all.
- Hindley & Seldin, *Lambda-Calculus and Combinators: An Introduction* (2008). Accessible textbook covering Church encoding, beta reduction, fixed points.
- Michaelson, *An Introduction to Functional Programming Through Lambda Calculus* (2011). Gentle path from lambda calculus to practical FP.
- Benjamin Pierce, *Types and Programming Languages* (2002), chapters 5-9. Church booleans, numerals, pairs, recursion via fixed-point combinators.

Curry-Howard correspondence:

- Howard, "The Formulae-as-Types Notion of Construction" (1969/1980). Types are propositions, programs are proofs.

- Wadler, "Propositions as Types" (2015, Communications of the ACM). The most readable introduction.

Category theory connections:

- Bartosz Milewski, *Category Theory for Programmers* (2019, free online). Accessible introduction with Haskell examples, translatable to R.
- Mac Lane, *Categories for the Working Mathematician* (1971). The standard reference.

Recreational mathematics in R:

- Project Euler. Mathematical puzzles solvable in any language.
- `Ryacas` package: symbolic math via Yacas CAS.
- `gmp` package: arbitrary precision arithmetic.
- `numbers` package: prime factorization, GCD, modular arithmetic.

Historical:

- Leibniz (1679): the dream of a calculus of reasoning.
- Frege (1879): formal logic, *Begriffsschrift*.
- Church (1936): lambda calculus.
- Curry (1934 onward): combinatory logic, Curry-Howard correspondence.
- McCarthy (1960): LISP, first practical lambda calculus.
- Steele & Sussman (1975): Scheme, "Lambda the Ultimate" papers.
- Chambers (1976 onward): S language.
- Ihaka & Gentleman (1993): R, S reimplemented with Scheme's scoping.

31 Connecting to other languages

Call `sum()` in R. You are calling C. Call `qr()` and you are calling LAPACK Fortran routines that predate R itself by decades. The entire numerical core of the language, from matrix multiplication to sorting to random number generation, lives in compiled code that R merely orchestrates from above, translating your high-level expressions into fast, low-level operations that crunch numbers the way hardware prefers. R was designed to be a glue language, a thin layer of expressiveness sitting on top of compiled muscle. So what happens when you need muscle of your own?

This chapter covers the practical interfaces: how to call C, C++, Rust, Python, and Fortran from R, with complete working examples for each. Chapter 29 introduced the SEXP type system and memory model; Section 28.6 in Chapter 28 showed when compiled code is worth the trouble. Here we go deeper into each interface, starting with the one that underlies all the others.

31.1. C via `.Call()`

`.Call()` is R's native foreign function interface, the bedrock on which every other approach (Rcpp, extendr, all of them) ultimately rests. When Rcpp generates a wrapper or extendr produces a binding, what R actually loads and dispatches is a `.Call()` entry point. Understanding this interface means understanding the foundation that everything else stands on. Section 29.10 in Chapter 29 introduced the three mechanisms for calling compiled code (`.Primitive()`, `.Internal()`, `.Call()`); here we focus on `.Call()` with complete examples.

A C function callable from R takes `SEXP` arguments and returns a `SEXP`, where `SEXP` is a pointer to R's internal object representation (see Chapter 29). Any R objects you allocate inside the function must be shielded from the garbage collector using `PROTECT`, then released with `UNPROTECT` before returning. Get the bookkeeping wrong and things go sideways fast.

Here is a complete example: a C function that computes the sum of a numeric vector.

```
// sum_c.c
#include <R.h>
#include <Rinternals.h>

SEXP sum_c(SEXP x) {
    int n = length(x);
```

```
    double *px = REAL(x);
    double total = 0.0;
    for (int i = 0; i < n; i++) {
        total += px[i];
    }
    return ScalarReal(total);
}
```

Compile and load it from R:

```
system("R CMD SHLIB sum_c.c")
dyn.load("sum_c.so")  # sum_c.dll on Windows
.Call("sum_c", as.numeric(1:1000))
```

`R CMD SHLIB` invokes the system C compiler with the correct flags and include paths, `dyn.load()` loads the shared library into R's process, and `.Call()` dispatches to the function by name.

Notice what happens inside the function. `REAL(x)` extracts the underlying C `double*` array from a numeric SEXP; `ScalarReal()` wraps a C double back into a SEXP. Because this function does not allocate any new R objects, there is nothing to `PROTECT`. If it did allocate (say, a result vector via `allocVector(REALSXP, n)`), you would need to `PROTECT` that allocation and `UNPROTECT(1)` before returning.

That `PROTECT`/`UNPROTECT` protocol is the source of most bugs in hand-written C extensions. Forget a `PROTECT` and the garbage collector frees your object mid-computation, producing a segfault that may not surface until weeks later in someone else's R session. Add a `PROTECT` without a matching `UNPROTECT` and you overflow the protection stack. The count passed to `UNPROTECT` must match the number of `PROTECT` calls exactly, which is precisely the kind of tedious, error-prone bookkeeping that higher-level interfaces were invented to eliminate. But the accessor macros underneath those interfaces are still worth knowing.

Other accessors follow the same pattern: `INTEGER(x)` for integer vectors, `LOGICAL(x)` for logical vectors, `STRING_ELT(x, i)` for string vectors (which are arrays of `CHARSXP` pointers, not C strings), `VECTOR_ELT(x, i)` for list elements.

> **Opinion**
>
> Writing raw C against R's API is rarely the right choice for new code. The reason to learn it is to read existing code: base R, data.table, and hundreds of CRAN packages use `.Call()` directly, and knowing the interface makes their source legible in a way that no amount of documentation can substitute for.

Exercises

1. Write a C function that takes an integer vector and returns its maximum value. Compile it with `R CMD SHLIB`, load it, and test it from R. Use `INTEGER()` instead of `REAL()`.

2. Modify the `sum_c` function to return a length-1 numeric vector allocated with `allocVector(REALSXP, 1)` instead of using `ScalarReal()`. You will need `PROTECT` and `UNPROTECT`. Verify it produces the same result.

31.2. C++ via Rcpp

The `PROTECT`/`UNPROTECT` dance is exactly the kind of ceremony that makes writing raw C against R's API feel like defusing a bomb. What if something else handled the wiring? Rcpp wraps R's C API in C++ classes that manage type conversion and memory automatically: no protection macros, no `SEXP` arithmetic, no accessor juggling. You write ordinary C++ and Rcpp translates it into the `.Call()` machinery underneath.

The same sum function, rewritten:

```
// sum_rcpp.cpp
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double sum_rcpp(NumericVector x) {
    int n = x.size();
    double total = 0.0;
    for (int i = 0; i < n; i++) {
        total += x[i];
    }
    return total;
}
```

The `// [[Rcpp::export]]` attribute tells `Rcpp::sourceCpp()` to generate the `.Call()` wrapper automatically. From R:

```
Rcpp::sourceCpp("sum_rcpp.cpp")
sum_rcpp(as.numeric(1:1000))
```

One function call compiles, links, loads, and registers the function; the turnaround from edit to test is seconds. The convenience extends beyond compilation, too, because Rcpp handles type conversion on both sides of the boundary.

Rcpp provides wrapper classes for all common R types: `NumericVector`, `IntegerVector`, `CharacterVector`, `LogicalVector`, `List`, `DataFrame`, `NumericMatrix`. These proxy the underlying SEXP without copying, so you pay no conversion cost on the way in, and return values are converted back to R objects automatically.

Sugar expressions are Rcpp's vectorized operations, mirroring R's vectorized functions in C++:

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
NumericVector abs_diff(NumericVector x, NumericVector y) {
    return abs(x - y);  // sugar: vectorized, no explicit loop
}

// [[Rcpp::export]]
LogicalVector above_threshold(NumericVector x, double threshold) {
    return x > threshold;  // sugar: vectorized comparison
}
```

Sugar covers `abs`, `sum`, `mean`, `min`, `max`, `ifelse`, `which`, `any`, `all`, `pow`, `sqrt`, and many more. When sugar is sufficient, your C++ code looks almost identical to R but runs at compiled speed.

RcppArmadillo adds the Armadillo linear algebra library:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::export]]
arma::vec solve_system(arma::mat A, arma::vec b) {
    return arma::solve(A, b);
}
```

Armadillo provides matrix decompositions (QR, SVD, Cholesky, eigenvalues), sparse matrix support, and an expression template engine that fuses operations to avoid temporaries. If your bottleneck is linear algebra beyond what R's BLAS provides, RcppArmadillo is the natural tool.

When Rcpp shines: tight loops over vector elements, element-wise operations that resist vectorization in R, recursive algorithms like tree traversal or dynamic programming, and any situation demanding fine-grained control over iteration. Where Rcpp cannot help is I/O-bound code or code that already calls optimized C internally. You cannot make `sum()` faster by rewriting it in Rcpp, because `sum()` is already C. So what about a language that offers the same low-level performance but catches your memory bugs before they happen?

Exercises

1. Write an Rcpp function that computes the running maximum of a numeric vector (each element is the max of all elements up to that index). Compare its speed with `cummax()` from base R using `bench::mark()`.
2. Write an Rcpp function that takes a numeric vector and returns the indices of all values greater than the mean. Use a loop, not sugar. Then rewrite it using sugar (`which(x > mean(x))`). Which version is faster?

3. Using RcppArmadillo, write a function that computes the ordinary least squares coefficients (X^T X)^{-1} X^T y for a matrix X and vector y. Compare with R's `lm.fit()`.

31.3. Rust via extendr

C and C++ give you `PROTECT`/`UNPROTECT` and hope you get the count right. Rust takes a different approach entirely: its type system prevents memory errors at compile time, enforcing ownership rules where every value has exactly one owner, references carry explicit lifetimes, and the compiler flatly rejects code that could produce dangling pointers or use-after-free bugs. No garbage collector needed, no protection stack to manage, no segfaults lurking in code that "usually works."

The `extendr` crate bridges Rust and R, and the R package `rextendr` provides the interactive workflow:

```
rextendr::rust_function("
fn sum_rust(x: &[f64]) -> f64 {
    x.iter().sum()
}
")

sum_rust(as.numeric(1:1000))
```

`rust_function()` compiles a single Rust function and loads it into R, similar to `Rcpp::cppFunction()`. For larger projects, `rust_source()` compiles an entire Rust file.

A slightly more involved example: computing the nth Fibonacci number iteratively.

```
rextendr::rust_function("
fn fib(n: i32) -> i32 {
    if n <= 1 { return n; }
    let mut a = 0i32;
    let mut b = 1i32;
    for _ in 2..=n {
        let tmp = a + b;
        a = b;
        b = tmp;
    }
    b
}
")

fib(10)
#> [1] 55
```

No `PROTECT`, no `UNPROTECT`, no `SEXP`. The type conversion between R and Rust is handled by extendr's `#[extendr]` macro (used in source files) or inferred by `rust_function()`.

Rust's ownership model says every value has exactly one owner; when ownership transfers, the old name becomes invalid. R's copy-on-modify semantics (Section 28.3) say an object is shared until someone modifies it, at which point R copies it so the original is preserved. The two mechanisms look different — Rust enforces ownership at compile time; R defers to runtime copying — but they guarantee the same thing: a value will not change while someone else is looking at it. That guarantee is why Rust backends like the Polars data frame library can hand data to R through extendr without defensive copying.

But does the safety justify the ecosystem trade-off?

Rcpp vs extendr: Rcpp has over 2,600 reverse dependencies on CRAN and has been battle-tested for 15 years; its documentation, Stack Overflow coverage, and library of examples are unmatched in the R world. extendr is younger, with a smaller ecosystem and fewer production examples. The trade-off is safety. Rcpp inherits C++'s memory model, where segfaults and undefined behavior remain possible if you make mistakes. Rust eliminates those entire categories of bug at compile time, which matters more the larger and more complex the compiled codebase becomes.

For package development, `rextendr::use_extendr()` sets up the directory structure and build configuration. The `polars` package (R bindings to the Polars data frame library) and `gifski` (GIF encoding) are production examples of extendr-based packages on CRAN.

> **Opinion**
>
> If you are starting a new package today and the compiled code is non-trivial (more than a few hundred lines), Rust is worth serious consideration. The upfront cost of learning ownership semantics pays for itself in bugs you never have to debug. For quick one-off functions or small performance patches, Rcpp's lower friction and larger community still win.

Exercises

1. Install `rextendr` and write a Rust function that counts the number of values in a numeric vector that exceed a given threshold. Call it from R and verify the result.
2. Compare the compile time of `Rcpp::cppFunction()` and `rextendr::rust_function()` for equivalent simple functions. Which has faster turnaround?

31.4. Python via reticulate

Calling Python from R is not about speed. Python's interpreter is slower than R's for numerical work. The reason to call Python is access: scikit-learn, PyTorch, TensorFlow, Hugging Face Transformers, spaCy, and hundreds of other libraries either have no R equivalent or their R equivalents lag behind by months or years. When a new deep learning architecture drops, the Python implementation comes first. Always.

The `reticulate` package embeds a Python interpreter inside R's process, with no inter-process communication overhead; R and Python share the same memory space.

Importing modules:

```
library(reticulate)

np <- import("numpy")
pd <- import("pandas")

x <- np$array(c(1, 2, 3, 4, 5))
np$mean(x)
#> [1] 3
```

The `$` operator accesses Python attributes and methods. R vectors convert to NumPy arrays automatically, and NumPy arrays convert back to R vectors.

Sourcing Python scripts:

```
# helpers.py contains:
# def normalize(x):
#     return (x - x.mean()) / x.std()

source_python("helpers.py")
normalize(c(1, 2, 3, 4, 5))
```

`source_python()` executes a Python file and makes its top-level functions available in R's global environment as regular R functions.

Type conversion rules: R numeric vectors become NumPy arrays, R data frames become Pandas DataFrames, R lists become Python dicts, R `TRUE`/`FALSE` become Python `True`/`False`, and R `NULL` becomes Python `None`. These conversions happen automatically in most cases, and for large arrays the conversion is zero-copy when the memory layout is compatible (both R and NumPy store doubles as contiguous 64-bit IEEE 754 values).

Calling scikit-learn from R:

```
sklearn <- import("sklearn")
linear_model <- import("sklearn.linear_model")

X <- matrix(rnorm(200), ncol = 2)
y <- X[, 1] * 3 + X[, 2] * -1 + rnorm(100, sd = 0.5)

model <- linear_model$LinearRegression()
model$fit(X, y)
model$coef_
#> [1]  2.98 -1.02
```

The model object lives in Python, but you interact with it from R using `$`. Predictions, coefficients, scores: all accessible through the same operator.

Managing Python environments: reticulate can use system Python, virtualenvs, or conda environments. Specify which Python to use before loading reticulate:

```
Sys.setenv(RETICULATE_PYTHON = "/usr/bin/python3")
# or
reticulate::use_virtualenv("myproject")
# or
reticulate::use_condaenv("myenv")
```

Call this before `import()`. Once the Python interpreter starts, you cannot switch to a different one within the same R session.

Exercises

1. Use reticulate to import Python's `collections` module and call `Counter` on a character vector. Verify the result matches R's `table()`.
2. Create a NumPy array of 1 million random values using `np$random$standard_normal()`. Pass it to an R function (e.g., `mean()`). Does reticulate copy the data or share it? Use `bench::mark()` with varying sizes to find out.

31.5. Fortran

Every linear model you have ever fitted in R ultimately ran Fortran code. Every call to `qr()`, `svd()`, `chol()`, and matrix multiplication (`%*%`) dispatches to BLAS and LAPACK routines written in a language that predates C by over a decade, and those routines have been optimized by numerical analysts for longer than most programming languages have existed. Fortran is also the oldest language in R's foreign function toolkit. Understanding its interface explains something that puzzles people who benchmark R against "faster" languages: R's numerical performance is competitive precisely because R does not do the numerical work itself.

The `.Fortran()` interface passes R vectors to Fortran subroutines by copying them in and out:

```
! dot_product.f90
subroutine dotprod(x, y, n, result)
    implicit none
    integer, intent(in) :: n
    double precision, intent(in) :: x(n), y(n)
    double precision, intent(out) :: result
    integer :: i
    result = 0.0d0
    do i = 1, n
```

```
        result = result + x(i) * y(i)
    end do
end subroutine
```

```
system("R CMD SHLIB dot_product.f90")
dyn.load("dot_product.so")

result <- .Fortran("dotprod",
                   x = as.double(1:5),
                   y = as.double(6:10),
                   n = 5L,
                   result = double(1))
result$result
#> [1] 130
```

`.Fortran()` returns a named list with all arguments, including outputs. This copy-in-copy-out semantics is straightforward but wasteful for large data; the newer `.Call()` interface with C wrappers around Fortran code avoids the copies.

You will rarely write new Fortran for R. The interface matters for two reasons: reading legacy code (many statistical packages on CRAN have Fortran backends dating to the 1990s), and understanding why R's numerical performance is strong despite its interpreted overhead. When someone says "R is slow," they are talking about R's interpreter loop, not the compiled Fortran that does the actual linear algebra. But with four modern interfaces available (C, C++, Rust, Python) plus Fortran, which one should you actually reach for?

31.6. When to use what

The answer depends on why you need another language in the first place.

You need speed in a tight loop: profile first (Section 28.1). If the bottleneck is a loop that cannot be vectorized, use Rcpp (largest ecosystem, fastest iteration cycle) or Rust via extendr (memory safety, better for large codebases). Raw C via `.Call()` if you want zero dependencies.

You need a library that only exists in Python: use reticulate. This includes deep learning (PyTorch, TensorFlow), NLP (spaCy, Transformers), and computer vision. Do not rewrite Python libraries in R; call them.

You need numerical linear algebra beyond base R: RcppArmadillo for dense matrices, RcppEigen for sparse matrices. Or call LAPACK directly via `.Fortran()` if you need a specific routine.

You are building a package with substantial compiled code: consider Rust if the team knows it, or is willing to learn. The compile-time safety checks catch bugs that would otherwise surface as sporadic segfaults in users' R sessions. For smaller amounts of compiled code, Rcpp is fine.

You have legacy Fortran code: wrap it with `.Fortran()` or write a thin C wrapper and use `.Call()`.

The ordering from Section 28.7 still applies: pure R first, vectorize, pre-allocate, switch engines (data.table, Arrow, DuckDB), then compiled code. Calling another language is a cost, one that adds build dependencies, complicates installation, and makes debugging harder. Pay that cost only when the benefit is clear.

> **Opinion**
>
> R was designed from the start to sit on top of compiled code. The foreign function interface is the architecture, not a feature bolted on later.

31.7. Packages worth studying

Real-world packages show how these interfaces work at scale, and each of the following is open source; reading their `src/` directory teaches more than any tutorial can.

C backends:

- `data.table`: the core grouping, joining, and sorting engine is C. The `src/` directory is a masterclass in writing high-performance C against R's API. `fread()` and `fwrite()` are C implementations of CSV reading and writing that outperform most alternatives.
- `stringi`: wraps the ICU (International Components for Unicode) C library. Provides fast, correct string operations covering Unicode normalization, collation, regex, and transliteration.

C++ backends:

- `arrow`: R bindings to the Apache Arrow C++ library. Columnar in-memory format for zero-copy data exchange between systems.
- `torch`: R bindings to LibTorch (PyTorch's C++ backend). No Python dependency; the C++ library is called directly via `.Call()` with custom binding code.
- `dplyr`: the core verbs dispatch to C++ for grouped operations via `vctrs` and internal C code.

Rust backends:

- `polars`: R bindings to the Polars DataFrame library, built with extendr. A complete data manipulation engine written in Rust, exposed to R through generated `.Call()` wrappers.
- `gifski`: GIF encoding library, also via extendr. A small, clean example of a Rust-powered R package.

Python bridges:

- `tensorflow` and `keras3`: use reticulate to call TensorFlow/Keras. The R API mirrors the Python API closely.
- `spacyr`: wraps spaCy for natural language processing.

The pattern across all of these is the same: R provides the user-facing API (function names, argument handling, documentation, S3/S4 dispatch), while the compiled backend provides the computation. The interface layer is thin. A high-level language for expressiveness orchestrating a low-level language for speed: that is the architecture R was built for.

In Chapter 1, this book described two traditions: Church's lambda calculus (expressions and functions) and Turing's machine (instructions and state). R descends from Church, but it calls into Turing's world every time it invokes C, Fortran, or Rust. The foreign function interfaces in this chapter are the bridge between those two models. You write the logic in R — composing functions, passing closures, piping transformations — and the performance-critical inner loops run in a language that manipulates memory directly. Church's model gives you the expressiveness and composability to think clearly about what the code should do. Turing's model gives you the control over memory layout and instruction scheduling to make it fast. R's foreign function interfaces sit at the boundary, translating between a world of expressions and a world of addresses. The `.Call()` interface is where Church's abstractions meet Turing's machine, and the fact that R was designed with that boundary in mind is what makes the translation practical rather than painful.

What happens when you apply the same principle not to a single function but to an entire project?

31.8. References and sources

C (the foundation):

- R Core Team, *Writing R Extensions*, chapter 5 ("System and foreign language interfaces"). The official guide to `.C()`, `.Call()`, `.External()`.
- `.Call()` is the modern interface: pass SEXPs, return SEXPs, full control. `.C()` is the old interface (copies data, limited types). `.External()` is rarely used.
- `R_RegisterCCallable()` / `R_GetCCallable()`: sharing C functions between packages without linking.

C++ via Rcpp:

- Dirk Eddelbuettel, *Seamless R and C++ Integration with Rcpp* (2013). The standard reference.
- Dirk Eddelbuettel & Romain Francois, "Rcpp: Seamless R and C++ Integration" (2011, JSS).
- Hadley Wickham, *Advanced R* (2e), chapter 25.

Rust via extendr:

- extendr project (extendr.github.io). Rust extensions for R, inspired by PyO3.
- The `rextendr` package vignettes cover both interactive use and package integration.

Python via reticulate:

- Kevin Ushey et al., "reticulate: Interface to Python" (CRAN). Full documentation at rstudio.github.io/reticulate.

Fortran:

- R Core Team, *Writing R Extensions*, section 5.2. The `.Fortran()` interface.
- LAPACK Users' Guide (netlib.org). The linear algebra library that R calls internally.

32 Building packages

You email `analysis.R` to a colleague. They run it and get an error because they don't have the `janitor` package. They install it, try again, and hit a different wall: your script calls `source("helpers.R")`, a file they don't have. You send it. They put it in the wrong directory. Two days later, they give up. The problem isn't your code; it's that scripts have no way to declare what they need, no way to carry their own context, no mechanism for self-description. Packages solve all three problems at once, and once you see the structure, you can build one in ten minutes.

32.1. Why packages

A package is the unit of shareable, testable, documented R code. Scripts depend on `source()` calls, hardcoded paths, and the hope that the right libraries happen to be installed; packages carry their dependencies, their documentation, and their tests as a single installable unit that either works or tells you exactly why it doesn't.

Compare the email scenario with what happens when you share a package. Your colleague runs `pak::pak("username/mypackage")`, and everything installs: functions, documentation, dependencies. They type `?my_function` and see how to use it. No back-and-forth. No guessing.

Even if you never publish to CRAN, a package gives you:

- **Forced documentation**: you have to describe what your function does.
- **Automated testing**: you can prove it works.
- **Dependency management**: you declare what you need.
- **Namespace control**: your names do not collide with other packages' names.

In Chapter 7, you saw that functions are values. A package is a named collection of functions with metadata, and that metadata is what separates "a folder of scripts" from "something someone else can install and use." The metadata has a specific shape, which raises an immediate question: what does it look like?

> **Opinion**
>
> If you have written the same function in three scripts, it belongs in a package. Packages are not just for CRAN.

The complete reference for everything in this chapter is *R Packages* (2nd edition) by Hadley Wickham and Jenny Bryan, freely available at r-pkgs.org. This chapter gives you enough to build your first package. That book gives you enough to build your twentieth.

32.2. The anatomy of a package

The minimum viable package has three components: `DESCRIPTION`, `NAMESPACE`, and an `R/` directory. A package is a namespace of functions: the `R/` directory holds function definitions, `NAMESPACE` controls which ones are visible, and `DESCRIPTION` records metadata. A standard layout looks like this:

```
mypackage/
  DESCRIPTION       # metadata: name, version, authors, dependencies
  NAMESPACE         # what you export, what you import (auto-generated)
  R/                # your functions
  man/              # documentation (auto-generated by roxygen2)
  tests/            # test files
  vignettes/        # long-form documentation
  data/             # included datasets
  .Rbuildignore     # files to exclude from the built package
  LICENSE           # license file
```

DESCRIPTION is the most important file. It declares the package name, title, version, authors, license, and dependencies, and every field matters because CRAN checks it, `install.packages()` reads it, and other packages' `Imports` reference it. A minimal example:

```
Package: mypackage
Title: What My Package Does (One Line, Title Case)
Version: 0.1.0
Authors@R: person("Jane", "Doe", email = "jane@example.com",
                  role = c("aut", "cre"))
Description: A longer description of what the package does. This can
    span multiple lines. It should explain the purpose, not list
    functions.
License: MIT + file LICENSE
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.1
```

NAMESPACE controls what is visible to users (exports) and what you borrow from other packages (imports). Never edit it by hand; let roxygen2 generate it. Two hooks let you run code when the package loads: `.onLoad()` fires when the namespace is loaded (even by `::` access), while `.onAttach()` fires only when someone calls `library()`. Use `.onLoad()` for setup that must happen before any function runs (registering S3 methods, setting default options,

initializing connections) and `.onAttach()` only for startup messages. Both go in `R/zzz.R` by convention. For full details, see R Packages (2e), chapter 10.

The namespace is an environment (the concept from Section 7.4), and `NAMESPACE` defines what names are visible in that environment. When you call `dplyr::filter()`, R looks in dplyr's namespace environment; when you call just `filter()`, R searches the environment chain, and which `filter` it finds depends on what is attached. In lambda calculus terms, a package namespace is a closure at the module level: functions inside can reference each other because they share the namespace environment, but the outside world only sees what is explicitly exported. The `NAMESPACE` file marks the same boundary that a lambda abstraction marks between bound and free variables.

32.3. Creating a package

One function call scaffolds everything:

```
usethis::create_package("path/to/mypackage")
```

This creates the directory, `DESCRIPTION`, `NAMESPACE`, `R/`, `.Rbuildignore`, and `.gitignore`. The package is immediately loadable, which means you can start writing functions right away.

The development loop has four verbs:

```
devtools::load_all()    # simulate installing the package (Ctrl+Shift+L)
devtools::document()    # regenerate man/ and NAMESPACE from roxygen
(Ctrl+Shift+D)
devtools::test()        # run all tests (Ctrl+Shift+T)
devtools::check()       # run R CMD check (Ctrl+Shift+E)
```

`load_all()` is the key to fast iteration. It sources all files in `R/`, makes exports available, and simulates a fresh install without actually installing, so the cycle becomes: edit a function, hit `Ctrl+Shift+L`, test immediately, repeat. An installed package is immutable: once built, its functions cannot be edited in place. `load_all()` gives you a mutable development loop on top of that immutable target, so you iterate freely and then freeze the result when you install. Without it, you'd have to rebuild and reinstall the entire package after every change, which is the kind of friction that kills momentum.

The `usethis::use_*()` functions handle setup tasks:

```
usethis::use_r("bmi")              # create R/bmi.R
usethis::use_test("bmi")           # create tests/testthat/test-bmi.R
usethis::use_package("dplyr")      # add dplyr to Imports
usethis::use_mit_license()         # add MIT license
usethis::use_readme_rmd()          # add README.Rmd
usethis::use_github_action()       # add CI/CD
```

Each one does one task correctly. No boilerplate to remember. But the functions you write remain invisible to users until you document them, and documentation has its own conventions.

> **Opinion**
>
> Never create package files by hand. `usethis` knows the right boilerplate. You focus on the code.

Exercises

1. Create a package called `mymath` using `usethis::create_package()`. Add a file `R/square.R` with a function `square <- function(x) x^2`. Load it with `devtools::load_all()` and test that `square(5)` returns 25.
2. Run `devtools::check()` on your package. How many errors, warnings, and notes do you get? Read the output carefully.

32.4. Writing documentation with roxygen2

Pure functions are the easiest to document: their contract is fully specified by inputs and outputs, so `@param` and `@return` capture everything a caller needs to know. Functions with side effects need additional prose explaining what state they modify and when.

Documentation lives above your function as special comments (`#'`):

```
#' Calculate body mass index
#'
#' Computes BMI from weight and height using the standard formula.
#'
#' @param weight Weight in kilograms.
#' @param height Height in meters.
#' @return A numeric vector of BMI values.
#' @export
#' @examples
#' bmi(70, 1.75)
bmi <- function(weight, height) {
  weight / height^2
}
```

Key tags:

- `@param`: describe each argument. Type + meaning + constraints.
- `@return`: what the function returns.
- `@export`: makes the function visible to users. Without it, the function is internal (accessible via the triple-colon operator but not `::`).
- `@examples`: runnable code. `R CMD check` executes these.

- `@seealso`, `@family`: cross-references to related functions.
- `@inheritParams`: borrow parameter documentation from another function.

Markdown in roxygen is enabled by adding `Roxygen: list(markdown = TRUE)` to `DESCRIPTION`, which lets you use `**bold**`, `*italic*`, `` `code` ``, and `[function()]` for cross-links.

`devtools::document()` converts roxygen comments to `.Rd` files in `man/` and updates `NAMESPACE`. You write roxygen; you never touch `man/` directly. Good documentation earns users, but even the best-documented function needs other packages to work with.

> **Opinion**
>
> Document the "why", not just the "what". `@param x A numeric vector` tells me the type. `@param x Body mass in grams; must be positive` tells me how to use it.

A common pattern in real packages is documenting multiple related functions on the same help page using `@rdname`:

```
#' Arithmetic operations
#'
#' @param x A numeric vector.
#' @return A numeric vector.
#' @name arithmetic
NULL

#' @rdname arithmetic
#' @export
square <- function(x) x^2

#' @rdname arithmetic
#' @export
cube <- function(x) x^3
```

Both `?square` and `?cube` now point to the same help page, which is useful when functions are closely related and easier to understand side by side. But how do you manage the other packages your functions depend on?

Exercises

1. Add roxygen documentation to the `square()` function from the previous exercise. Include `@param`, `@return`, `@export`, and `@examples`. Run `devtools::document()` and view the help with `?square`.
2. Create a second function `cube()` in the same package. Document both using `@rdname` so they share a help page. Run `devtools::document()` and verify with `?square`.

32.5. Dependencies

Your package will use functions from other packages. How you declare that relationship determines whether your package installs cleanly or collapses into a tangle of missing symbols.

Imports: packages your code needs to work. Listed in `DESCRIPTION`. Use `package::function()` in your code:

```
# Good: explicit
clean_data <- function(df) {
  dplyr::filter(df, !is.na(value))
}
```

Suggests: packages needed only for tests, vignettes, or examples. Not installed automatically when a user installs your package.

Depends: rarely appropriate. Attaches the package when yours is loaded, meaning `library(mypackage)` also runs `library(dependency)`, which pollutes the user's namespace. Almost always wrong; use `Imports` instead.

The rule is simple: use `Imports` and call functions with `::`. This makes dependencies explicit and avoids namespace collisions. If typing `dplyr::filter()` everywhere feels verbose, you can import specific functions into your namespace with the roxygen tag `@importFrom dplyr filter`, which writes `importFrom(dplyr, filter)` into `NAMESPACE` and lets you call `filter()` without the prefix. Use this sparingly: one or two heavily-used functions per dependency, not every function you call.

```
# Bad: which filter? stats::filter or dplyr::filter?
filter(data, x > 0)

# Good: unambiguous
dplyr::filter(data, x > 0)
```

`usethis::use_package("dplyr")` adds to `Imports` in `DESCRIPTION`. `usethis::use_package("ggplot2", type = "Suggests")` adds to `Suggests`.

When you use a Suggested package in tests or vignettes, guard the code:

```
test_that("plotting works", {
  skip_if_not_installed("ggplot2")
  p <- ggplot2::ggplot(data, ggplot2::aes(x, y)) + ggplot2::geom_point()
  expect_s3_class(p, "ggplot")
})
```

`skip_if_not_installed()` prevents the test from failing on machines where the Suggested package is not available.

Minimize dependencies. Every dependency is a potential breakage point, and if you only need one function from a package, consider whether you can write it yourself. A useful diagnostic: `usethis::use_package()` will refuse to add a package that is not installed, catching typos early, and `devtools::check()` will warn if you import a package but never use it. Dependencies declare what your package needs; tests prove that what you built actually works.

32.6. Testing with testthat

`usethis::use_testthat(edition = 3)` sets up the test infrastructure: `tests/testthat/`, `tests/testthat.R`, and the necessary `DESCRIPTION` fields.

`usethis::use_test("bmi")` creates `tests/testthat/test-bmi.R`. Tests are organized in `test_that()` blocks:

```
test_that("bmi computes correctly", {
  expect_equal(bmi(70, 1.75), 70 / 1.75^2)
  expect_length(bmi(c(70, 80), c(1.75, 1.80)), 2)
})

test_that("bmi rejects invalid input", {
  expect_error(bmi("a", 1.75))
})
```

Key expectations:

- `expect_equal()`: compare with tolerance (good for floating-point results).
- `expect_identical()`: compare exactly (good for integers, strings, logicals).
- `expect_true()`, `expect_false()`: boolean checks.
- `expect_error()`, `expect_warning()`, `expect_message()`: check for conditions.
- `expect_length()`, `expect_named()`: structural checks.
- `expect_snapshot()`: capture output and compare to a stored reference. Good for complex output that is hard to specify by hand.

`devtools::test()` runs all tests. `devtools::test_active_file()` runs just the file you are editing.

Pure functions are also the easiest to test: same inputs always produce the same output, so each `expect_equal()` call is a complete specification of behavior with no setup or teardown required.

How many tests should you write? Enough to cover the normal case, the edge cases, and the error cases. For a function like `bmi()`, that means correct output for typical input, correct output for vectorized input, correct behavior for zero or negative values, and an error for non-numeric input. Four or five tests per function is a reasonable starting point, and writing them forces you to think about your function's contract more carefully than any amount of staring at the implementation would.

> **Opinion**
>
> Test the contract, not the implementation. Your test should pass even if you rewrite the function body, as long as the inputs and outputs stay the same.

Exercises

1. Write tests for the `square()` function: test that `square(3)` is 9, `square(-2)` is 4, `square(0)` is 0, and `square(c(1, 2, 3))` returns `c(1, 4, 9)`.
2. Add a test that checks `square()` returns a numeric vector. (Hint: `expect_type()`.)

32.7. Vignettes

Function documentation is reference: you look up a specific function when you already know what you're looking for. Vignettes are the opposite. They show a newcomer how the pieces fit together, walking through a workflow from start to finish, and the difference between the two is the difference between a dictionary and a story.

```
usethis::use_vignette("getting-started")
```

This creates a template in `vignettes/`. R Markdown vignettes (`.Rmd`) are the standard: code chunks run during package build, and the output is HTML. Quarto vignettes (`.qmd`) are the newer option, with more features but requiring Quarto as a system dependency.

A good package has at least one vignette: "Getting Started" or "Introduction to mypackage." Write it as if the reader has never seen your package before and has five minutes to decide whether to keep reading.

A vignette template looks like this:

```
---
title: "Getting Started with mypackage"
output: rmarkdown::html_vignette
vignette: >
  %\VignetteIndexEntry{Getting Started with mypackage}
  %\VignetteEngine{knitr::rmarkdown}
  %\VignetteEncoding{UTF-8}
---

## Installation

Install from GitHub:

```r
pak::pak("username/mypackage")
```
```

32.8. Basic usage

Load the package and run a simple example:

```
library(mypackage)
result <- bmi(70, 1.75)
result
```

```
The YAML header is boilerplate. The content is yours. Documentation and vignettes make a package usable, bu
```

```
## R CMD check {#sec-r-cmd-check}

`R CMD check`\index{R CMD check} runs roughly 50 tests against your package: does the `NAMESPACE` match the
```

```
::: {.cell}

```{.r .cell-code}
devtools::check()

:::
```

Three levels of feedback:

- **ERROR**: something is broken. Must fix.
- **WARNING**: something is wrong. Must fix for CRAN; should fix regardless.
- **NOTE**: something is unusual. Fix if you can; explain if you can't.

The goal: 0 errors, 0 warnings, 0 notes. Each of those roughly 50 tests exists because someone, somewhere, shipped a broken package, and the strictness reflects the cost of breakage on CRAN, where 20,000+ packages depend on each other. Every NOTE exists because someone shipped that exact mistake to CRAN and broke something downstream.

Common issues and how to fix them:

- **"Undocumented arguments"**: add `@param` tags in your roxygen comments.
- **"Undefined global functions or variables"**: you are calling a function from another package without `::`. Use `dplyr::filter()` instead of `filter()`, or add `@importFrom dplyr filter` to your roxygen.
- **"Non-standard file/directory found"**: add the file to `.Rbuildignore` with `usethis::use_build_ignore("filename")`.
- **"No visible binding for global variable"**: common with tidyverse code that uses unquoted column names. Fix with `utils::globalVariables()` or use `.data$column` from rlang.
```

Run `check()` often during development, not just at the end. Each run should find at most one or two issues; if you wait until the end, you face dozens of notes at once, which is demoralizing and nearly impossible to debug systematically. A clean check means your package is well-formed, but a check that passes on your machine and fails on Linux is still a problem, which is why the next question matters: how do you get your package into other people's hands, and how do you make sure it works on their machines too?

Exercises

1. Introduce a deliberate error in your package: remove the `@param` tag for one argument. Run `devtools::check()` and find the NOTE. Fix it and re-check.
2. Add `dplyr::filter` to a function without adding dplyr to `Imports`. Run `devtools::check()`. What feedback do you get?

32.9. Sharing your package

GitHub: push your package to GitHub. Users install with `pak::pak("username/mypackage")` or `devtools::install_github("username/mypackage")`. Lowest barrier to sharing.

pkgdown: `usethis::use_pkgdown_github_pages()` builds a website from your documentation, README, and vignettes, with automatic deployment through GitHub Actions. Professional documentation for free.

CRAN: the official repository. `devtools::submit_cran()`. Strict review: 0 errors, 0 warnings, ideally 0 notes, plus human reviewers who check CRAN policies. Worth the effort for packages with a broad audience.

R-universe (`r-universe.dev`): automated package hosting with a lower barrier than CRAN but still discoverable. Point it at your GitHub repo, and it builds binaries for all platforms automatically.

Version numbers follow semantic versioning (semver.org): MAJOR.MINOR.PATCH. PATCH for bug fixes, MINOR for new backward-compatible features, MAJOR for breaking changes. The tidyverse follows this strictly: when dplyr goes from 1.0 to 1.1, your code will not break; when it goes to 2.0, check the changelog.

The progression is natural. You build a package for yourself, put it on GitHub so colleagues can use it, add pkgdown so new users can find their way around, and eventually submit to CRAN if the package solves a general enough problem. Each step adds effort but also reach.

For GitHub-hosted packages, continuous integration is nearly free:

```
usethis::use_github_action("check-standard")
```

This adds a GitHub Actions workflow that runs `R CMD check` on Linux, macOS, and Windows every time you push. If the check fails, GitHub shows a red X next to the commit; if it passes, you get a green checkmark and a badge for your README. The real value is

catching platform-specific bugs you would never find on your own machine: path separators (Windows uses `\`, everything else uses `/`), case-sensitive file systems (Linux is case-sensitive, macOS and Windows are not), and system library availability.

A package with CI, documentation, and a one-line install command turns a two-day email chain into a single `pak::pak()` call.

A closure captures variables in its enclosing environment and exposes only the returned function (Chapter 18). A package does the same thing at a larger scale: internal functions, data, and dependencies live in the namespace; only the functions listed in `NAMESPACE` are visible outside. `devtools`, `usethis`, and roxygen automate the mechanics.

Exercises

1. Push a package to GitHub and install it on another machine (or in a fresh R session) using `pak::pak()`. Does it install cleanly?

> **Opinion**
>
> Start with GitHub. Add pkgdown when your package has users. Submit to CRAN when it is stable and general-purpose. CRAN gives your package visibility, credibility, and a guarantee that it installs cleanly across platforms. If your package solves a real problem, aim for CRAN.

33 Reproducibility and workflow

You finished an analysis six months ago, and now a reviewer wants you to re-run it with one variable removed. You open the project, hit "Source," and nothing works: a package updated, a file path points to a folder that no longer exists, and you cannot remember whether you were supposed to run the cleaning script before or after the imputation script. The analysis produced correct results once. It will never produce them again.

This chapter walks through five tools (`here`, `renv`, Quarto, `targets`, Git) that prevent exactly this situation, giving each one enough coverage to start using it today while pointing to deeper references when you need them. Each layer eliminates a specific way that analyses break.

33.1. What reproducibility means

Same code plus same data should equal the same results. That is reproducibility, and if you squint, it looks like referential transparency at the project level: your analysis script is an expression, and if that expression is reproducible, you can replace it with its value (the results) without changing meaning. A non-reproducible analysis is a function with hidden side effects; it depends on state you never declared.

Every tool in this chapter addresses one category of hidden state. `here` makes file paths explicit, removing the dependency on which directory you happened to open. `renv` pins package versions to the project, so a colleague installing your code six months from now gets the same library you tested against. Quarto binds code to output in a single document, making it impossible for the prose and the numbers to drift apart. `targets` declares dependencies between pipeline steps and only reruns what has changed, turning a sequential script into a dependency graph that skips unnecessary work. Git records the reasoning behind each change, not just snapshots. Together these tools enforce the functional principle of referential transparency at the scale of a project: swap one execution for another and get identical results.

This is not replicability, which asks a bigger question: same research question, new data, similar conclusions. Reproducibility is the lower bar. Most analyses fail it anyway.

A 2016 Nature survey found that 70% of researchers had tried and failed to reproduce another scientist's experiment, and a 2019 study found that only 26% of published R-based analyses could be independently reproduced from the provided code. Reproducing work requires infrastructure, and most analysts do not build infrastructure until something has already gone wrong.

Why it breaks:

- Hardcoded file paths (`"C:/Users/me/Desktop/data.csv"`).
- Unrecorded package versions (dplyr 1.0 behaves differently from dplyr 1.1).
- Manual steps between scripts ("run this, then open Excel, then paste the table…").
- Random seeds not set.
- Outputs separated from the code that produced them.

The fix is structural, not behavioral. Do not rely on discipline ("I will remember to run script 3 first"); build a system that makes the wrong thing hard. Each tool in this chapter eliminates one failure mode: `here` eliminates path problems, `renv` eliminates version problems, Quarto eliminates the code/output sync problem, `targets` eliminates the run-order problem, and Git eliminates the "which version was it?" problem. The first one costs nothing to adopt.

> **Opinion**
>
> If your analysis requires a README that says "run these scripts in this order, but skip step 4 on Tuesdays," it is not reproducible. It is a ritual.

33.2. Projects and file paths

Every path you hardcode is a bet that your directory structure will never change. You will lose that bet. An RStudio project (`.Rproj` file) defines a working directory so that everything becomes relative to one root, and when you open the project, the working directory is set automatically.

`here::here()` builds paths relative to that root:

```
# Works regardless of where the script lives in the project
data <- read.csv(here::here("data", "penguins.csv"))
```

`here::here("data", "penguins.csv")` produces something like `"C:/Users/me/projects/analysis/data/penguins.csv"`, but you never type that path. It works on any OS (forward slashes on Windows too), in any subdirectory, in Quarto documents, in test files, and in interactive sessions. It even works when you `source()` a script from a different directory, because it finds the project root, not the script's location.

Avoid `setwd()`, which creates a hidden dependency on your computer's directory structure. Avoid absolute paths, which break on every other machine. And never write `source("C:/Users/me/Desktop/functions.R")`, because that ties your code to one computer and one human's folder layout.

The `here` package detects the project root (the directory containing `.Rproj`, `.here`, or `.git`) and builds every path relative to it. A clean project layout looks like this:

```
my_analysis/
  my_analysis.Rproj
  data/               # raw data (read-only)
  R/                  # functions
  analysis/           # scripts or Quarto documents
  output/             # results, figures, tables
  renv.lock           # dependency lockfile
```

Raw data is sacred: never modify it. Read it, transform it in code, write outputs elsewhere. If someone asks "where did this number come from?", you can trace it from the output file back through the code to the raw data, and that traceability breaks the moment code and data live in different places.

A common mistake is placing data-cleaning code in a different project from the analysis, so that reproducing the analysis requires finding and running a separate project first. Keep everything in one project, or use `targets` (Section 33.5) to formalize the dependency.

Another common mistake: using `rm(list = ls())` at the top of a script to "clean up." This clears your workspace but does not restart R, so hidden state (loaded packages, modified options, changed working directories) persists. Instead, restart R (Ctrl+Shift+F10 in RStudio) for a true clean slate. Clean paths solve the "where is my file?" problem; but what about the "which version of dplyr was I using?" problem?

> **Opinion**
>
> If you would not email your project folder to a collaborator and expect it to work, it is not organized well enough.

Exercises

1. Create an RStudio project with the layout above. Place a CSV file in `data/`. Write a script in `analysis/` that reads it using `here::here()`. Verify it works by opening the project fresh and running the script.
2. Try `here::here("data", "penguins.csv")` from both the project root and a subdirectory. Does it return the same path?
3. Open an old script of yours that uses `setwd()` or absolute paths. Rewrite it to use `here::here()`. Does it still work when you move the project to a different folder?

33.3. renv: locking dependencies

`install.packages("dplyr")` today gives you a different version than six months from now, and when your code breaks or your results silently change, nothing in your source files will explain why.

`renv` isolates your project's package library:

```
renv::init()       # create a project-local library
renv::snapshot()   # record exact versions in renv.lock
renv::restore()    # install exactly the versions in the lockfile
```

`renv::init()` creates a private library for this project, isolating your package versions from every other project on your machine. `renv::snapshot()` writes `renv.lock`, a JSON file listing every package and its exact version: your dependency manifest, which you should commit to Git. `renv::restore()` reads that lockfile and installs exactly those versions, so a collaborator who clones your repo and runs `renv::restore()` gets the same packages you used.

The workflow: `init()` once, `snapshot()` after installing or updating packages, commit `renv.lock` to Git. That is all.

renv does not version R itself. If the R version matters, document it; Docker solves this completely (see Section 33.8), but is beyond this book. Should you commit `renv.lock` to Git? Always. It is small (a few KB of JSON), and it is the entire point of renv. Without the lockfile in version control, a collaborator cannot restore your environment. The `renv/` directory itself, which contains the actual installed packages, is typically git-ignored.

> **Opinion**
>
> `renv.lock` is boring. That is the point. Boring is reproducible.

One practical detail worth knowing: when you switch between projects that use renv, you do not need to do anything special, because each project has its own library and renv activates automatically when you open the project. Packages installed in one project are invisible to another, which prevents the classic problem of updating a package for one analysis and breaking a different one. Code and dependencies are now pinned; but what about the prose, the figures, the narrative that wraps around your analysis?

Exercises

1. Run `renv::init()` in a project. Install a package with `install.packages()`. Run `renv::snapshot()`. Open `renv.lock` and find the package and its version number.
2. Delete the package from your library (simulate a fresh machine). Run `renv::restore()`. Verify the package is back.
3. Open `renv.lock` in a text editor. What information does it store besides package names and versions?

33.4. Quarto: code and prose together

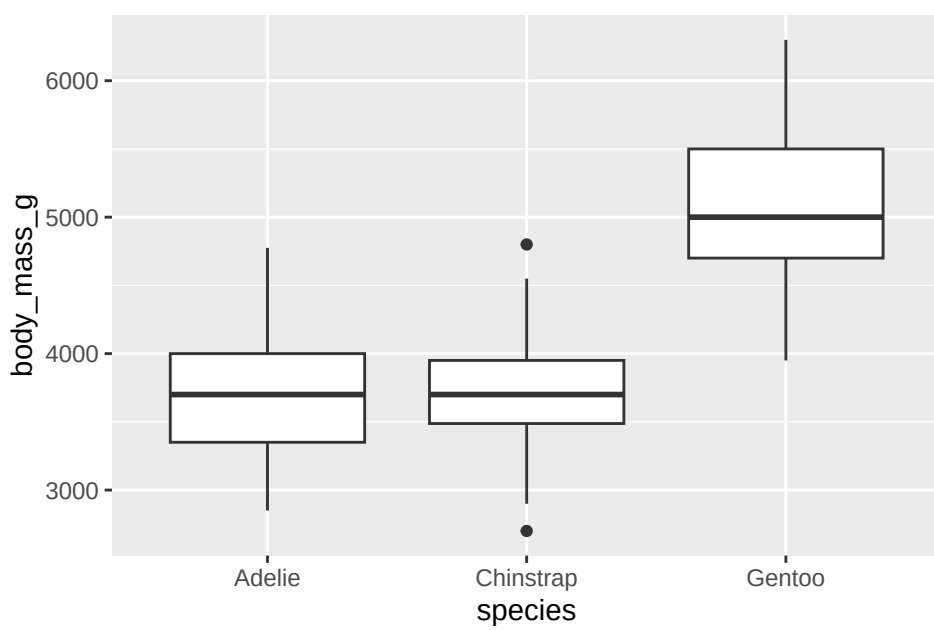
You have a script that produces three figures. You have a Word document that describes those figures. You update the script, re-run it, forget to update the Word document, and now the text describes figures that no longer exist. The architecture guarantees drift: code and prose live in different files, so they drift apart.

Quarto documents (`.qmd`) eliminate the gap by combining narrative text, code chunks, and their output in a single file. When you render the document, the code runs and the results appear inline:

```
---
title: "Penguin Analysis"
format: html
---

## Body mass by species
```

```
library(palmerpenguins)
#>
#> Attaching package: 'palmerpenguins'
#> The following objects are masked from 'package:datasets':
#>
#>     penguins, penguins_raw
library(ggplot2)
penguins |>
  ggplot(aes(species, body_mass_g)) +
  geom_boxplot()
#> Warning: Removed 2 rows containing non-finite outside the scale range
#> (`stat_boxplot()`).
```



No separate script that makes the figures, no separate Word document that describes them. If the data chang
render and everything updates. To render the document, run from the terminal:

```
::: {.cell}

```{.r .cell-code}
quarto render analysis.qmd           # defaults to the format in the YAML header
quarto render analysis.qmd --to pdf  # override the output format
quarto render analysis.qmd --to docx

:::
```

The `format` field in the YAML header sets the default (html, pdf, docx, revealjs for slides), and the `--to` flag overrides it. HTML needs no extra software. PDF requires a LaTeX installation; `quarto install tinytex` handles that. Word output produces a `.docx` that collaborators who do not use R can read and comment on. In RStudio, the "Render" button (Ctrl+Shift+K) does the same thing without the terminal.

Quarto is the successor to R Markdown, and the core idea (literate programming) dates back to Donald Knuth in 1984: write programs as documents where the prose explains the code, not the other way around. The lineage runs from WEB (Pascal/TeX) through Sweave (2002), knitr (2012), R Markdown, and finally Quarto (2022), each generation bringing better tooling and broader language support. If you know R Markdown, you already know Quarto. The main differences: Quarto uses `#|` for chunk options instead of the chunk header, supports Python, Julia, and Observable in addition to R, and has better defaults for academic output (cross-references, citations, callouts).

Output formats include HTML, PDF, Word, presentations, websites, and books (this book is written in Quarto). The format is flexible, but the real control comes from chunk options, which you place at the top of each chunk with the `#|` prefix:

`echo: false` hides the code and shows only the output. `eval: false` shows the code but does not run it. `fig-cap` adds a caption. `cache: true` caches results so unchanged chunks do not re-run. These options give you fine control over what the reader sees versus what runs behind the scenes.

For academic writing, Quarto supports citations natively. Place a `.bib` file in your project and reference entries with `@key`:

```
---
title: "My Analysis"
bibliography: references.bib
---

As shown by @wickham2019, tidy data principles simplify analysis.
```

The citation is rendered in the output and a reference list is appended automatically, so there are no more manually typed reference sections drifting from the actual citations. Quarto keeps code and prose in sync within a single document; but when your analysis spans multiple scripts with complex dependencies, a different tool keeps the steps themselves in sync.
```

Exercises

1. Create a `.qmd` file with a title, a code chunk that loads a dataset, and a code chunk that makes a plot. Render it to HTML. Change the data and re-render.
2. Add `echo: false` to a code chunk. What changes in the rendered output?

33.5. targets: pipeline automation

Your analysis has five steps: clean data, fit model, run diagnostics, make figures, render report. You run them by hand, in order, every time. One day you change the model but forget to re-run the diagnostics. The figures now describe a model that no longer exists. How long before you notice?

`targets` prevents this by making you declare the pipeline: each step (target) is a function call with explicit inputs, and `targets` tracks what changed so it only re-runs what needs re-running.

```
# _targets.R
library(targets)

tar_option_set(packages = c("dplyr", "ggplot2"))

list(
  tar_target(raw_data, read.csv(here::here("data", "penguins.csv"))),
  tar_target(clean_data, clean_penguins(raw_data)),
  tar_target(model, lm(body_mass_g ~ species, data = clean_data)),
  tar_target(fig, plot_results(model))
)
```

`tar_make()` runs the pipeline. `tar_visnetwork()` visualizes the dependency graph. `tar_read(model)` retrieves a cached result.

Each target is a function call. Functions are the unit of computation (Chapter 7); targets are the unit of caching. If `raw_data` has not changed, `clean_data` does not re-run. If you change `plot_results()`, only `fig` re-runs.

The caching works precisely because the functions are pure: given the same inputs, `clean_penguins()` returns the same output, so `targets` can skip it when the inputs have not changed. A function with side effects (one that reads from a database, modifies a global variable, or depends on the current time) would break the caching, because the same inputs would no longer guarantee the same output. `targets` makes the functional discipline from Chapter 18 pay off at the project level: pure functions compose into pipelines that `targets` can reason about, cache, and selectively re-execute.

Not every project needs `targets`. A single Quarto document handles simple analyses perfectly well. Reach for `targets` when you have long-running steps, many interdependent outputs, or pipelines that change frequently.

The real power shows when your pipeline is expensive: if a model takes an hour to fit, you do not want to re-fit it every time you tweak a color on a plot. `targets` caches the model result and only re-fits when the model code or its inputs change, so the plot target re-runs in seconds because it reads the cached model. `tar_visnetwork()` produces a dependency graph showing which targets are up to date (green), outdated (blue), or errored (red), which makes debugging complex pipelines far easier than staring at a collection of numbered scripts.

> **Opinion**
>
> `targets` is overkill for a homework assignment and necessary for a thesis chapter. Know where your project falls.

One design principle worth internalizing: each target should be defined by a function, not raw code. Instead of writing the transformation inline, write a function in `R/` and call it from the target. This makes targets testable (you can test the function independently) and readable (the pipeline reads like a table of contents):

```
# R/functions.R
clean_penguins <- function(data) {
  data[!is.na(data$body_mass_g), ]
}

plot_results <- function(model) {
  plot(model, which = 1)
}
```

```
# _targets.R
source("R/functions.R")
list(
  tar_target(raw, read.csv(here::here("data", "penguins.csv"))),
  tar_target(clean, clean_penguins(raw)),
  tar_target(model, lm(body_mass_g ~ species, data = clean)),
  tar_target(fig, plot_results(model))
)
```

Exercises

1. Define a small `targets` pipeline with three steps: read data, compute a summary, make a plot. Run it with `tar_make()`. Change the summary function and run `tar_make()` again. Observe which targets re-run and which are skipped.
2. Run `tar_visnetwork()` on your pipeline. What do the colors mean?

33.6. Version control with Git

You have a working analysis. You try a new approach, and it breaks everything. You hit Ctrl+Z forty times, but the file is not quite back to where it was, and you are not sure which of the six files you changed. If only you had saved a snapshot before experimenting.

Git is that snapshot system. Every change is recorded as a *commit*: a frozen image of your project at one moment, with a message describing what changed and why. You can go back to any previous snapshot, compare any two, and branch your work into parallel lines that merge back together.

Why Git for data analysis:

- **Undo mistakes without losing work.** Every commit is a checkpoint you can return to. Accidentally deleted a function? `git checkout -- file.R` brings it back. Broke your analysis and cannot figure out what changed? `git diff` shows exactly which lines differ from the last working state.
- **Track the evolution of your analysis.** Three months from now, a reviewer asks why you removed a covariate. `git log` shows the commit where you removed it, with a message explaining the decision. Without Git, that reasoning is lost.
- **Collaborate without emailing files back and forth.** Multiple people can work on the same project, each on their own *branch*, and Git merges their changes. Conflicts (two people editing the same line) are surfaced explicitly, not silently overwritten.
- **Share and cite.** GitHub and GitLab give your project a permanent URL. Zenodo can mint a DOI for a specific release, making your code citable in papers.

The daily workflow

Git has many commands. You need six:

```
git status                              # what has changed?
git add file.R                          # stage a file for the next commit
git commit -m "Add bootstrap analysis"  # record the snapshot
git log --oneline                       # see recent history
git diff                                # see unstaged changes line by line
git push                                # send commits to GitHub
```

The mental model: you make changes to files, `git add` selects which changes to include in the next snapshot, `git commit` takes the snapshot with a message, and `git push` sends it to a remote server. `git pull` does the reverse, fetching commits from the remote and incorporating them into your local copy.

RStudio has a built-in Git pane that shows modified files, lets you stage changes, write commit messages, and push to GitHub without touching the terminal. For initial setup, `usethis::use_git()` initializes a repository, and `usethis::use_github()` creates a remote on GitHub and pushes your code in one step.

Branching

Branches let you try something without risking the main line of work:

```
git branch try-new-model     # create a branch
git checkout try-new-model   # switch to it
# ... make changes, commit ...
git checkout main            # switch back
git merge try-new-model      # incorporate the branch's changes
```

If the experiment works, merge it. If it does not, delete the branch. The main branch is untouched either way, your code stays the same regardless of which experiment succeeded.

What belongs in Git

Commit: code, documentation, `renv.lock`, `_targets.R`, `.gitignore`, small data files (under a few MB), Quarto source files.

Do not commit: large data files, generated output (figures, HTML reports, `_targets/` cache), secrets and passwords, `.Rhistory`, `.RData`, `.DS_Store`.

`.gitignore` tells Git what to skip. A good starting point for R projects:

```
.Rhistory
.RData
.Rproj.user
_targets/
docs/
*.html
*.pdf
```

`usethis::use_git_ignore()` helps set it up.

Commit messages

Write commit messages that describe *why* you changed something, not *what* you changed. Git already records every added and deleted line; your message adds the reasoning that the diff cannot show.

Good: "Remove species interaction term (AIC worse by 4.2)." Bad: "Update analysis.R." Good: "Fix off-by-one in bootstrap loop causing n+1 resamples." Bad: "Bug fix."

Your future self will search through `git log` when something breaks, and those messages are the only documentation of your decision-making process.

Going further

This section covers enough Git to track your work, collaborate, and recover from mistakes. It does not cover rebasing, cherry-picking, bisecting, or other advanced operations. Jenny Bryan's *Happy Git and GitHub for the useR* (happygitwithr.com) is the best reference for R users and covers installation, SSH keys, merge conflicts, and common workflows in detail.

33.7. set.seed() and session info

Any analysis involving randomness (simulation, resampling, train/test splits) needs `set.seed()`:

```
set.seed(42)
sample(1:100, 5)
#> [1] 49 65 25 74 18
```

Without `set.seed()`, every run produces different numbers. With it, the same seed always produces the same sequence. Place it at the top of your script or Quarto document; the specific number does not matter (42, 123, 2024, anything), but it must be fixed and documented.

One subtlety: R 3.6.0 changed the default random number generator, so code using `set.seed()` before and after that version may produce different sequences even with the same seed. If exact reproducibility across R versions matters, specify the RNG kind: `set.seed(42, kind = "Mersenne-Twister", normal.kind = "Inversion")`.

At the end of reports, record your environment:

```
sessionInfo()
# or
sessioninfo::session_info()
```

This captures R version, OS, and package versions. If results differ on another machine, the session info is the first place to look. Small practices, both of them, but they close gaps that the larger tools leave open.

Exercises

1. Run the same `sample()` call twice without `set.seed()`. Do you get the same result? Now add `set.seed(123)` before each call. What changes?
2. Run `sessionInfo()` and find: your R version, your operating system, and the version of a package you use frequently.

33.8. The reproducibility stack

Here is when to reach for each tool:

1. **RStudio projects + here**: always. Zero cost, immediate benefit.
2. **Git**: always, for anything beyond a one-off exploration.
3. **renv**: for any project that needs to produce the same results later.
4. **Quarto**: for any analysis that produces a report or document.
5. **targets**: for multi-step pipelines with long-running computations.
6. **Docker**: for complete environment reproducibility (R version, system libraries). Beyond this book, but worth knowing it exists.

You do not need all of these for every project. Start with projects + here + Git, add renv when versions matter, add Quarto when you write reports, add targets when your pipeline grows.

A decision guide:

If your project…	You need
Has any R code at all	RStudio project + here
Will exist for more than a week	Git
Uses packages that update	renv
Produces a report or paper	Quarto
Has steps that take >30 seconds	targets
Must run on a different OS	Docker or Nix

Each layer solves one failure mode. Together, they make "works on my machine" a non-issue.

The common thread across all of them is automation: `here` automates path construction, `renv` automates dependency tracking, Quarto automates the link between code and output, `targets` automates pipeline execution, and Git automates change tracking. Each requires a one-time setup cost and then runs automatically.

33.9. Beyond renv: full-stack reproducibility

`renv` captures R package versions. It does not capture R itself, system libraries (libcurl, GDAL, libxml2), or the C compiler that built them. When your code depends on any of these (and spatial code, for instance, almost always depends on GDAL), `renv` alone leaves a gap.

Docker fills it by bundling everything into a container: OS, R, system libraries, packages, your code. The Rocker project (rocker-project.org) provides pre-built R images:

```
# Dockerfile
FROM rocker/r-ver:4.4.0
```

```
RUN install2.r dplyr ggplot2
COPY . /analysis
CMD ["Rscript", "analysis/main.R"]
```

A Dockerfile is a recipe that always produces the same environment. Build once, run anywhere. The trade-off is complexity: Docker adds a layer of tooling (images, containers, registries) that takes time to learn.

Nix takes a different approach. Where Docker isolates by containerization, Nix isolates by functional purity: the Nix package manager treats every package as a pure function from its inputs (source code, dependencies, compiler flags) to its output (the built artifact). Fix all inputs, get a deterministic output. This is referential transparency applied to software builds, the same principle that makes pure functions predictable in R: same inputs, same output, no hidden state.

The `rix` R package makes Nix accessible from R:

```
library(rix)

rix(
  r_ver = "4.4.0",
  r_pkgs = c("dplyr", "ggplot2", "palmerpenguins"),
  system_pkgs = NULL,
  ide = "rstudio",
  project_path = "."
)
```

This generates a `default.nix` file that pins everything: R version, package versions, system libraries, even the C compiler. Running `nix-build` on any machine with Nix installed produces an identical environment. Where `renv.lock` captures one layer (R packages), `default.nix` captures all of them.

The practical trade-off: Nix has a steep learning curve and limited Windows support (it runs natively on Linux and macOS, or via WSL on Windows), while Docker is more widely adopted and better supported across platforms. Both achieve full-stack reproducibility; they differ in mechanism and philosophy. Docker says "ship the whole machine." Nix says "describe the machine as a function and let anyone rebuild it."

For most R users, `renv` is sufficient. Add Docker or Nix when your results depend on system-level components, when you need to guarantee reproducibility across operating systems, or when a collaborator reports "it doesn't work on my machine" and the problem turns out to be something no R package can fix. Which layer you pin depends on how far the reproducibility guarantee needs to reach.

Afterword

Chapter 1 put four numbers on a table and told two stories about adding them. Mr. State held a running total. Mr. Fold pressed pairs together until one number remained. Both got 18. That split between stepping through state and folding over structure ran through every chapter that followed: `for` loops and `Reduce()`, Turing machines and lambda calculus. Church and Turing proved in 1936 that the two models compute exactly the same class of functions. Any question you can ask of one, you can ask of the other.

Curry published on combinatory logic in the 1930s. Howard arrived at the same structure from proof theory three decades later, without knowing about Curry's work. What they both found was that typed lambda calculus and natural deduction in logic follow the same rules. A function of type `A -> B` is a proof that A implies B; a pair of type `(A, B)` is a proof that both A and B hold. If a program type-checks, the types constitute a theorem, and the type checker is verifying the proof. The correspondence was not something anyone set out to build. Two formal systems, designed for completely different purposes, turned out to be the same system.

Voevodsky pushed the question one level deeper. He proposed that equality is not a yes-or-no judgement but a structure: two values are equal if there is a path connecting them, and that path is itself a type, which can have its own paths (equalities between equalities). The entire framework of homotopy theory (the branch of topology that studies shapes up to continuous deformation) maps onto type theory. A path in a topological space is a geometric object; in homotopy type theory, that same path is a term you can type-check and compute with. What looked like a metaphor ("programs are *like* proofs," "types are *like* spaces") kept turning out to be an identity. Homotopy type theory is young, and where it leads is open.

In 1945, Eilenberg and Mac Lane introduced category theory to track the relationships between algebra, topology, and logic. What they found was that the objects mattered less than the maps between them. Saunders Mac Lane called it "abstract nonsense." He meant it as a compliment. The vocabulary (functors, monoids, natural transformations) is forbidding, but you have been using the structures since chapter 5. `map(x, sqrt)` returning a list of the same length is a functor. Composing functions with `|>` gives you a category where objects are types and arrows are functions. `Reduce()` over an associative operation with an identity element is a monoid. `pivot_longer()` and `pivot_wider()` without losing information is a natural isomorphism.

If you go looking you will find more of these connections, between R and Haskell, between type theory and topology, between the code you write on a Tuesday afternoon and mathematics that predates computers by centuries.

The connections are still forming. R 4.1 gave the language a native pipe (`|>`) and a one-character lambda (`\(x)`). R 4.4 moved the null-coalescing operator (`%||%`) into base R and patched the internal dispatch machinery so that S7, a class system built entirely out of

functions, could sit on top of it. duckplyr routes dplyr verbs through DuckDB without changing a line of user code, because the verbs were designed as a functional interface: data frame in, data frame out, no side effects. The Polars data frame engine, written in Rust, connects to R through extendr, and Rust's ownership model maps onto R's copy-on-modify semantics closely enough that the two languages share data without defensive copying. The tidyverse's AI toolkit (ellmer for calling language models, ragnar for retrieval-augmented generation, mcptools for exposing R functions as tools that language models can call) is built on the same pattern: pure functions composed into pipelines. Church published the lambda calculus in 1936. Ninety years later, the architecture has not needed replacing.

I have been down this rabbit hole for years. I am nowhere near the bottom.

Thank you for reading.

Index

+ operator overloading, 310
.Call interface, 371
.Call(), 333
.Internal(), 349
.Primitive(), 349
«- (super-assignment), 213
[(single bracket), 104
[[(double bracket), 104
$ (dollar sign), 104
%>% (magrittr pipe), 168

absolute path, 140
abstract syntax tree (AST), 300
accumulate(), 246
across(), 159, 230
adverb, 239
aes(), 185
aesthetic, 185
algorithm, 96
all.equal(), 55
anonymous function, 66
apply family, 223
argument, 39
arrange(), 149
as_tibble(), 121
assertion, 289
assignment, 21
associativity, 244
atomic vector, 27

Backus, John, 14
bang-bang operator, 304
base case, 251
bench::mark(), 323
benchmarking, 323
beta reduction
    in R, 354
Big-O notation, 87
binary search, 88, 257
binding, 21
Bioconductor, 283
Boole, George, 76
Boolean, 73
bquote(), 305
byte compilation, 335

C API, 371
call object, 299
call-by-name, 265
call-by-need, 266
call-by-value, 265
case_when(), 85
catastrophic cancellation, 56
category theory, 363
cbind(), 120
Chambers, John, 16
character, 125
CharacterVector, 373
Church boolean, 354
Church encoding, 362
Church numeral, 356
Church, Alonzo, 9
Church-Turing thesis, 10, 98
CI/CD, 392
class, 277
closure, 70, 213
CLOSXP, 339
Codd, Edgar, 113
code generation, 307
coercion, 30
column, 113
computation, 7
condition, 295
condition number, 59
constructor, 280
contract, 287
control flow, 73
coordinate system, 186
copy-on-modify, 16
    performance, 329
count(), 157
cppFunction(), 373
CRAN, 392
CSV, 140
cumprod(), 249
cumsum(), 249
Curry, Haskell, 50
Curry-Howard correspondence, 51, 362
currying, 41

data frame, 113

data masking, 271
data.frame(), 115
Date, 134
default argument, 40
defensive programming, 288
deparse(), 301
dependency graph, 401
DESCRIPTION, 384
Design by Contract, 287
dispatch, 277
divide and conquer, 256
domain-specific language (DSL), 309
double, 29
dplyr (package), 149
dynamic lookup, 210

Eiffel (language), 287
embrace ({{}}), 272
enclosing environment, 212
encoding, 126
enquo(), 302
Entscheidungsproblem, 96
environment, 25, 207
ENVSXP, 339
error, 289
error handling, 239
eval(), 270, 300
evaluation, 19
Excel, 143
execution environment, 211
expr(), 299
expression, 9, 19
expression builder, 310
expression object, 300
extendr (package), 375

facet, 185
factor, 130
file path, 140
filter(), 149
first-class function, 13, 65
fixed point, 260
fixed-point combinator, 360
flame graph, 324
floating-point, 53
fold, 243
for loop, 222
forcats (package), 132
force(), 235
formula interface, 310
Frege, Gottlob, 368
function, 39
    as argument, 66
    as return value, 69
function composition, 166
function factory, 233
functor, 363

garbage collection, 14, 21
generational garbage collection, 344
generic function, 277
Gentleman, Robert, 17
geom, 185
geom_bar(), 192
geom_histogram(), 190
geom_line(), 192
geom_point(), 186
ggplot2 (as DSL example), 310
ggplot2 (package), 185
Git, 403
GitHub Actions, 392
global environment, 208
global string pool, 341
grammar of graphics, 185
grep(), 127
group_by(), 156
gsub(), 127
Gödel, Kurt, 263

halting problem, 96
hash table, 90
Haskell (language), 265
helper, 280
here(), 139, 396
heterogeneous, 103
higher-order function, 13, 66
Hilbert matrix, 59
Hilbert, David, 7
Howard, William, 50
Hutton, Graham, 249

identity element, 244
IEEE 754, 53
if/else, 83
ifelse(), 84
Ihaka, Ross, 17
ill-conditioned, 59
incompleteness theorem, 263
infix operator, 315
inheritance, 281
integer, 29
IntegerVector, 373
INTSXP, 339
invariant, 287
iteration, 221

join, 180
JSON, 107

Kleene, Stephen, 129

labs(), 194
lambda calculus, 9
    in R, 353
lapply(), 66, 223
layer, 186
lazy evaluation, 265
lazy evaluation trap, 234
left fold, 243
left_join(), 180
Leibniz, Gottfried, 367
levels, 132
LGLSXP, 339
linked list
    Church encoding, 359
LISP, 263
Lisp, 14
list, 103
    nested, 107
LISTSXP, 338
lockfile, 398
log-sum-exp, 63
logic gate, 76
logical operator
    AND, 75
    NOT, 75
    OR, 75
long format, 174
lubridate (package), 135

machine epsilon, 53
magrittr (package), 168
Map(), 228
map(), 224
map2(), 227
map_chr(), 224
map_dbl(), 224
map_int(), 226
map_lgl(), 226
mark-and-sweep, 344
matrix decomposition, 61
McCarthy, John, 14, 263
McIlroy, Doug, 165
memoization, 238
memory allocation, 328
merge sort, 256
message, 289
metaprogramming, 299
method, 277
Meyer, Bertrand, 287
microbenchmark(), 324
missing value, 35
monoid, 244
    category theory, 364
Mr. Fold (allegory), 8
Mr. State (allegory), 7
Mr. Tape (allegory), 7
multiple dispatch, 283
mutable state, 214
mutate(), 149

n(), 157
NA, 35
name masking, 210
NAMESPACE, 384
native pipe, 165
new_class(), 284
new_generic(), 284
NextMethod(), 281
non-standard evaluation (NSE), 270
Norvig, Peter, 217
NP-hard, 100
NULL, 28
numerical stability, 57
NumericVector, 373

O(1), 88
O(n log n), 88
O(n squared), 88
O(n), 88
operator overloading, 312

P vs NP, 99
package, 383
pair
    Church encoding, 358
pairlist, 338
parallel, 335
parameter, 39
parent environment, 207
parse(), 301
partial application, 233
paste(), 126
pattern matching, 128
pipe, 165
pipeline, 167, 401
pivot_longer(), 175
pivot_wider(), 178
pkgdown (package), 392
placeholder, 168
pmap(), 227
point-free, 167
POSIXct, 134
possibly(), 239
postcondition, 287
pre-allocation, 328
precondition, 287
primitive recursive function, 263

product type, 114
profiling, 323
promise, 265
proof by induction, 50
property, 284
PROTECT(), 371
purrr (package), 224
Python integration, 377

Quarto, 399
quietly(), 240
quosure, 273
quote(), 299
quoting, 273

R API, 371
rapply(), 258
rbind(), 119
Rcpp (package), 333, 373
RDS, 145
read.csv(), 141
read_csv(), 141
readr (package), 141
readRDS(), 145
readxl (package), 143
REALSXP, 339
recursion, 47, 251
recursive case, 251
recursive data structure, 103
recycling, 33
Reduce(), 243
reduce(), 246
reference counting, 338
referential transparency, 21
regex, 128
regular expression, 128
relational algebra, 149
relational model, 113
relative path, 139
renv (package), 397
REPL, 19
reproducibility, 395
reticulate (package), 377
return value, 41
Rice's theorem, 98
right fold, 245
rlang (package), 273, 299
rounding error, 54
row, 113
roxygen2 (package), 384
Rprof(), 324
RStudio project, 396
Rust integration, 375

S language, 16
S3, 279
S3 class, 312
S4, 283
S7, 283
safely(), 239
sapply(), 65, 223
saveRDS(), 145
scale, 185
scan, 245
Schönfinkel, Moses, 167
Scheme, 15
scoping
    dynamic, 15
    lexical, 15
search path, 209
seed, 405
select(), 149
separate(), 182
set.seed(), 405
setwd(), 139
SEXP, 337
    C level, 371
SEXPREC, 337
Shannon, Claude, 77
short-circuit evaluation, 76
side effect, 22
slice(), 159
sloop (package), 279
slot, 283
sourceCpp(), 373
space complexity, 87
splice operator, 304
split-apply-combine, 230
SQL, 161
stack overflow, 251
stat, 185
Steele, Guy Lewis Jr., 15
stopifnot(), 46, 287
str(), 107
string, 125
stringr (package), 127
STRSXP, 339
subsetting, 34
    logical, 34
substitute(), 270, 302
summarise(), 149
Sussman, Gerald Jay, 15
switch(), 86
symbol, 301

tail recursion, 254
tail-call optimization, 254
targets (package), 401
testthat (package), 389

theme, 185
thunk, 255
tibble, 120
tidy data, 173
    one observation per row, 173
tidy evaluation, 273
tidyr (package), 175
time complexity, 87
trampoline, 255
truth table, 76
tryCatch(), 294
Turing machine, 8
Turing, Alan, 8
type tag, 338
TYPEOF(), 339

undecidable, 97
unite(), 183
UNPROTECT(), 371
use_testthat(), 389
UseMethod(), 279
UTF-8, 126

validator, 280
vapply(), 223
VECSXP, 339
vector
    character, 27
    logical, 27
    numeric, 27
vectorization, 16, 31
    performance, 326
version control, 403
vignette, 390

walk(), 229
warning, 289
wide format, 174
workflow, 406
working directory, 139
write_csv(), 142

Y combinator, 260, 360

Z combinator, 360